

CROSS-LAYER SYSTEMS OPTIMIZATION FOR
FAULT-TOLERANT QUANTUM COMPUTERS

by

Satvik Maurya

A dissertation submitted in partial fulfillment of
the requirements for the degree of

Doctor of Philosophy

(Electrical and Computer Engineering)

at the

UNIVERSITY OF WISCONSIN-MADISON

2026

Date of final oral examination: July 23, 2026

The dissertation is approved by the following members of the Final Oral Committee:

Swamit Tannu, Computer Sciences, University of Wisconsin-Madison

Parmesh Ramanathan, Electrical and Computer Engineering, University of
Wisconsin-Madison

Aws Albarghouthi, Computer Sciences, University of Wisconsin-Madison

Frederic T. Chong, Computer Sciences, University of Chicago

Michael Beverland, Research Scientist, IBM Quantum

Dedicated to my brother, Sarthak.
You made the world a better place.

And to my parents,
for their love, patience, and belief in me.

Per aspera ad astra.

Acknowledgments

This dissertation could not have been possible without the help, guidance, and support of numerous people. I will do my best to thank each and every one of them here.

First and foremost, I would like to thank my parents for their love and belief in me. They have supported every decision I have made, and helped me overcome every challenge I have faced. From cultivating my interest in science at an early age to driving me around to get to and from classes and tests for the (infamous) Indian college entrance exams, they have toiled and sacrificed to give me the foundations that have made me who I am today. I dedicate this dissertation to them.

This dissertation could have never been written without the guidance, support, and the faith put in me by my advisor, Swamit Tannu. His patience in training me at the very beginning (especially with making figures, I hope I am better at them now), and the freedom he gave me to find and pursue my own research interests created a research environment I could thrive in. I have learned a ton while working with him, and I will forever be grateful.

I am extremely grateful to have had my partner Lisa through a lot of my time in graduate school. From hearing me rant about oblivious reviewers who had the audacity to reject my papers to trying to explain decoding and quantum error correction, she has given me all the love and support I could have asked for. I am also grateful to Lisa's family for all their love and support, and I consider myself very lucky to have all of them in my life.

I am lucky to have had so many great friends during my time in Madison. Logan, who met me reluctantly at the very beginning, is now one of my closest friends, and I have enjoyed all the fun and trips we've had along with Matt and Lane. He has always been supportive and helpful, and I'm lucky to have him as a friend. Abtin and Amanda have been a huge part of this journey, even though they maintain that I did not exist before the Fall of 2022 since that is when I first met them. We have gone to conferences, trivia, and even Disneyland together, and have become my closest friends the past few years. Along with

Amberlee and Eli, we have enjoyed all that Madison has to offer and I am grateful to have shared this journey with them. Playing Counter-Strike late at nights with Lane, Issac, and Tanner; hanging out and discussing every topic under the sun with Avani, Robert, Wihan, and Aidana; grabbing drinks and playing squash with Tanuj: I have enjoyed each and every moment with all of you and I thank you all.

I owe a lot to Michael Beverland and am grateful for all his help, mentoring, and training in the art of decoding. Michael has often gone out of his way and taken time out of his busy schedule to speak with me, answer all the questions I've ever had, and deal with some of my not-so-smart moments. Collaborating with Michael was an unbelievably rich learning experience for me, and has opened many doors for me.

I have collaborated with some wonderful people during my PhD. Ben Lienhard, who took the time to respond to my email asking for his readout dataset – thank you, that email and subsequent discussions were the genesis of HERQULES and several other papers on qubit readout. Scott Lekuch took a chance on me during my first internship at IBM, and I am grateful to have been given the freedom to pursue my research interests during my internship. I am grateful to Alper Buyuktosunoglu and David Trilla for agreeing to working with me on a (non quantum) side project during an IBM internship that ended up being a pretty good hardware prefetcher.

I would like to thank my final defense committee – Aws Albarghouthi, Parmesh Ramathan, Fred Chong, and Michael Beverland – for agreeing to be readers of this dissertation and attending my defense. I have had great discussions with each of them, and they have all inspired me in different ways.

Finally, I would like to thank the people at the University of Wisconsin-Madison and the city of Madison. I have spent the last five and a half years in this wonderful city, and considering that I came here knowing very little about it, I think I am truly lucky to have experienced this city and university. From biking around the lakes, great restaurants (like the G.O.A.T wings place – Chicken Licks), wonderful and kind people, to sailing in Lake Mendota – UW-Madison and the city of Madison have given me the best five and a half years of my life so far. On, Wisconsin!

Satvik Maurya
Madison, WI
July 2026

Contents

Acknowledgments	iii
Contents	v
List of Tables	x
List of Figures	xii
Abstract	xxiii
1 Introduction	1
1.1 A Brief History of Quantum Computing	1
1.1.1 Genesis	1
1.1.2 Breakthroughs in device physics	2
1.1.3 Quantum in the cloud	3
1.1.4 Experimental demonstrations of quantum error correction (QEC) .	3
1.1.5 The scaling era	4
1.2 Contributions	5
1.2.1 Scalable qubit control and readout (Chapter 3)	7
1.2.2 Accelerated decoders for QEC (Chapter 4)	8
1.2.3 Systems for quantum computers (Chapter 5)	9
1.3 Organization	10
2 General Background	12

2.1	The Quantum and Classical Worlds	12
2.2	Qubits	13
2.3	The Bloch Sphere	14
2.4	Quantum Gates	14
2.4.1	Measuring qubits	21
2.4.2	Quantum circuits	21
2.5	Quantum Computers are Noisy	21
2.5.1	Gate errors	22
2.5.2	Decoherence	22
2.5.3	Measurement errors	22
2.5.4	Leakage	22
2.5.5	Crosstalk	23
2.6	Qubit Technologies	23
2.7	Noisy Intermediate-Scale Quantum (NISQ) Era	23
2.8	Quantum Error Correction	25
2.8.1	The repetition code	26
2.8.2	Stabilizer formalism	30
2.8.3	Decoding	31
2.8.4	Threshold theorem	35
2.8.5	Some common QEC codes	36
2.8.6	Logical operations	39
2.9	Classical Control for Quantum Computers	40
3	Optimizing Qubit Control and Readout	42
3.1	Introduction	42
3.1.1	Contributions	42
3.1.2	Organization	43
3.2	Superconducting Qubit Control and Readout	43
3.2.1	Superconducting qubits	43
3.2.2	Control	44

3.2.3	Readout	46
3.3	Waveform Memory Compression	48
3.3.1	Waveform memory bottleneck	50
3.3.2	Waveform compression	54
3.3.3	COMPAQT microarchitecture	60
3.3.4	Evaluating compressed waveform memories	64
3.3.5	Scalability of COMPAQT with ASICs	68
3.3.6	Conclusions	69
3.4	Hardware-Efficient Machine Learning-based Qubit Readout	70
3.4.1	Scaling high-fidelity readout	71
3.4.2	Enabling high-fidelity readout	73
3.4.3	Impact on quantum error correction	82
3.4.4	Conclusions	83
3.5	Summary	84
4	Accelerated Decoding for QEC	85
4.1	Introduction	85
4.1.1	Contributions	86
4.1.2	Organization	86
4.2	The Decoding Problem	87
4.2.1	Quantum degeneracy	87
4.2.2	Real-time decoding	88
4.3	Clustering Decoder	90
4.3.1	Cluster representation	91
4.3.2	Cluster initialization	92
4.3.3	Cluster growth	94
4.3.4	Cluster merge	94
4.3.5	Cluster validity and solution	95
4.3.6	Algorithm output	97
4.4	Results	98

4.5	Summary	99
5	Systems for Quantum Computers	101
5.1	Introduction	101
5.1.1	Contributions	102
5.1.2	Organization	102
5.2	Logical Operations and Windowed Decoding	103
5.2.1	Logical operations	103
5.2.2	Windowed decoding	106
5.3	Decoder Provisioning and Scheduling	109
5.3.1	Decoder allocation is non-trivial	110
5.3.2	Coarse-grained scheduling	113
5.3.3	Fine-grained scheduling	117
5.3.4	Evaluations	121
5.4	Synchronizing Logical Qubits	127
5.4.1	Sources of Desynchronization	128
5.4.2	Synchronizing two patches	129
5.4.3	Synchronizing k patches	137
5.4.4	Evaluations	137
5.4.5	The <i>Extra Rounds</i> and <i>Hybrid</i> policies	141
5.4.6	Synchronization in neutral atom platforms	142
5.5	Summary	143
6	Related Work	145
6.1	Low-Level Control Hardware	145
6.1.1	Qubit control	145
6.1.2	Qubit readout	147
6.2	Decoding	149
6.3	Systems for Quantum Computers	151
7	Conclusion and Future Work	152

7.1	Summary	152
7.2	Future Work	154
7.3	Closing Remarks	157
A	Tools and Frameworks	158
A.1	HERQULES	158
A.1.1	Training	158
A.1.2	Inference	159
A.2	decoder-bench: Benchmarking QEC Decoders	159
A.2.1	Trace Generation	160
A.2.2	Trace Evaluation	163
A.3	Elastic Decoders and Scheduling	163
A.3.1	Scheduling	163
A.3.2	Capacity planning	164
	Bibliography	166

List of Tables

2.1	Common quantum gates with circuit symbols and unitary matrices.	20
2.2	Qubit technologies and their attributes.	24
2.3	Effect of each single-qubit bit-flip error on the three-qubit encoded state and the syndrome ancilla outcomes. Ancilla a measures the parity Z_1Z_2 ; ancilla b measures the parity Z_2Z_3 . A syndrome bit of 1 indicates odd parity, signalling an error.	28
2.4	Syndrome lookup table for the three-qubit bit-flip code. Each syndrome uniquely identifies the location of a single bit-flip error. Applying the same X gate at the inferred location restores the encoded state.	29
3.1	Parameters used to estimate memory capacity and bandwidth for controlling one qubit.	51
3.2	Implemented DCT variants.	57
3.3	RB fidelity for uncompressed baseline pulse and compressed pulses with window size of 16.	60
3.4	Hardware resources needed for the IDCT engine of DCT-W and int-DCT-W. .	62
3.5	Number of qubits that can be supported by an FPGA based design with uncompressed waveform memory and a design with compressed memory using int-DCT-W for different window sizes.	63
3.6	Minimum, maximum, and average compression ratios with int-DCT-W (WS=16). 65	
3.7	FPGA resource usage for the baseline and a single IDCT engine for int-DCT-W for different window sizes. The percentages represent the percent utilization of the total resources. The designs were synthesized for the same SoC - Xilinx zc7u7ev.	67

3.8	Qubit-readout accuracy of baseline [149], different designs, and cumulative accuracy $F_{5Q} = \sqrt[5]{F_1 F_2 F_3 F_4 F_5}$	76
3.9	Mean absolute values of the cross-fidelity (F^{CF}) for different hamming distances (lower is better).	81
4.1	FPGA resource utilization estimates for different decoders and code types. . .	99
5.1	Number of logical errors for different code distances d for the example shown in Figure 5.18(b).	133
5.2	Comparison of the idling period, extra rounds, and LER of different synchronization policies for the given configuration of patch cycle times and initial slack τ . The <i>Hybrid</i> policy achieves a significantly better LER by reducing the idling period and the number of extra rounds run.	135
5.3	System parameters used in evaluations.	138
5.4	Average reduction in the LER for all policies compared to the <i>Passive</i> policy for $d = 11, 13, 15$	142
5.5	Additional rounds needed for synchronization with the <i>Hybrid</i> policy – these result in an increase in the LER.	142

List of Figures

1.1	The quantum computing control stack. Layers shaded in blue represent the contributions of this dissertation (Chapters 3–5).	5
1.2	Generalization scope of each contribution. <code>COMPACT</code> and <code>HERCULES</code> are anchored in superconducting (SC) hardware but their principles reach other qubit technologies; the clustering decoder, decoder scheduling, and synchronization policies are technology-agnostic and generalize across QEC codes beyond the ones used for evaluation. Solid segments denote the evaluated scope; lighter segments denote the scope to which the underlying principle generalizes. (LS: lattice surgery; NA: neutral atom; BB: bivariate bicycle.)	6
2.1	Bloch sphere representation of a qubit state.	15
2.2	The Pauli-X gate maps $ 0\rangle$ to $ 1\rangle$ via a π rotation around the X-axis. The gray vector on the right shows the initial state before the gate.	16
2.3	The Hadamard gate maps $ 0\rangle$ to $ +\rangle = \frac{1}{\sqrt{2}}(0\rangle + 1\rangle)$, corresponding to a π rotation around the $(X+Z)/\sqrt{2}$ axis (red).	17
2.4	Quantum measurement collapses the qubit $ \psi\rangle$ into a classical bit, yielding outcome 0 or 1. The single wire carries quantum state; the double wire carries a classical value.	21
2.5	General NISQ algorithm workflow. A parameterized quantum circuit is executed and measured N times (shots). The bitstring outcomes are tallied and normalized to produce a probability distribution over measurement results, which a classical optimizer uses to update the circuit parameters θ	24

2.6	A general QEC cycle: physical qubits are encoded into logical qubit(s) by executing syndrome generation circuits, after which a subset of qubits are measured to generate a syndrome. These syndromes are measured over multiple rounds before being passed to the decoder.	25
2.7	Difference in the decoding graphs with a (a) Code-capacity noise model; (b) Circuit-level noise model.	33
2.8	(a) A surface code logical qubit: black nodes represent data qubits, nodes annotated with X and Z represent check qubits that measure X and Z parities, respectively. (b) Syndrome generation circuit of the surface code logical qubit. H: Hadamard gate, MR: Measurement and reset operation, two-qubit gates shown are CNOTs.	37
2.9	(a) A color code logical qubit mapped on a square lattice; (b) A portion of the lattice used for encoding logical qubits using the bivariate bicycle (BB) code, a qLDPC code. Notice the long range interactions between data and check qubits. For both color and BB codes, the black nodes represent data qubits, and colored nodes represent check qubits that measure stabilizers.	37
2.10	Overview of the classical control hardware for a quantum computer: the control system sits between the quantum processor and the user/application layer, and is responsible for translating programs into low-level instructions that can be executed on the quantum processor, as well as reading and measuring the state of the qubits, and running the error correction protocols.	40
3.1	(a) Parallel LC oscillator; (b) Josephson junction used for introducing nonlinearities and anharmonicity.	44
3.2	(a) An X-gate and its corresponding microwave pulse (in blue) The envelope of the pulse is shown in red (b) A typical pipeline for generating control pulses.	44
3.3	The qubit readout pipeline comprises 1) readout-probe-tone generation, 2) acquisition of qubit-state-dependent phase shift, and 3) readout-signal processing.	46

3.4	(a) Evolution of readout traces for ground and excited states in the 2D I-Q plane – the point annotated with ‘t = 0’ represents the first time bin of the readout trace, and every subsequent point comes later at a granularity of 50ns; (b) Mean values of multiple readout traces for both states showing the distinction between them.	47
3.5	Waveform (WF) memory (a) capacity and (b) bandwidth (BW) requirements (c) Peak and average bandwidth for select benchmarks (d) For an RFSoc platform, if only capacity is considered a constraint, the number of qubits that can be supported is >200. If bandwidth is taken as a constraint, this number drops by 5x to less than 40.	52
3.6	Architectural overview of COMPAQT.	54
3.7	(a) Compression Ratio (old size / new size) for five waveforms on IBM Guadalupe (b) Overall Compression Ratio for qft-4 (c) Average Mean Squared Error (MSE) for all waveforms of qft-4.	55
3.8	Input waveform and its DCT, the vertical green line shows the start of RLE where $ \text{sample} < \text{threshold}$	56
3.9	Randomized Benchmarking of two qubit system with the uncompressed baseline gate pulses and compressed gate pulses with int-DCT-W on IBM Guadalupe. .	59
3.10	Decompression pipeline for waveforms compressed with DCT-W/int-DCT-W (WS=8).	61
3.11	Histogram of the number of samples per window in the compressed waveform (including the RLE codeword) for 132 waveforms from IBM Guadalupe.	61
3.12	(a) Interleaved waveform samples in different BRAMs to boost bandwidth (b) Reduction in number of BRAMs for a waveform with maximum of three samples per window (including RLE code word) (c) Compressed window with two non-zero samples interleaved in three BRAMs (d) Interleaved memory organization for a pulse that requires four samples for a worst-case window.	61
3.13	Compression ratios of the basis gates for all 16 qubits of IBM Guadalupe using int-DCT-W with WS=16.	64
3.14	COMPAQT fidelity normalized to baseline fidelity on IBM Guadalupe. The baseline fidelities are annotated beneath the benchmark names.	66

3.15	Degradation of clock frequency compared to baseline for different decompressed waveform memory architectures.	67
3.16	Power dissipated by the cryogenic controller with the uncompressed and compressed memory.	68
3.17	Increasing the number of logical qubits that can be supported; (a) The peak concurrent operations for a $d=3$ code; (b) The maximum logical qubits that can be supported by different Xilinx RFSoc based designs.	69
3.18	Goal: Fast hardware readout discriminator.	71
3.19	Overview of the baseline [149] design.	72
3.20	Overview of pipeline designs; (a) Baseline – Feed forward Neural network (FNN) on ADC data, without dimensionality reduction; (b) MF-NN: dimensionality reduction with matched filter (MF), then as input to FNN (smaller network than the baseline); (c) MF-RMF-NN: Additional feature with relaxation matched filter (RMF) over and above MF-NN.	74
3.21	(a) Matched Filter (MF) training: Ground state traces (Tr_0) and excited state traces(Tr_1) are processed to get MF envelopes; (b) The dot product of the traces and MF envelopes is computed using multiply-and-accumulate (MAC) units; (c) The computed dot product is input for a Feed Forward Neural Network (FNN); (d) Comparison of LUT utilization between MF-NN and MF-RMF-NN (remaining component utilization is relatively low).	74
3.22	(a) Plot of I vs. Q values along with centroids of ground and excited-state traces shown as mean-trace values (MTV); (b) The traces corresponding to a ground state (0), excited state (1), and relaxation ($1 \rightarrow 0$).	79
3.23	MF-RMF-NN design overview.	80
3.24	Comparison of misclassification of ground ('0') and excited ('1') states for all qubits between MF-NN and MF-RMF-NN designs.	81
3.25	Logical error rate per round vs. physical gate error rate for surface code of distance 7 for different values of the averaged readout error (ϵ_R). The logical error rate is equal to the physical error rate at the dash-dot line.	82
3.26	(a) FPGA resource utilization for HERQULES; (b) Reduction in the syndrome generation cycle time (lower is better) for a surface-17 circuit [262] with gate durations from different systems (Google, IBM).	83

4.1	Linking low-level control and readout hardware with the decoder: the syndrome generation cycle (shown in program form) is repeatedly executed by the control pipeline, and syndromes are generated by the readout hardware which are then fed to the decoder.	86
4.2	Fault-tolerant T_L gate via magic state injection. The data qubit $ \psi_L\rangle$ is entangled with the ancilla $ A_L\rangle = T_L +\rangle$ via a logical CNOT, after which the ancilla is measured to yield a classical outcome m (double lines denote classical wires). A classically-controlled Clifford correction S_L^m is applied to recover $T_L \psi_L\rangle$, but the data qubit must stall (dashed segment) until the decoder resolves the accumulated syndromes and returns an updated Pauli frame.	89
4.3	The bitmap representation of a cluster assigns a bit to each node in the Tanner graph $\mathcal{G}$; the left bits correspond to syndrome nodes and the right bits to fault nodes. The shaded box highlights a cluster containing nodes c_0, f_0, f_1, f_2, f_3 , which are assigned a value of 1 in the cluster's bitmap, with all other nodes assigned 0. Note that in practice the full decoding graph is much larger than this example.	92
4.4	Mapping pre-decoder outputs to cluster decoder inputs. Confident errors ($\Lambda_j \geq \Lambda_{\text{accept}}$) are flipped to form the partial correction $\tilde{F}_{\text{pre}}$, reducing the syndrome weight, while uncertain faults ($\Lambda_{\text{erase}} \leq \Lambda_j < \Lambda_{\text{accept}}^+$) form the erasure set $\mathcal{E}$. .	93
4.5	(a) Example of cluster growth: all vertices adjacent to any vertex in the current cluster (highlighted) are added to the cluster. (b) Gateware implementation: columns of H^{ext} corresponding to 1 entries in b_{init} are selected; a column-wise OR reduction over these columns yields b'	95
4.6	Cutoff-time performance curves for FPGA-tailored decoders. Logical error rate versus FPGA cycle budget for Relay [191], cluster decoding, and standard OSD [16], with runs exceeding the cycle cutoff counted as failures. Relay achieves the lowest error rates across all tested budgets.	98
5.1	The fundamental split and merge operations of lattice surgery used for logical computation.	104

- 5.2 Logical T_L gate via Pauli product measurement. A joint measurement of $Z_L \otimes X_L$ is performed on the data logical qubit and the magic state ancilla $|A_L\rangle = T_L|+_L\rangle$. In the surface code, this PPM is implemented by a lattice surgery merge of the two patches. The classical outcome m determines whether a Clifford correction S_L^m must be applied to the data qubit. 105
- 5.3 (a) Streams of syndromes are processed in windows represented by *blocks* which are fixed decoding volumes: each block consists of a region in which corrections are committed, while buffer regions are used for merging corrections with overlapping windows; (b) Sliding windowed decoding (SWD) is inherently sequential: window W_{i+1} can only be processed after W_i has been processed; Parallel windowed decoding (PWD) can be (c) temporal and/or (d) spatial: in either case, W_i, W_{i+2} are treated as independent tasks and processed independently in parallel at time step t_j . W_{i+1} , which includes overlaps with W_i, W_{i+2} , is processed in the next time step. 107
- 5.4 (a) A full system using hardware (FPGA/GPU/ASIC) decoders: qubits are measured using the readout system, which produces syndromes that are buffered until a window has been collected. Then, the decoder uses these collected syndromes to determine a correction, which is communicated to software; (b) Effect of the normalized decoder latency t_D on the slowdown in processing 5d rounds of syndromes using SWD. The buffer region size is the normalized size of the buffer regions required ($n_W = 3d, n_{com} = d$ would yield an overlap of 0.66). 109
- 5.5 (a) Using simulations, effect of the normalized decoder latency t_D on the slowdown in processing 5d rounds of syndromes using Temporally parallel windowed decoding (PWD); (b) Logical qubits arranged in the EDPC layout with ancilla bridges of varying sizes to facilitate LS operations; (c) PWD is applicable in both space *and* time – this leads to an a variable and sudden increase in the number of decoders required for implementing PWD (assuming $t_W = 3d$). 111

5.6	(a) Distribution of the number of patches merged to perform lattice surgery-based logical measurements for different workloads – most workloads require short-range interactions on average, but some large outliers are possible due to routing constraints; (b) Distribution of the decoder demand over time for different benchmarks – these error bars capture the fluctuations in active logical patches that require decoding during the computation.	112
5.7	Distinction between coarse- and fine-grained scheduling. (Left) Coarse-grained decoder scheduling handles the prioritization between spatial and temporal decoding tasks. This includes handling critical decodes, which will always have some spatial component to them due to lattice surgery. (Right) Any decoders left over after handling critical decoding tasks are assigned to the temporal decoding tasks associated with logical qubits not involved in any operation in that time step. Scheduling decoders among this subset of qubits is what we refer to as fine-grained scheduling.	114
5.8	(a) Decoding volume of a single logical patch idling for a few rounds—only one decoder is needed; (b) Idling for multiple rounds creates a linear growth in windows, requiring temporally parallel windowed decoding (PWD); (c) A lattice surgery merge between distant patches temporarily increases decoder demand due to spatial parallelism, which may coincide with temporal parallelism (overlapping windows omitted for clarity); (d) Spatial PWD is not robust to decoder shortages—if two of three required decoders are unavailable, one must either decode the merged patch monolithically or serialize the protocol, both slow and undesirable. Deferring the task is also problematic, as the next slice may contain a critical decode that must wait for the deferred task, further slowing execution.	115

5.9	(a) (Left) Logical error rates (LER) for various lattice surgery configurations with and without spatially parallel window decoding (PWD) at circuit-level noise $p = 0.1\%$. (Right) Spatial parallelism does not affect LER but reduces decoder latency when merged patches are separated by $\geq 3d$ data qubits, assuming sufficient decoders for all distances (for $d = 11$, window size = $3d$). (b) Temporal PWD remains robust under decoder resource constraints: with only one of three decoders available, the decoding task can be processed incrementally by the single decoder. This creates a qubit backlog, which can be rapidly cleared once additional decoders become available via temporal parallelism.	116
5.10	(a) Illustration of the longest undecoded sequence – Q_1 has the longest undecoded sequence before the last decode (each square is a decoding block/window); (b) MFD policy – undecoded qubits are sorted according to the number of critical decodes they are involved in after time slice t and $M - C$ qubits are selected from this sorted list; (c) RR policy – the $M - C$ qubits decoded in time slice t are not decoded in time slice $t + 1$; (d) MLS policy – undecoded qubits are sorted according to their undecoded sequence lengths and $M - C$ qubits are selected from this sorted list.	119
5.11	Total slices executed for each benchmark.	122
5.12	Peak and average concurrency for non-Clifford operations for all benchmarks – concurrency is sporadic.	123
5.13	Number of decoders allocated for the baselines: Baselines-1 and 2 provision for the worst- and average-case decoder demands respectively.	123
5.14	(a) The relative reduction (higher is better) in the number of decoders required by the MLS policy to prevent any slowdowns compared to the two baselines. Baseline-2 provisions for fewer decoders than what is required by the MLS policy; (b) Slowdown in computation when setting the number of decoders to Baseline-2 for the MLS and RR fine-grained policies; (c) The relative reduction in the minimum number of decoders required (higher is better) when using the MFD and MLS policies over the baseline RR policy. The MFD policy requires more decoders to prevent any slowdowns, while the MLS policy can reduce the number of required decoders by up to 19%;	125

- 5.15 (a) Density histogram of latencies sampled from the PyMatching [114] decoder: the distribution can be approximated by a log-normal distribution with long tail latencies ($d = 9, p = 0.1\%$); (b) The density histogram used for our simulations with t_D sampled from a log-normal distribution. 126
- 5.16 Relative increase in the number of optimal decoders required by the MLS policy to prevent slowdowns with longer non-uniform decoder latencies. 127
- 5.17 (a) Teleporting logical qubits between different codes will require synchronization since every code has a different number of CNOTs in its syndrome generation cycle and hence has a different cycle duration; (b) Dropouts result in some qubits and/or coupler links being unusable (top). This can be circumvented by time-multiplexing of some qubits to construct syndrome generation circuits that have a duration that is longer than (but not a multiple of) the original duration (bottom). 128
- 5.18 (a) Patches P and P' are desynchronized by a slack of τ . This corresponds to both patches being in different phases of their syndrome generation cycles (of duration $T_{\text{cycle}} = T_P, T_{P'}$); (b) P and P' can be synchronized by idling P for a period equal to τ right before Lattice Surgery (*Passive*), or by breaking τ into smaller segments and adding those idle periods after every round (*Active*) before Lattice Surgery. 130
- 5.19 Circuits used to emulate the (a) *Passive* and (b) *Active* policies. The delay t_a is repeated N times to yield a net delay of t_p , resulting in both circuits having the same total runtime. Both circuits were run on 20 qubits on IBM Brisbane [120] for 20,000 shots. (c) Experimental mean fidelity across all 20 qubits for $N = 20,200$ with different values of t_p ($t_a = t_p/N$). 131
- 5.20 (a) Logical error rate is influenced by the syndrome hamming weight – higher weights result in more logical errors ($d = 15, p = 0.001$); (b) Syndrome hamming weights for all rounds before and after Lattice Surgery with *Passive* and *Active* synchronization ($\tau = 500\text{ns}$). The dashed vertical line shows the round during which Lattice Surgery is performed. *Passive* synchronization results in an almost 2x higher hamming weight during Lattice Surgery, resulting in a higher chance of logical errors compared to *Active* synchronization. 132

5.21	The <i>Active-intra</i> policy, where the slack is distributed within the last syndrome measurement round before Lattice Surgery.	133
5.22	Extra Rounds: for patch P' with a cycle time of T_P' and P with a cycle time of T_P , synchronization before Lattice Surgery can be achieved by running $P(P')$ for $m(n)$ extra rounds; <i>Hybrid</i> policy: a small idling period is added after every round and P is run for z extra rounds.	134
5.23	Different example configurations to show how running P for m extra rounds to achieve synchronization can be advantageous in some cases and impossible in others with the syndrome generation cycle times of P, P' as T_P, T_P' (ns) and an initial slack τ (ns).	135
5.24	τ vs. $T_{P'}$, colored by the number of extra rounds (z). T_P was fixed at 1000ns. $\epsilon = 400$ ns allows more $\tau, T_P/T_{P'}$ combinations to be optimized with the <i>Hybrid</i> policy. White space indicates no solution was found.	136
5.25	<i>Active</i> synchronization of k patches – every pair can be synchronized concurrently.	137
5.26	Two surface code patches P, P' are initialized, with some synchronization slack added to P . After $d + 1$ rounds, they are synchronized and then merged, after which the merged patch is run for another $d + 1$ rounds. The observables $X_P, X_{P'}, Z_P Z_{P'}, X_P X_{P'}$ of the merged patch can be used by Stim to compute the Logical Error Rate (LER). Note that for the Z basis Lattice Surgery, the $X_P X_{P'}$ observable corresponds to the logical measurement between P and P' and is computed by measuring the green X measure qubits [84].	138
5.27	Reduction in the logical error rate for one synchronization when using <i>Active</i> synchronization instead of <i>Passive</i> for different code distances with a system configuration similar to (a)-(b) IBM systems, and (c)-(d) Google systems for both X and Z basis Lattice Surgery with $p = 10^{-3}$. The synchronization slack (τ) was set at 500ns and 1000ns.	139
5.28	Logical error rate of the $X_P X_{P'}$ and X_P observables with Z basis Lattice Surgery after $2d + 2$ rounds for an ideal system that does not require any synchronizations and a system either using <i>Active</i> or <i>Passive</i> synchronization. The worst-case synchronization slack of $\tau = 1000$ ns was chosen for this experiment with an IBM configuration ($p = 10^{-3}$).	140

5.29	(a) Reduction in the LER compared to the <i>Passive</i> policy when the <i>Active</i> policy distributes its slack over $d + 1 + R$ rounds rather than $d + 1$ rounds; (b) Influence of the number of rounds run on the LER (without any slack/idling).	141
5.30	Reduction in the LER compared to the <i>Passive</i> policy for different initial slacks τ and tolerances ϵ (in ns). The <i>Hybrid</i> policy works more effectively for larger τ	142
5.31	Reduction in the LER compared to the <i>Passive</i> policy with the use of the <i>Active</i> and <i>Hybrid</i> policies on a neutral atom system. Running additional rounds, even with the <i>Hybrid</i> policy, is detrimental to the LER.	143
A.1	decoder-bench: Given a circuit and noise model, decoder-bench generates the parity check matrix, priors, the observable matrix, and samples syndromes, observables using Stim. A hash map is used for ensuring only unique syndromes (and observables) are sampled.	161
A.2	Compiler workflow – our rewrite pass changes the IR schedule based on the number of available decoders.	163

Abstract

Quantum computers promise to solve problems that today's machines cannot – simulating molecules to design new drugs, modeling new materials, breaking the cryptography that secures the modern internet. What is easy to miss is how much of that promise rests on classical (non-quantum) hardware. A useful quantum computer cannot run without quantum error correction (QEC), and error correction demands a classical control system that drives every qubit, reads it out, decodes its errors, and keeps thousands of qubits working in concert—all in real time, continuously, for the entire length of a computation. As quantum machines grow, this classical system grows with them, and threatens to become the bottleneck that determines whether they scale at all. This dissertation optimizes that control system from the ground up, so that the classical machine can keep pace with the quantum one it serves.

First, we target the low-level control and readout hardware that communicates with the qubits—manipulating their states and measuring them. COMPAQT is a compressed waveform memory architecture for scalable qubit control. Superconducting qubits are manipulated using shaped microwave pulses (waveforms) stored in on-chip memory; as the qubit count grows, streaming these waveforms to the digital-to-analog converters simultaneously becomes a bandwidth bottleneck that limits how many qubits a single controller can drive. COMPAQT reduces this bandwidth using a hardware-friendly discrete cosine transform paired with run-length encoding, enabling lossless-in-practice compression that allows a single FPGA-based controller to drive $5\times$ more qubits, and reduces the waveform memory footprint in cryogenic ASIC controllers where dissipated power is tightly constrained. HERQULES is a scalable qubit readout discriminator that achieves high classification accuracy without complex hardware. Readout errors in superconducting processors are dominated by two effects: thermal relaxation, in which a qubit decays from the excited to the ground state mid-measurement, and crosstalk between frequency-multiplexed readout

channels. HERQULES addresses both with a two-level hierarchy: matched filters detect the characteristic IQ-trace signature of a $1 \rightarrow 0$ relaxation event, and a small on-chip neural network—consuming less than 8% of FPGA LUTs—handles residual crosstalk, together achieving a 16.4% improvement in readout fidelity over the state-of-the-art neural network discriminator.

Second, we present an FPGA-tailored clustering decoder for decoding quantum low-density parity check (qLDPC) codes (and in general, any stabilizer code). Syndromes generated for QEC must be processed by the decoder within a tight latency budget. This latency budget arises due to the need for executing non-Clifford operations on the quantum computer. When these non-Clifford operations are executed, the decoder must be up to date with the latest syndrome generated right before the non-Clifford operation. This necessitates real-time operation of the decoder, and requires the decoder to, on average, decode errors per error correction round within this latency budget. For superconducting qubits, this budget can be in the order of 1-2 μ s, making accelerated hardware decoders necessary. We present a clustering decoder that uses hardware-friendly computational structures such as systolic arrays, bitmaps, and fast bitwise operations for decoding, allowing it to be implemented with $\sim 6\%$ FGPA LUT usage for the $[[144, 12, 12]]$ bivariate-bicycle qLDPC code.

Third, we introduce system management frameworks for QEC: scheduling decoders and orchestrating synchronized logical operations. Accelerated decoders are a finite, expensive resource—a single FPGA decoder can consume the resources of an entire board and decoder demand is highly non-deterministic: routing distances and lattice surgery operations can cause demand to spike by up to $50\times$ for just two logical qubits. We propose a two-level scheduling framework that dynamically allocates a shared pool of decoders across logical qubits, prioritizing critical decodes that gate non-Clifford computation and using a longest-queue-first fine-grained policy that reduces the number of decoders needed to prevent program slowdowns by up to 40% compared to worst-case provisioning. Separately, logical operations between qubits require their syndrome generation cycles to be temporally aligned; heterogeneous codes, fabrication defects, and modular architectures all break this alignment. We show that distributing synchronization idle time across many QEC rounds (the *Active* policy) rather than concentrating it immediately before an operation reduces the logical error rate by up to $2.4\times$, and a *Hybrid* policy that combines distributed idling with additional syndrome rounds extends this to $3.4\times$ for codes with mismatched cycle times. Both decoder scheduling and synchronization works were evaluated with a surface code and

lattice surgery architecture in mind, along with the limitations imposed by superconducting qubits. However, they are broadly applicable ideas, and can be just as easily extended for other QEC codes, general code surgery, and other qubit technologies.

We also present all software tools and frameworks that were used to evaluate the ideas presented in this dissertation. First, we define the algorithms implemented for HERQULES, including the semi-supervised classification subroutine and the training and inference flows for qubit state classification. Next, we introduce `decoder-bench`, a decoder benchmarking framework that is capable of evaluating any decoder for a broad set of quantum error correcting codes. And finally, we highlight the workflow and heuristics used for provisioning and scheduling decoders using compiler passes.

Chapter 1

Introduction

1.1 A Brief History of Quantum Computing

A brief look back in time shows how quantum computing arrived at its current moment, and why the problems this dissertation addresses are the ones that matter now.

1.1.1 Genesis

The idea of exploiting quantum mechanics for computation emerged in the 1980s. Paul Benioff showed that quantum-mechanical systems could simulate Turing machines [18]. Independently, Richard Feynman observed that simulating generic quantum systems on a classical computer demand exponential resources, and Feynman argued that a machine built from quantum elements might simulate nature more efficiently [79]. David Deutsch made these intuitions precise in 1985 by defining a universal quantum Turing machine and the quantum circuit model [67]. The advantage of using quantum computers was further sharpened in the Deutsch-Jozsa algorithm [68].

The 1990s saw a rapid rise in the popularity of quantum computers. Peter Shor's 1994 polynomial-time algorithms for integer factoring and discrete logarithms threatened the foundations of public-key cryptography [226] which spurred more research in subsequent years. Lov Grover's 1996 algorithm gave a quadratic speedup for unstructured search [106], further establishing the potential of the field.

At this point, critics pointed out that noise and decoherence (the process by which qubits lose their properties) would render any physical devices useless. This criticism was

immediately addressed by Shor and Steane independently with the first quantum error correcting codes [227, 239]. The threshold theorem later established that arbitrarily long quantum computations are possible provided the physical error rate lies below a constant threshold [2, 138]. David P. DiVincenzo’s criteria defined the conditions necessary for building a quantum computer capable of running algorithms like Grover search [73]. In the early 2000s, Kitaev’s topological codes (the surface code in particular) supplied a hardware-friendly route to fault tolerance [136]. This path has been the dominant basis of architectural planning ever since.

1.1.2 Breakthroughs in device physics

Translating these abstract (and at that time, theoretical) concepts into hardware required physical systems in which quantum information could be initialized, manipulated, and read out with sufficient fidelity; requirements defined by DiVincenzo’s criteria [73].

The earliest demonstrations used nuclear magnetic resonance (NMR), the same physics that underlies medical MRI, in which the spin states of atomic nuclei within a molecule serve as qubits. This approach culminated in 2001 with the factoring of the number 15 using Shor’s algorithm; NMR was an invaluable testbed but could not be scaled to many qubits [261].

Trapped ions soon became a leading platform. Building on the Cirac–Zoller gate proposal of 1995, these systems hold individual charged atoms in place with electromagnetic fields and control them with lasers, and they have achieved some of the most accurate single- and two-qubit operations of any technology [188].

Superconducting circuits advanced especially rapidly. After the first demonstration of a charge qubit in 1999, the central challenge was building qubits that could hold their quantum state long enough to compute with. The transmon—a charge qubit redesigned to be far less sensitive to electrical noise—together with the circuit quantum electrodynamics (circuit QED) architecture, which couples qubits to on-chip microwave resonators, extended *coherence times* (how long a qubit retains its quantum information before it decoheres) by orders of magnitude in the late 2000s [140, 194]. This progress made superconducting qubits the leading approach for many large-scale efforts.

In parallel, several other platforms matured into serious competitors: neutral atoms held in arrays of tightly focused laser beams (optical tweezers), single-electron spins in

semiconductors, and photonic systems that encode qubits in light. Each strikes a different balance among qubit connectivity, coherence, and ease of manufacturing. Across roughly two decades, the field moved from single, short-lived qubits to processors with dozens and then hundreds of physical qubits, with steady gains in fidelity that finally brought several platforms below the error threshold required for quantum error correction.

1.1.3 Quantum in the cloud

A pivotal change in the way how researchers and the general public engaged with quantum computers came about in 2016, when IBM launched its quantum cloud service, providing free access to small superconducting qubit systems. Comparable services followed from Amazon Braket, Microsoft Azure Quantum, and others, and a rich ecosystem of open-source software development kits such as Qiskit, Cirq, PennyLane, and their peers lowered the barrier to entry still further.

This period coincided with what Preskill named the era of Noisy Intermediate-Scale Quantum (NISQ) devices: machines large enough to be interesting but too noisy for full error correction [209]. The response was a class of hybrid, variational algorithms, the variational quantum eigensolver and the quantum approximate optimization algorithm chief among them, designed to extract value from imperfect hardware with classical assistance. Easy access to real quantum computers also opened the way for computer science researchers to study compilers, error mitigation strategies, and low-level pulse optimizations. This era also produced the first claims of quantum computational advantage [11].

1.1.4 Experimental demonstrations of quantum error correction (QEC)

For most of the NISQ period, error correction existed largely on paper and in small proof-of-principle experiments. The goal of QEC is to combine many noisy *physical* qubits into fewer, far more reliable *logical* qubits. An early milestone was reaching the *break-even point*, at which such an encoded logical qubit finally outlives its best individual physical qubit; this was first achieved using bosonic codes, which spread the information across the many energy levels of a single superconducting microwave cavity rather than across separate qubits [200]. Attention then turned to the surface code—the leading QEC scheme—and to related families

of codes. In 2023, IBM demonstrated the heavy-hex code on its hardware [246], while Google refined the surface code and showed something pivotal: increasing the number of physical qubits used to encode a logical qubit actually *reduced* its error rate [5].

This is the defining signature of working error correction: adding redundancy yields a more reliable qubit, rather than merely more noise to track. Google reinforced the result in 2024 on its 105-qubit Willow chip [6], and color codes (an alternative code family) were subsequently demonstrated on the same hardware [145].

Other platforms pushed in a complementary direction: encoding many logical qubits at once and computing with them. Neutral-atom arrays ran algorithms on dozens of logical qubits using *transversal gates*—operations applied to all the physical qubits of a code in parallel, a naturally fault-tolerant strategy—and a Microsoft–Quantinuum collaboration produced logical qubits whose error rates fell far below those of their underlying physical qubits [25, 53]. By 2025 these threads came together: a single neutral-atom device combined all the essential ingredients of fault-tolerant computing—error-corrected memory operating below threshold, logical entangling gates, a route to universal computation via quantum teleportation, and the ability to reuse qubits within a running computation [26].

1.1.5 The scaling era

With promising experimental demonstrations in the past few years, we have now entered the era of scaling – building large-scale quantum computers with thousands of qubits for running quantum error correction. Building these large-scale systems will enable the development of more useful algorithms that exploit the advantages offered by quantum computers. Aggressive roadmaps for achieving these utility-scale systems have emerged from Google [102], IBM [122], Quantinuum [210], and QuERA [211]. Resource estimation studies further lend motivation to these roadmaps. For example, the total physical qubit cost of running Shor’s factoring algorithm dropped by $20\times$ for superconducting qubits from 2021 [97] to 2025 [96] due to advances in QEC theory and algorithms, and Cain et al. showed that factoring could be performed with as few as 10,000 reconfigurable atomic qubits [37]. This last pivotal result is aided by advances in quantum low-density parity check (qLDPC) codes [29, 280], which increases the number of logical qubits encoded per physical qubit (code rate). With further advances in QEC, these physical qubit requirements will reduce further, allowing fault-tolerant quantum computers to be built in the next decade, if not the

next half-decade.

1.2 Contributions

The scaling era introduces not just more qubits, but an entire classical control infrastructure that must scale alongside the quantum hardware. As with any computing paradigm, these challenges are best understood through the lens of abstraction: each layer in the quantum computing stack addresses a distinct set of problems, building on the guarantees offered by the layer below. Figure 1.1 illustrates this stack for a fault-tolerant quantum computer and marks the layers addressed by this dissertation.

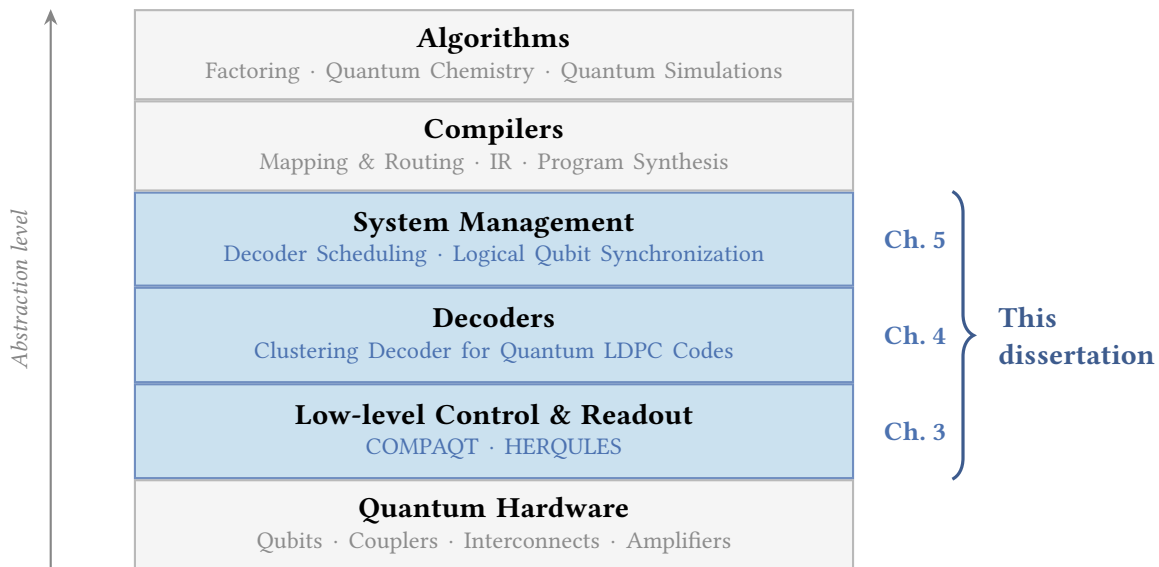


Figure 1.1: The quantum computing control stack. Layers shaded in blue represent the contributions of this dissertation (Chapters 3–5).

At the foundation sits the quantum hardware: the physical qubits, couplers, and interconnects that store and process quantum information. Directly above, the *low-level control and readout* layer generates the microwave or optical pulses that implement quantum gates and measures the qubits’ states at the end of each error correction cycle. The syndrome bits produced by these measurements are passed upward to the *decoder* layer, which identifies and corrects physical errors before they accumulate into logical faults. Above the decoders, a *system management* layer orchestrates the full ensemble of logical qubits: it allocates a finite pool of accelerated hardware decoders across competing logical qubits and ensures that

patches participating in joint operations remain temporally aligned. Compilers above this layer translate high-level quantum programs into sequences of logical operations compatible with the underlying QEC code and device layout, and at the very top, quantum algorithms exploit the reliable logical qubits that the lower layers collectively provide. This dissertation addresses the three layers highlighted in Figure 1.1—low-level control and readout, decoding, and system management. These layers also constitute the classical control system of any fault-tolerant quantum computer. The general theme across this dissertation is to make the classical control system more scalable for facilitating fault-tolerant quantum computing.

While Figure 1.1 shows *where* each contribution sits in the stack, it does not convey *how broadly* each one applies. The contributions differ in this regard. COMPAQT and HERQULES are anchored in a specific qubit technology—superconducting qubits—but the principles behind them reach outward to other technologies. Conversely, the clustering decoder, decoder scheduling, and synchronization policies are anchored in a specific QEC code or setting used for evaluation, yet are agnostic to the underlying qubit technology and generalize across code families. Figure 1.2 makes this distinction explicit: for each contribution, the solid segment marks the scope in which it is *evaluated* in this dissertation, while the lighter segment marks the scope to which its underlying principle *generalizes*.

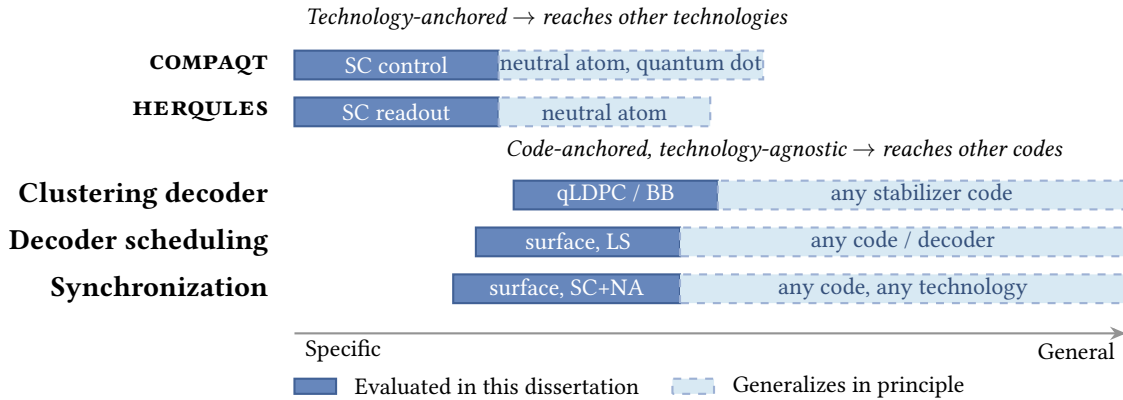


Figure 1.2: Generalization scope of each contribution. COMPAQT and HERQULES are anchored in superconducting (SC) hardware but their principles reach other qubit technologies; the clustering decoder, decoder scheduling, and synchronization policies are technology-agnostic and generalize across QEC codes beyond the ones used for evaluation. Solid segments denote the evaluated scope; lighter segments denote the scope to which the underlying principle generalizes. (LS: lattice surgery; NA: neutral atom; BB: bivariate bicycle.)

We now summarize the contributions of this dissertation.

1.2.1 Scalable qubit control and readout (Chapter 3)

The first set of contributions of this dissertation focus on the lowest level of the quantum computing stack. As the number of physical qubits increase, the classical hardware required to manipulate their states (control) and read them out (readout) must scale proportionally. Current experimental setups use ad-hoc, brute-force approaches for scaling this low-level control hardware, and while this may not be problematic for systems with less than a thousand qubits, utility-scale quantum computers requiring more than ten thousand qubits will require more refined scaling techniques. We propose two optimizations to the control and readout pipeline of superconducting qubits, called COMPAQT and HERQULES respectively.

COMPAQT: Compressed waveform memory for scalable qubit control. Every gate applied to a superconducting qubit is implemented as a shaped microwave pulse whose envelope (the *waveform*) is stored in on-chip memory and streamed to a digital-to-analog converter (DAC) at sampling rates exceeding 4 GS/s per channel. As the qubit count grows, the memory bandwidth required to drive all DACs simultaneously becomes the primary bottleneck: a hundred-qubit machine requires over 2 TB/s of waveform memory bandwidth, far exceeding what a single RFSoc or ASIC controller can sustain. COMPAQT resolves this by compressing waveforms using a hardware-friendly integer discrete cosine transform (DCT) paired with run-length encoding, decompressing samples on-the-fly at DAC speed. Because the compression is lossy, we characterize the fidelity loss carefully and show it is negligible for practical gate sets. The bandwidth reduction translates directly to scalability: a single RFSoc-based controller running COMPAQT can drive $5\times$ more qubits than an uncompressed baseline, and in cryogenic ASIC controllers, where every milliwatt counts, the smaller waveform memory footprint reduces dissipated power within the dilution refrigerator’s strict cooling budget.

HERQULES: High-fidelity hardware efficient qubit readout using low-dimensional traces. Qubit readout is among the most error-prone operations in superconducting processors: on a 127-qubit IBM system, assignment error rates span from $\sim 0.1\%$ to over 10% per qubit, driven by two dominant mechanisms. The first is *thermal relaxation*: during

the readout pulse, a qubit in the excited state can decay to the ground state, producing a signal trace whose shape is distinctly different from a qubit that was in the ground state all along. The second is *readout crosstalk*: when many qubits share a multiplexed readout channel, their resonator signals interfere and distort one another’s classified state. Existing neural-network discriminators can handle both effects but require networks large enough to run in software off the control FPGA, adding communication latency that is incompatible with fast QEC feedback. HERQULES breaks this accuracy–latency tradeoff with a two-stage hierarchy: lightweight matched filters identify the characteristic signature of a mid-readout $1 \rightarrow 0$ relaxation event, and a very small fully-connected neural network—consuming less than 8% of the LUTs on a commercial FPGA—handles residual crosstalk correction entirely on-chip. Together, these stages achieve an overall readout fidelity 16.4% better than the state-of-the-art neural network discriminator while remaining orders of magnitude more resource-efficient.

While both optimizations were developed and evaluated on superconducting qubit systems, their underlying principles extend naturally to other qubit technologies: modulated waveform envelopes are used for gate control in neutral atom and quantum dot systems, making COMPAQT’s compression strategy directly applicable, and the hierarchical matched-filter approach in HERQULES generalizes to any readout paradigm in which state-dependent signal traces can be characterized and filtered, such as neutral atoms [190].

1.2.2 Accelerated decoders for QEC (Chapter 4)

Every syndrome generated by the readout hardware must be decoded before the next round of syndrome measurements completes—a hard budget of roughly $1\ \mu\text{s}$ on superconducting platforms. If the decoder consistently exceeds this budget, unprocessed syndromes accumulate without bound and stall the computation; non-Clifford gates impose an even stricter constraint, since the Pauli frame cannot be updated until the decoder has resolved the accumulated syndrome and returned a correction, forcing the data qubit to stall. Quantum LDPC codes compound this difficulty: their dense check matrices make minimum-weight perfect matching inapplicable, and belief propagation (BP) frequently fails to converge on degenerate codes where many distinct low-weight error configurations share the same syndrome. We present a clustering decoder tailored for FPGA implementation that uses BP as a *pre-decoder*: faults with high BP confidence are accepted immediately, while uncertain

faults seed an initial set of clusters together with unsatisfied syndrome nodes. Clusters are grown and merged using bitwise operations on compact bitmap representations—a format that maps directly onto FPGA logic—and cluster validity is decided by a systolic linear-system solver over $\mathbb{F}_2$ that achieves linear runtime complexity. The resulting decoder occupies only 6% of the LUTs on a Xilinx VU19P FPGA for the gross bivariate-bicycle qLDPC code—significantly less than competing approaches—and generalizes to any stabilizer code whose decoding graph can be expressed as a binary check matrix.

1.2.3 Systems for quantum computers (Chapter 5)

Elastic decoder scheduling for large-scale fault-tolerant systems. A single logical qubit requires a dedicated real-time decoder running continuously; at scale, a fault-tolerant machine running hundreds of logical qubits would naively require hundreds of accelerated hardware decoders—each consuming nearly the full resources of an FPGA or ASIC. This provisioning problem is compounded by the non-uniformity of decoder demand: patches idling as logical memory need far less decoding capacity than patches participating in an active lattice surgery operation, and the routing distances required by that surgery can cause decoder demand to spike by up to $50\times$ for just two logical qubits. Static allocation strategies fail at both extremes: provisioning for the worst case wastes hardware that sits idle most of the time, while provisioning for the average case introduces slowdowns of up to $10\times$ when demand spikes. We address this with a two-level scheduling framework that treats decoders as a shared, finite pool allocated dynamically across logical qubits. Coarse-grained scheduling prioritizes critical spatial decodes arising from non-Clifford gates—where decoding latency directly gates computation—over non-critical temporal decoding tasks. Fine-grained scheduling then distributes remaining decoder capacity using a *minimize longest undecoded sequence* (MLS) policy that greedily prevents any logical qubit from falling too far behind. Across 27 fault-tolerant benchmarks, MLS reduces the number of decoders required to prevent any program slowdown by up to 40% compared to worst-case provisioning and up to 19% compared to round-robin scheduling, with 10–40% savings preserved even under realistic log-normal decoder tail latency distributions.

Synchronization policies for interacting logical qubits. A logical operation between two patches—a lattice surgery merge, a magic state injection, a logical CNOT—requires the

syndrome generation cycles of both patches to be aligned in time before the merge can begin. In a homogeneous system this alignment is guaranteed, but practical fault-tolerant machines will inevitably be heterogeneous: magic state factories based on color or qLDPC codes have different cycle times than surface code data qubits, circuits adapted for fabrication defects may run at non-standard rates, and chiplet-based modular architectures introduce boundary latencies that accumulate over time. When two patches are desynchronized by a slack τ , the simplest response is to idle the leading patch until the lagging one catches up—but concentrated idling spikes the syndrome Hamming weight during the subsequent lattice surgery operation, directly increasing the logical error rate. We show that distributing the same total slack evenly across the d syndrome rounds preceding the merge—the *Active* policy—suppresses this Hamming weight spike and achieves up to $2.4\times$ reduction in logical error rate compared to concentrated idling. When the two patches have different cycle times, a *Hybrid* policy that combines a small number of additional syndrome rounds with distributed idling reduces the residual slack further, achieving up to $3.4\times$ LER reduction at code distance $d = 15$. On neutral atom platforms, where long coherence times make idling errors comparatively cheap, Active or Passive synchronization is always preferable—the extra rounds required by the Hybrid policy increase the logical error rate rather than reducing it.

These scheduling and synchronization approaches were developed within a specific context of surface code lattice surgery. However, they can be generalized beyond the surface code, and beyond superconducting qubits as well. For example, the decoder scheduling policies presented in this dissertation use insights based on the serial nature of quantum programs and how logical operations can make decoder demand non-deterministic. These insights will hold true for other QEC codes and decoder configurations as well, allowing this work on making decoders elastic broadly applicable. Similarly, the synchronization policies can be applied for any combination of codes for any qubit technology. While we show the trade-offs of the different policies primarily for superconducting qubits, our evaluations cover neutral atom platforms as well.

1.3 Organization

This dissertation is organized as follows:

- We provide necessary background in Chapter 2.
- In Chapter 3, we present COMPAQT and HERQULES: two optimizations to the control and readout pipelines for superconducting qubits to make them more scalable (based on [177] and [176]).
- In Chapter 4, we present an FPGA-tailored clustering decoder for quantum LDPC codes (based on [173]).
- In Chapter 5, we present two system management techniques for managing decoders and orchestrating synchronized logical operations between logical qubits (based on [174] and [178]).
- We highlight work related to this dissertation in Chapter 6.
- Finally, we summarize this dissertation and discuss potential future work in Chapter 7.
- Appendix A highlights some software tools and frameworks developed for evaluating different parts of this dissertation.

Chapter 2

General Background

Quantum computing promises to accelerate the computation of certain classes of problems. First proposed by Richard Feynman in 1982 [79], quantum computing has advanced rapidly in the last decade. In this chapter, we provide a general background on quantum computing, including the basics of qubits and quantum gates, the challenges of noise in quantum computers, the principles of quantum error correction, and the role of classical control in quantum computing. This background will set the stage for the more specific topics covered in subsequent chapters.

2.1 The Quantum and Classical Worlds

Classical physics describes the world at the scale of everyday objects: matter has definite properties, systems evolve predictably, and the outcome of any measurement can, in principle, be known with certainty given enough information about the system. This deterministic picture underlies all of classical computing, where information is represented by physical states that are always well-defined. Quantum mechanics, developed in the early 20th century to account for phenomena classical theory could not explain, replaces definite states with probability amplitudes: systems can exist in superpositions of configurations, certain observables take only discrete values, and measurement is intrinsically probabilistic. This underlying distinction in physics is what motivates the corresponding distinction in computing hardware. Throughout this dissertation, the terms classical and quantum refer to categories of hardware rather than to physical theories. Classical hardware denotes any

computing substrate whose state evolution is described by deterministic or probabilistic logic over definite bit values: general-purpose CPUs, GPUs, FPGAs, ASICs, memory systems, interconnects. Quantum hardware denotes, specifically, the qubit substrate itself — the physical devices whose state is described by quantum amplitudes and which execute the operations of a quantum program. Under this convention, a fault-tolerant quantum computer is a hybrid system: a quantum processor surrounded by, and continuously interacting with a substantial classical infrastructure.

2.2 Qubits

Devices we use every day, such as smartphones and laptops, utilize *bits* as the fundamental units of information. These classical bits are binary in nature — they can be either 0 or 1 at any given time. Analogous to these classical bits, quantum computers use *quantum bits* (qubits) as their fundamental units of information.

Qubits are two-level systems, where the first state corresponds to the computational ‘0’ state and the second state corresponds to the computational ‘1’ state. In quantum mechanics, these ‘levels’ correspond to energy levels, where the lower energy level is the ground state of a qubit and the higher energy level is an excited state. Qubits are mathematically described using Dirac’s Bra-Ket notation, a reflection of the linear algebraic formalism used in quantum mechanics which underpins quantum computing. Using the Bra-Ket notation, the computational states of a qubit are $|0\rangle$ and $|1\rangle$, and arbitrary qubit states can be represented as $|\psi\rangle = a|0\rangle + b|1\rangle$. A general qubit state $|\psi\rangle = a|0\rangle + b|1\rangle$ encodes information through two distinct properties of its complex coefficients $a, b \in \mathbb{C}$. Writing each in polar form as $a = |a|e^{i\phi_a}$ and $b = |b|e^{i\phi_b}$, the moduli $|a|$ and $|b|$ determine the measurement probabilities ($\text{Pr}[0] = |a|^2$, $\text{Pr}[1] = |b|^2$), while the *relative phase* $\phi = \phi_b - \phi_a$ encodes additional quantum information. Only the relative phase is physically observable: multiplying the entire state by a global phase $e^{i\alpha}$ leaves all measurement outcomes unchanged. Errors on qubits can corrupt either property. A *bit-flip error* swaps the amplitudes of $|0\rangle$ and $|1\rangle$, while a *phase-flip error* negates the relative phase between them. For example, the state $|\psi\rangle = \frac{1}{\sqrt{2}}(|0\rangle + e^{i\pi/4}|1\rangle)$ has equal measurement probabilities for both basis states and a relative phase of $\pi/4$; a phase-flip error would transform this to $\frac{1}{\sqrt{2}}(|0\rangle - e^{i\pi/4}|1\rangle)$, leaving probabilities unchanged but corrupting the phase. As we discuss in Section 2.8, quantum error correction must protect against both error types.

Unlike classical bits, qubits can exist in a *superposition* of states, where superposition refers to qubits existing in a linear combination of states rather than either of $|0\rangle$ or $|1\rangle$. This means that a qubit with an arbitrary state $|\psi\rangle = a|0\rangle + b|1\rangle$ is probabilistic – if this qubit in superposition is measured repeatedly, it would measure as $|0\rangle$ with a probability of $|a|^2$ and $|1\rangle$ with a probability of $|b|^2$. Since qubits represent two-level systems, $|a|^2 + |b|^2 = 1$.

Qubits can also be *entangled*, which is a unique property of quantum systems. Entanglement refers to the phenomenon where the state of one qubit is dependent on the state of another qubit, even if they are physically separated. For example, consider two qubits in the state $|\psi\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$. This state is an example of a *Bell state*, which is a maximally entangled state of two qubits. In this state, if we measure the first qubit and find it in the state $|0\rangle$, we immediately know that the second qubit is also in the state $|0\rangle$, without measuring it. Similarly, if we measure the first qubit and find it in the state $|1\rangle$, we know that the second qubit is also in the state $|1\rangle$. This correlation between the states of the two qubits is a hallmark of entanglement and is a key resource for quantum computing. Entanglement allows quantum computers to perform certain computations more efficiently than classical computers, as it enables quantum algorithms to explore a larger computational space than classical algorithms.

2.3 The Bloch Sphere

The state of a qubit can be represented by Bloch spheres, as shown in Figure 2.1. The computational basis states of a qubit ($|0\rangle$, $|1\rangle$) lie on the north and south poles of the sphere. An arbitrary qubit state $|\psi\rangle = a|0\rangle + b|1\rangle$ is then a vector pointing to a particular point on the Bloch sphere. Any change/modification to the qubit's state then rotates this vector along one or more axes (X/Y/Z) of the Bloch sphere.

2.4 Quantum Gates

Quantum gates are the fundamental unitary¹ operations that drive quantum algorithms. Quantum gates can operate on one or more qubits at a time. When a quantum gate is applied

¹A unitary operator U is one whose inverse is equal to its complex-conjugate transpose such that $UU^* = U^*U = I$

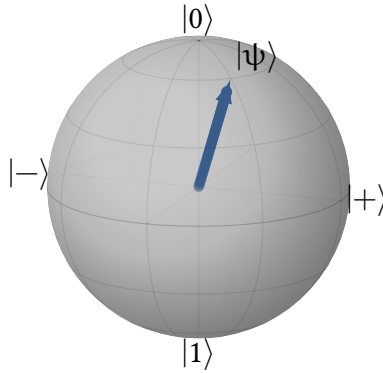


Figure 2.1: Bloch sphere representation of a qubit state.

on multiple qubits, it is generally used as an entangling operation. Entanglement refers to creating correlated qubit states, and two/multi-qubit gates are essential. We will describe some commonly used single- and two-qubit gates, including commonly used groups such as Paulis and Cliffords.

A qubit in an arbitrary state $|\psi\rangle = a|0\rangle + b|1\rangle$ can be represented linear algebraically as:

$$|\psi\rangle = \begin{pmatrix} a \\ b \end{pmatrix} \quad (2.1)$$

Here, the rows of the matrix correspond to the possible basis states of the qubit. For a single qubit, those states are $|0\rangle$ and $|1\rangle$. In general, for an n -qubit state, there will be 2^n possible basis states, and thus rows in the matrix.

2.4.0.1 Single-qubit gates

Single-qubit gates operate on a single qubit, and result in rotation of the qubit's vector around one or more axes of the Bloch sphere. Some commonly used single-qubit gates are:

The Pauli group: Pauli gates form a fundamental set of operations that rotate a qubit's state on the Bloch sphere by 180 degrees. The three Pauli gates are the X, Y, and Z gates, which rotate the qubit's state along the X, Y, and Z axis of the Bloch sphere respectively. The unitary representations of the three Pauli gates are:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}; \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}; \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \quad (2.2)$$

As an example, consider a qubit in an initial ground state of $|\psi\rangle = |0\rangle$. The effect of a Pauli-X gate applied on this qubit will be as follows:

$$|\psi\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}; \quad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

$$X|\psi\rangle = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

The effect of a Pauli-X gate can also be seen through the Bloch sphere representation of a qubit, as shown in Figure 2.2. The X gate is the quantum equivalent of the NOT gate used in conventional digital logic.

Pauli gates have special properties, the chief among which is that they anti-commute: $ZX = iY = -XZ$. This property will be important later when we discuss quantum error correction.

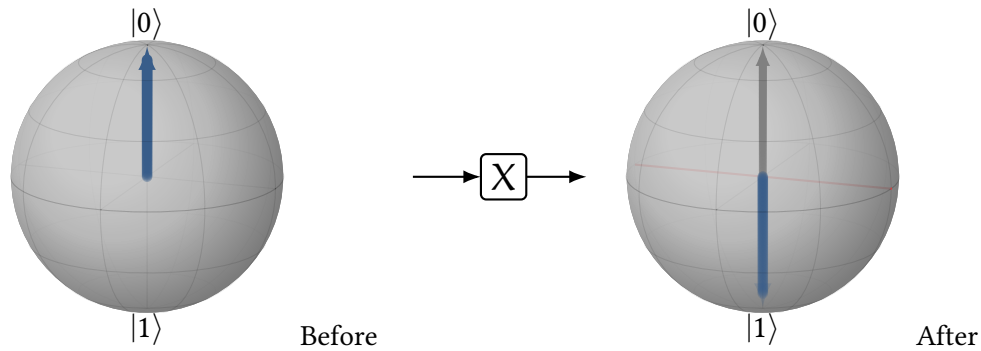


Figure 2.2: The Pauli-X gate maps $|0\rangle$ to $|1\rangle$ via a π rotation around the X-axis. The gray vector on the right shows the initial state before the gate.

The Hadamard gate: The Hadamard gate performs a rotation of 180 degrees around the $(X + Z)/\sqrt{2}$ axis of the Bloch sphere. Its unitary representation is:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

The Hadamard gate is used to create superposition states. As an example, we show the effect of the Hadamard gate on an initial state of $|\psi\rangle = |0\rangle$:

$$|\psi\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}; \quad H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

$$H|\psi\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

A final state of $|\psi\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}$ shows that both basis states $|0\rangle, |1\rangle$ are equally probable with a probability of $|a|^2 = |b|^2 = 1/2$. This represents an equal superposition state.

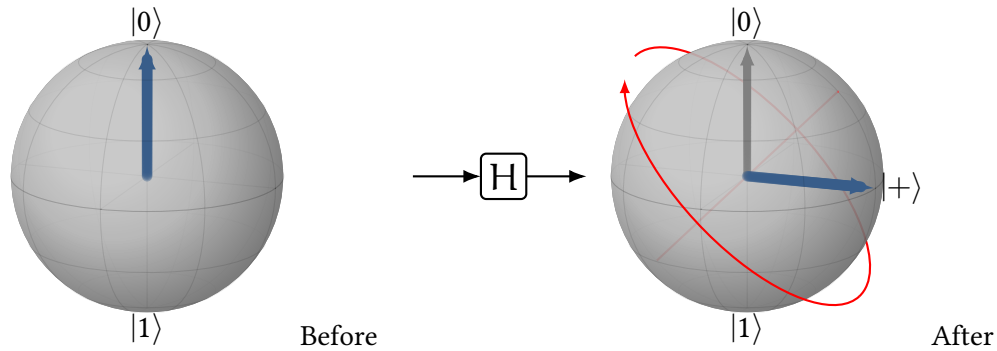


Figure 2.3: The Hadamard gate maps $|0\rangle$ to $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$, corresponding to a π rotation around the $(X+Z)/\sqrt{2}$ axis (red).

The Bloch sphere visualization of the H gate is shown in Figure 2.3. The H gate rotates the qubit's vector to a point on the equatorial plane – a measurement of this state would yield $|0\rangle$ and $|1\rangle$ 50% of the times, since the vector of the qubit's state is equidistant from both poles on the Bloch sphere.

The S gate: The S gate is a pure phase gate, as in it has no change on the measured state of a qubit. The S gate rotates the state of a qubit by 90 degrees around the Z-axis of the Bloch sphere. Since it only rotates the state by half of that of the Pauli-Z gate, it follows the identity $S^2 = Z$. The unitary representation of an S gate is:

$$S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}$$

The T gate: The T gate is an operation given by the unitary:

$$T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\frac{\pi}{4}} \end{pmatrix}$$

It results in a rotation of 45 degrees around the Z axis of the Bloch sphere. The T gate is a non-Clifford gate, which means that it cannot be efficiently simulated on a classical computer.

2.4.0.2 Two-qubit gates

Two-qubit gates are crucial for introducing interactions between qubits. The joint state of two qubits can be represented as: $|\phi\rangle = a|00\rangle + b|01\rangle + c|10\rangle + d|11\rangle$. The matrix representation of this state is then:

$$|\phi\rangle = \begin{pmatrix} a \\ b \\ c \\ d \end{pmatrix}$$

Here, each row of the matrix corresponds to the $|00\rangle$, $|01\rangle$, $|10\rangle$, and $|11\rangle$ states respectively.

The CNOT gate: The controlled-NOT (CNOT or CX) gate is a two-qubit gate that uses one qubit as the control and the second qubit as the target. The CNOT gate applies a Pauli-X gate on the target if and only if the control qubit is in state $|1\rangle$. The unitary matrix of the CNOT gate is:

$$CX = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

As an example, let's consider an initial joint two-qubit state $|\phi\rangle = |10\rangle$. Here, the first qubit (the control) is in state $|1\rangle$ and the target qubit is in state $|0\rangle$. The application of the CX gate then affects this initial state as:

$$|\phi\rangle = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix}; \quad CX = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

$$CX|\phi\rangle = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}$$

The final state $|\phi\rangle = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}$ corresponds to the state $|11\rangle$ – the target qubit has been

flipped from $|0\rangle$ to $|1\rangle$.

2.4.0.3 Clifford and non-Clifford gates

Clifford gates are an important classification of quantum gates because they can be simulated efficiently on a classical computer. In general, a gate U belongs to the Clifford group if $UPU^\dagger$ belongs to the Pauli group, for some Pauli gate P . Here, $U^\dagger$ represents the conjugate transpose (Hermitian adjoint) of the unitary U . Some common Clifford gates are the H , CX , and S gates. For example, $HXH^\dagger = Z$.

Non-Clifford gates are gates for which there exists at least one Pauli P for which $UPU^\dagger$ does not belong to the Pauli group. The most commonly used non-Clifford gate is the T gate. The T gate's conjugation on Pauli- X demonstrates its non-Clifford nature:

$$TXT^\dagger = \frac{1}{\sqrt{2}}(X + Y) \quad (2.3)$$

This result is not a Pauli operator, confirming T is non-Clifford. As mentioned earlier, non-Clifford gates are important because they enable quantum computers to perform computations that are believed to be intractable for classical computers. In other words, they ensure universality of the quantum gate set, which is a necessary condition for a quantum computer to be able to perform any arbitrary quantum computation. A key feature

of non-Clifford gates is that they cannot be efficiently simulated on a classical computer, as stated in the Gottesman-Knill theorem [105], which is why they are essential for achieving quantum advantage.

2.4.0.4 Summary

A summary of commonly used gates and their symbols, unitaries are given below in Table 2.1.

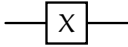
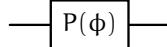
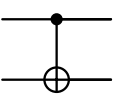
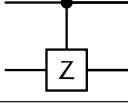
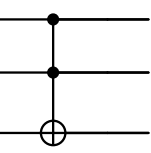
Gate	Circuit	Unitary Matrix
X		$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$
Y		$\begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$
Z		$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$
H		$\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$
S		$\begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}$
P(ϕ)		$\begin{bmatrix} 1 & 0 \\ 0 & e^{i\phi} \end{bmatrix}$
T		$\begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix}$
CX		$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$
CZ		$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix}$
SWAP		$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$
Toffoli		$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$

Table 2.1: Common quantum gates with circuit symbols and unitary matrices.

2.4.1 Measuring qubits

Qubits can exist in a variety of superposition and entangled states. However, when measured, the state of every qubit collapses to 0 or 1 with some probability. As shown in Figure 2.4, measurements of qubits are depicted by a "meter" symbol. In general, qubits are generally measured in the Z basis, which means that the state of the qubit collapses to the nearest pole on the Bloch sphere. Other measurement bases are X and Y, which correspond to measurements about the X and Y axes of the Bloch sphere.

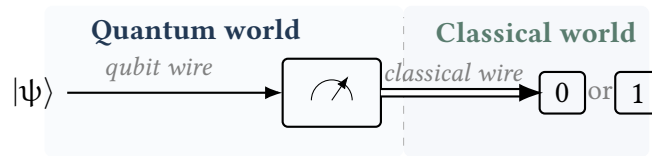
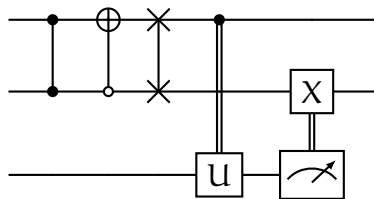


Figure 2.4: Quantum measurement collapses the qubit $|\psi\rangle$ into a classical bit, yielding outcome 0 or 1. The single wire carries quantum state; the double wire carries a classical value.

2.4.2 Quantum circuits

Quantum circuits combine gates and measurements to implement quantum algorithms. An example is shown below, which combines two-qubit gates and operations conditioned on measurement outcomes:



2.5 Quantum Computers are Noisy

Quantum devices are extremely error-prone. Error rates on quantum computers are orders of magnitude higher than that in conventional digital hardware. For example, a conventional DRAM chip fails once every few billion accesses, while the best quantum devices face an error once in a few hundreds to thousands of operations. This disparity in error-rates is the primary reason for the need of quantum error correction (QEC).

Noise affecting qubits has different classifications. We summarize some well-studied noise sources below.

2.5.1 Gate errors

Every gate (X, Y, Z, CX etc.) has some error associated with it. Most single-qubit gates have high fidelities (fidelity is the opposite of the error rate, it measures the degree of success of the operation) on most qubit technologies. Two-qubit gates are relatively more error-prone than single-qubit gates.

2.5.2 Decoherence

Qubits are inherently unstable because they are open quantum systems that interact with their environment. Decoherence refers to the gradual loss of stored quantum information over time. Two primary mechanisms are studied: relaxation and dephasing. Relaxation (characterized by time constant T_1) causes a qubit in the $|1\rangle$ state to spontaneously decay to $|0\rangle$. Dephasing (characterized by time constant T_2) causes the relative phase between $|0\rangle$ and $|1\rangle$ to fluctuate randomly, destroying superpositions. Even idle qubits are subject to decoherence, which limits the maximum useful duration of any quantum computation.

2.5.3 Measurement errors

Readout operations on qubits are also imperfect. In many technologies, measuring a qubit in $|0\rangle$ can yield an outcome “1” with some probability, and similarly a qubit in $|1\rangle$ may be misclassified as “0”. In addition to such mis-classification, imperfect reset operations can leave residual population in the wrong state, effectively introducing preparation errors. These errors directly corrupt the outcomes of quantum algorithms and, more critically, can mislead QEC decoders during syndrome extraction.

2.5.4 Leakage

Ideally, qubits operate within a two-dimensional Hilbert space spanned by $|0\rangle$ and $|1\rangle$. However, most physical qubits naturally have access to higher energy levels or additional states. Leakage occurs when a qubit leaves the computational subspace and occupies one

of these unwanted states. Unlike standard bit-flip or phase-flip errors, leakage cannot be handled directly by conventional QEC codes, and leaked qubits can disrupt multi-qubit gates they participate in. Specialized techniques such as leakage-reduction units or active resets are often needed to mitigate this effect.

2.5.5 Crosstalk

In practice, qubits are not perfectly isolated from one another. Crosstalk refers to unwanted interactions between qubits or their control lines. For example, driving a gate on one qubit can unintentionally induce a small rotation on a neighboring qubit, or residual couplings can cause correlated errors across multiple qubits. Crosstalk is especially problematic because most QEC codes assume that errors occur independently across qubits. When errors become correlated, the effective logical error rates are significantly worse than predicted by simple independent-noise models.

2.6 Qubit Technologies

Similar to the early days of conventional computing when digital logic used different underlying technologies such as BJTs and MOSFETs, qubit technologies today are also varied. A summary of major qubit technologies is given below:

At present, there is no clear winner among the different qubit technologies. However, superconducting and neutral atom technologies are at the forefront of all technologies. Superconducting qubits offer fast gate times and can leverage existing fabrication methods for scaling, but are very error-prone. Neutral atoms on the other hand are slower, but offer significantly better error rates.

2.7 Noisy Intermediate-Scale Quantum (NISQ) Era

The current generation of quantum computers is often referred to as Noisy Intermediate-Scale Quantum (NISQ) devices. These systems typically have between 50 and a few hundred qubits, but suffer from high error rates and limited coherence times. NISQ devices are not yet capable of performing error correction, and thus cannot run long quantum algorithms reliably. However, they provide a valuable platform for exploring quantum algorithms,

Qubit Type	Key Players	Highlights and Advantages
Superconducting	IBM, Google, Rigetti	Fast gate speeds and scalable fabrication. IBM's Condor chip surpassed 1,000 qubits.
Trapped Ion	IonQ, Quantinuum	High-fidelity gates and long coherence times. IonQ achieved >99.9% two-qubit gate fidelity.
Photonic	PsiQuantum	Operates at room temperature with potential for massive scalability. Utilizes ultra-low-loss photonic chips.
Topological	Microsoft (research)	Employs Majorana zero modes for error-resilient qubits. First 8-qubit prototype demonstrated in 2025.
Neutral Atom	QuEra, PASQAL	Uses arrays of optically trapped atoms. Offers flexible connectivity and scalability via Rydberg interactions.

Table 2.2: Qubit technologies and their attributes.

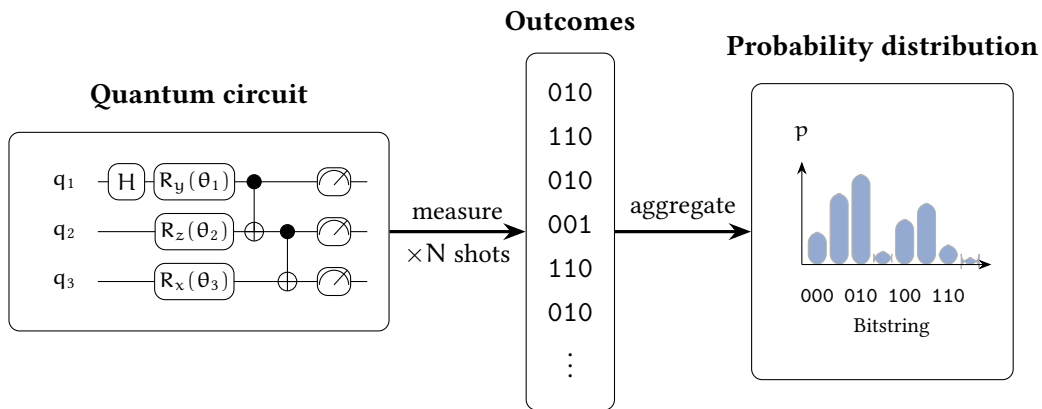


Figure 2.5: General NISQ algorithm workflow. A parameterized quantum circuit is executed and measured N times (shots). The bitstring outcomes are tallied and normalized to produce a probability distribution over measurement results, which a classical optimizer uses to update the circuit parameters θ .

benchmarking hardware performance, and developing error mitigation techniques. NISQ algorithms include variational quantum eigensolvers (VQE) for chemistry simulations, quantum approximate optimization algorithms (QAOA) for combinatorial problems, and quantum machine learning models. Figure 2.5 shows a general overview of how NISQ programs are executed: a classical optimizer (such as Adam, COBYLA) sweeps different values of a set of parameters, and each configuration is run for thousands of shots to generate a probability distribution. The optimizer keeps adjusting parameters until the distribution is as close as possible to the ideal. While NISQ devices are unlikely to achieve quantum advantage for practical applications, they are an essential stepping stone towards the development of fault-tolerant quantum computers.

2.8 Quantum Error Correction

Quantum error correction (QEC) is a set of techniques designed to protect quantum information from errors due to noise and decoherence. QEC works by encoding logical qubits into a larger number of physical qubits, allowing for the detection and correction of errors without directly measuring the quantum information. While a thorough, theoretical treatment of QEC is beyond the scope of this chapter, we will briefly introduce the concept and its importance in quantum computing.

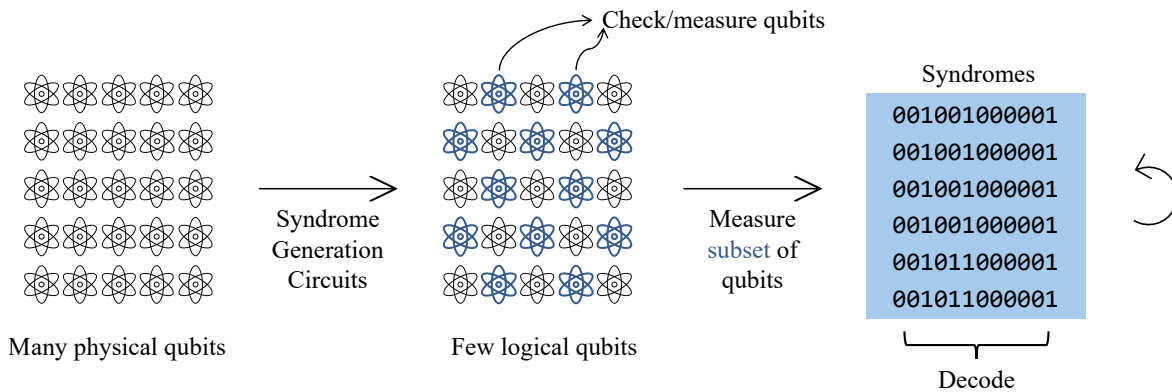


Figure 2.6: A general QEC cycle: physical qubits are encoded into logical qubit(s) by executing syndrome generation circuits, after which a subset of qubits are measured to generate a syndrome. These syndromes are measured over multiple rounds before being passed to the decoder.

Figure 2.6 shows a general overview of the QEC process. The first step is to encode the

quantum information of a logical qubit into multiple physical qubits using an encoding circuit specific to the chosen QEC code. Once encoded, errors are detected by repeatedly running a syndrome generation circuit, which measures certain properties of the physical qubits — known as syndromes — without collapsing the quantum information. Syndrome measurements are performed by measuring a subset of the physical qubits. This subset of qubits are commonly referred to as check or measure qubits, while the rest are called data qubits. Measuring the check qubits yields a syndrome, which is a classical bit string that indicates the presence of errors that may have occurred on the data qubits. Multiple rounds of syndrome measurements are typically performed to improve the reliability of error detection, as individual measurements can themselves be noisy. The collected syndromes are then fed into a classical decoder, which uses the syndrome information to infer the most likely error that occurred and determines the appropriate correction to apply to the data qubits. It is important to note that the syndrome generation and measurement process and the classical decoder are decoupled such that the syndrome generation cycle repeats until the computation ends and does not wait for the decoder. The decoder thus operates asynchronously and in parallel with the syndrome generation cycle, and must be fast enough to keep up with the rate of syndrome generation. The ultimate goal of QEC is to maintain the integrity of the logical qubits over time, allowing for the execution of long quantum algorithms that would otherwise be disrupted by noise and errors.

QEC codes are derived from classical error correcting codes used commonly in digital communications and storage devices like DRAMs. However, the no-cloning theorem [269], which states that quantum information cannot be copied, imposes unique constraints on the design of quantum codes. This is further exacerbated by the fact that encoded states cannot be directly measured to detect errors, as this would collapse the quantum state and destroy the information being protected. To provide better intuition on how QEC works, we will briefly describe the simplest quantum error correcting code, the three-qubit repetition code, which can correct a single bit-flip error on any one of the three physical qubits it uses.

2.8.1 The repetition code

The three-qubit repetition code is not a practical or complete code for QEC. This is because it can only correct for bit-flip errors and not phase-flip errors, which are equally common in

quantum systems. In the 3-qubit repetition code, the logical states are encoded as

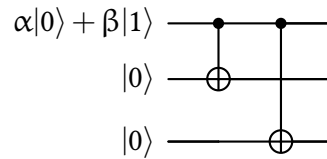
$$|0_L\rangle = |000\rangle, \quad |1_L\rangle = |111\rangle.$$

This is the direct analog of the classical 3-bit repetition code. A general state $\alpha|0\rangle + \beta|1\rangle$ is encoded as

$$\alpha|000\rangle + \beta|111\rangle,$$

so that the quantum information is spread redundantly across three physical qubits.

A simple encoding circuit for the three-qubit repetition code is:



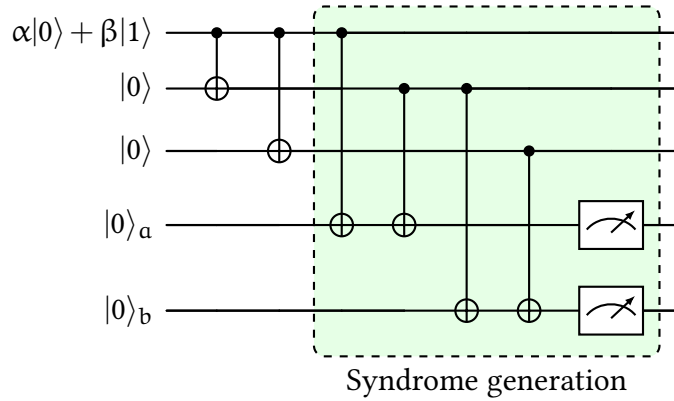
Here, two CNOT gates copy the state of the first qubit onto two ancilla qubits, producing the encoded $|\psi\rangle_L = \alpha|000\rangle + \beta|111\rangle$ state. To take the state of the logical qubit from one to the other $|0\rangle_L \leftrightarrow |1\rangle_L$, three individual qubits must be flipped since $|0\rangle_L = |000\rangle$ and $|1\rangle_L = |111\rangle$. This is essentially the binary distance between the two codewords or logical states of the code. This distance d defines the number of errors that the code can correct. In this case, $d = 3$ and the code can correct for $\lfloor (d-1)/2 \rfloor = 1$ error.

Since all three qubits are used to encode the logical information $|\psi\rangle$, we introduce two additional ancilla qubits to measure syndromes of the code. Suppose a single bit-flip error X occurs on one of the three qubits, e.g., on the second qubit:

$$\alpha|000\rangle + \beta|111\rangle \xrightarrow{X_2} \alpha|010\rangle + \beta|101\rangle.$$

The encoded information is still recoverable, because two of the qubits still carry the correct value. The key is to measure the *parity* between qubits to detect the presence and location of the error. The parity of the first and second qubits is measured by the operator $Z_1 Z_2$, which yields a syndrome bit of 0 if the two qubits are the same and 1 if they are different. Similarly, the parity of the second and third qubits is measured by $Z_2 Z_3$. The syndrome

measurement circuit for the three-qubit repetition code is shown below.



In the circuit above, ancillas a and b are used to measure the syndromes. a measures the parity between the first and second data qubits, while b measures the parity between the second and third data qubits. Table 2.3 shows the effect of each single-qubit bit-flip error on the encoded state and the measured syndrome bits a and b . From the syndrome bits, we can infer the location of the error and apply the appropriate correction, as shown in Table 2.4. The error-correcting limitations of this code is further highlighted by the fact that a two-qubit error, e.g., X_1X_2 , produces the same syndrome as a single-qubit error X_3 .

Error	Data qubits state	a	b
None	$\alpha 000\rangle + \beta 111\rangle$	0	0
X_1	$\alpha 100\rangle + \beta 011\rangle$	1	0
X_2	$\alpha 010\rangle + \beta 101\rangle$	1	1
X_3	$\alpha 001\rangle + \beta 110\rangle$	0	1

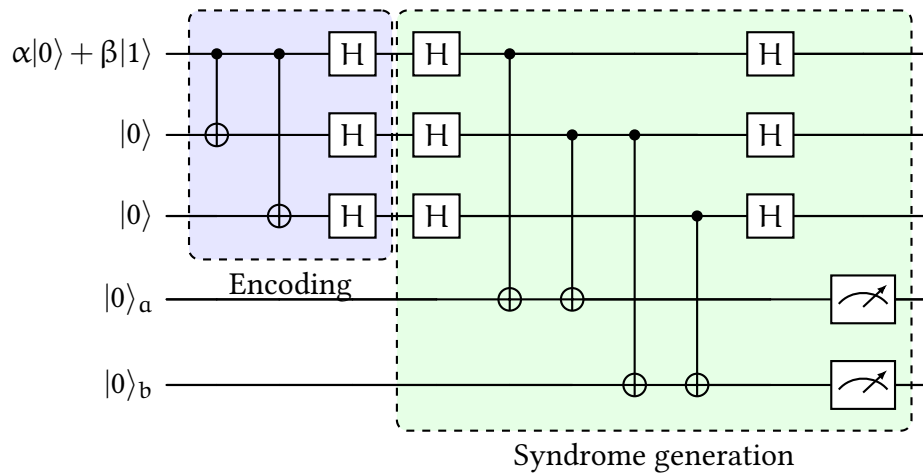
Table 2.3: Effect of each single-qubit bit-flip error on the three-qubit encoded state and the syndrome ancilla outcomes. Ancilla a measures the parity Z_1Z_2 ; ancilla b measures the parity Z_2Z_3 . A syndrome bit of 1 indicates odd parity, signalling an error.

The three-qubit repetition code illustrated above can only detect and correct bit-flip errors. However, qubits are also susceptible to phase-flip errors. We now show how the

a	b	Inferred error	Correction
0	0	None	—
1	0	X_1	Apply X_1
1	1	X_2	Apply X_2
0	1	X_3	Apply X_3

Table 2.4: Syndrome lookup table for the three-qubit bit-flip code. Each syndrome uniquely identifies the location of a single bit-flip error. Applying the same X gate at the inferred location restores the encoded state.

three-qubit repetition code can be adapted to detect and correct phase-flip errors.



In the circuit above, the earlier repetition code is adapted to detect and correct phase-flip errors instead of bit-flip errors. First, the encoding circuit is modified by adding a layer of Hadamard gates after the two CNOT gates. This transforms the logical states to $|\psi\rangle_L = \alpha|+++\rangle + \beta|---\rangle$, where $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ and $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$. In this new basis, a phase-flip error Z on any one of the three qubits manifests as a bit-flip error in the $|+\rangle/|-\rangle$ basis. Thus, the syndrome generation circuit is modified to measure the parity in the $|+\rangle/|-\rangle$ basis by adding Hadamard gates before and after the CNOT gates that couple the data qubits to the ancillas. The syndrome measurement and correction process is then analogous to the bit-flip code, but now it detects and corrects phase-flip errors instead of bit-flip errors.

Summary. The three-qubit repetition code is a simple example of a quantum error correcting code that can correct for either bit-flip or phase-flip errors, but not both simultaneously. More sophisticated codes, such as the Shor code, Steane code, surface code, and quantum low-density parity-check (qLDPC) codes, are designed to correct for both types of errors and can achieve higher distances, allowing for the correction of multiple errors. We will now briefly introduce stabilizer formalism, which is an efficient framework for describing most quantum error correcting codes that are members of class called stabilizer codes.

2.8.2 Stabilizer formalism

The stabilizer formalism was first introduced by Daniel Gottesman [104]. A stabilizer code is defined by a set of operators, called stabilizers, that leave the encoded logical states invariant, i.e., $K|\psi\rangle = |\psi\rangle$. For example, the single qubit state $|\psi\rangle = |0\rangle$ is stabilized by the operator Z , since $Z|0\rangle = |0\rangle$. Many extremely useful multi-qubit states, such as Bell states and Greenberger-Horne-Zeilinger (GHZ) states, are also stabilizer states. As an example, the three-qubit GHZ state $|\text{GHZ}\rangle = \frac{1}{\sqrt{2}}(|000\rangle + |111\rangle)$ is stabilized by the operators Z_1Z_2 , Z_2Z_3 , and $X_1X_2X_3$. More formally, the stabilizers of a code form an *abelian subgroup* $\mathcal{S}$ of the n -qubit Pauli group $\mathcal{P}_n$, where each element is a tensor product of single-qubit Pauli operators. The requirement that stabilizers commute with one another ensures a joint $+1$ eigenspace exists; this joint eigenspace is the *codespace*, the set of all states satisfying $K_i|\psi\rangle = |\psi\rangle$ for every generator $K_i \in \mathcal{S}$. An n -qubit stabilizer code with $n - k$ independent generators encodes k logical qubits into the codespace, and the $n - k$ generators provide an exponentially more compact description than the 2^n amplitudes of a general quantum state.

Error detection. The connection to QEC follows from how Pauli errors interact with stabilizers. Every Pauli error E either commutes or anti-commutes with each stabilizer K_i . If E commutes with K_i , the errored state $E|\psi\rangle$ still satisfies $K_i(E|\psi\rangle) = E(K_i|\psi\rangle) = E|\psi\rangle$, yielding measurement outcome $+1$. If instead E anti-commutes with K_i (i.e., $EK_i = -K_iE$), then

$$K_i(E|\psi\rangle) = -E(K_i|\psi\rangle) = -(E|\psi\rangle),$$

so measuring K_i on the errored state yields -1 . The *syndrome* is the vector of measurement outcomes across all $n - k$ stabilizer generators, encoded as a classical bit string (0 for $+1$, 1 for -1). A non-trivial syndrome reveals which stabilizers anti-commute with the error, and

the decoder uses this information to identify and correct the error. Crucially, measuring the stabilizers does not disturb the encoded logical information, since K_i acts as identity on any state already in the codespace.

Logical operators. Any Pauli operator that commutes with all stabilizers in $\mathcal{S}$ but is not itself an element of $\mathcal{S}$ is a *logical operator*: it acts non-trivially on the encoded qubits without producing any syndrome. An undetectable logical error is therefore a Pauli operator that lies outside $\mathcal{S}$ but commutes with every stabilizer generator. The minimum weight of any such operator is the *distance* d of the code; a code of distance d can correct any combination of errors on fewer than $\lfloor (d-1)/2 \rfloor$ physical qubits. Every encoded logical qubit has a logical $\bar{X}$ and a logical $\bar{Z}$ operator, which satisfy the same anti-commutation relation $\{\bar{X}, \bar{Z}\} = 0$ as physical Pauli operators and implement the corresponding logical operations on the codespace.

Connection to the repetition code. The three-qubit repetition code introduced in Section 2.8.1 is a stabilizer code. Its two stabilizer generators are exactly the parity-check operators Z_1Z_2 and Z_2Z_3 , and the ancilla measurement outcomes a and b in Table 2.3 are the corresponding syndrome bits. A bit-flip error X_i anti-commutes with any Z acting on the same qubit, which is why each single-qubit error produces a distinct non-zero syndrome. The logical $\bar{X} = X_1X_2X_3$ commutes with both stabilizer generators (each generator shares two qubits with $X_1X_2X_3$, and the two anti-commutations cancel) but is not in $\mathcal{S}$, confirming it is a weight-3 logical operator and that the code has distance $d = 3$ against bit-flip errors. The logical $\bar{Z} = Z_1$ (equivalently Z_2 or Z_3 , since they differ by elements of $\mathcal{S}$) commutes with both generators and maps $|0_L\rangle \mapsto |0_L\rangle$ and $|1_L\rangle \mapsto -|1_L\rangle$.

2.8.3 Decoding

A stabilizer code can be defined by two classical binary matrices: the X-type parity check matrix H_X and the Z-type parity check matrix H_Z (we restrict ourselves to CSS codes for now). These matrices specify the parity check matrix H of the code as

$$H = \begin{bmatrix} H_X \\ H_Z \end{bmatrix}$$

Each row of H_X corresponds to an X-type stabilizer generator, and each row of H_Z corresponds to a Z-type stabilizer generator. Each column of H corresponds to a physical qubit, and a 1 in the matrix indicates that the corresponding stabilizer generator acts non-trivially (i.e., with an X or Z) on that qubit. A logical action matrix A is also defined for the code, where each row corresponds to a logical operator and each column corresponds to a physical qubit.

Given a syndrome σ , if the actual error vector that affected the qubits is F , the decoder's task is to find an error vector F' over $\mathbb{F}_2$ satisfying $HF' = \sigma$. Each single-qubit error location has an associated weight, typically derived from its log-likelihood under the noise model. The minimum-weight decoding paradigm seeks the error vector F' satisfying this relation whose total weight—the sum of weights of all active error locations—is minimized. This approximates maximum-likelihood decoding, which would instead identify the most probable equivalence class of errors consistent with σ by marginalizing over all errors within each class; minimum-weight decoding instead finds the most probable individual error vector, and serves as a computationally tractable approximation to maximum-likelihood decoding.

In general, it is not necessary that $F = F'$, since the decoder is successful if $A(F \oplus F') = 0$, i.e., F and F' differ at most by a stabilizer and thus implement the same logical action.

2.8.3.1 Noise models

Minimum-weight decoding requires the knowledge of error probabilities and processes that can cause check qubits to flip. For QEC simulations, this is generally defined by the noise models selected for a given QEC code and experiment. We now describe the three most commonly used noise models:

Code-capacity model. The code-capacity noise model is the simplest noise model and is used to determine the upper bound (performance limits) on the effectiveness (error suppression) of a QEC code. In the code-capacity noise model, only data qubits can experience errors at the very start of a syndrome generation cycle. Every other operation (measurements, gates) is error-free.

Phenomenological model. The phenomenological noise model is more realistic since it accounts for measurement errors on the check qubits. At the start of every syndrome generation round, data qubits experience an error with some predefined probability too.

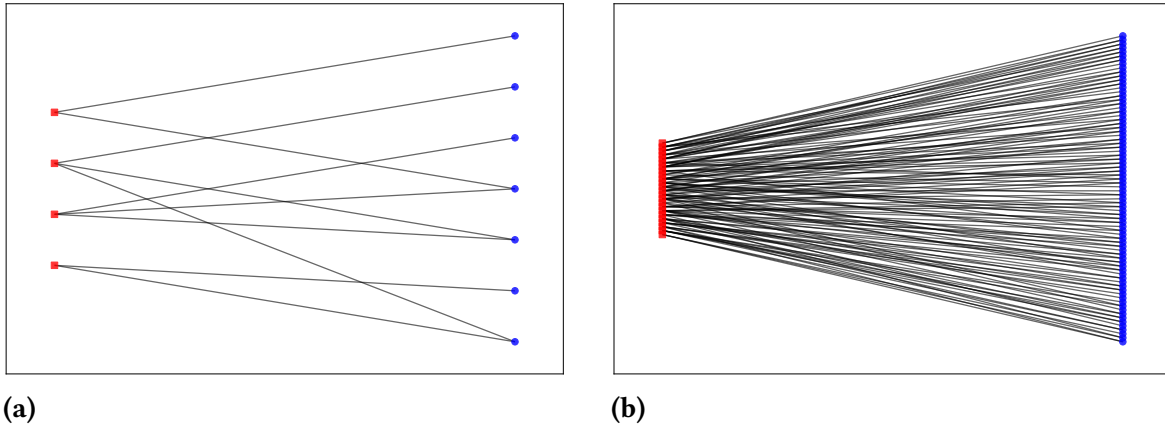


Figure 2.7: Difference in the decoding graphs with a (a) Code-capacity noise model; (b) Circuit-level noise model.

However, this noise model does not account for errors due to all the different gates in the system.

Circuit-level model. Circuit-level noise models are the most realistic noise models since they account for noise not only due to data qubit decoherence but also due to every gate and measurement in the syndrome generation circuit.

2.8.3.2 Decoding graphs

The choice of QEC code and the noise model determines the complexity of the decoding problem. In the code-capacity model, the simple assumptions mean that it can be decoded in a single round of measurements since there are no measurement or gate errors. For the phenomenological and circuit-level noise models, the addition of measurement errors makes the decoding problem more complicated. Every noise model changes the number of error mechanisms that affect the code, and thus increases the complexity of the decoding problem.

The combined effect of the code and noise model is captured formally by the *detector error model* (DEM). Rather than operating on physical qubits directly, the DEM reformulates the decoding problem in terms of *detectors*—parities of measurement outcomes across one or more syndrome rounds—and *faults*—physical error mechanisms that flip one or more detectors. This yields a fault-space parity check matrix H_{DEM} , where each row corresponds to a detector and each column corresponds to a fault, with a 1 indicating that the fault flips that detector. Each fault is assigned a weight derived from its log-likelihood. The

decoding task then becomes finding the minimum-weight fault vector F' over $\mathbb{F}_2$ satisfying $H_{\text{DEM}}F' = \sigma$. The DEM thus provides a unified representation of code and noise model that a decoder can consume without needing to reason about the underlying circuit or stabilizer structure directly.

The DEM has a natural graphical representation as a *decoding graph*, in which nodes correspond to detectors and (hyper)edges correspond to faults, connecting the detectors they flip and weighted by their log-likelihoods. When every fault flips at most two detectors, the decoding graph is an ordinary graph and the minimum-weight decoding problem reduces to minimum-weight perfect matching (MWPM), which can be solved efficiently. Faults that flip more than two detectors give rise to hyperedges, requiring more general hypergraph decoders. The structure of the decoding graph i.e., its density, connectivity, and the prevalence of hyperedges, depends directly on the noise model. For example, Figure 2.7 shows the differences in the decoding graph for a code-capacity noise model (Figure 2.7(a)) and a circuit-level noise model (Figure 2.7(b)): the latter contains significantly more nodes and edges, reflecting the additional fault mechanisms introduced by gate and measurement errors across multiple syndrome rounds.

2.8.3.3 Types of decoders

The minimum-weight decoding problem can be solved by a variety of algorithmic strategies, each with different trade-offs between accuracy, latency, and scalability.

SAT-based decoders. SAT-based decoders reformulate the decoding problem as a Boolean satisfiability or maximum-satisfiability (MAX-SAT) instance and invoke an off-the-shelf solver. Because the underlying problem is NP-hard in general [117], this approach is exact but does not scale to large codes or real-time operation; it is most useful as an offline reference for benchmarking other decoders.

Search-based decoders. Search-based decoders enumerate candidate error patterns explicitly, using heuristics such as beam search or branch-and-bound to prune the exponentially large search space. They are flexible and applicable to arbitrary codes and noise models but remain computationally expensive for large code distances.

Message-passing decoders. Message-passing decoders, most commonly belief propagation (BP), operate on the Tanner graph of the parity-check matrix by iteratively exchanging probability estimates between qubit and stabilizer nodes until convergence. BP is fast and naturally parallelizable, and performs well on sparse LDPC codes, but degrades on graphs with short cycles, which are prevalent in topological codes.

Ordered statistics decoding. Ordered statistics decoding (OSD) post-processes the output of belief propagation by ranking qubits by reliability, solving a reduced subsystem exactly using the most reliable positions, and then enumerating low-weight corrections among the remaining positions. OSD achieves near-maximum-likelihood accuracy at the cost of additional classical computation, and BP-OSD hybrids are among the best-performing decoders for qLDPC codes.

Clustering decoders. Clustering decoders partition the syndrome graph into small local clusters of flipped detectors and decode each cluster independently. The union-find decoder is the canonical example: it grows clusters using a union-find data structure and achieves nearly linear time complexity, making it highly scalable. The trade-off is a modest reduction in accuracy relative to MWPM.

Neural network decoders. Neural network decoders replace explicit algorithmic rules with a trained model—ranging from feedforward networks to recurrent or graph neural networks—that maps syndrome patterns directly to predicted corrections. They can implicitly capture complex noise correlations without an explicit noise model, but require large training datasets, often generalize poorly across code distances, and can be difficult to deploy within the tight latency budgets of real-time QEC.

2.8.4 Threshold theorem

The *fault-tolerance threshold theorem* [1, 139] is the foundational result that makes scalable quantum error correction practically meaningful. Informally, it states that if the physical error rate p of every gate and measurement is below a critical value p_{th} , called the *threshold*, then the logical error rate p_L can be made arbitrarily small by increasing the code distance d , at the cost of a polynomial increase in the number of physical qubits.

More concretely, for a code of distance d operating below threshold, the logical error rate scales as

$$p_L \propto \left(\frac{p}{p_{\text{th}}} \right)^{\lfloor (d+1)/2 \rfloor}. \quad (2.4)$$

Each additional increment of distance therefore suppresses the logical error rate by another factor of (p/p_{th}) , which is less than one when $p < p_{\text{th}}$. This exponential suppression is what distinguishes fault-tolerant quantum computing from the NISQ regime: rather than accumulating errors with circuit depth, a fault-tolerant system can in principle run arbitrarily long computations by simply encoding at a high enough distance.

The threshold value depends on both the QEC code and the noise model. Under the circuit-level depolarizing noise model, the surface code has a well-studied threshold of approximately $p_{\text{th}} \approx 1\%$ [85]. State-of-the-art superconducting and trapped-ion devices are now approaching or exceeding this threshold for their best gate operations, making the fault-tolerant regime a near-term experimental target. The threshold theorem therefore serves as a design criterion for hardware: all components of the control stack — gates, measurements, reset operations, and the classical decoder — must collectively keep the effective per-operation error rate below p_{th} .

2.8.5 Some common QEC codes

We now briefly describe some commonly studied QEC codes that are also used throughout the rest of this dissertation.

2.8.5.1 Surface code

The surface code is a topological quantum error correcting code defined on a two-dimensional lattice of qubits. It is one of the most widely studied and experimentally implemented QEC codes due to its high error threshold and local stabilizer structure, which makes it well-suited for physical implementations with nearest-neighbor interactions. Figure 2.8 shows a logical qubit encoded in a surface code patch and its syndrome generation circuit. This particular configuration of the surface code is called the rotated surface code, and is the most resource efficient variant of the surface code. As can be seen from the figure, the surface code requires only nearest-neighbor interactions, and qubits are arranged in a 2D grid. The X and Z-type check/measure qubits are interleaved and detect Z and X errors, respectively. The syndrome

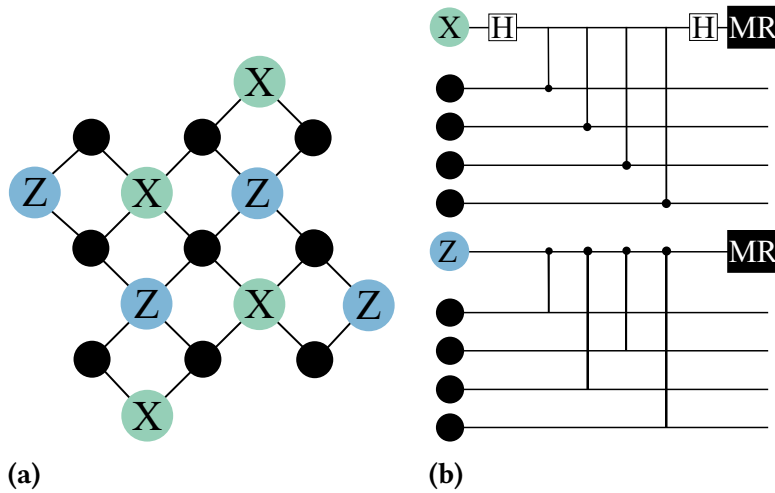


Figure 2.8: (a) A surface code logical qubit: black nodes represent data qubits, nodes annotated with X and Z represent check qubits that measure X and Z parities, respectively. (b) Syndrome generation circuit of the surface code logical qubit. H: Hadamard gate, MR: Measurement and reset operation, two-qubit gates shown are CNOTs.

generation circuit of the surface code is shown in Figure 2.8(b). This circuit is replicated across the entire lattice, and is repeated for multiple rounds to generate syndromes for decoding.

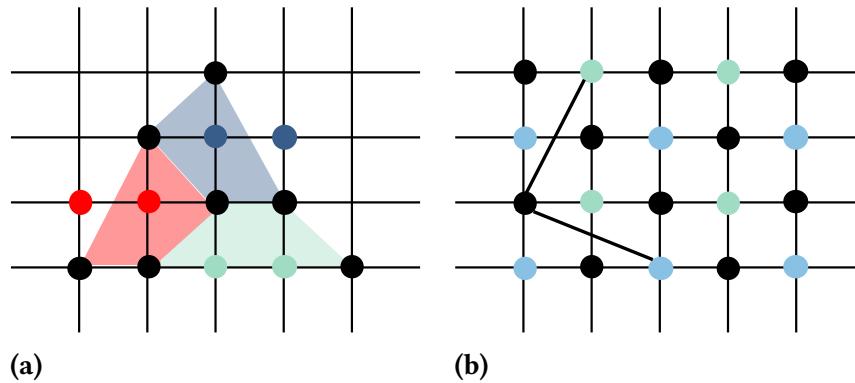


Figure 2.9: (a) A color code logical qubit mapped on a square lattice; (b) A portion of the lattice used for encoding logical qubits using the bivariate bicycle (BB) code, a qLDPC code. Notice the long range interactions between data and check qubits. For both color and BB codes, the black nodes represent data qubits, and colored nodes represent check qubits that measure stabilizers.

2.8.5.2 Color codes

Color codes are another class of topological quantum error correcting codes defined on a two-dimensional lattice, similar to the surface code. However, color codes have a more complex stabilizer structure that allows for the transversal² implementation of a larger set of logical gates, including the entire Clifford group. Figure 2.9(a) shows a logical qubit encoded in a color code defined on a square lattice. In this configuration, the check qubits are colored red, green, and blue, and each color corresponds to a different type of stabilizer generator that measures a specific combination of X and Z operators on the data qubits. This particular construction of the color code is adapted for implementation on a square lattice [145] with nearest-neighbor interactions, but color codes can also be defined on more general lattices with different connectivity.

2.8.5.3 qLDPC codes

Quantum low-density parity-check (qLDPC) codes are a class of quantum error correcting codes that generalize classical LDPC codes to the quantum setting. qLDPC codes are defined by sparse parity check matrices, which means that each stabilizer generator acts on a small number of qubits, and each qubit is involved in a small number of stabilizers. This sparsity can lead to efficient decoding algorithms and potentially lower overhead compared to more traditional codes like the surface code. In particular, the surface code can only encode a single logical qubit per lattice, while qLDPC codes can encode multiple logical qubits in the same number of physical qubits, thus reducing the overhead for quantum error correction. This advantage comes at the cost of more complex stabilizer structures and potentially more challenging decoding problems, especially under realistic noise models. Figure 2.9(b) shows a portion of the lattice used for encoding logical qubits using the bivariate bicycle (BB) code [30, 280], a specific type of qLDPC code. The long-range interactions increase the connectivity requirements on the quantum chip, making this class of codes more difficult to implement, despite their reduced qubit overhead.

²Transversal gates can be applied directly on the physical qubits in the code rather than using elaborate constructions specific to the code.

2.8.6 Logical operations

A QEC code encodes k *logical qubits* into n physical qubits by restricting the joint state of the system to a 2^k -dimensional subspace of the full 2^n -dimensional Hilbert space, called the *code space*. The encoded quantum information lives entirely within this subspace; the remaining dimensions are used by the code to detect and correct errors, as described in the preceding sections. The key abstraction introduced by QEC is that algorithms and programs can be written entirely in terms of logical qubits and logical operations, without reference to the underlying physical qubits or the error correction machinery running beneath them.

A *logical gate* is a physical circuit applied to the n physical qubits of a code block that realizes a desired unitary on the encoded information while leaving the code space invariant. Analogously, *logical initialization* prepares a code block in the logical $|0\rangle_L$ or $|+\rangle_L$ state, and a *logical measurement* extracts a classical bit from the logical qubit without collapsing the physical qubits unnecessarily. Together, these operations form a *logical instruction set*: the interface between a quantum program and the physical hardware.

Not all logical gates are equally straightforward to implement fault-tolerantly. For many QEC codes, logical Clifford gates admit *transversal* implementations, in which the same single-qubit physical gate is applied independently to every qubit in the code block. Because each physical gate touches only one qubit, errors cannot propagate across multiple qubits within the same block, making transversal implementations inherently fault-tolerant. However, the Eastin–Knill theorem proves that no QEC code can have a transversal gate set that is universal for quantum computation [76]. The logical T gate — which, together with Clifford gates, forms a universal gate set — cannot be implemented transversally on most topological codes. Instead, it is realized via *magic state distillation*: many noisy copies of an ancilla state $|A\rangle = T|+\rangle$ are distilled into a single high-fidelity copy, which is then consumed to enact the logical T gate via a teleportation-based circuit. Magic state distillation is resource-intensive and is expected to dominate the physical qubit overhead of early fault-tolerant computers, which is why other techniques such as magic state cultivation are likely to be used first.

Two-qubit logical gates, such as a logical CNOT, require physical interactions between the qubits of two separate code blocks. Executing such a gate therefore demands that the two logical qubits be synchronized: their QEC cycles must be coordinated so that the inter-block operation can be applied at a point where both code blocks are in a consistent state. This is

qualitatively different from single-qubit logical gates, which act entirely within one code block and can proceed independently of other logical qubits.

These considerations define the scope of *logical-level control*: translating a quantum algorithm’s stream of logical instructions into the physical circuits, magic state requests, and inter-block synchronization events that the classical control hardware must execute.

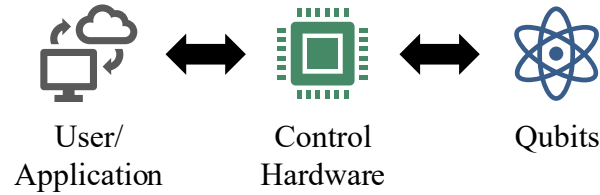


Figure 2.10: Overview of the classical control hardware for a quantum computer: the control system sits between the quantum processor and the user/application layer, and is responsible for translating programs into low-level instructions that can be executed on the quantum processor, as well as reading and measuring the state of the qubits, and running the error correction protocols.

2.9 Classical Control for Quantum Computers

Quantum computers require a classical control system to interact with qubits, and execute different subroutines that enable computation using those qubits. The complexity of the classical control system depends on the qubit technology, and will become more complex and involved as the number of qubits scale up and we transition from the NISQ era to the era of fault-tolerant quantum computing. Figure 2.10 shows an overview of the classical control hardware for a quantum computer, which sits between the quantum processor and the user/application layer. Different qubit technologies have different low-level control requirements. For example, superconducting qubits require microwave pulses for executing gates, while trapped ion qubits require laser pulses. Some responsibilities of the control system for fault-tolerant quantum computing include:

- **Running QEC syndrome generation circuit subroutines:** the control system is responsible for executing the syndrome generation circuits for each logical qubit, and repeating these circuits for multiple rounds to generate syndromes for decoding.

- **Maintain logical frames:** Once the decoder determines a correction, the correction is not always applied directly on the physical qubits. Instead a frame is maintained by the control system which tracks the logical state of the qubits.
- **Executing logical operations:** On top of the syndrome generation circuits, the control system is also responsible for executing logical operations on the logical qubits, which is necessary for running quantum algorithms.
- **Routing and decoding syndromes:** the control system is responsible for routing the generated syndromes to the decoder, and running the decoder to determine the appropriate corrections to update the logical frame.
- **Controlling physical qubits:** All operations that must be performed on the physical qubits must be translated to signals that the qubits can understand. This translation layer forms the lowest level of the control system.
- **Measuring qubits:** Measurement (readout) of physical qubits is necessary for generating syndromes, and this is also part of the low-level control system. The syndromes generated are then passed to the decoder for processing.

Having highlighted the role of the classical control system for running applications on quantum computers, we will now discuss contributions made as part of this dissertation to various levels of the control stack.

Chapter 3

Optimizing Qubit Control and Readout

3.1 Introduction

Qubit control and readout form the lowest level of the classical control system for quantum computers. Typically, qubit control refers to the process of manipulating the states of the qubits, while readout refers to the act of measuring the qubits. High fidelity control and readout of physical qubits is essential for Near-Term Intermediate Quantum Computing (NISQ) systems as well as Fault-Tolerant Quantum Computing (FTQC) systems. Qubit control and readout systems are dependent on the underlying qubit technology. For example, superconducting qubits require microwave pulses for control and readout, while neutral atom based systems utilize lasers. FTQC systems capable of running complex, useful applications will require control and readout systems for thousands of qubits, which makes it essential for them to be scalable while preserving the quality of smaller scale systems. In this chapter, we will focus on the control and readout of superconducting qubits. In particular we will focus on making control and readout more scalable without sacrificing precision. This forms the first research thrust of this dissertation, where low-level control system components are optimized for scalability to facilitate FTQC.

3.1.1 Contributions

The contributions of this chapter are summarized below:

- We introduce COMPAQT: A compressed waveform memory architecture for the

control of superconducting qubits.

- We introduce HERQULES: A machine-learning (ML) based qubit readout state discriminator that mitigates crosstalk and thermal relaxation errors that occur during the readout of superconducting qubits.

3.1.2 Organization

This chapter is organized as follows:

- In Section 3.2, we introduce the control and readout control systems for superconducting qubits.
- In Section 3.3, we show how COMPAQT makes qubit control more scalable using compressed waveform memories.
- In Section 3.4, we describe the architecture of HERQULES and show how detecting thermal relaxations during readout boosts the readout fidelity without requiring complex hardware.
- We summarize this chapter in Section 3.5.

3.2 Superconducting Qubit Control and Readout

We first briefly introduce superconducting qubits and how they are controlled and measured.

3.2.1 Superconducting qubits

Superconducting qubits are one of the most promising qubit technologies, offering fast gate operations (orders of magnitude faster than neutral atom or trapped-ion systems) and compatibility with existing semiconductor fabrication processes.

Superconducting qubits are weak anharmonic oscillators. Anharmonicity — the property that energy level spacings are non-uniform — is essential for isolating the two lowest energy levels as a well-defined qubit: without it, control pulses targeting the $|0\rangle \leftrightarrow |1\rangle$ transition would also drive unintended transitions to higher energy levels. Intuitively, a superconducting qubit can be understood as a variation of the harmonic LC oscillator shown

in Figure 3.1(a). The key difference is that the linear inductor is replaced by a nonlinear circuit element called a Josephson junction (JJ), as shown in Figure 3.1(b). The Josephson junction requires a superconducting material to function, which is the origin of the name *superconducting qubits*.

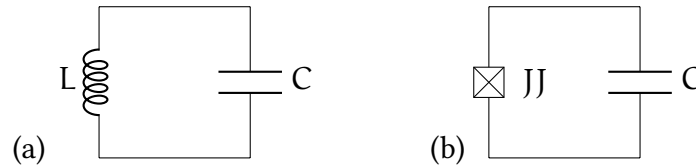


Figure 3.1: (a) Parallel LC oscillator; (b) Josephson junction used for introducing non-linearities and anharmonicity.

Since they are oscillators, superconducting qubits have a resonant frequency that is either fixed at fabrication or tunable at runtime, depending on the qubit design. This resonant frequency is typically in the microwave range (5–8 GHz), which is why microwave pulses are used for control and readout. For a comprehensive treatment of superconducting qubit physics, we refer the reader to [143]. We now briefly overview control and readout for this qubit technology.

3.2.2 Control

Quantum algorithms can be expressed as quantum circuits, a sequence of gate operations applied on qubit variables. On superconducting architectures, the gates are implemented using microwave pulses as qubit devices (such as transmons) are essentially oscillators that respond to signals in the frequency range of 4-10 GHz. Quantum control refers to the

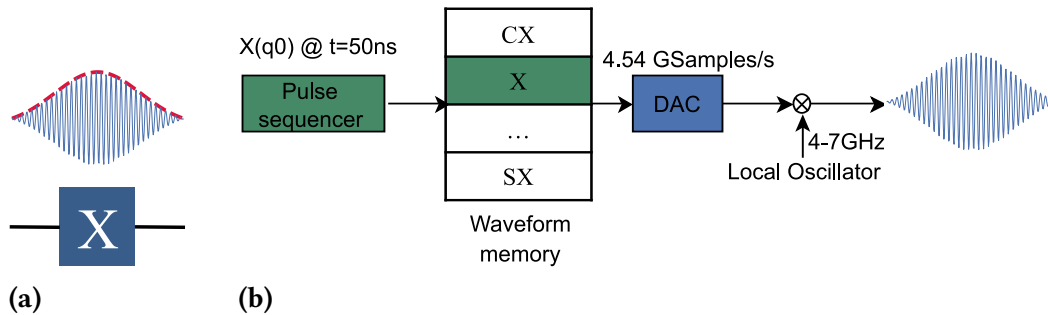


Figure 3.2: (a) An X-gate and its corresponding microwave pulse (in blue) The envelope of the pulse is shown in red (b) A typical pipeline for generating control pulses.

synthesis of these microwave pulses to implement various quantum gates. These microwave pulses can have different amplitudes, shapes, and duration depending on the type of gate and the qubit architecture. The state of a qubit is represented by a vector on a Bloch sphere [196]. When a pulse is applied, the state rotates about the X/Y/Z axis of the Bloch sphere. Since Z rotations can be implemented in software [184], microwave pulses are composed of two components that control the qubit state evolution about the X and Y-axis of the Bloch sphere. Consider the circuit shown in Figure 3.2(a), with X-gate applied on qubit q0, which inverts the qubit state. Physically, this gate is implemented as a pulse with a frequency equal to the qubit resonant frequency (shown in blue). The envelope of the pulse refers to its shape (shown as dotted line), and we will refer to it as a waveform henceforth.

The physical implementation of two-qubit (2Q) gates such as CNOT/CZ depends on qubit architecture [143]. For example, IBM machines use fixed-frequency qubits, wherein 2Q gates are implemented using Cross-Resonance (CR) gates [46, 215] whereas the frequency-tunable qubits used by Google implement a 2Q gate called the iSWAP gate [11, 143]. Both implementations may require additional pulses for the coupler that enables interactions between neighboring qubits. In addition to standard gates, custom pulses can significantly boost application fidelity. For example, Toffoli gates have been demonstrated on superconducting hardware [135, 243]. Furthermore, Shi et al. proposed to use custom pulses to enable multi-qubit operators to reduce circuit depth and improve fidelity [225], and Xie et al. showed that carefully re-shaping the waveforms can reduce ZZ-crosstalk [274].

Figure 3.2(b) shows a simplified control pipeline used to generate microwave gate pulses to manipulate the state of the qubit. A pulse sequencer is used to trigger the execution of gates (an X gate for qubit q0 in this case). The pulse envelopes are stored in the waveform memory, and they consist of the Inphase (I) and Quadrature (Q) components for rotating the qubit about the X- and Y-axis of the Bloch sphere. Since DACs have a fixed sampling rate (sampling rate of IBM machines annotated in the figure), the waveform memory interface should supply data with a bandwidth that matches the sampling rate to ensure signal integrity. Higher DAC sampling rates are desirable for high-fidelity gates. The waveforms are converted to pulses at an intermediate frequency by a DAC. The resultant pulse is then mixed with a microwave carrier generated by a local oscillator (LO) to produce the final microwave pulse.

State-of-the-art quantum control hardware uses discrete FPGAs, DACs, and ADCs placed at room temperature. Controllers can be made more scalable with the use of RFSoc platforms

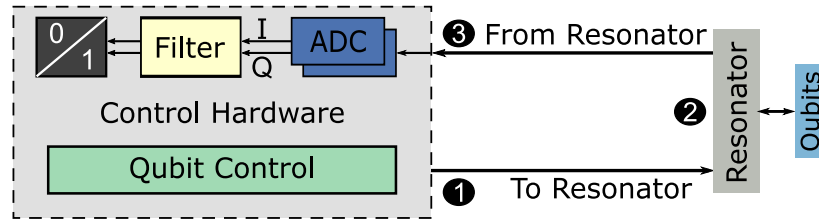


Figure 3.3: The qubit readout pipeline comprises 1) readout-probe-tone generation, 2) acquisition of qubit-state-dependent phase shift, and 3) readout-signal processing.

that integrate a CPU, FPGA, DACs, and ADCs on the same chip. The CPU is interfaced with the control hardware to calibrate qubits and load experiments, while the qubits are actively controlled using an FPGA. As suggested in [241], a single RFSoc can control more than 100 qubits with the help of frequency division multiplexing (FDM), making this approach more scalable in terms of cost and complexity. The high-bandwidth DACs/ADCs on-chip eliminate the need for external analog mixers; pulses can be directly synthesized at the desired frequency. Other ways of implementing the control computer include ASICs that can integrate large numbers of discrete components but are expensive to design and manufacture.

3.2.3 Readout

Readout refers to the process of determining the state of a qubit. After a measurement, the qubit is typically in the ground state ('0') or excited state ('1'). Superconducting qubits are dispersively coupled to circuit elements called resonators which interact with the qubits for the purpose of readout. Readout involves multiple steps, as visualized in Figure 3.3: **①** a microwave pulse is driven into the resonator by the control hardware; **②** the incident microwave pulse undergoes a phase shift that depends on the qubit state; **③** the control hardware analyzes the phase shift to discriminate between the ground and excited states. Superconducting qubit readout involves multiple analog components along with robust digital signal processing to achieve appreciable accuracy. In this paper, we focus on improving step **③** of the readout process, in terms of both accuracy and scalability.

To determine whether a qubit is in state '0' or '1', the microwave signal transmitted or reflected off the readout resonator is typically frequency-downmodulated, digitized, and then processed in the control hardware. The incoming microwave signal is quadrature modulated. The In-phase(I) and Quadrature(Q) components of the signal are retrieved via

analog mixing before they are digitized by two Analog-to-Digital Converters (ADCs). ADCs used for readout have sampling rates ranging from 250–1000 MSamples/sec, which makes the data generated by the ADCs for long readout pulses challenging to manage without additional dimensionality reduction. A (mode) matched filter [11, 110, 220, 266] is used to reduce the incoming I and Q data stream from the ADCs to a single value. The filtered values are then used to discriminate the qubit state using techniques such as clustering or Support Vector Machines (SVMs) [168].

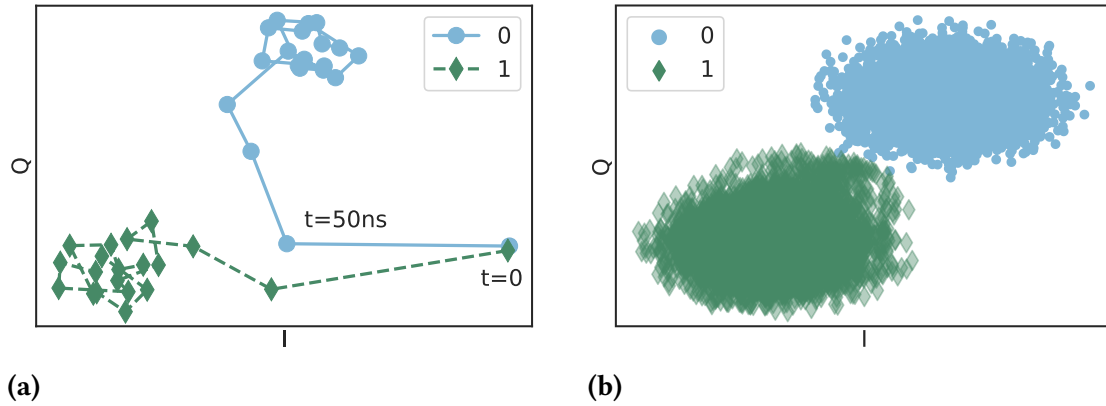


Figure 3.4: (a) Evolution of readout traces for ground and excited states in the 2D I-Q plane – the point annotated with ‘t = 0’ represents the first time bin of the readout trace, and every subsequent point comes later at a granularity of 50ns; (b) Mean values of multiple readout traces for both states showing the distinction between them.

Figure 3.4 illustrates the signal generated during the readout process and its subsequent processing. In Figure 3.4(a), the output of the ADCs (I and Q channels) is represented on a 2D plane showing how the qubit-readout trace evolves over time (each IQ-pair corresponds to a period of 50ns). The temporal mean of one such trace corresponds to a single point in Figure 3.4(b), which represents the Mean Trace Value (MTV). The MTV of a trace Tr is given by the equation: $MTV = \frac{1}{\text{len}(Tr)} \sum_{t=0}^{\text{len}(Tr)} Tr(t)$. In the case of frequency-multiplexed readout where multiple qubits are measured using the same physical channel [11, 45, 110, 149, 266], the readout trace obtained from the ADCs can be demodulated to retrieve the traces of the constituent qubits. Digital demodulation of the readout signal requires the extraction of the readout signals for each qubit. For this work, digital demodulation is achieved by multiplying the frequency-multiplexed readout signal with an oscillating signal at a frequency specific to the readout resonator. The result is then averaged over intervals of 50ns.

Readout errors. High-fidelity readout is essential to enabling accurate computation on quantum computers. Unfortunately, qubit readout is among the most error-prone operations in state-of-the-art superconducting quantum processors. For example, on the 127-qubit IBM Washington quantum computer, readout error rates range from $\sim 0.1\%$ for some qubits to $>10\%$ for other qubits [123]. The primary sources of qubit-readout errors are described below.

Relaxation errors: Most superconducting qubits cannot sustain the excited state ('1') for more than a few tens to hundreds of microseconds. Therefore, relaxations from the excited to the ground state are likely to occur during a long latency operation like readout [206].

Crosstalk errors: In the case of frequency-multiplexed readout, interactions (crosstalk) between the readout pulses can result in errors during classification. Frequency-multiplexed readout dramatically improves the scalability of systems since less hardware is needed for readout, which increases the importance of mitigating crosstalk errors.

Excitation errors: Errors due to undesired qubit excitations are similar to relaxation errors, except that they occur when the readout pulse excites a qubit [266] resulting in an incorrect measurement.

Errors due to environmental noise: Multiple filters and amplifiers are used to increase the signal-to-noise ratio (SNR) of the readout signal [166, 195, 223]. Unfortunately, these components do not perfectly compensate for noise on the readout signal.

3.3 Waveform Memory Compression

Superconducting quantum computers use microwave pulses to perform operations on qubits. A typical quantum program that uses tens of qubits requires hundreds of such pulses. The control hardware sends these pulses through coaxial cables to qubits at cryogenic temperatures, while the control hardware works at room temperature. Today, all quantum computers employ brute-force approaches to scaling the control hardware where discrete, off-the-shelf components like Field Programmable Gate Arrays (FPGAs), Digital to Analog Converters (DACs), and Analog to Digital Converters (ADCs) are replicated depending on

the number of qubits in the target quantum processor. For example, the control hardware of the Sycamore chip [11] required 200+ DACs, 9 ADCs, and 30+ FPGAs for control and readout of 53 qubits. As the number of qubits increases, this brute-force scaling of control hardware will become infeasible in terms of both cost and complexity.

Increasing integration – i.e., decreasing the number of discrete hardware components used for quantum control will make qubit control more scalable. At present, there are two broad approaches for increasing integration: (1) Using off-the-shelf RFSocS (2) Building custom Qubit Control ASICs. RFSocS integrate an FPGA, a processor, DACs, and ADCs on the same chip and are typically used for telecommunication applications. RFSocS can synthesize signals with tens of gigahertz bandwidth. Recently proposed “QICK” and “ICARUS-Q” platforms leverage RFSocS to control qubits [204, 241]. In theory, RFSocS are capable of supporting 100+ qubits per board (with the help of frequency-division multiplexing (FDM)) [241]. Compared to designing ASICs that can integrate a large number of DACs and ADCs on a chip, RFSoc based solutions offer flexibility that is crucial for near term qubit control as quantum hardware is rapidly evolving. As discussed in [241], using FDM with high-bandwidth DACs and ADCs available on these RFSocS can increase the number of qubits controlled by one board, thus reducing the total cost and complexity of the setup. However, controlling more than thousands of qubits would require custom control hardware and new quantum-classical interfaces. We envision the following transition for qubit control as we scale the number of qubits.

FPGA controllers $\rightarrow$ RFSOC controllers $\rightarrow$ ASIC controllers

To scale beyond thousands of qubits, we will need qubit controller that can operate at cryogenic temperatures [14, 69, 88, 127, 249]. For cryogenic control chips, the cooling capacity of the dilution refrigerator results in the “Power Wall” – the average power dissipated by these chips should not exceed the rated cooling power of the dilution refrigerator. To this end, several architectural and design proposals envision scalable qubit control using ultra-low power CMOS and SFQ technologies [14, 69, 88, 127, 181, 248].

With increasing integration, potentially more qubits can be supported per control chip, reducing both complexity and cost. However, the memory bandwidth and capacity required to generate the pulses will scale linearly with the number of qubits as each qubit device requires a unique pulse shape (waveform) to enable high fidelity gates [77, 218, 238]. Furthermore, for performing multi-qubit gates, most architectures use a coupler between

qubits, and for reading the qubit, the readout resonators are used, and even these non-qubit components use device-specific pulses to enable high-fidelity operations. As a result, the number of unique pulses grows super-linearly with the increasing number of qubits. For example, we estimate, on IBM quantum computers, each qubit device requires about 18KB to store single-qubit, two-qubit, and readout pulses. To support more gates, we require additional memory capacity. We estimate that a hundred-qubit quantum computer would require up to 5MB of memory for pulse shapes of basic gates. Moreover, we will require significantly more memory capacity to support complex gates. For example, we estimate that a single Toffoli gate can require >20KB to store a pulse applied on three qubits [135].

Memory bandwidth is the primary bottleneck in scaling the number of qubits. For example, on IBM quantum computers, two DACs per qubit (with a sampling rate of 4.54 Gigasamples/sec each) are required to perform a single-qubit gate. We need a waveform memory to stream samples with more than 16 GB/s bandwidth to such DACs. Furthermore, the peak bandwidth required scales linearly with the number of qubits. To perform concurrent gates on a hundred-qubit machines, we need more than 2 TB/s. Current qubit control setups such as the one used for Google's Sycamore chip solve this problem using distributed waveform memory across 30+ FPGAs. However, distributed control does not scale practically with increasing number of qubits.

We now show how waveform memory bandwidth limits scalability, and how compression can make qubit control more scalable.

3.3.1 Waveform memory bottleneck

We now show the impact of linearly scaling the bandwidth demand of qubit control on FPGA-based systems. We use parameters described in the Table 3.1 to model capacity and bandwidth requirements, which are derived from IBM [9, 123] and Google [11, 103, 182] systems.

We estimate memory capacity (MC) and bandwidth (BW) required for controlling one qubit -

$$MC = \sum_{i=0}^{n_{sq}} f_s \cdot N_s \cdot \tau_i + \sum_{j=0}^{d \cdot n_{tq}} f_s \cdot N_s \cdot \tau_j + f_s \cdot N_s \cdot \tau_{\text{readout}}$$

Table 3.1: Parameters used to estimate memory capacity and bandwidth for controlling one qubit.

Vendor	Sampling Rate(f_s)	Sample Size(N_s)	Gate Set	Gate Latencies(τ)	Connectivity	Memory (per qubit)
IBM	4.54GS/s	32-bits	X, SX, CX	1Q:30ns; 2Q:300ns; Readout:300ns	Heavy hexagonal	18KB
Google	1GS/s	28-bits	fsim, iSWAP, phased XZ	1Q:25ns; 2Q:30ns; Readout:500ns	Grid	3KB

The product of sampling frequency of DAC (f_s), sample size (N_s), and gate latency(τ) is the memory required for storing one gate waveform. We can perform a of total n_{sq} types of single qubit gates and n_{tq} types of two qubit gates. Furthermore, two qubit waveforms are unique for every pair of qubits, and thus for a qubit with d neighboring qubits, we will need $d * n_{tq}$ waveforms. In addition to gates, each qubit will need a readout pulse with latency $\tau_{readout}$. As shown in Table 3.1, IBM uses two single-qubit gates and one type of two-qubit gate, which require a total of 18KB of waveform memory for every qubit device. Note that the sample size s accounts for both I and Q channels. The required memory bandwidth of waveform buffer is determined by the sampling frequency and bit-width of DAC as shown below.

$$BW = f_s \cdot s$$

3.3.1.1 Scaling capacity and bandwidth demand

Capacity scaling. Every qubit device requires a unique waveform, which is determined during the calibration process. It is essential to build control engine that supports the device specific waveforms to enable high gate fidelity. As a result, required memory capacity scales linearly with number of qubits. In addition to storing waveforms for qubits themselves, some architectures require waveforms for couplers that connect qubits to improve fidelity of two qubit (2Q) gates. The capacity required by the coupler waveforms depends on the connectivity, and this must be considered when determining the total capacity. Coupler waveforms are unique for every pair of qubits. As shown in Figure 3.5(a), the required waveform memory capacity scales linearly with the number of qubits in the system (some

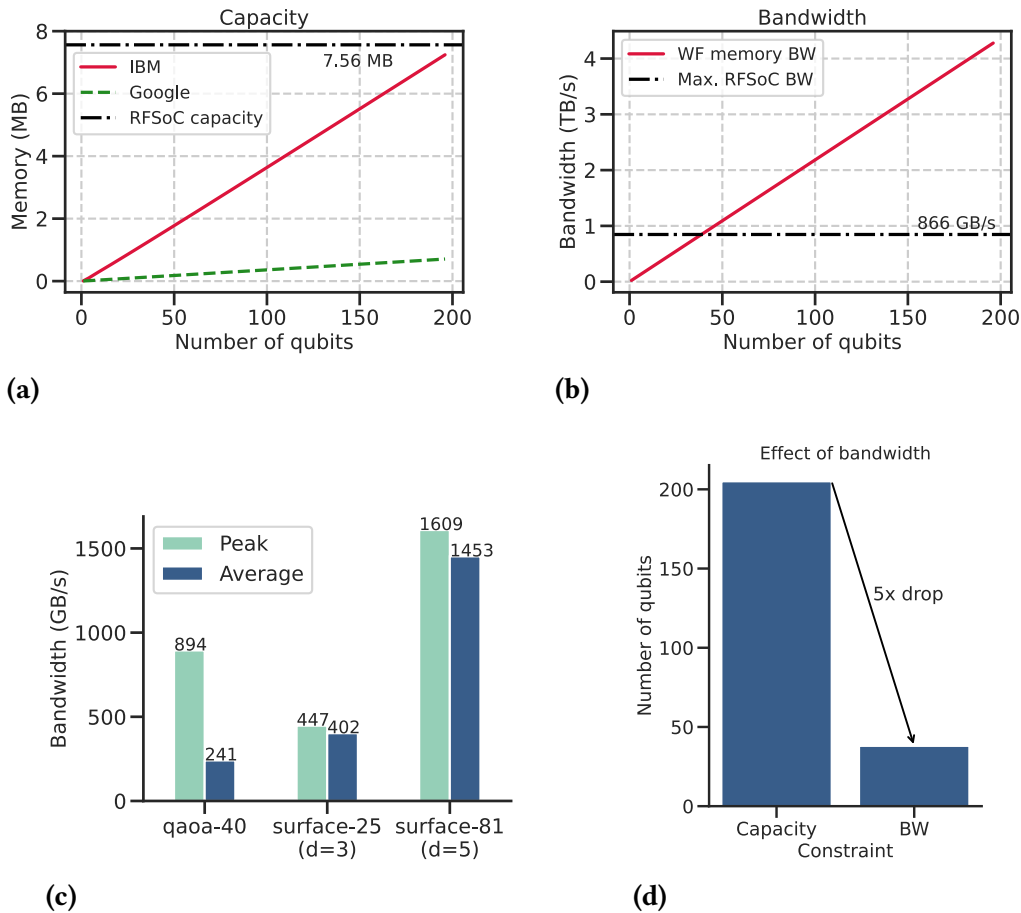


Figure 3.5: Waveform (WF) memory (a) capacity and (b) bandwidth (BW) requirements (c) Peak and average bandwidth for select benchmarks (d) For an RFSoc platform, if only capacity is considered a constraint, the number of qubits that can be supported is >200 . If bandwidth is taken as a constraint, this number drops by 5x to less than 40.

approximations made to account for coupler waveforms). The RFSoc capacity includes both Block-RAM (BRAM) and UltraRAM (URAM) blocks [275] and has been added as a reference for understanding how the capacity requirements scale (both BRAMs and URAMs will be collectively referred as BRAMs from here on).

Bandwidth scaling. Figure 3.5(b) shows the linear scaling of the required memory bandwidth to drive all qubits concurrently with the 6GS/s DACs found on RFSocs. Using such high-bandwidth DACs eliminates the need for external analog mixing, requiring frequent re-calibration. Google/IBM systems will show a similar, albeit lesser bandwidth scaling since

those systems employ brute-force scaling with discrete DACs and FPGAs. The reference RFSoc bandwidth¹ shows that a single RFSoc can concurrently control less than 40 qubits, as FPGA clocks are significantly slower than DAC sampling rates. For example, On the QICK platform, the DAC is 16x slower compared to waveform memory.

Circuit scalability. To understand the impact of waveform memory bandwidth, we estimate the peak and average bandwidth required for representative circuits that users would want to run in the near future. Average bandwidth is determined by how many gates are performed concurrently on an average, whereas peak bandwidth is determined by the maximum concurrency needed for executing the circuit. Shown in Figure 3.5(c), which plots the peak and average bandwidth for three benchmarks – Quantum Approximate Optimization Algorithm (QAOA) with 40 qubits, a distance-3 surface code with 25 qubits, and a distance-5 surface code with 81 qubits. For surface codes, the difference between peak and average bandwidth is small as quantum error correction codes such as the surface code [85] are designed to run concurrent gates as any delay in generating and measuring quantum error syndromes can significantly degrade the protection offered by the error correction code. NISQ applications such as QAOA are not bandwidth intensive on average. However, the last step of all NISQ circuits involves the concurrent measurement of all qubits, requiring the maximum possible waveform memory bandwidth. Unfortunately, serializing measurements is not an option as it can significantly degrade readout fidelity.

3.3.1.2 Meeting capacity and bandwidth demand

The demand for capacity and bandwidth of waveform memory scales linearly with the number of qubits. This presents a challenge for scaling quantum control computers, irrespective of whether it is implemented on an RFSoc or an ASIC. On RFSocs, the internal memory bandwidth and on-chip memory capacity can limit how many qubits we can control. We can boost the memory bandwidth by interleaving (or banking) the memory but due to rigid constraints on Block RAMs, the peak internal bandwidth is limited.

On cryogenic ASICs, power is the primary constraint and thus providing the necessary capacity and bandwidth within a limited power budget is challenging. Furthermore, tech-

¹Determined by assuming a 16x difference in clock speeds of the FPGA and DAC and considering 1260 BRAMs [275].

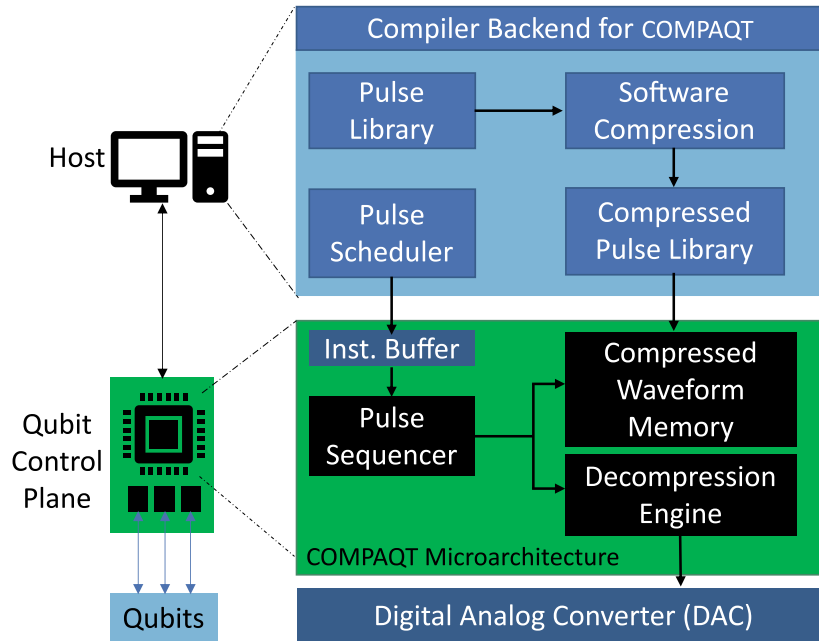


Figure 3.6: Architectural overview of COMPAQT.

niques to improve scalability of qubit control, such as Frequency Division Multiplexing (FDM), cannot be supported if the waveform memory is too small or if it cannot provide the necessary bandwidth. For example, the QICK framework can potentially control 100+ qubits per RFSoc board with FDM. However, before waveforms are mixed onto a single channel for FDM, the waveforms for all the multiplexed qubits must be stored and then individually generated, which means that the waveform memory must have sufficient capacity and bandwidth for all qubits. As shown in Figure 3.5(d), the bandwidth on RFSoc platforms is insufficient to control more than 40 qubits.

3.3.2 Waveform compression

In this section, we first give an overview of COMPAQT, and then discuss different methods that can be used to enable compression and their impact on the gate fidelity.

3.3.2.1 COMPAQT: insight and overview

Insight. We observe that waveforms are highly compressible as they are designed to have a tight bandwidth and low spectral leakage, which means they are smooth and change

slowly in time-domain. We leverage this property to alleviate the high waveform memory bandwidth. We present COMPAQT: Compressed Waveform Memory Architecture for Scalable Qubit Control. By compressing the waveforms, we reduce the required capacity, and by using a low-latency decompression engine, we can quickly expand the compressed waveform to meet the desired memory bandwidth demand.

Design overview. Figure 3.6 shows overview of COMPAQT, where waveforms are compressed in the software and then transferred to the controller waveform memory. The pulse sequencer can then play waveforms by decompressing each sample and then passing it to the DAC. COMPAQT leverages the read-only nature of the Waveform Memory, which is loaded with waveforms at the end of a calibration cycle. To reduce capacity and bandwidth overhead, COMPAQT compresses the waveforms such that only a few samples are needed for representing the entire waveform. Since waveform memory is a ROM, compressing the waveforms has no hardware overhead and can be done in purely software while decompressing the waveforms must be done entirely on hardware. Note that there could be other (and better) compression techniques, the techniques used in this paper are purely to demonstrate the compressibility of waveforms and the advantages compression offers for improving scalability.

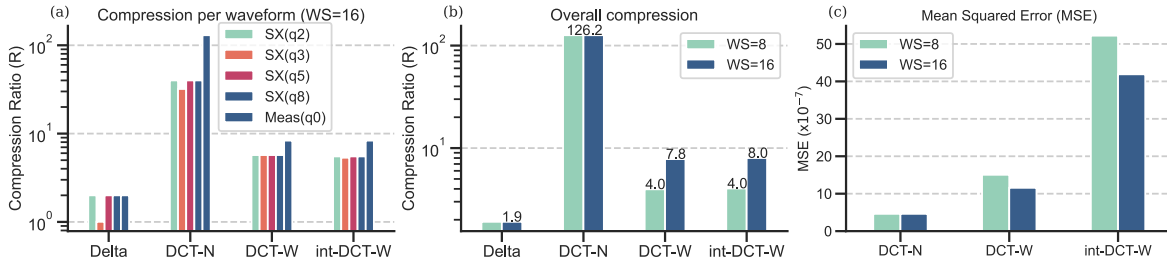


Figure 3.7: (a) Compression Ratio (old size / new size) for five waveforms on IBM Guadalupe (b) Overall Compression Ratio for qft-4 (c) Average Mean Squared Error (MSE) for all waveforms of qft-4.

3.3.2.2 Selecting the compression algorithm

Base-delta and dictionary-based compression are widely studied algorithms to maximize on-chip memory capacity [7, 205]. However, traditional memory compression algorithms are not effective in compressing waveform data. Delta compression reduces the number

of bits by storing only the difference between adjacent samples. However, waveforms can have zero crossings or sign changes. If samples are represented by signed data types (fixed or floating point), the zero crossings will result in a difference that occupies the entire bit-field of the original samples. Furthermore, any non-uniform deltas will add complexity to decompression. Our evaluations on IBM hardware, show that the size of waveforms with no zero-crossings can be reduced by up to 2x as deltas generally occupy half the original bit-width of the sample. However, there is no reduction for waveforms with one or more zero crossings, as shown in Figure 3.7(a). Moreover, in base-delta compression there is a sequential dependence between samples, which limits its utility for improving bandwidth. Similarly, other widely used dictionary-based compression schemes are not effective, as waveform sample values can have arbitrary values, which rarely repeat.

We propose to use **transform-based compression**, which is widely used for audio and video codecs. These compression schemes leverage the Discrete Cosine Transform (DCT) that transforms the input data from the spatial domain to the frequency domain [3], where data can be represented with significantly less memory [214, 247]. Specifically, the DCT of the input signal results in high-energy components in the first few samples after which the magnitude of the transformed samples becomes significantly smaller. The original signal can be recovered by computing the inverse (IDCT) of the transformed sequence.

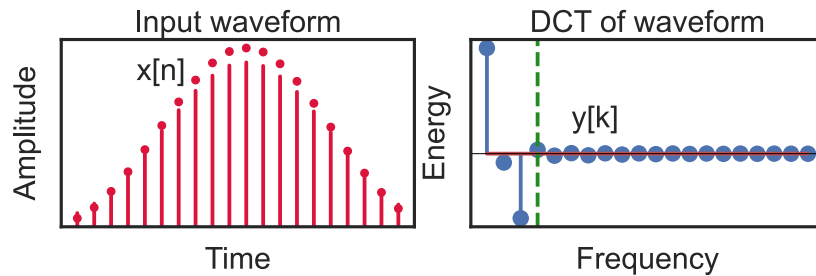


Figure 3.8: Input waveform and its DCT, the vertical green line shows the start of RLE where $|\text{sample}| < \text{threshold}$.

3.3.2.3 DCT-based waveform compression

Discrete cosine transform. The N-point DCT of a signal $x[n]$ and its inverse (IDCT) can be computed using Equations 3.1 and 3.2 respectively.

$$y[k] = \frac{1}{\sqrt{N}} \sum_{n=0}^{N-1} x[n] \cos \left[\frac{\pi(2n+1)k}{2N} \right] \quad (3.1)$$

$$x[k] = \frac{y[0]}{\sqrt{N}} + \sqrt{\frac{2}{N}} \sum_{n=1}^{N-1} y[n] \cos \left[\frac{\pi(2k+1)n}{2N} \right] \quad (3.2)$$

In COMPAQT, decompression is performed on hardware, and since it is part of the critical path during waveform synthesis, it must have low latency and a minimal hardware footprint. If a waveform with N samples is compressed with an N -point DCT, decompressing it would be complex as 1) N can vary for different waveforms, and 2) N can be very large – a two-qubit CR waveform on IBM systems can have 1000+ samples. For this reason, we use a windowed DCT scheme in which the input waveform is broken into windows of a fixed size called a *window size* (WS). The windowed DCT scheme shall hereon be abbreviated as DCT-W while the N -point scheme shall be abbreviated as DCT-N.

The compression algorithm using the DCT is as follows: The DCT of a window of the input waveform is computed (for DCT-N, the window would be the entire waveform) after which a thresholding operation is performed, which sets samples with low magnitudes equal to zero. Finally, Run-Length Encoding (RLE) [28] is performed on the transformed window. RLE replaces all the zeros with a single codeword that contains two fields: 1) a signature identifying it as the RLE codeword and 2) the number of zeros that have been encoded to yield the compressed waveform.

Table 3.2: Implemented DCT variants.

Variant	Description	Hardware Complexity
DCT-N	N-point DCT, WS = length of waveform	High
DCT-W	Windowed DCT, WS = 8, 16	Moderate
int-DCT-W	Windowed integer-DCT, WS = 8, 16	Low

The compression using DCT-N has been visualized in Figure 3.8 for single-qubit gates. We use typical Derivative Removal by Adiabatic Gate (DRAG) waveforms used in most superconducting systems to implement single qubit gates. RLE is started only when the transformed waveform after thresholding is consistently zero. This process is done separately for I and Q channels, and to simplify the decompression, the number of samples per window after compression are kept the same for both channels.

Integer-DCT. The DCT/IDCT in its original form requires a minimum number of fixed/floating-point multipliers that make hardware implementations very resource intensive. The 8-point DCT/IDCT (WS=8) requires a minimum of 11 multipliers [162]. Larger transforms need resources, with the 16-point DCT/IDCT requiring a minimum of 26 multipliers.

To make waveform compression using DCT scalable for quantum control applications, we also implemented an integer DCT consistent with the HEVC standard [247]. Since this variant of the DCT was aimed at making hardware implementations as efficient as possible, we do not use the arbitrary N-point transform. The windowed integer DCT will henceforth be abbreviated as `int-DCT-W`. Since the DCT coefficients are scaled by a constant factor, the input waveform is scaled by the same factor in software (this can be done in hardware too). The constant scaling factor S for an N -point DCT is given by $S = 2^{6 + \frac{\log_2 N}{2}}$. All DCT variants are summarized in Table 3.2. We used two different window sizes to evaluate the compression of waveforms using DCT.

Fidelity-Aware Thresholding. Each gate pulse is unique and a uniform threshold during compression can cause fidelity loss for some qubits. To enable high fidelity gates, we find optimal threshold for every gate pulse. We use the Algorithm 1, to tune the threshold to obtain the target gate fidelity. We observe that MSE between decompressed and uncompressed pulses are highly co-related to the gate fidelity, and we can use MSE for tuning the threshold at compile time. We can take a step further and integrate the Fidelity-Aware compression within the gate calibration loop.

3.3.2.4 Efficacy of DCT compression

The DCT-based compression scheme is lossy and its effectiveness depends on two factors: 1) Compressibility of the waveforms. 2) Fidelity of decompressed waveforms. To evaluate compressibility, we measure the reduction in memory size to store all waveforms used for a benchmark circuit. For evaluating the effect on fidelity, we ran two-qubit Randomized Benchmarking [167] on various IBM machines.

Compressibility. Figure 3.7(a) shows the reduction ($R = \text{old size}/\text{new size}$) in required memory for five representative waveforms of `qft-4` benchmark derived from [147] implemented on the IBM Guadalupe machine. Figure 3.7(b) shows the overall reduction in memory for `qft-4`. Note that measurement and 2Q gates are longer and more compressible than 1Q gates which affects the total compression ratio for `qft-4`. A window size of 8 has

Algorithm 1: Fidelity-Aware Compression

Input: W_{in} (Gate Pulse), ε (Target Error)

Output: W_{out} (Compressed Pulse)

```

1 Function Get_Compressed_Pulse( $W_{in}$ ,  $\varepsilon$ ):
2   while  $mse \leq \varepsilon$  do
3      $Y = \text{int-DCT}(W_{in}, W_{out})$  // Compute DCT
4     for  $s$  in  $Y$  do
5       if  $s < \text{threshold}$  then  $s = 0$ 
6     end
7      $W_{out} = \text{int-iDCT}(Y)$ 
8      $mse = \text{get\_MSE}(W_{in}, W_{out})$ 
9      $\text{threshold} = \text{threshold}/2$ 
10    if  $\text{threshold} < 10^{-6}$  then return -1
11    // No solution found
12  end
13 return  $W_{out}$ 

```

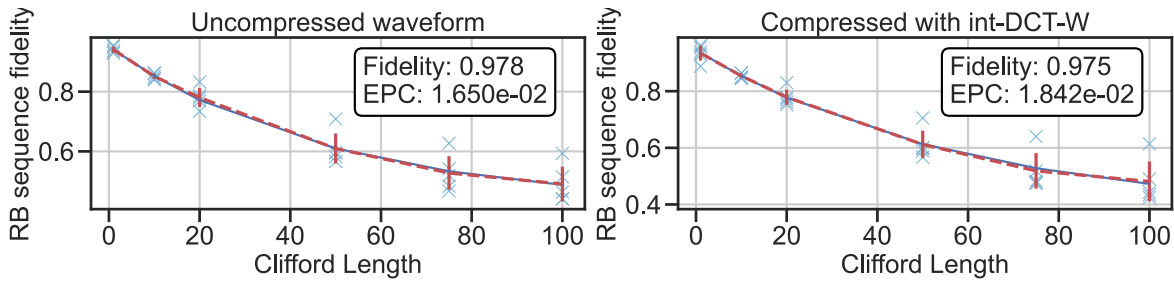


Figure 3.9: Randomized Benchmarking of two qubit system with the uncompressed baseline gate pulses and compressed gate pulses with int-DCT-W on IBM Guadalupe.

the least reduction because RLE is limited to a maximum of 8 samples at a time. Increasing the window size makes the IDCT design complex. Thus, more compression can be achieved at the cost of hardware complexity. Delta compression shows an overall reduction in size for the qft-4 circuit but not all waveforms are compressible due to the overhead of supporting zero crossing as shown in Figure 3.7(a).

Impact on gate fidelity. Distortion in the waveforms can reduce gate fidelity. To understand whether compression distorts the waveforms, we compute Mean Squared error (MSE) between the original waveforms and the decompressed waveforms. Figure 3.7(c) shows the average MSE over all waveforms used for the qft_n4 benchmark for three variants of

DCT-based compression. The MSE lies between 10^{-5} and 10^{-7} , and it is highest (but still negligible) for the int-DCT-W variant since it incorporates integer approximations before the thresholding step in the algorithm.

Moreover, to test the gate fidelity, first we compress and then decompress waveforms by emulating the decompression engine design, and then we use these decompressed waveforms to drive qubits on IBM quantum computers. Figure 3.9 shows the RB fidelity with increasing number of Clifford gates and average Error Per Clifford (EPC) which averages the total error over the number of gates. Table 3.3 summarizes the fidelity of two-qubit Randomized Benchmarking (RB) on three IBM machines. The results show that even though DCT based compression is lossy, the decompressed waveform is similar to the original, because of which there is minimal effect on the fidelity of the gates.

Table 3.3: RB fidelity for uncompressed baseline pulse and compressed pulses with window size of 16.

Design	2Q RB Fidelity IBM Bogota	2Q RB Fidelity IBM Guadalupe	2Q RB Fidelity IBM Hanoi
Baseline	0.980	0.978	0.987
DCT-N	0.982	0.977	0.989
DCT-W	0.983	0.976	0.986
int-DCT-W	0.983	0.975	0.988

3.3.3 COMPAQT microarchitecture

With COMPAQT, waveforms are compressed offline in software after calibration, and then stored on the control hardware. Thus, microarchitecturally, only the decompression pipeline will live in the control system for real-time operation.

3.3.3.1 Decompression pipeline

Decompression is a two-step process, as shown in Figure 3.10. In ❶, the codeword generated by RLE is decoded to yield all the encoded zeros and then in ❷, the IDCT of the resultant vector produces the decompressed waveform. To enable hardware efficient design, we use the windowed IDCT algorithm in COMPAQT. If compression was done using a window size of sixteen, the samples generated by the RLE decoder and IDCT would scale accordingly.

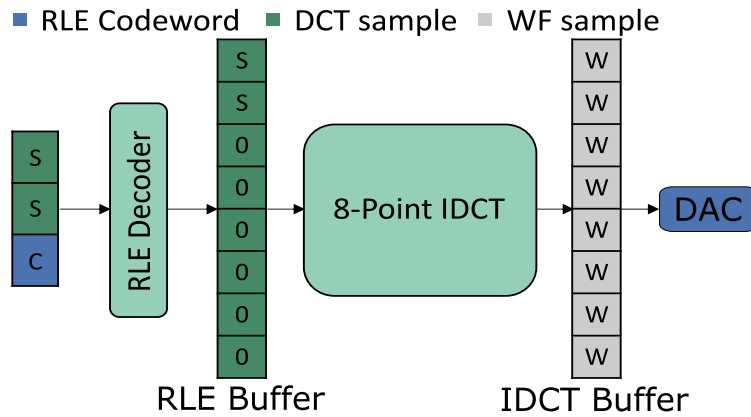


Figure 3.10: Decompression pipeline for waveforms compressed with DCT-W/int-DCT-W (WS=8).

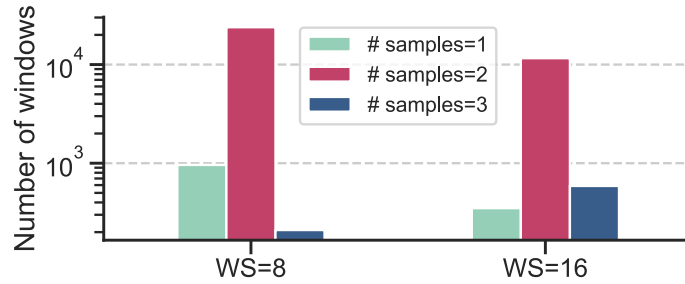


Figure 3.11: Histogram of the number of samples per window in the compressed waveform (including the RLE codeword) for 132 waveforms from IBM Guadalupe.

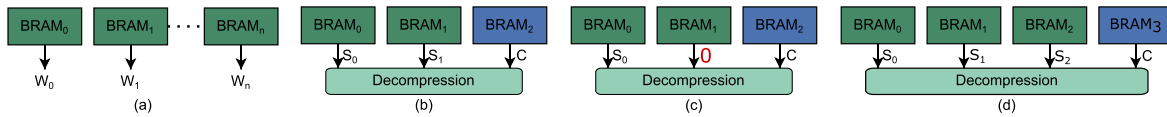


Figure 3.12: (a) Interleaved waveform samples in different BRAMs to boost bandwidth (b) Reduction in number of BRAMs for a waveform with maximum of three samples per window (including RLE code word) (c) Compressed window with two non-zero samples interleaved in three BRAMs (d) Interleaved memory organization for a pulse that requires four samples for a worst-case window.

Compression reduces the waveform capacity, but, more importantly, by storing a small number of samples and expanding them in the RLE buffer, DCT based compression enables higher bandwidth. As shown in Figure 3.10, the three compressed waveform samples yield eight waveform samples after decompression, thereby boosting the waveform memory bandwidth. In our design, for simplicity and efficient hardware synthesis, we design compressed

waveform memory with uniform width, which is determined by the size of the worst-case compressed window. For example, the width of the compressed DCT samples (green+blue squares) in Figure 3.10 is three samples, including the RLE codeword. Figure 3.11 shows the histogram of the number of samples in every window for `int-DCT-W` showing that waveforms are highly compressible. Based on an empirical analysis, the worst-case number of samples the design must cater to is thus three as shown by the histogram. The design of the decompression pipeline will be complex if we enable each window to have a variable number of samples. In `COMPACT`, we choose a uniform input buffer size which sacrifices compressibility to enable a significant performance boost.

3.3.3.2 Decompression engine design

In the DCT decompression pipeline, the RLE decoder depends on the RLE codeword to decompress all zeros and feed them to the IDCT stage. The signature in the RLE codeword can be used to identify the codeword, and if the RLE codeword specifies that c_n zeros have been encoded, the last c_n inputs of the IDCT stage can be set to 0.

A major advantage of using `int-DCT-W` over `DCT-W` is that the multipliers used for computing the inverse transform can be replaced with shift-and-add operations, drastically reducing hardware complexity and improving performance. We encourage readers to peruse existing designs for `DCT-W` [33, 78, 224] and `int-DCT-W` [44, 232, 283] (IDCT circuits are simply the reverse of DCT circuits).

Table 3.4: Hardware resources needed for the IDCT engine of `DCT-W` and `int-DCT-W`.

Variant	Window size(WS)	Multipliers	Adders	Shifters
DCT-W	8	11	29	0
int-DCT-W		0	50	26
DCT-W	16	26	81	0
int-DCT-W		0	186	128

Table 3.4 summarizes the differences in requirements for implementing the IDCT in hardware for both `DCT-W` and `int-DCT-W` with a window size of eight. The `DCT-W` design is based on Loeffler’s algorithm and is the least expensive implementation [162]. For `int-DCT-W`, the

multiplications are converted to shift-and-add operations [232]. This optimization enables the IDCT engine to have a constant latency of one clock cycle.

3.3.3.3 Banked compressed waveform memory

Typically, the RFSoc FPGA clock is slower (≈ 0.3 GHz) than the DAC sampling frequency (≈ 6 GHz). As a result, even in the baseline uncompressed systems, feeding data to the DACs at the peak sampling rate can become challenging. To solve this problem, waveform samples for a single waveform are interleaved in multiple BRAMs. By banking waveform memory, we can compensate for the difference in frequencies between the FPGA and the DAC as shown in Figure 3.12(a). However, this approach is not scalable and eventually hits a BRAM bandwidth wall due to the finite number of BRAMs available. In COMPAQT, we use memory interleaving with compressed waveforms, as fewer BRAMs are needed for a waveform with the decompression pipeline.

Figure 3.11 showed that regardless of the window size, the number of samples in the compressed waveform is ≤ 3 for int-DCT-W. Figure 3.12(b) is an example showing how if the waveform samples were originally interleaved in multiple BRAMs, the number of BRAMs needed after compression with WS=16 would be just three (if WS=8 was used, the number of BRAMs would have been six, since two 8-point IDCT modules would be needed). The decompression block in the figure is the pipeline shown in Figure 3.10 and the results after decompression can be stored on distributed memory in the RFSoc until they are used. Figure 3.12(c) shows how the input to the decompression engine is zero if the number of samples in the window is less than the maximum number and Figure 3.12(d) show a case where the maximum number of samples per window is four.

Table 3.5: Number of qubits that can be supported by an FPGA based design with uncompressed waveform memory and a design with compressed memory using int-DCT-W for different window sizes.

	Uncompressed	WS=8	WS=16
Number of qubits (normalized)	1	2.66	5.33

We reduce the number of concurrent reads from BRAMs required to control one qubit. As a result, RFSocs can drive significantly more qubits. Table 3.5 shows how different

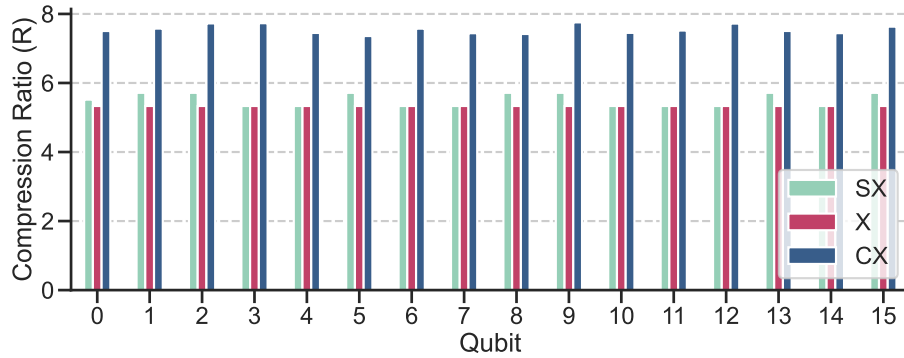


Figure 3.13: Compression ratios of the basis gates for all 16 qubits of IBM Guadalupe using int-DCT-W with WS=16.

window sizes increase the number of qubits that can be controlled by an RFSoc based controller. This gain is independent of the ratio of FPGA clock frequency and DAC sampling rate as long as the clock frequency ratio is a multiple of the window size. If the ratio is not a multiple of the window, then the gain will be slightly lower. As an example, consider the case where the ratio is 6x. For such a system, uncompressed waveforms would require interleaving of samples between six BRAMs per waveform. If compressed waveforms with WS=8 were used, the number of BRAMs needed per waveform would be three (assuming the same distribution as shown in Figure 3.11). The gain in the number of qubits for such a system would be 2x, which is slightly less than the case where the ratio is 8x. The ratio between the DAC and FPGA was 16x in QICK due to which it can theoretically support about 36 qubits. Using COMPAQT with WS=8, number of qubits can be increased to about 95 qubits, and for WS=16, we can drive 191 qubits concurrently. Although the availability of DACs and the limit of Frequency Division Multiplexing (FDM) may result in lesser qubits, the theoretical bounds of the control computer will no longer bottleneck the scalability of the FPGA platforms.

3.3.4 Evaluating compressed waveform memories

3.3.4.1 Gate compressibility

We evaluated the compressibility of the gate pulses using COMPAQT for five IBM quantum computers. Table 3.6 reports the minimum, average, and maximum compressibility of a

control pulse used on IBM hardware. Despite the diversity in the pulse shapes, we observe that all control pulses are highly compressible. For example, as shown in Figure 3.13, on average, we can compress the control pulses by more than 5x for each qubit device on the IBM-Guadalupe. Note that in Figure 3.13, the compression ratios for CNOT gates are averaged over all CNOT gates that can be performed using a qubit device, whereas Table 3.6 reports the minimum and maximum compression ratio for an individual gate pulse on the IBM hardware. We observe that on all platforms the SX-gate has the lowest compression ratio of 5.33 with our design.

Table 3.6: Minimum, maximum, and average compression ratios with int-DCT-W (WS=16).

Machine	Compression Ratio (R)		
	Min.	Max.	Avg.
IBM Toronto	5.33	8.11	6.49
IBM Montreal	5.33	8.02	6.45
IBM Mumbai	5.33	8.05	6.47
IBM Guadalupe	5.33	8.02	6.48
IBM Lima	5.33	7.92	6.33

3.3.4.2 Gate fidelity with COMPAQT

To evaluate whether compressed waveforms degrade the fidelity of quantum circuits, we run circuits for 80K shots with and without waveform compression and compute the normalized fidelity by taking the ratio of the fidelities of COMPAQT and the uncompressed baseline.

Figure 3.14 shows the normalized benchmark fidelities for WS=8 and WS=16 with int-DCT-W on IBM Guadalupe. Norm. fidelity close to one means both the baseline and COMPAQT have identical fidelities, indicating no degradation due to lossy compression. For window size of WS=16 we see no fidelity degradation but for the WS=8, there are significant fidelity losses for some benchmarks due to distortions introduced at the boundaries of consecutive windows. These distortions can be reduced by using overlapping windows to compress the waveform. Note that in some cases normalized fidelity is slightly greater than one, indicating COMPAQT is more reliable. This can be attributed to the variability in the experiments. Note that the fidelity for the QAOA benchmarks was determined by

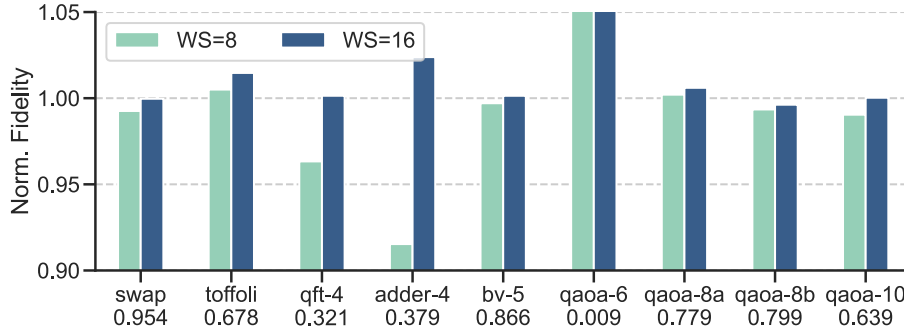


Figure 3.14: COMPACT fidelity normalized to baseline fidelity on IBM Guadalupe. The baseline fidelities are annotated beneath the benchmark names.

computing the normalized fidelity [109, 163]. We observe that the fidelity degradation is less than 0.5% for the largest QAOA benchmark using 10 qubits with a window size of 16.

3.3.4.3 Scalability of COMPACT with RFSOCs

To evaluate the scalability of COMPACT on RFSOC platforms, we evaluate the maximum clock frequency, resource utilization and maximum number of concurrent gates that can be executed for key benchmark circuits.

Clock frequency. We use QICK [241] as the baseline design with a maximum frequency of 294 MHz. For COMPACT, the maximum achievable frequency drops by more than 33% to 195 MHz for the pipelined DCT-W engine, as shown in Figure 3.15. This degradation results from complex design and long critical path due to multipliers. In comparison, the unpipelined int-DCT-W design introduced a worst-case degradation of 10%, which can be pipelined to enable a design with no clock frequency degradation. These results can be scaled to arbitrary number of qubits as to control multiple qubits the individual qubit control block will be instantiated in parallel having a little to no effect on the critical path.

Resource utilization. Table 3.7 shows the usage of critical FPGA resources like Look-Up Tables (LUTs) and Flip-Flops (FFs) of the baseline design (QICK) and a single IDCT engine for int-DCT-W with different window sizes. The baseline design can control a single qubit, but it includes components like the AXI interface that will not scale linearly with the number of qubits. The resources are not enough to accommodate 100+ qubits. The overall

Table 3.7: FPGA resource usage for the baseline and a single IDCT engine for int-DCT-W for different window sizes. The percentages represent the percent utilization of the total resources. The designs were synthesized for the same SoC - Xilinx zc7u7ev.

Design	LUT Utilization	FF Utilization
Baseline	3386 (1.4%)	6448 (1.4%)
int-DCT-W (WS=8)	601 (0.26%)	266 (0.05%)
int-DCT-W (WS=16)	1954 (0.85%)	671 (0.15%)
int-DCT-W (WS=32)	9063 (3.93%)	1197 (0.26%)

resource usage per module will be lower than 1% for RFSocCs, as they have significantly more resources than the device used for this evaluation (the RFSocC used by QICK has 400K+ LUTs and 800K+ FFs). Compared to BRAMs, LUTs and FFs are more readily available on FPGA and so using a compressed memory architecture trades memory units (BRAMs) with system logic (compute) per qubit in FPGAs. The design of the IDCT hardware used for these evaluations can be further optimized by pipelining.

Although increasing window size can improve compression efficiency, using a larger window size such as WS=32 with int-DCT-W uses significantly more resources than WS=16, which makes it a sub-optimal design as a single iDCT engine uses more than 4% of the total LUTs. In summary, int-DCT with WS=32 reduces clock frequency and adds complexity, limiting the scalability.

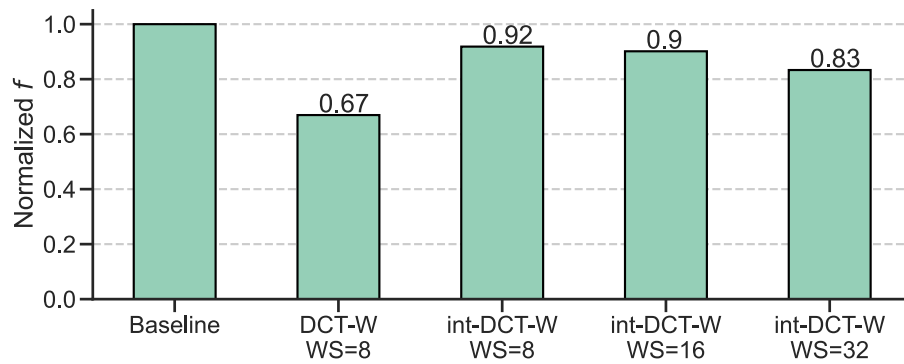


Figure 3.15: Degradation of clock frequency compared to baseline for different decompressed waveform memory architectures.

3.3.5 Scalability of COMPAQT with ASICs

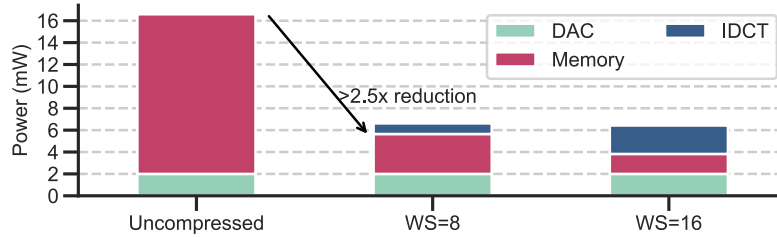


Figure 3.16: Power dissipated by the cryogenic controller with the uncompressed and compressed memory.

The intrinsic BRAM bandwidth on FPGA-fabric limits their scalability. Compressing waveform memory can help improve such systems. However, unlike BRAMs, SRAMs used in ASICs can operate at much higher frequencies and deliver significantly more bandwidth. However, for ASICs designed to work at cryogenic temperatures, it is crucial for the overall architecture to be as power efficient as possible due to tiny power budget at 4K. While the cooling capacity of dilution refrigerators will probably have to be expanded in the future to accommodate more qubits, the power budget will still be limited and minimizing the power dissipated by digital components like waveform memory can help optimize other components of qubit control that are critical for high-fidelity gates. For example, 50% of the total power (23mW) in the cryogenic chip presented in [88] was dissipated by digital components (including the digital frontend of the DACs).

3.3.5.1 Scalability of QEC experiments

Demonstrating a small set of logical qubits that run quantum error correction is a crucial milestone in achieving fault-tolerant quantum computation. Surface code is a leading candidate for implementing QEC [85] and its effectiveness have been demonstrated experimentally for low-distance codes [4, 144, 170, 281]. As systems scale and add more qubits to move towards fault-tolerant systems, the number of logical qubits that a single controller can support will thus be an important metric.

Figure 3.17(a) shows the maximum number of concurrent operations at any given time during the syndrome generation cycle in two variants of a distance three surface code patch. More than 80% of the physical qubits to create one logical qubit are driven concurrently,

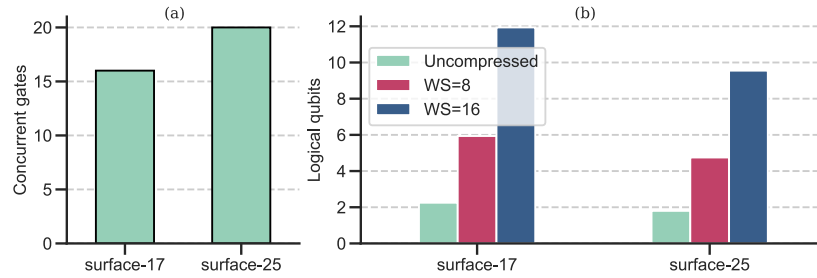


Figure 3.17: Increasing the number of logical qubits that can be supported; (a) The peak concurrent operations for a $d=3$ code; (b) The maximum logical qubits that can be supported by different Xilinx RFSoc based designs.

which makes it important for a controller hardware to be able to support concurrency in as many qubits as possible. Figure 3.17(b) shows that COMPAQT can control 5x more logical qubits compared to an uncompressed baseline.

3.3.6 Conclusions

As qubits scale beyond the 100-qubit mark, scaling the control hardware with brute-force methods will be infeasible. Integration using RFSocs and cryogenic ASICs will be required to enable scale qubit control hardware. However, in this paper, we show that the internal memory bandwidth used for playing waveforms in RFSoc-based systems becomes a bottleneck. Furthermore, for controller ASICs, the power dissipated by the waveform memory is a significant portion of the total power and optimizing the memory architecture will enable better scalability in such systems. To solve these problems with waveform memory in different types of controllers, we present COMPAQT, a compressed waveform memory architecture that compresses waveforms in software before they are loaded on the controller. In hardware, waveforms are decompressed just before they are needed. Our results show that for RFSoc-based systems, the number of qubits that can be supported increases by more than 5x compared to the baseline. For CMOS ASICs, the reduced number of accesses required per waveform reduces power dissipation by more than 2.5x. Compressed waveform memory has minimal effect on the gate fidelity, as shown on real quantum hardware, while enabling more scalable controllers.

3.4 Hardware-Efficient Machine Learning-based Qubit Readout

Due to the complexity of the readout process, which involves multiple analog components like filters and amplifiers, reducing readout errors cannot be achieved by merely optimizing a single part in the pipeline. Improvements to readout hardware, such as Purcell filters and Josephson Traveling Wave Parametric Amplifiers [110, 166, 223] have improved qubit-readout accuracy. Such hardware enhancements are beyond the scope of this paper. In contrast, statistical methods that mitigate readout errors on a distribution of readout results [31] have been proposed to help applications of the Noisy Intermediate Scale Quantum (NISQ) era, where the same program is run thousands of times to generate a probability distribution to reveal the most likely result. However, such statistical methods do not improve the readout accuracy for single-shot readout, i.e., the accuracy of a single measurement. In this work, we focus on *improving single-shot qubit-state discrimination*.

Apart from improving the analog components of the readout hardware, single-shot qubit-readout accuracy can be improved by designing better discriminators. The state-of-the-art deep neural network-assisted discriminator demonstrated by Lienhard et al. [149] achieved a significantly higher readout accuracy for a group of frequency-multiplexed qubits compared to other discriminators, such as matched filters and support vector machines. This was achieved by using the digitized readout signal at the intermediate frequencies as an input to a large Feed Forward Neural Network (FNN) and training it to classify the state of all qubits. Using the readout signal directly from the analog-to-digital converter without any digital preprocessing, the FNN effectively utilizes all available information during each measurement to achieve better qubit-readout accuracy. However, such a discriminator architecture introduces a substantial overhead – the size of the neural network prevents it from being implemented in a scalable manner on programmable hardware like FPGAs, which are used as control hardware for most quantum computers based on superconducting qubits today. Thus, the discriminator must be implemented in software, and the readout signal has to be transferred from the control hardware to software every time a qubit measurement is performed.

Most discriminators implemented on hardware are relatively simple and involve some form of filtering to reduce the noisiness or dimensionality of the readout data received from the qubits. Filters such as (mode) matched filters [11, 220, 266] are commonly used to

reduce the digitized readout signal received from the ADC into a single value that can be used for discrimination. On the other hand, the deep neural network discriminator [149] demonstrated that there is a plethora of information in the unfiltered readout signal that can be used for more accurate discrimination. However, using a software discriminator introduces a significant latency overhead to the readout operation due to the expense of transferring the high-dimensional readout traces to software.

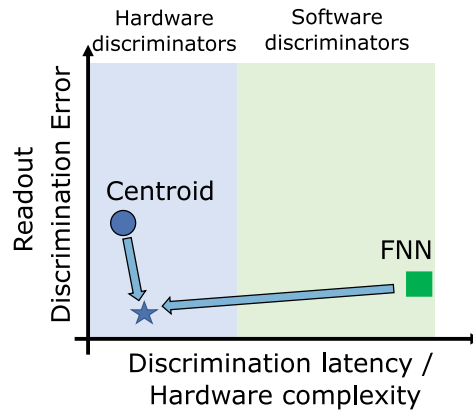


Figure 3.18: Goal: Fast hardware readout discriminator.

Figure 3.18 summarizes the problem – hardware discriminators are fast but inaccurate, while highly accurate ML-based discriminators are slow and computationally expensive. We seek to find a hardware discriminator that is scalable, and achieves accuracies at par or better than ML-based discriminators.

3.4.1 Scaling high-fidelity readout

3.4.1.1 Single-shot qubit readout fidelity

Efforts to improve qubit-readout fidelities can be divided into two broad categories: (a) statistical error mitigation over an ensemble of measurement results, and (b) improving the readout accuracy per measurement (single-shot). Statistical error mitigation uses a trained error profile to reduce errors in the probability distribution of the measurement (obtained after running the same experiment for many shots). Measurement error mitigation [31, 124] is an example of such an approach, along with an alternative where neural networks are used to learn the error profile [134].

Methods to improve single-shot readout accuracy are beneficial for both near-term and long-term applications. Active error mitigation methods that use ancillary qubits for mitigating error via redundancy [10, 90, 111] and methods that optimize the readout pipeline, such as deep learning assisted readout [149] are ways to improve the single-shot qubit readout accuracy. Since any improvements to the readout pipeline that result in better readout fidelity automatically benefit active readout error mitigation, we limit our discussion to techniques such as the Feed Forward Neural Network (FNN) discriminator presented by Lienhard et al. [149] that optimize the readout pipeline.

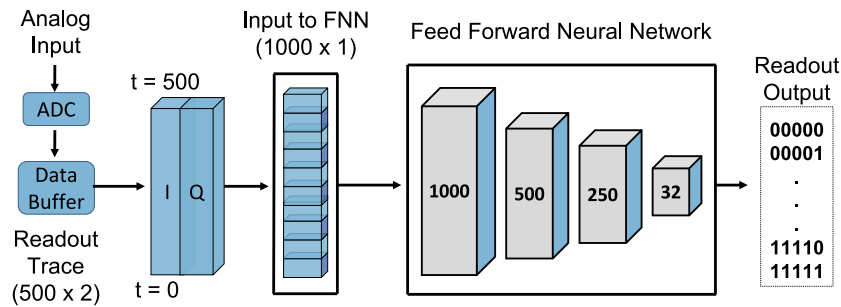


Figure 3.19: Overview of the baseline [149] design.

3.4.1.2 Baseline design

An overview of the baseline FNN (1000-500-250-32) design proposed by Lienhard et al. [149] is illustrated in Figure 3.19. The intermediate-frequency readout signal is digitized and buffered before being sent to software classifier. Every readout trace has 500 elements for both I and Q channels, with each element corresponding to a 2ns interval (total readout duration is 1μs). The 1000 elements corresponding to a single trace are then used as inputs to an FNN. The baseline design does not demodulate the readout signal. It uses all samples from the ADC to minimize undersampling, and thus the input layer consists of 1000 neurons. The FNN has 32 outputs corresponding to a five-qubit system's 2^5 basis states.

3.4.2 Enabling high-fidelity readout

We present *High Fidelity Hardware-efficient Qubit Readout using Low-dimensional Traces* (HERQULES)², a holistic architecture that focuses on improving the fidelity, reducing the latency, and enhancing the scalability of superconducting qubit readout³.

3.4.2.1 Design overview

The baseline design [149] uses a large Feed Forward Neural Network (FNN) trained on the readout traces at intermediate frequencies from the ADC to discriminate qubit state. This architecture learns features corresponding to crosstalk, qubit decay, and other non-idealities to achieve high-fidelity qubit readout. Unfortunately, large FNNs require significant computational and memory resources, forcing designers to run them in software, as shown in Figure 3.20(a).

To enable efficient design, we present HERQULES that reduces the dimensionality of the readout trace using matched filters (MF) and feeds this data to a significantly smaller, hardware-efficient FNN to infer the qubit states as shown in Figure 3.20(b). A MF reduces the dimensionality of the demodulated time traces to a single dimension with two classes maximally separated. The reduction in the dimension of the input data allows for a smaller FNN, as shown in Figure 3.20(b). Furthermore, HERQULES introduces additional classifiers into the readout pipeline to detect qubit relaxation. An additional MF-based discriminator is tasked to catch qubit relaxation (relaxation matched filter (RMF)) as shown in Figure 3.20(c). HERQULES shows that by feeding the FNN with other specialized classifiers, such as MFs, we can compensate for non-idealities occurring in frequency-multiplexed qubit readout architectures.

In the rest of this section, we further describe our design to explain the training (done in software) and the inference (in hardware) processes for HERQULES.

²Available online: <https://github.com/satvikmaurya/HERQULES>

³This work [176] was done in collaboration with Chaithanya Naik Mude, Benjamin Lienhard, William D. Oliver, and Swamit Tannu. Chaithanya Naik Mude helped with setting up the baseline design using PyTorch, and Benjamin Lienhard provided the readout datasets used in this work.

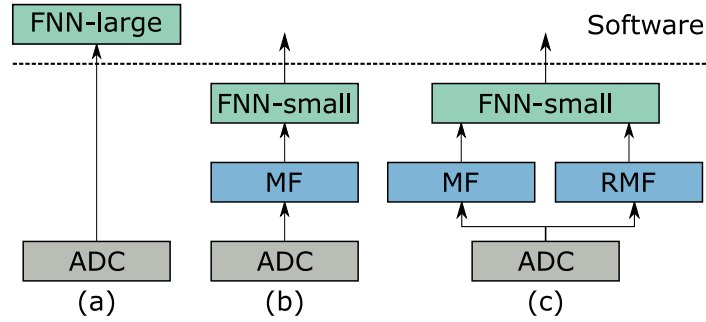


Figure 3.20: Overview of pipeline designs; (a) Baseline – Feed forward Neural network (FNN) on ADC data, without dimensionality reduction; (b) MF-NN: dimensionality reduction with matched filter (MF), then as input to FNN (smaller network than the baseline); (c) MF-RMF-NN: Additional feature with relaxation matched filter (RMF) over and above MF-NN.

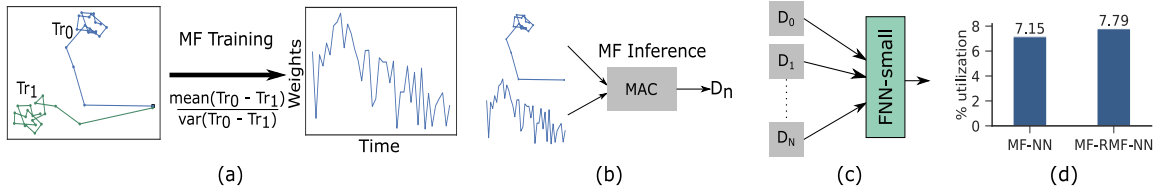


Figure 3.21: (a) Matched Filter (MF) training: Ground state traces (Tr_0) and excited state traces (Tr_1) are processed to get MF envelopes; (b) The dot product of the traces and MF envelopes is computed using multiply-and-accumulate (MAC) units; (c) The computed dot product is input for a Feed Forward Neural Network (FNN); (d) Comparison of LUT utilization between MF-NN and MF-RMF-NN (remaining component utilization is relatively low).

3.4.2.2 Dimensionality reduction using matched filters (MF)

The FNN discriminator used every time-bin element of the readout trace received from the qubits as an input feature, resulting in an input layer of 1000 neurons for a $1\mu s$ readout trace [149]. However, using such a neural network, non-idealities, such as the effects of crosstalk, can be mitigated to improve the combined readout accuracy for a frequency-multiplexed readout scheme. To retain the advantages offered by neural networks in mitigating readout crosstalk while reducing the amount of data used to discriminate the state of a qubit, we use a combination of a kernel for dimensionality reduction and a significantly smaller neural network for mitigating non-idealities. We call this design MF-NN.

MFs maximize the signal-to-noise ratio (SNR) for signals with linearly added Gaussian

noise and in the absence of state transitions and have been shown to be optimal for single qubit readout [220] (see Appendix A). Many experimental and commercial superconducting quantum computing systems utilize MFs for qubit-state discriminators [32, 48, 255]. As described below, MFs are straightforward in training and inexpensive to implement in hardware, making them ideal for dimensionality reduction with preserving features that maximize the separation between two classes. Note that a Matched Filter (also referred to as Linear Discriminant Analysis [164, 259]) is a lightweight supervised learning method that requires labeled data.

MF training. The readout signal is 2D (I and Q values) and varies with time. The MF envelope is derived by computing the ratio of the mean and variance of the difference between the two classes, visualized in Figure 3.21(a). A separate MF envelope is trained for both I and Q components of the readout signal but for brevity, we refer to both envelopes in the singular. Since this envelope is unique for every qubit, the number of envelopes that need to be trained for readout scales linearly with the number of qubits. MF's training overhead is significantly lower than the baseline FNN, making HERQULES agile and efficient.

MF inference. The MF output is essentially a dot product between the trained envelope env and the demodulated trace Tr being discriminated. For both I and Q components, the result of the matched filter is thus $\sum_{j \in I, Q} \sum_{t=0}^T env_j(t) \cdot Tr(t)$, where T is the readout duration. Since the envelopes of both I and Q components are summed together, the final result is a single scalar value. The MF operation can be implemented in hardware using inexpensive multiply-and-accumulate (MAC) units, as shown in Figure 3.21(b).

3.4.2.3 Feed-forward neural network architecture

Matched filtering allows a readout time trace to be reduced to a single value. Typically, this value is utilized to discriminate between two states through thresholding. Here, however, we use the filtered values as inputs to a Feed Forward Neural Network (FNN). In general, for the readout of a frequency-multiplexed group of N qubits, N MF outputs form the inputs to a neural network with N neurons in the input layer, as shown in Figure 3.21(c). Since the size of a group of multiplexed qubits is relatively small (5 to 10 qubits), this drastically reduces the size of the neural network and simplifies the design.

Table 3.8: Qubit-readout accuracy of baseline [149], different designs, and cumulative accuracy $F_{5Q} = \sqrt[5]{F_1 F_2 F_3 F_4 F_5}$.

Design	Qubit 1	Qubit 2	Qubit 3	Qubit 4	Qubit 5	F_{5Q}	F_{4Q}
Baseline	0.969	0.753	0.943	0.946	0.97	0.912	0.957
mf	0.968	0.734	0.891	0.934	0.956	0.892	0.937
MF-SVM	0.968	0.738	0.895	0.928	0.953	0.892	0.936
MF-NN	0.969	0.740	0.901	0.936	0.957	0.896	0.940
MF-RMF-SVM	0.981	0.752	0.959	0.957	0.986	0.923	0.970
MF-RMF-NN	0.985	0.754	0.966	0.962	0.989	0.927	0.975

FNN training. The FNN can be trained only *after* the MFs for all the qubits in the -frequency-multiplexed group of qubits have been trained. The training of the FNN can then be performed utilizing the same training set as the one used to train the MF. The FNN is trained with a cross-entropy loss with two hidden layers. The overall FNN architecture used is $N \rightarrow 2N \rightarrow 4N \rightarrow 2^N$, where N denotes group size in the frequency-multiplexed group of qubits.

FNN inference. The trained FNN can be used to obtain discriminated values for all N qubits in the group. The benefits of using a smaller FNN in hardware are shown in Figure 3.21(d), where the LUT usage on an FPGA for the MF-NN design drops to below 8% for a single FNN catering to five qubits.

3.4.2.4 Improving readout fidelity with qubit relaxation detectors

So far, we have used an MF to reduce the dimensionality of the readout time traces acquired after the ADC and a neural network to mitigate non-idealities. However, Table 3.8 shows that the use of a smaller FNN can not match the accuracy of the baseline FNN design [149]. We observe that the qubit-readout accuracy achieved by the smaller neural network is greatly affected by relaxations ($1 \rightarrow 0$ transitions). Typically, during measurements, qubits can undergo undesired state transitions. Excited-state relaxation is a stochastic process and its probability increases exponentially in the measurement operation latency. To enhance readout fidelity, we focus on detecting qubit relaxation to reduce the diminishing impact on the qubit-readout accuracy.

Insight. Relaxations during readout cause a bias, i.e., the readout fidelity of qubits in the ground ('0') state exceed the one of qubits in the excited ('1') state. This bias has led to work such as invert and measure, where this state-dependent bias is used to improve the overall readout accuracy [10, 90, 235, 250]. However, all prior works focus on reducing the readout bias using statistical techniques applied to a large number of qubit measurements and do not improve the single-shot qubit-readout fidelity, which is essential for quantum error correction. The critical insight of our design is -

The readout trace of a qubit relaxing during readout differs from error-free traces for the ground and excited states.

Using this insight, we train an MF to detect relaxations ($1 \rightarrow 0$ transitions) during readout to accurately estimate the state of the qubit prior to the measurement.

3.4.2.5 Identifying relaxation traces during training

Training a machine learning model to detect qubit relaxation in a supervised manner is challenging, as qubit relaxation is an uncontrolled stochastic process. Unlike typical supervised training used for readout classifiers, where a qubit is prepared in a known state to generate a training dataset, creating labeled relaxation traces is implausible. Moreover, the relaxation traces are device dependent. To solve this problem, we propose a computationally efficient and accurate unsupervised algorithm that automatically identifies the relaxations and classifies qubit-readout traces as true ground states ('0'), excited states ('1'), or qubit-state relaxations ($1 \rightarrow 0$) traces.

We propose Algorithm 2 to identify relaxation traces. For every qubit-readout trace in the baseline dataset, we have labels '0' and '1.' This labeled dataset is typically generated during the device calibration. The proposed Algorithm 2, further refines these labels into trusted ground ('0'), excited ('1'), and qubit-state relaxation ($1 \rightarrow 0$) time traces without running any additional calibration experiments. For example, we use the baseline dataset consisting of N_{train} traces, with each trace being a 2D matrix (one dimension each for the I and Q components) with a length equal to the readout duration. From the baseline dataset, Algorithm 2 estimated 4.3%, 8.9%, 11.6%, 6.5% of the total traces were relaxation traces for qubits 1, 3, 4, and 5 respectively (the lack of distinguishability results in noisy results for qubit 2).

Algorithm 2: Identify relaxation traces in training set

Input: Tr_{in} : Readout traces from training set
Output: Tr_{relax} : Relaxation traces

```

1 Function getRelaxationTraces( $\text{Tr}_{\text{in}}$ ):
2    $\text{Tr}_{\text{in}}$ :  $N_{\text{train}}$  traces, each trace a 2D vector
3    $\text{traces}_0 = \text{Tr}_{\text{in}}[0]$  // ('0') traces
4    $\text{traces}_1 = \text{Tr}_{\text{in}}[1]$  // ('1') traces
5   for  $i$  in  $N_{\text{train}}$  do
6      $\text{mean}_0[i] = \text{mean}(\text{traces}_0[i])$ 
7      $\text{mean}_1[i] = \text{mean}(\text{traces}_1[i])$ 
8   end
9    $\text{centroid}_0 = \text{mean}(\text{mean}_0)$ 
10   $\text{centroid}_1 = \text{mean}(\text{mean}_1)$ 
11   $\text{radius} = \text{distance}(\text{centroid}_0, \text{centroid}_1)/2$ 
12  for  $\text{tr}_1$  in  $\text{traces}_1$  do
13    if  $\text{distance}(\text{tr}_1, \text{centroid}_0) \leq \text{radius}$  then
14      // Append index of original trace
15       $\text{Tr}_{\text{relax}}.\text{append}(\text{index}(\text{tr}_1))$ 
16    end
17 return  $\text{Tr}_{\text{relax}}$ 

```

For labeling relaxation traces, we first reduce the dimensionality of each qubit-readout trace by computing the *Mean Trace Value (MTV)*, which is a mean value of I and Q components for the trace. The key insight behind this algorithm is that traces that relax to the ground state have a *MTV* similar to traces that are correctly identified as '0,' as depicted in Figure 3.22(a) showing the *MTV* of all traces in the training set. The measurements that are labeled as '1' but lie in the region dominated by the '0' measurements are thus either (a) traces corresponding to relaxations that occurred *during* readout, (b) traces corresponding to relaxations that occurred *before* readout, or (c) initialization errors. For training purposes, (b) and (c) present the same behavior. To keep the identification simple, we assume that all cases are due to reason (a), even though this may bias the process. Thus, to detect relaxation errors, we define a circular region for both classes, where the centers are the means of all points corresponding to each class (centroids), and the radius is half the distance between the two centroids.

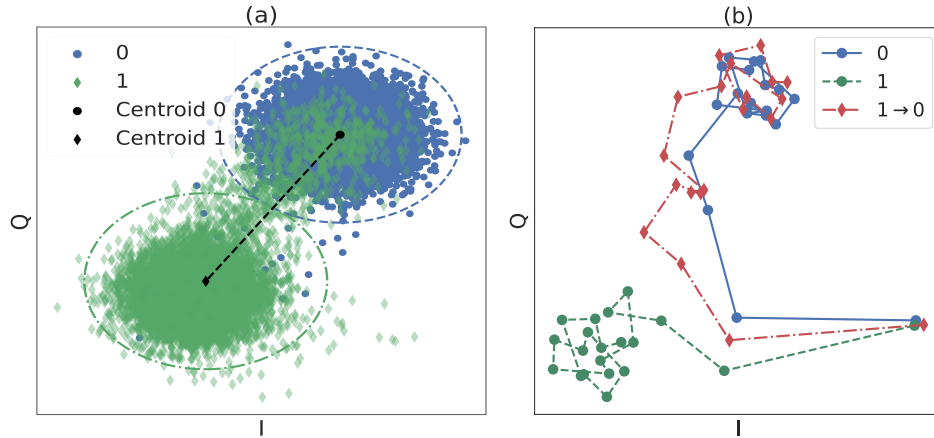


Figure 3.22: (a) Plot of I vs. Q values along with centroids of ground and excited-state traces shown as mean-trace values (MTV); (b) The traces corresponding to a ground state (0), excited state (1), and relaxation (1→0).

3.4.2.6 Adding features for better accuracy

Once traces corresponding to qubit-state relaxations have been identified, they can be used to train a classifier that can distinguish between relaxations and true ‘0’ traces. Choosing the correct filter/detector is the next step. As shown in Figure 3.22(b), the time evolution of a trace corresponding to a qubit-state relaxation is different from a trace corresponding to a reliable ground state measurement. We use this observation to create another MF for every qubit in the system, here referred to as a Relaxation Matched Filter (RMF). The RMF is trained using the relaxation traces (Tr_{relax}) identified from the training set and the ground-state traces (Tr_0) with the formula $\text{mean}(\text{Tr}_{\text{relax}} - \text{Tr}_0) / \text{var}(\text{Tr}_{\text{relax}} - \text{Tr}_0)$. This is thus different from the general MFs, which are trained using the ground and excited-state traces.

The second part of our design involves expanding the inputs of the FNN used in the MF-NN design to accommodate the output of the RMF. Since there are N RMFs for N qubits in a frequency-multiplexed group of qubits, the input layer of the FNN increases from N to 2N neurons. This intuitively corresponds to adding more features to the input data for the neural network to use, thus changing its role from just identifying crosstalk to using the data from both kinds of MFs to yield an optimal result. Depicted in Figure 3.23, we call this design MF-RMF-NN. The MF-NN design can also be derived from MF-RMF-NN by removing the RMFs and reducing the input layer of the neural network to 5 neurons.

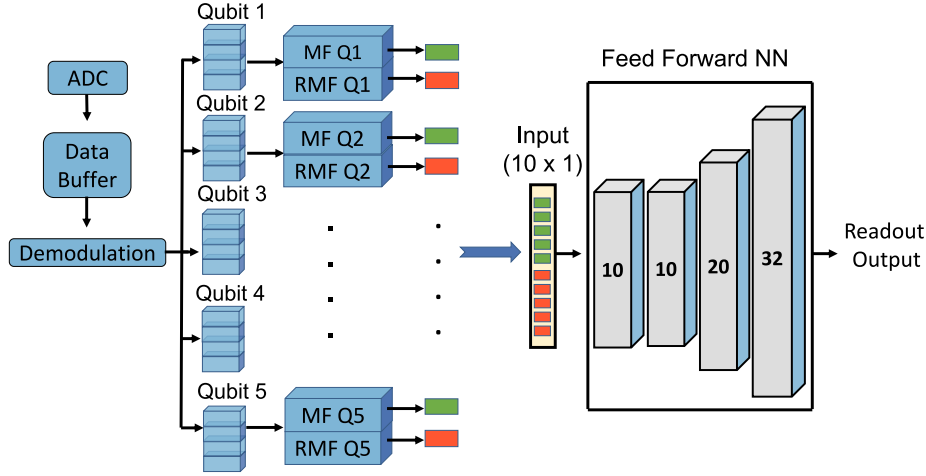


Figure 3.23: MF-RMF-NN design overview.

The benefit in discrimination accuracy by adding the RMF and modifying the role of the FNN is summarized in Table 3.8. For almost all qubits, we see a minimum of 1.5% increase in qubit-readout accuracy with MF-RMF-NN, with qubit 5 reaching 98.9% accuracy. Qubit 2’s lack of improvement can be explained by the fact that there was no separation between the ground and excited states, due to which the algorithm used for determining relaxation traces yielded very noisy results. We achieve substantial gains in accuracy without using a significant amount of hardware – as shown in Figure 3.21(d), the increase in LUT usage on an FPGA due to an increase in the size of the input layer and additional MFs is marginal, utilization increases from 7.15% to 7.79% to support five-qubit readout. Compared to the FNN implemented in the baseline, which at best requires an entire FPGA, HERQULES allows high-fidelity and scalable qubit readout. Overall, we see a 1.44% improvement in cumulative accuracy (F_{5Q}) over the baseline, which corresponds to a **16.4%** ($= \frac{92.66-91.22}{100-91.22}$) relative improvement in the readout infidelity. If qubit 2 is omitted⁴, then the improvement to the cumulative accuracy (F_{4Q}) is 1.85% over the baseline, corresponding to a **42.9% relative improvement** in inaccuracy. The precision (recall) of the predictions made by MF-RMF-NN for the five qubits were 98.6% (98.4%), 78.6% (69.7%), 97.1% (96.2%), 95.5% (97%), and 98.8% (99.1%) respectively.

⁴The distinguishability of the states of qubit 2 is limited due to the experimental setup [149].

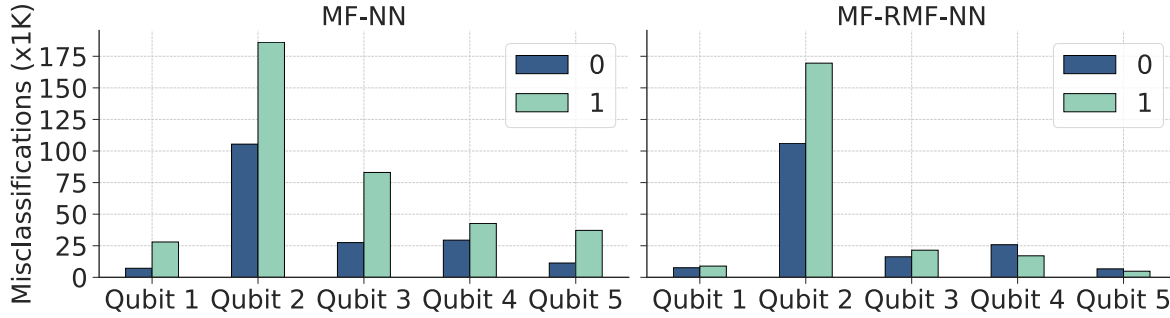


Figure 3.24: Comparison of misclassification of ground ('0') and excited ('1') states for all qubits between MF-NN and MF-RMF-NN designs.

3.4.2.7 Crosstalk mitigation

To illustrate the benefit of neural networks for qubit-state discrimination, we perform cross-fidelity [110, 149] studies. Cross-fidelity is a measure of correlations between parallel qubit measurements, thus capturing information on correlated errors occurring due to crosstalk. Cross-fidelity is defined as: $F_{ij}^{CF} = 1 - [P(e_i|0_j) + P(g_i|1_j)]$, ($i \neq j$); where 0_j (1_j) is the preparation of the qubit j in the ground (excited) state and g_i (e_i) is the measured qubit i in the ground (excited) state. Cross-fidelity captures the crosstalk between readout resonators i, j (since every qubit is connected to its own resonator). Table 3.9 shows that using the neural network reduces crosstalk in all cases, especially for a hamming distance of one where readout crosstalk dominates (qubit-qubit interactions dominate errors for hamming distances 2 and 4) where the reduction is more than 3x compared to a design that uses a linear Support Vector Machine (SVM) instead of a neural network. Note that suppressing crosstalk is essential as correlated errors are hard to detect and correct.

Table 3.9: Mean absolute values of the cross-fidelity (F^{CF}) for different hamming distances (lower is better).

Design	$\langle F_{j=i\pm 1}^{CF} \rangle$	$\langle F_{j=i\pm 2}^{CF} \rangle$	$\langle F_{j=i\pm 3}^{CF} \rangle$	$\langle F_{j=i\pm 4}^{CF} \rangle$
Baseline	0.002	0.005	0.002	0.0003
MF	0.0108	0.015	0.0021	0.0008
MF-NN	0.0071	0.011	0.003	0.0003
MF-RMF-SVM	0.011	0.0077	0.0024	0.0006
MF-RMF-NN	0.0031	0.0062	0.0008	0.0005

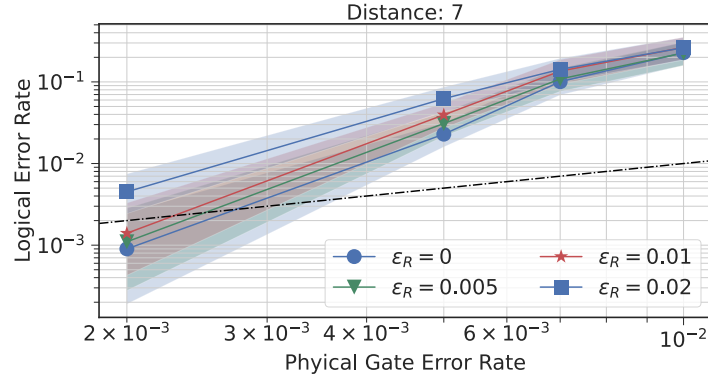


Figure 3.25: Logical error rate per round vs. physical gate error rate for surface code of distance 7 for different values of the averaged readout error (ϵ_R). The logical error rate is equal to the physical error rate at the dash-dot line.

3.4.2.8 Why does the RMF improve accuracy?

By training the RMF to separate qubit-state relaxation traces and ground-state traces, we provide the FNN with additional features of the qubit-readout signal. Figure 3.24 shows empirically how additional features provided by the RMF help reduce the number of misclassifications made by the MF-RMF-NN discriminator for the excited ('1') state for all qubits, which improves the overall readout accuracy. For qubit 2, the information provided by the RMF was noisy and the improvement was marginal.

3.4.3 Impact on quantum error correction

Effect of readout errors. The unreliability of today's NISQ-era quantum computational systems necessitates using Quantum Error Correction (QEC) codes to increase their usefulness. Surface codes [87] are among the most promising QEC protocols. To evaluate the effect of readout error on surface codes and to show that reducing readout errors is just as important as reducing the physical gate error rate to the efficacy of the surface code, we use the Stim [94] stabilizer simulator with different values of average readout error rate ($\epsilon_R = 0\%, 0.5\%, 1\%, \text{ and } 2\%$). As shown in Figure 3.25, a 1% increase in ϵ_R can cause the logical to surpass the physical gate error rate, which undermines the protection promised by QECs.

Scalability. The FNN used by the baseline design [149] cannot be efficiently implemented in hardware, making its use for the surface code impractical. This is because the signal

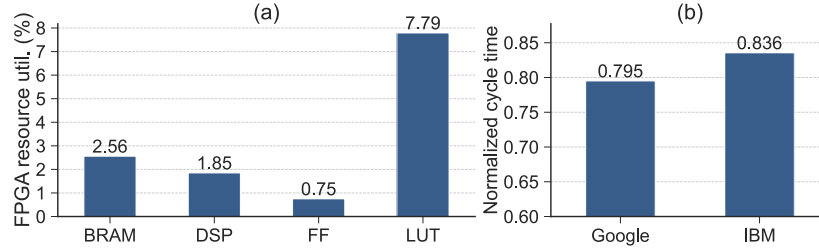


Figure 3.26: (a) FPGA resource utilization for HERQULES; (b) Reduction in the syndrome generation cycle time (lower is better) for a surface-17 circuit [262] with gate durations from different systems (Google, IBM).

output stream from the ADC for every group of multiplexed qubits needs to be transferred to software for discrimination, thus requiring a tremendous amount of bandwidth for large numbers of qubits while increasing the latency of discrimination. In comparison, HERQULES can be efficiently implemented in hardware, with resource utilization numbers shown in Figure 3.26(a). Assuming that 80% of resources are available for readout on an RFSoc controller like QICK [240], a single RFSoc can potentially read out more than 50 qubits with extremely low latency. This is in stark contrast to the baseline design which at best can read out a single group of multiplexed qubits per RFSoc. FPGAs such as the Virtex Ultrascale+ have larger fabrics, but do not offer the scalability advantages and cost-effectiveness offered by RFSocs [240, 255] for quantum control applications.

3.4.4 Conclusions

HERQULES is a high-fidelity readout discriminator for superconducting qubits with a scalable, hardware-efficient architecture. HERQULES achieves up to 2% improvement (97% $\rightarrow$ 99%) in single-qubit readout accuracy compared to the state-of-the-art baseline design while using a small and lightweight neural network that can be easily implemented on current and future hardware platforms. HERQULES can thus be implemented on conventional FPGA control hardware used in commercial and experimental quantum computers to enable low-latency discrimination with very high accuracy. We envision rich architectural design space to enable readout pipelines with higher fidelity and low latency readout and features that can be used for fault-tolerant quantum computers.

3.5 Summary

In this chapter, we presented COMPAQT and HERQULES, two optimizations to the low-level qubit control and readout pipelines for superconducting qubits. COMPAQT enables scalable qubit control by compressing waveforms required for implementing quantum gates. By using a lightweight discrete cosine transform (DCT) based compression algorithm, waveforms can be compressed efficiently without an appreciable loss in gate fidelity. Waveform compression reduces the bandwidth requirements of the waveform memory, since decompression can directly provide samples to the digital-to-analog converters (DACs) at the required bandwidth. Compression thus reduces the memory capacity *and* bandwidth requirements, which makes FPGA- or ASIC-based controllers more scalable and power efficient. In the context of fault-tolerant quantum computers, COMPAQT can allow a single RFSoc-based controller to control $5\times$ more logical qubits than a baseline system that does not use compression. HERQULES on the other hand targets the qubit readout pipeline. By using a combination of matched filters, HERQULES can (1) detect $1 \rightarrow 0$ transitions occurring during readout (thermal relaxations) and (2) use a significantly smaller neural network for accurate state discrimination and crosstalk mitigation. This allows HERQULES to attain an overall readout fidelity 16.4% better than the state-of-the-art neural network discriminator, while consuming less than 8% of the total LUTs on a commercial FPGA. HERQULES is thus scalable and can allow for lower logical error rates on fault-tolerant quantum computers.

Chapter 4

Accelerated Decoding for QEC

4.1 Introduction

Implementing fault-tolerant quantum computers requires fast, accurate decoders. As shown in Chapter 1, decoders sit directly above the low-level control hardware used for controlling and measuring qubits. Figure 4.1 shows the position of the decoder relative to the control and readout components of the classical control system. Syndromes generated by the readout hardware are buffered before being fed to the decoder. The decoder identifies the qubit(s) which experienced errors from the batch of syndromes it decoded. What makes decoding an interesting and challenging problem are the constraints: (a) Decoding must be fast and should, on average, take less time than the syndrome generation cycle time to prevent a *backlog* of syndromes, and (b) decoding must be accurate – it should be able to identify the correct pattern of errors from the syndrome *almost* every time. Latency and accuracy are conflicting requirements due to the complexity of the decoding problem. For classical error correcting codes, decoding has been proved to be NP-complete [20]. For QEC codes, optimal (maximum likelihood) decoding for general quantum codes has been shown to be NP-hard [117], and #P-complete for stabilizer codes [126].

In this chapter, we build on the background given in Section 2.8.3 by elaborating on the constraints within which decoders must operate in. We then describe a heuristic decoder tailored for FPGAs for quantum low-density parity check (qLDPC) codes and show how it performs relative to message passing decoders.

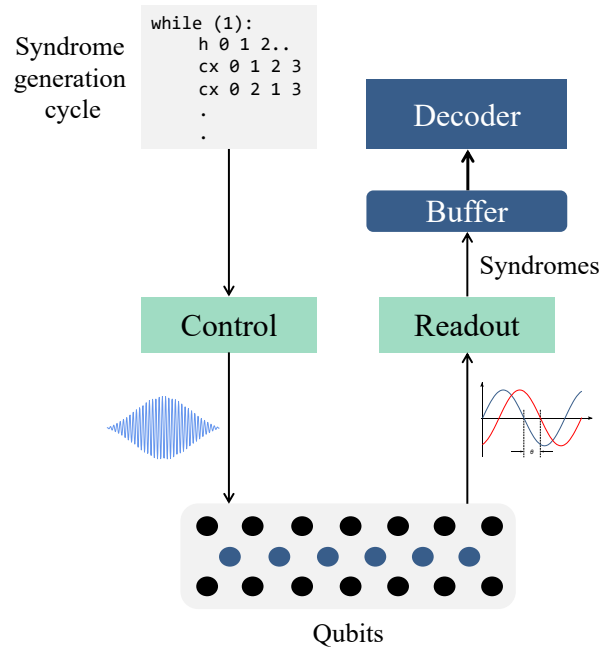


Figure 4.1: Linking low-level control and readout hardware with the decoder: the syndrome generation cycle (shown in program form) is repeatedly executed by the control pipeline, and syndromes are generated by the readout hardware which are then fed to the decoder.

4.1.1 Contributions

We make the following contributions in this chapter:

- We introduce a clustering decoder, an FPGA-tailored version of the generalized union-find decoding algorithm that uses Belief Propagation (BP) as a pre-decoder.

4.1.2 Organization

This chapter is organized as follows:

- We describe the decoding problem and its constraints in Section 4.2.
- In Section 4.3, we present the clustering decoder.
- In Section 4.4, we show the performance of the clustering decoder for bivariate-bicycle qLDPC codes.
- Finally, we summarize this chapter in Section 4.5.

4.2 The Decoding Problem

Decoding was introduced in Section 2.8.3. To recap, decoding of QEC codes requires access to the check matrix H , the logical action matrix A , and a set of probabilities $\mathbf{p}$ corresponding to each column of H . These weights are often called *priors*. H , and therefore A and the priors $\mathbf{p}$ are dependent on the noise model associated with the decoding problem. H is an $M \times N$ matrix, where there are M detectors and N faults derived from the code and the error model. $H_{ij} = 1$ specifies detector i being flipped by fault j if it occurs. A is a $k \times N$ matrix, where there are k logical qubits. $\mathbf{p}$ is a list of N probabilities, where p_j is the probability of fault j occurring. In this dissertation, we always use the circuit-level noise model, where every operation (including idling qubits) can experience errors.

The decoding problem can be defined as follows: The syndrome generation circuit is executed, and a set of (unknown) faults F occurs with probability $\Pr(F)$. The syndrome bitstring $\sigma(F) = HF$ is known, even though F is not. The syndrome is passed to a decoder $\mathcal{D}$, which is a classical algorithm with knowledge of H , A , and $\mathbf{p}$. The decoder outputs a correction $\hat{F} = \mathcal{D}(\sigma(F))$. If $\alpha(F) + \alpha(\hat{F}) = A(F + \hat{F}) = 0$, then we say that the decoder succeeds; otherwise, it fails. When the decoder fails, it essentially allows an unknown change in the state of the logical qubit, which is called a logical error. The figure of merit that is commonly used to quantify the accuracy of a decoder is called the logical error rate, which is essentially the failure rate of the decoder over thousands of syndromes for a given noise model and QEC code. Note that it is not necessary that $\hat{F} = F$ in order for the decoder to succeed.

Having defined the decoding problem, we now describe two factors that contribute towards making decoding a hard (but interesting) problem.

4.2.1 Quantum degeneracy

Quantum degeneracy is a feature of quantum codes with no counterpart in classical error correction, and it fundamentally reshapes the decoding problem. In a well-designed classical code, the minimum-weight error consistent with a given syndrome is unique, so decoding reduces to finding the single most-likely error pattern and applying the corresponding correction. Quantum codes behave differently: as noted above, the decoder succeeds whenever $A(F \oplus \hat{F}) = 0$, so it need only identify an error in the same logical equivalence class as F , rather than F itself.

A stabilizer code is called *degenerate* if its stabilizer group contains non-identity elements of weight strictly less than the code distance d . Whenever this holds, distinct low-weight errors can share the same syndrome while acting identically on the logical state, differing only by a stabilizer element. The surface code is the canonical example: because its check operators have weight four, two error configurations that produce the same syndrome but follow distinct, homologous paths between the resulting defects differ only by products of plaquette stabilizers, and are therefore logically interchangeable. The decoder need not determine which configuration actually occurred—only the correct equivalence class.

Degeneracy is double-edged. On one hand, the multiplicity of valid corrections works in the decoder’s favor: any correction in the right equivalence class succeeds, not only the one that exactly reproduces the physical error. On the other hand, many decoding algorithms inherited from classical coding theory—belief propagation (BP) in particular—implicitly assume a unique most-likely error vector. When a degenerate code presents many equally probable, logically equivalent errors, BP can oscillate between these solutions and fail to converge, degrading its accuracy well below that of (degenerate) maximum-likelihood decoding, which correctly weighs each equivalence class as a whole. This degeneracy-induced failure of BP is a primary motivation for post-processing techniques such as ordered statistics decoding (OSD), which uses the soft reliability estimates from BP to construct a valid low-weight correction—and, in higher-order variants, to search systematically over nearby candidates—even when BP itself fails to converge.

4.2.2 Real-time decoding

During the course of the computation on a fault-tolerant quantum computer, syndromes for more than 10^{10} QEC cycles can be generated for running useful programs. The decoder must, on average, process each syndrome within the time budget set by one syndrome generation cycle—the time required to execute one round of stabilizer measurements, typically on the order of $1\ \mu\text{s}$ on superconducting platforms. If the decoder consistently takes longer than this budget, unprocessed syndromes accumulate in a queue faster than they are drained, creating an unbounded backlog that eventually stalls the computation indefinitely.

A further, and often stricter, latency constraint arises from the implementation of non-Clifford logical gates. Logical Clifford gates are well-behaved with respect to the Pauli frame: any Pauli error commuting through a Clifford gate is simply re-labeled on the

other side, so the frame update is tracked purely in software without requiring a physical correction to be applied to the qubits mid-circuit. Non-Clifford gates break this property. On most topological codes, logical non-Clifford gates cannot be implemented transversally and instead require the consumption of a *magic state*—a specially prepared ancilla qubit. Figure 4.2 shows the canonical example: the logical T_L gate, realized by entangling the data qubit $|\psi_L\rangle$ with the ancilla $|A_L\rangle = T_L|+\rangle$ via a logical CNOT and then measuring the ancilla. The classical outcome m of the measurement determines whether a logical Clifford correction S_L^m must be applied to the data qubit to recover $T_L|\psi_L\rangle$. Crucially, Pauli errors do not commute cleanly through a non-Clifford gate, so this correction cannot be deferred past the next T_L gate in the circuit: the data qubit must stall until the decoder has processed the accumulated syndrome information and returned an updated Pauli frame. The decoder thus operates under a hard latency deadline set by the rate of magic state injection. This is generally referred to as the *backlog problem* [253].

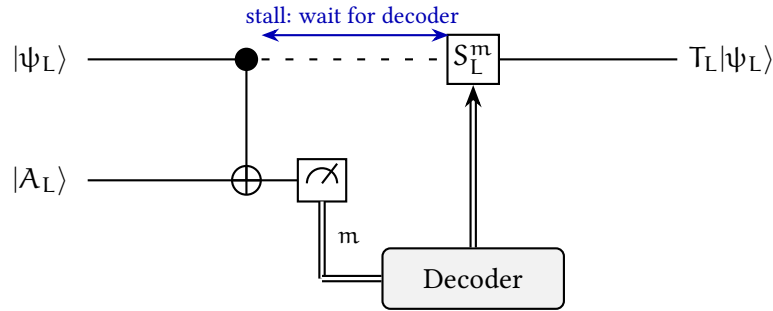


Figure 4.2: Fault-tolerant T_L gate via magic state injection. The data qubit $|\psi_L\rangle$ is entangled with the ancilla $|A_L\rangle = T_L|+\rangle$ via a logical CNOT, after which the ancilla is measured to yield a classical outcome m (double lines denote classical wires). A classically-controlled Clifford correction S_L^m is applied to recover $T_L|\psi_L\rangle$, but the data qubit must stall (dashed segment) until the decoder resolves the accumulated syndromes and returns an updated Pauli frame.

Quantum degeneracy increases the complexity of the decoding problem such that it can reduce the accuracy of certain decoders. The real-time decoding requirement imposes the latency constraint that makes fast, accurate decoding hard. Due to technologies like superconducting qubits having syndrome generation cycle times in the order of $1\mu\text{s}$, simply implementing decoders in software (or more specifically on CPUs) will not be enough to meet the strict latency requirements. This limitation has spurred research in *accelerated* decoders – decoding algorithms implemented on FPGAs/ASICs capable of meeting the

latency requirements of superconducting qubits while maintaining reasonably high accuracies [172, 272, 284]. We now present the clustering decoding algorithm tailored for implementation on FPGAs which trade off some accuracy for fast decoding times. This decoder were implemented for the bivariate-bicycle qLDPC code, but is generalized in the sense that it can be used for any stabilizer code.

4.3 Clustering Decoder

The clustering decoder¹ is based on the Union-Find algorithm, first introduced in [63] for the surface code. Clustering decoders iteratively grow connected subgraphs of the Tanner graphs defined by H until every cluster can be decoded independently. Clustering is a divide-and-conquer approach – rather than decoding over the entire graph, it can be performed over smaller graphs containing all or a subset of the flipped detectors. We use a generalized version of the clustering algorithm, similar to the one presented in [61]. Algorithm 3 describes the clustering decoder that we implement in this work. An initial syndrome (and optionally an erasure set) is used for generating the clusters. These clusters are grown iteratively until the support for the detectors exists entirely within the cluster, i.e., the cluster is valid. If any clusters overlap with each other, they are merged. Once all clusters are valid, the decoded result is a combination of the solutions of all independent clusters.

For tailoring this clustering algorithm for FPGAs, the following components must be concretely defined:

- Cluster representation: how clusters are defined in logic components on the FPGA.
- Cluster initialization: how clusters are initialized from a syndrome.
- Cluster growth and merge: how clusters are grown and merged on the FPGA.
- Cluster validity and final solution: how clusters are determined to be valid and solved.

We now define these components for the clustering decoder.

¹This work is based on [173], in collaboration with Michael Beverland, Thilo Maurer, Markus Bühler, and Drew Vandeth. The algorithm used by this decoder (shown in subsequent pages) was conceptualized by Michael Beverland. The FPGA implementation and design was done by Satvik Maurya, along with all numerics.

Algorithm 3: Clustering Decoder

Input: H (decoding matrix),
 σ (syndrome),
 $\mathcal{E}$ (erasure set)
Output: $\hat{F}$ (correction)

```

1  $\mathcal{C} \leftarrow \{\{v\} : v \in \sigma \cup \mathcal{E}\}$  // one singleton cluster per syndrome/erasure
   node
2 while any cluster in  $\mathcal{C}$  is not IsValid() do
3    $G \leftarrow$  any invalid cluster; remove it from  $\mathcal{C}$ 
4    $G \leftarrow \text{grow}(G)$  // expand by one hop in the Tanner graph
5   for  $G' \in \mathcal{C}$  overlapping with  $G$  do
6     Remove  $G'$  from  $\mathcal{C}$ 
7      $G \leftarrow \text{merge}(G, G')$  // absorb into  $G$ ; repeat as  $G$  grows
8   end
9   Add  $G$  to  $\mathcal{C}$ 
10 end
11  $\hat{F} \leftarrow \mathbf{0}^N$ 
12 for  $G \in \mathcal{C}$  do
13    $\hat{F} \leftarrow \hat{F} \oplus \text{FindSolution}(H, \sigma, G)$  // solve linear system restricted
     to  $G$ 
14 end
15 return  $\hat{F}$ 

```

4.3.1 Cluster representation

The data structure used to represent clusters in an FPGA has a direct impact on the complexity of subsequent operations such as growth, merging, and validity checks. A straightforward approach would be to assign each node in the fault graph a unique ID and store, for each cluster, the list of its node IDs, but this makes merge detection in an FPGA cumbersome. Instead, as shown in Figure 4.3, we assign each node in the fault graph a fixed position in a *cluster bitmap*, which is a bitstring $b \in \mathbb{F}_2^{M+N}$ in which a bit value of 1 indicates that the corresponding node belongs to the cluster (in FPGA literature, such a representation is often called a *bitmap*). We assume N_{clus} dedicated registers, which along with FFs to contain the bitmap, include a FF which marks whether or not the cluster is in use, and a FF which marks whether or not an in-use cluster is valid. As we will see later, we also use the *interior faults bitmap* of a cluster, a length- N bitmap indicating the fault nodes whose neighboring check

nodes all lie within the cluster.

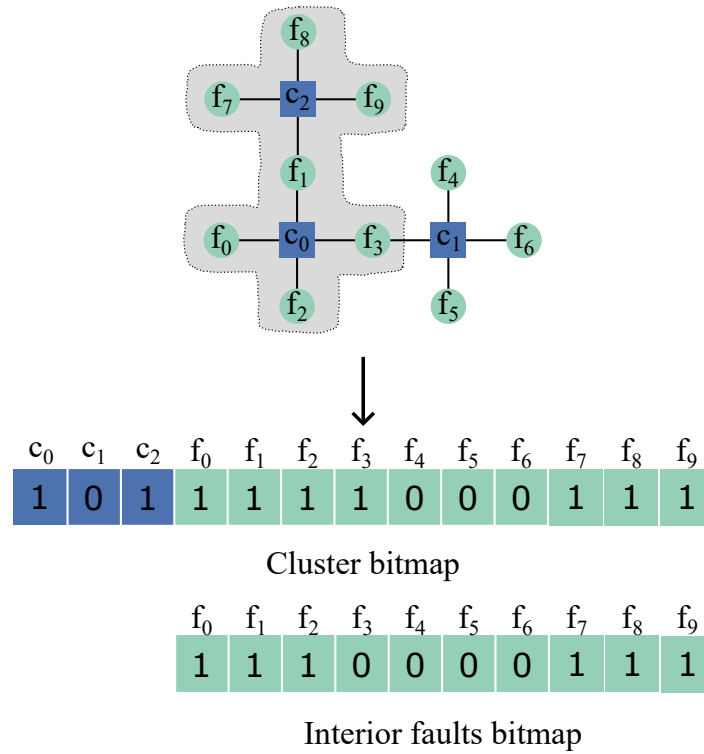


Figure 4.3: The bitmap representation of a cluster assigns a bit to each node in the Tanner graph $\mathcal{G}$; the left bits correspond to syndrome nodes and the right bits to fault nodes. The shaded box highlights a cluster containing nodes c_0, f_0, f_1, f_2, f_3 , which are assigned a value of 1 in the cluster’s bitmap, with all other nodes assigned 0. Note that in practice the full decoding graph is much larger than this example.

4.3.2 Cluster initialization

In our work, we use the belief propagation (BP) decoder as a predecoder. This means that not all syndromes are processed by the clustering decoder – only syndromes not successfully decoded by BP are passed to the clustering decoder. The BP pre-decoder produces log-likelihood ratios Λ_j indicating confidence that fault j occurred. Faults with $\Lambda_j \geq \Lambda_{\text{accept}}$ are treated as confident errors and flipped to form a *partial correction* $\tilde{F}_{\text{pre}}$, which reduces the syndrome weight and thus the number of initial clusters. Faults with $\Lambda_{\text{erase}} \leq \Lambda_j < \Lambda_{\text{accept}}$ are treated as uncertain and form the erasure set $\mathcal{E}$, initializing clusters over the corresponding fault nodes in the decoding graph (Figure 4.4).

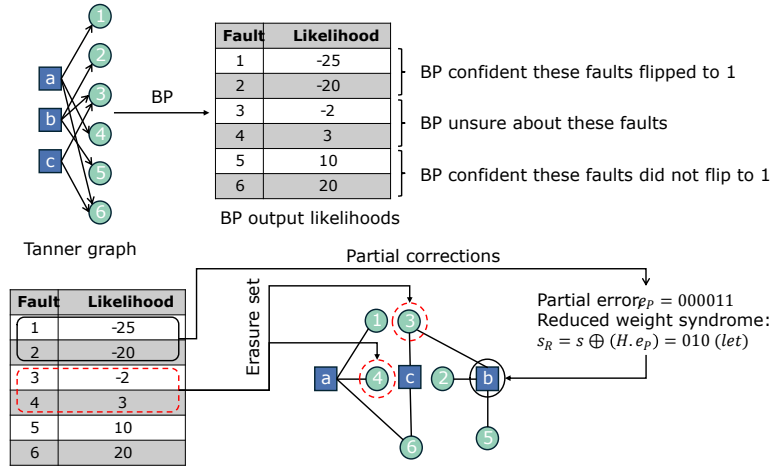


Figure 4.4: Mapping pre-decoder outputs to cluster decoder inputs. Confident errors ($\Lambda_j \geq \Lambda_{\text{accept}}$) are flipped to form the partial correction $\tilde{F}_{\text{pre}}$, reducing the syndrome weight, while uncertain faults ($\Lambda_{\text{erase}} \leq \Lambda_j < \Lambda_{\text{accept}}^+$) form the erasure set $\mathcal{E}$.

In the FPGA-tailored algorithm, the length- N list of pre-decoder outputs is stored in a chain of N nodes. Comparators threshold the pre-decoder outputs against the cutoffs to produce binary vectors for $\tilde{F}_{\text{pre}}$ and $\mathcal{E}$. Once $\tilde{F}_{\text{pre}}$ is obtained, the updated syndrome is computed via binary matrix-vector multiplication over $\mathbb{F}_2$ followed by entry-wise binary addition:

$$\sigma \leftarrow \sigma_{\text{original}} + (H\tilde{F}_{\text{pre}}),$$

both of which can be implemented efficiently in FPGAs.

At the beginning of the algorithm, clusters are initialized from the input syndrome σ and the erasure set $\mathcal{E}$ obtained from the pre-decoder output. We represent σ and $\mathcal{E}$ together as a length- $(M + N)$ bitmap b_{init} , where the first M bits correspond to check nodes and the remaining N bits correspond to fault nodes. Each ‘1’ in b_{init} seeds a unit-bitmap cluster; for example, $b_{\text{init}} = 10101$ produces unit bitmaps 10000, 00100, and 00001. For later use, we define the length- $(N + M)$ bitmap u to be all zeros except for a single one in the right-most position.

To extract each unit-bitmap initialization seed from b_{init} , we first set $b \leftarrow b_{\text{init}}$. We then iteratively isolate and remove the right-most one in b using the following loop:

1. Compute $c = b + u = \neg(10101) + 00001 = 01011$.
2. Compute $d = b \& c = 10101 \& 01011 = 00001$.

3. The result d is the desired unit-bitmap; store it.
4. Update $b \leftarrow b + d = 10101 + 00001 = 10100$ to clear the extracted bit.
5. Continue while $\text{OR_reduce}(b)$ is nonzero.

This procedure can be applied independently to the check-node and fault-node portions of b_{init} , allowing parallel initialization of syndrome and erasure clusters and achieving roughly a two-fold reduction in initialization time.

4.3.3 Cluster growth

Growing clusters requires knowledge of the underlying decoding graph, which is defined by the check matrix H . We assume the decoding matrix $H \in \mathbb{F}_2^{M \times N}$ is stored in RAM since its entries need to be accessed quickly. We also assume an ‘extended’ adjacency matrix $H^{\text{ext}} \in \mathbb{F}_2^{(M+N) \times (M+N)}$ by $H_{ij}^{\text{ext}} = 1$ iff $i = j$ or vertices i and j are neighbors in $\mathcal{G}$. Unlike H , the constant matrix H^{ext} does not require independent access of different columns or rows and can be embedded directly in an FPGA without using RAMs or FFs.

The cluster growth subroutine expands each cluster by adding all vertices adjacent (in the decoding graph $\mathcal{G}$) to any vertex currently in the cluster, as illustrated in Figure 4.5(a). In our FPGA-tailored design, we implement this operation using the extended adjacency matrix H^{ext} .

Given a cluster bitmap $b \in \mathbb{F}_2^{M+N}$, growth is performed as follows:

1. For each column of H^{ext} , perform a bitwise AND with b , yielding an intermediate matrix B whose rows match those of H^{ext} where $b = 1$ and are zero otherwise.
2. Apply a column-wise OR to B to obtain the grown bitmap b' .

4.3.4 Cluster merge

Clusters are merged whenever they share at least one vertex. In the bitmap representation introduced earlier, this condition can be detected via a simple bitwise operation: if the bitwise AND of two cluster bitmaps contains any 1, the clusters overlap and must be merged. Given two cluster bitmaps b and b' of length $(M + N)$, the merge procedure is:

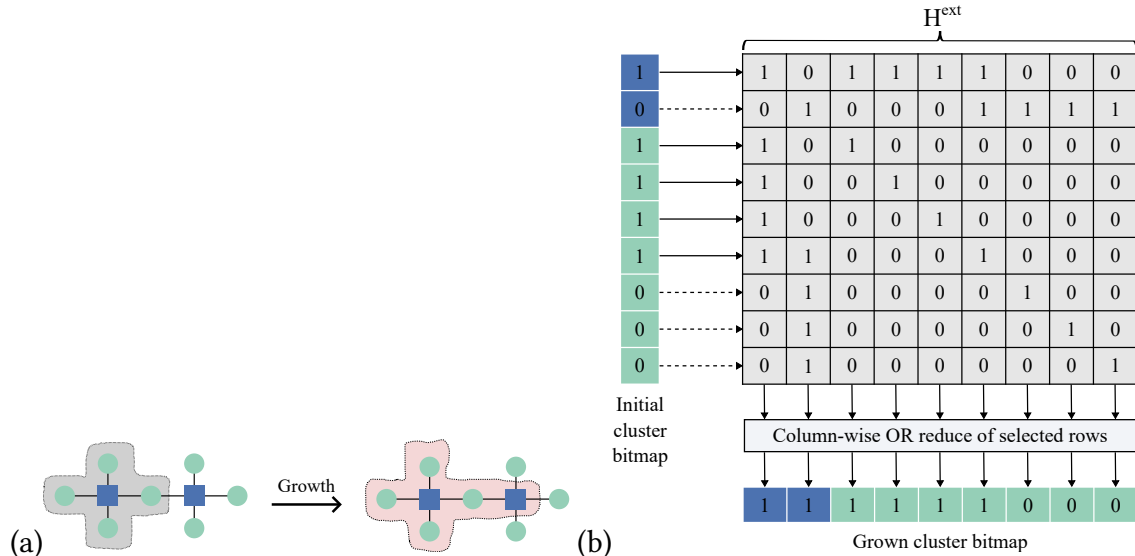


Figure 4.5: (a) Example of cluster growth: all vertices adjacent to any vertex in the current cluster (highlighted) are added to the cluster. (b) Gateway implementation: columns of H^{ext} corresponding to 1 entries in b_{init} are selected; a column-wise OR reduction over these columns yields b' .

1. **Merge predicate:** Compute $m = \text{OR_reduce}(b \ \& \ b')$. If $m = 1$, the clusters share at least one vertex.
2. **Merge operation:** If $m = 1$, set $b \leftarrow b \mid b'$ to obtain the merged bitmap, and assign the register that held cluster b' as empty.

In our cluster decoder implementation, one cluster is grown per update loop. Let $G^{(1)}$ be the grown cluster and $\{G^{(2)}, \dots, G^{(r)}\}$ the others. Before growth, each cluster represented a distinct connected component of $\mathcal{G}$ with disjoint bitmap support. After growth, $G^{(1)}$ may overlap with other clusters and must be merged with them. This is done by checking the merge predicate between $G^{(1)}$ and each of the $r - 1$ remaining clusters in turn, merging into $G^{(1)}$ whenever required.

4.3.5 Cluster validity and solution

Validity checking then proceeds in three steps:

1. **Find interior faults:** Verifying the validity of a cluster requires identifying those fault nodes whose neighboring checks all lie within the cluster.

2. **Index extraction:** Determine the row indices (enclosed check nodes) and column indices (interior fault nodes) from the first M bits of $\mathbf{b}$ and the column indices (interior fault nodes) from $\mathbf{b}_{\text{int}}$.
3. **Submatrix extraction:** From the fixed decoding matrix $\mathbf{H} \in \mathbb{F}_2^{M \times N}$, select the enclosed check node rows and interior fault node columns indexed above to form the submatrix $\mathbf{H}_{\text{sub}} \in \mathbb{F}_2^{m \times n}$.
4. **Solution existence test:** Use a linear systems solver to determine if $\mathbf{H}_{\text{sub}}\mathbf{x} = \mathbf{y}$ has a solution (since its of the form $\mathbf{Ax} = \mathbf{b}$).

Determine interior faults. Given a cluster bitmap $\mathbf{b} \in \mathbb{F}_2^{M+N}$, let $\mathbf{s} \in \mathbb{F}_2^M$ denote its checks sub-bitmap and $\mathbf{t} \in \mathbb{F}_2^N$ denote its faults sub-bitmap (the last N entries of $\mathbf{b}$). The interior faults bitmap is determined as follows:

1. Use the negated checks bitmap $\neg \mathbf{s}$ to select rows of $\mathbf{H}^{\text{ext}}$ corresponding to checks not in the cluster.
2. Apply a column-wise OR to these selected rows, yielding a non-interior faults bitmap $\mathbf{c} \in \mathbb{F}_2^N$.
3. Compute the interior faults bitmap as $(\neg \mathbf{c}) \& \mathbf{t}$.

Index extraction from the cluster bitmap. Determining the row and column indices for step (2) reduces to finding the indices of all 1-bits in a bitmap $\mathbf{b} \in \{0, 1\}^{M+N}$. Separate lists for checks and faults are required, and as such one can first separate $\mathbf{b}$ into $\mathbf{b}_S \in \{0, 1\}^M$ and $\mathbf{b}_F \in \{0, 1\}^N$ and handle them independently. A naïve sequential scan over all bits would be possible, but would require about as many cycles as there are bits. In what follows we briefly outline an alternative approach that reduces this slow-down (but we do not provide low-level algorithm details). One can separate the bits of $\mathbf{b}$ into consecutive buckets each containing Δ bits, and process the buckets in parallel. Within the i th bucket, the 1-bit indices $I_i \subset [i\Delta, (i+1)\Delta - 1]$ are found from among the Δ bits in the bucket and arranged into a sequential list. The lists indices obtained for each bucket are concatenated together to form a list of global indices.

Shift-based sub-matrix extraction. We have assumed H is stored in memory. Selecting arbitrary rows and columns via a large multiplexer would be fast but is impractical under FPGA timing constraints. Instead, we use a shift-based approach: rows are shifted upward until the top row matches a required row index, which is then copied to a temporary array. When a row index matches the first element, or head, of the list of indices in the buckets corresponding to the check nodes, the index is popped/removed from the buckets, making way for the next row to be found. After collecting all rows, columns are selected by shifting left and discarding columns corresponding to non-matching indices in the faults buckets. While a naïve approach could require mn cycles for an $m \times n$ submatrix, this shift-based method reduces the worst case to $M + N$ cycles.

Solving the linear system. The final step in validity checking is to solve the system $H_{\text{sub}}x = y$, where H_{sub} is the sub-matrix of H indexed by the cluster's enclosed check nodes and interior fault nodes, and y is the corresponding measurement-outcome vector. A cluster is valid if and only if this system admits at least one solution. Since clusters can have arbitrary shapes during decoding, H_{sub} may be highly under-determined ($n > m$) or over-determined ($m > n$), where m and n are the number of selected rows and columns, respectively. This requires linear solvers implemented on FPGAs that can solve arbitrary sized systems, such as the one presented in [119]. These solvers are systolic arrays, which make them fast and extremely compatible with FPGAs. We assume the use of such a systolic solver in this work. Note that checking for validity and finding the solution of the cluster are the same tasks in the context of the clustering decoder. Thus, when a cluster is found to be valid, the solution returned by the solver is the decoded solution of that cluster. This solution will only change if this valid cluster is then merged with a different cluster.

4.3.6 Algorithm output

At this point, every active cluster has a well-defined bitmap specifying its enclosed check and fault nodes. Each cluster's interior faults are then used to compute a valid per-cluster correction bitmap $\tilde{F}^{(1)}$, and the final correction $\tilde{F}$ is obtained by aggregating these L bitmaps via bitwise OR:

$$\hat{F} = \tilde{F}^{(1)} | \tilde{F}^{(2)} | \dots | \tilde{F}^{(L)}.$$

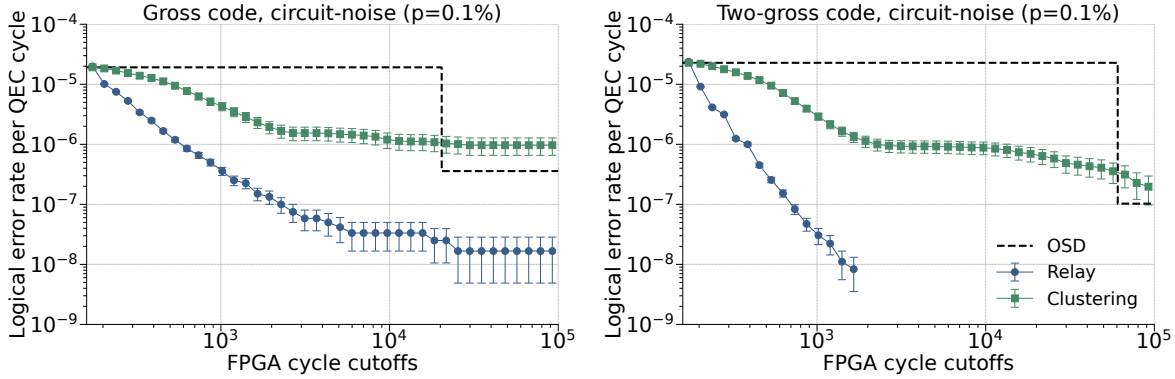


Figure 4.6: Cutoff-time performance curves for FPGA-tailored decoders. Logical error rate versus FPGA cycle budget for Relay [191], cluster decoding, and standard OSD [16], with runs exceeding the cycle cutoff counted as failures. Relay achieves the lowest error rates across all tested budgets.

Since clusters are disjoint by construction, this aggregation requires no further conflict resolution.

4.4 Results

We now evaluate the cluster decoder and compare it with Relay [191], the state-of-the-art decoder for qLDPC codes, and an FPGA implementation of the OSD algorithm [16]. Figure 4.6 summarizes the results. For a consistent comparison, we use the first leg of Relay as the pre-decoder for standard OSD and cluster decoding, and all performance data in Figure 4.6 are reported starting from the end of that leg. For any given FPGA cycle budget, Relay outperforms both OSD and the clustering decoder by several orders of magnitude in terms of accuracy (logical error rate). OSD takes a large number of cycles to complete, which explains its the plateau-like curve. Clustering can solve some decoding problems quicker than OSD, which is why it reaches a lower logical error rate for fewer cycles, but the floor of the OSD algorithm is lower than that of clustering for the gross code. For the two-gross code, clustering has a logical error rate floor that is close to that of OSD.

Overall, we find that the message-passing-based decoder Relay perform significantly better than OSD and clustering. The primary reason why clustering does not perform as well as OSD for large cycle budgets is that when clustering finds the solution of a valid cluster (which is also the decoded result of that cluster), it has no context of the priors

associated with the subgraph. This means that clustering often does not find the lowest weight error pattern that resulted in the set of defects contained within the cluster, which increases the logical error rate.

Decoder	Code	Flip-flops	LUTs	BRAMs	URAMs
OSD [16]	gross	25%	39%	25%	0%
Clustering	gross	9%	6%	25%	0.0%
Relay [191]	gross	7%	52%	1%	0%
Clustering	two-gross	69%	88%	53%	83%

Table 4.1: FPGA resource utilization estimates for different decoders and code types.

We estimate (without synthesis into an explicit FPGA implementation) the utilization of various FPGA resources, including flip-flops (FFs), two types of on-chip RAM (BRAM and URAM), and look-up tables (LUTs). Resource usage is reported as a percentage of the available capacity on a VU19P device (which contains 8,171,520 FFs, 2,160 BRAMs, 320 URAMs, and 4,085,760 LUTs), shown in Table 4.1.

4.5 Summary

We introduce an FPGA-tailored version of the algorithm in Ref. [61], which generalizes the Union-Find decoder [62] to arbitrary qLDPC codes. This decoder uses a systolic solver of linear systems of equations to test cluster solvability and extract local corrections, and incorporates hardware-specific optimizations to maximize throughput. It uses a pre-decoder: faults with high likelihoods are accepted immediately [38], while intermediate-likelihood faults are treated as erasures that seed the initial clusters together with unsatisfied syndrome nodes.

While the clustering decoder is severely outperformed by Relay, the divide-and-conquer approach of clustering presents interesting optimization avenues for future decoders since it significantly reduces the decoding problem size. Firstly, the growth stage of clustering could be optimized to grow non-uniformly by prioritizing nodes that are more likely to have experienced errors (higher values of p_j). Secondly, rather than just using the linear solver to find the decoded result of a cluster, a decoder like Relay can be used on the subgraph enclosed by a valid cluster. The advantage of doing this is that since clusters are generally

significantly smaller than the full decoding graph, decoders like Relay can achieve a far better solution than the linear solver very quickly.

The decoding work presented in this chapter sits between the two levels of the classical control stack explored in this dissertation. Below it, the low-level control and readout hardware described in Chapter 3 generates the syndromes that feed the decoder: the quality of those syndromes, and the rate at which they are produced, directly set the accuracy ceiling and the latency budget within which any decoder must operate. Above it, the quantum operating system must treat the decoder as a managed resource—deciding which decoder to invoke, how to allocate FPGA capacity across multiple logical qubits running concurrently, and how to synchronize logical qubits with one another when logical operations between them are required. These scheduling and synchronization problems form the subject of the next chapter.

Chapter 5

Systems for Quantum Computers

5.1 Introduction

In this chapter, we move further up the control system stack. In the previous chapter, we showed how syndromes generated by the readout system are decoded using the clustering decoder. However, we looked at a single decoder in isolation. This framing quietly carries an assumption—that a fault-tolerant quantum computer can be built by replicating these optimized components and letting them run independently.

That assumption breaks at scale. A practical fault-tolerant machine will not operate a single logical qubit but hundreds to thousands of them, each encoded in a large patch of physical qubits, each emitting a continuous stream of syndromes, and all of them participating in logical operations—lattice surgery merges and splits, magic state consumption, and code switching—that couple their execution together. The components we made fast and inexpensive in isolation now have to coexist within one machine, and in doing so they expose two problems that simply do not appear at the level of a single component: they contend for finite classical resources, and they must agree on when operations happen.

The first problem is resource contention. A real-time decoder, even an efficient one, is a substantial piece of classical hardware, and provisioning a dedicated decoder for every logical qubit¹ does not scale to machines with thousands of them. Decoder demand is also uneven and bursty across a program’s execution—many patches idle as logical memory while

¹For high-rate codes such as quantum LDPC codes, this would be a decoder per logical *block*. We treat the two similarly but make the distinction clear here.

a few drive an active logical operation—so a statically provisioned, one-decoder-per-qubit design pays for peak capacity that is rarely used. The decoders therefore become a shared, scarce resource that must be allocated across logical qubits over the course of a computation. This is, at its core, a scheduling problem.

The second problem is coordination. A logical operation requires the syndrome-generation cycles of the participating logical qubits to be aligned in time, yet a real system constantly drifts out of alignment: magic state production completes at data-dependent times, fabrication defects force some patches to deviate from a uniform cycle, and heterogeneous codes used to reduce overhead advance at different rates. Left unmanaged, the naive response is to idle the leading qubits until the rest catch up—but idling exposes them to additional rounds of physical error and inflates the logical error rate, eroding the very protection error correction is meant to provide. The system must instead keep concurrent logical qubits actively in step. This is a synchronization problem.

These problems are analogous to classical systems problems. For example, if every logical qubit’s/block’s syndrome generation cycle is considered a thread, then the any logical operation between two or more logical qubits will require the threads to be synchronized – similar to how critical sections in programs require synchronization between threads. Similarly, every thread requires access to compute for decoding – similar to how processes must be scheduled by the OS scheduler on a CPU. These parallels with classical operating system techniques is why we call this chapter “Systems for Quantum Computers”.

5.1.1 Contributions

The contributions of this chapter are summarized below:

- We introduce the notion of elastic decoders and study resource allocation of decoders for a full-fledged fault-tolerant quantum computer.
- We define the synchronization problem for performing fault-tolerant logical operations and present three primary synchronization policies.

5.1.2 Organization

This chapter is organized as follows:

- In Section 5.2, we provide relevant background on logical operations and windowed decoding.
- In Section 5.3, we show how decoders in a large-scale system can be provisioned and how decoding can be scheduled among them.
- In Section 5.4, we discuss the synchronization problem for FTQCs and define three synchronization policies.
- We summarize this chapter in Section 5.5.

5.2 Logical Operations and Windowed Decoding

In Chapter 2, we gave a high-level overview of logical operations. We expand on that background in this section and show how it impacts decoding.

5.2.1 Logical operations

In general, when a QEC code is used for encoding logical qubits, logical operations between those logical qubits are defined by the structure and restrictions imposed by the underlying code. Due to this dependence on the QEC code, there is no single recipe for performing logical operations; instead, a family of techniques has emerged, each suited to particular codes and hardware.

The first distinction within any set of logical operations is between single- and multi-qubit operations. Single-qubit logical operations are not interesting from a control systems standpoint since they are isolated to a single logical qubit. Multi-qubit logical operations on the other hand require interactions between logical qubits, which introduces interesting constraints and problems for the classical control system.

The most general method for performing multi-qubit logical operations is code surgery [50, 51]. However, if device and code constraints allow it, they can also some times be performed transversally. For example, logical CNOTs can be performed transversally on neutral atom systems for the surface code. The most popular variant of code surgery is called lattice surgery, which has been extensively studied for performing multi-qubit logical operations on the surface code. Due to its prevalence in QEC research, we focus on the surface code

and lattice surgery paradigm in this chapter, but note here that the ideas presented in the rest of this chapter will be applicable for other general code surgery methods as well.

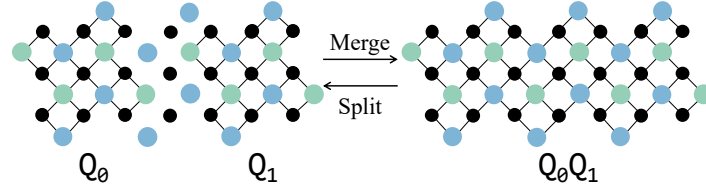


Figure 5.1: The fundamental split and merge operations of lattice surgery used for logical computation.

5.2.1.1 Lattice surgery

Lattice surgery [116] is the most common flavor of general code surgery and is restricted to 2D codes like the surface and color codes. Figure 5.1 shows the two primitives of lattice surgery – *merge* and *split*. A merge operation merges two or more logical qubits into a single logical qubit. A split operation splits a large logical qubit into two or more logical qubits. By performing a sequence of merge and split operations, a variety of logical operations can be performed to implement various gates and logical measurements.

The key insight underlying the lattice surgery paradigm is that a merge operation is equivalent, at the logical level, to a *Pauli product measurement* (PPM): merging two surface code patches along their X-boundary implements a joint measurement of the two-qubit logical operator $X_L^{(1)}X_L^{(2)}$, while merging along the Z-boundary measures $Z_L^{(1)}Z_L^{(2)}$. The merge takes d syndrome rounds to complete (where d is the code distance), after which the joint measurement outcome is available as a classical bit. A subsequent split then disentangles the merged patch, returning each logical qubit to an independent code block in a definite eigenstate of its local Pauli operator. Because any logical Clifford circuit can be decomposed into a sequence of single- and two-qubit PPMs [152], lattice surgery provides a universal basis for fault-tolerant logical computation on the surface code. The classical outcomes of these measurements are tracked in the *Pauli frame* and used to determine any Clifford corrections that must be applied to subsequent operations.

5.2.1.2 Logical measurements

A logical measurement extracts a classical bit from one or more logical qubits by measuring a Pauli observable on their code blocks. Single-qubit logical measurements are the simplest case: measuring all physical data qubits of a patch in the Z basis yields the logical Z_L outcome, and measuring in the X basis yields X_L . These measurements are destructive—the logical qubit is consumed in the process. Non-destructive logical measurements and measurements of joint multi-qubit Pauli operators are performed via lattice surgery: a merge couples the target logical qubit(s) to a fresh ancilla patch and extracts the PPM outcome, after which a split discards the ancilla.

The most important application of PPMs in the context of this dissertation is the implementation of the logical T_L gate, which requires consuming a magic state. As described in Section 2.8.6, the T_L gate cannot be implemented transversally on the surface code and instead requires a pre-distilled ancilla $|A_L\rangle = T_L|+_L\rangle$. The injection protocol is shown in Figure 5.2: a joint PPM of $Z_L \otimes X_L$ is performed between the data logical qubit and the magic state ancilla. In the surface code, this joint measurement is realized by a lattice surgery merge of the data patch with the magic state patch along the appropriate boundary. The measurement outcome m is a classical bit: if $m = 1$, a logical Clifford correction S_L must be applied to the data qubit before the next operation. Since S_L is a Clifford gate it can be tracked in the Pauli frame rather than physically applied, but its effect must be resolved before the next T_L gate in the circuit is injected—this is the hard decoder latency requirement discussed in Section 4.2.

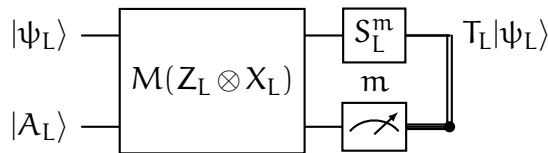


Figure 5.2: Logical T_L gate via Pauli product measurement. A joint measurement of $Z_L \otimes X_L$ is performed on the data logical qubit and the magic state ancilla $|A_L\rangle = T_L|+_L\rangle$. In the surface code, this PPM is implemented by a lattice surgery merge of the two patches. The classical outcome m determines whether a Clifford correction S_L^m must be applied to the data qubit.

5.2.1.3 Logical CNOTs

A logical CNOT is a two-qubit logical gate and the canonical example of an operation that requires physical interaction between two separate code blocks. In the lattice surgery paradigm, a logical CNOT between a control logical qubit A and a target logical qubit B is decomposed into three sequential PPMs using a fresh ancilla patch C initialized in $|+\rangle_L$ [152]:

1. Measure $X_L^{(A)} \otimes X_L^{(C)}$ (merge A and C along X-boundary)
2. Measure $Z_L^{(C)} \otimes Z_L^{(B)}$ (merge C and B along Z-boundary)
3. Measure $Z_L^{(C)}$ (split and discard ancilla)

Based on the three classical outcomes m_1, m_2, m_3 , Pauli frame corrections $X_L^{m_1}$ on B and $Z_L^{m_2}$ on A are applied, implementing the logical CNOT deterministically.

Because the logical CNOT couples the syndrome generation cycles of two independent code blocks, both patches must be at a compatible point in their respective QEC cycles before the merge can begin. If one patch is ahead of the other, the leading qubit must idle—running additional rounds of syndrome measurement without advancing the computation—until the lagging patch catches up. Idling is not free: each additional QEC round exposes the idling qubit to further physical errors, increasing the logical error rate. Managing this alignment between interacting logical qubits is the *synchronization problem*, which we discuss in depth in Section 5.4.

5.2.2 Windowed decoding

Logical qubits continuously generate syndromes that must be decoded. Since the syndrome stream is unbounded in time, it is not possible to collect all syndromes before decoding begins; instead, decoding is performed on *windows* of syndromes. Each window of size n_W syndrome rounds forms a three-dimensional decoding volume—two spatial dimensions covering the physical qubit layout of the patch, and one temporal dimension spanning the n_W rounds—visualized as a block in Figure 5.3(a). A block is divided into a *commit region* and surrounding *buffer regions*. Buffer regions are necessary because errors occurring near a window boundary can create syndrome defects that span two adjacent windows; the buffer provides enough context on each side for the decoder to correctly identify such errors.

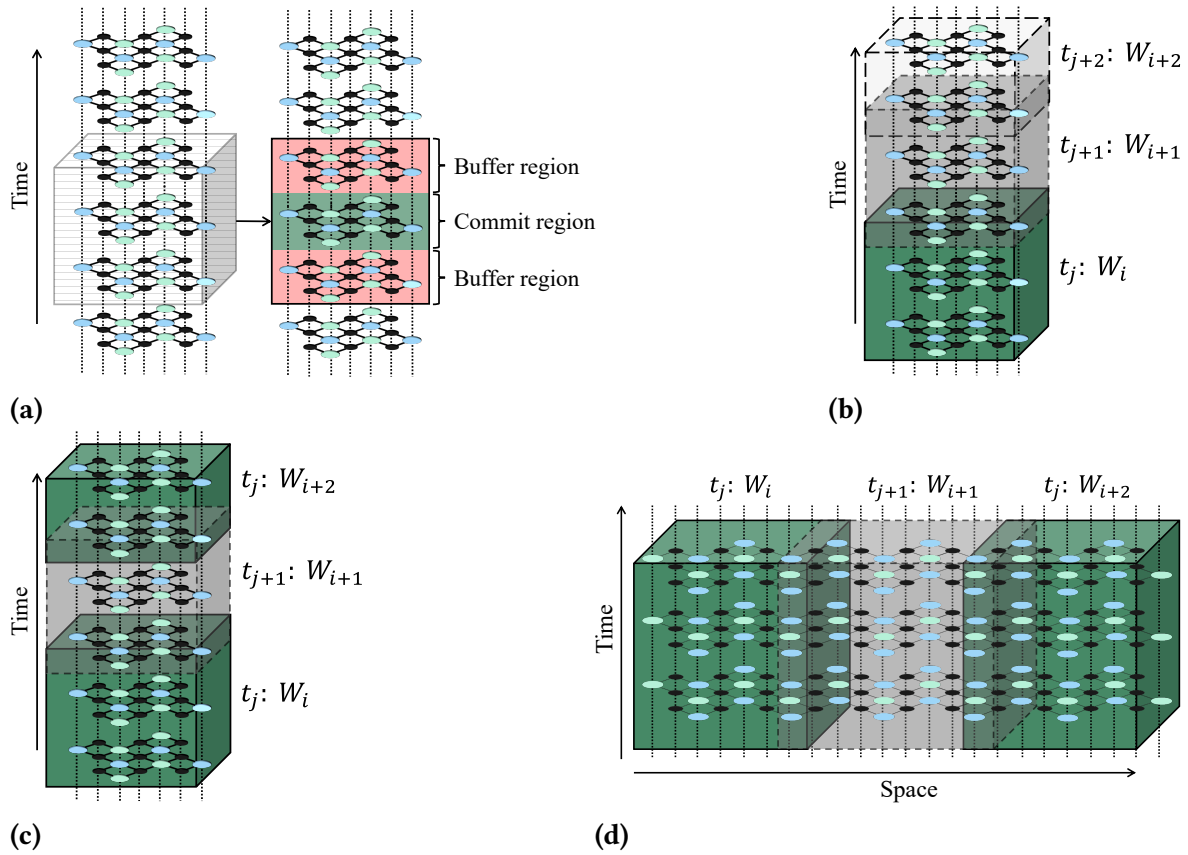


Figure 5.3: (a) Streams of syndromes are processed in windows represented by *blocks* which are fixed decoding volumes: each block consists of a region in which corrections are committed, while buffer regions are used for merging corrections with overlapping windows; (b) Sliding windowed decoding (SWD) is inherently sequential: window W_{i+1} can only be processed after W_i has been processed; Parallel windowed decoding (PWD) can be (c) temporal and/or (d) spatial: in either case, W_i, W_{i+2} are treated as independent tasks and processed independently in parallel at time step t_j . W_{i+1} , which includes overlaps with W_i, W_{i+2} , is processed in the next time step.

Corrections fully contained within the commit region are finalized and applied to the Pauli frame, while corrections in the buffer are deferred until the overlapping window is decoded. The size of the buffer is typically chosen to be at least $d/2$ rounds, so that any error chain of weight less than d is fully contained within at most one commit region. With this structure, the continuous syndrome stream can be decoded using one of the following paradigms.

5.2.2.1 Sliding windowed decoding (SWD)

SWD is inherently sequential, as shown in Figure 5.3(b). At time step t_j , the decoder processes window W_i . Once complete, the next window is processed at the following step. The overlap between W_i and W_{i+1} corresponds to their buffer regions.

5.2.2.2 Parallel windowed decoding (PWD)

PWD [233] extends SWD by exploiting temporal (Figure 5.3(c)) and spatial (Figure 5.3(d)) parallelism. In temporal PWD (Figure 5.3(c)), non-adjacent windows such as W_i and W_{i+2} share no buffer region and are therefore fully independent decoding tasks; they can be dispatched to two separate decoders simultaneously. Window W_{i+1} , which overlaps with both, is processed in the subsequent time step after its neighbors' buffer corrections are committed. Spatial PWD (Figure 5.3(d)) applies the same idea across the spatial extent of a large patch—for example, one formed by a lattice surgery merge—decomposing its decoding graph into independent sub-volumes that can be decoded in parallel [89].

PWD increases throughput at the cost of additional decoder hardware. For k simultaneously active windows, k decoders are required. In general, decoder requirements grow linearly with the number of independent decoding tasks in both the temporal and spatial dimensions. A key observation is that decoder demand is not uniform across the lifetime of a computation: patches participating in an active lattice surgery operation require more parallel decoding capacity than patches idling as logical memory. This non-uniformity is precisely what creates the opportunity for *decoder sharing*—the subject of Section 5.3—where a pool of decoders is dynamically allocated across logical qubits based on their instantaneous demand.

5.3 Decoder Provisioning and Scheduling

The first key question we want to answer in this section is: "Given an FTQC system and a workload to run on it, how many decoders will be needed?". Since fast technologies like superconducting qubits require fast, accelerated decoders built on FPGAs/ASICs, this is a real, hardware resource provisioning problem². Related to, but orthogonal to this question is the second key question we want to answer in this work: "Given a fixed number of available decoders in an FTQC system, how will decoding be scheduled across decoders and the logical qubits requiring them such that there are no program slowdowns?".

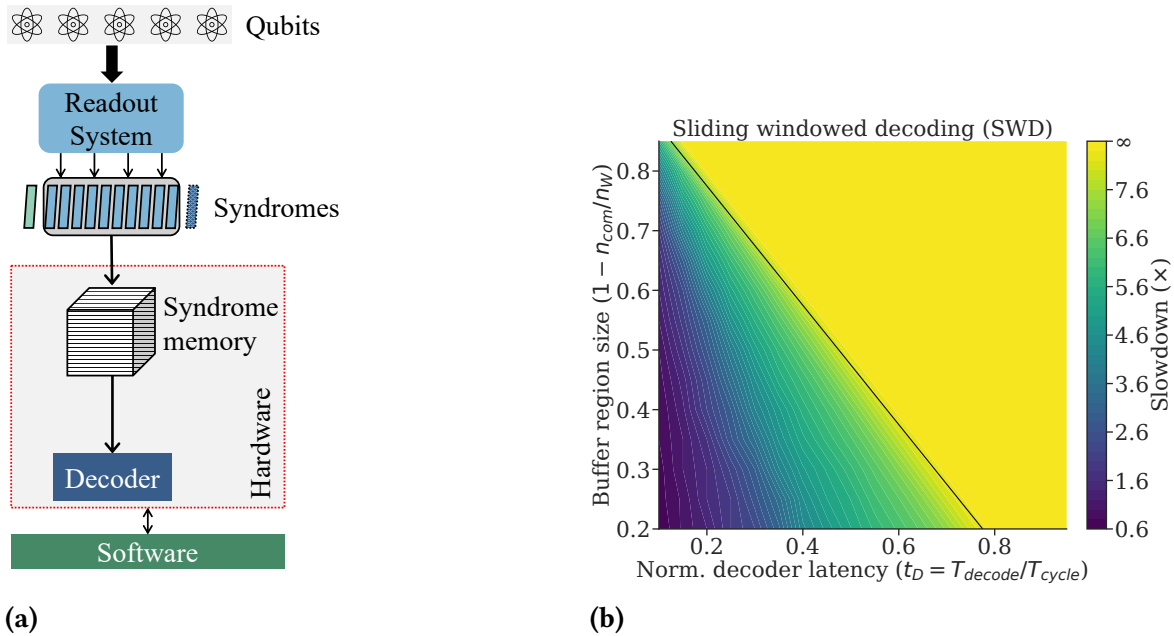


Figure 5.4: (a) A full system using hardware (FPGA/GPU/ASIC) decoders: qubits are measured using the readout system, which produces syndromes that are buffered until a window has been collected. Then, the decoder uses these collected syndromes to determine a correction, which is communicated to software; (b) Effect of the normalized decoder latency t_D on the slowdown in processing 5d rounds of syndromes using SWD. The buffer region size is the normalized size of the buffer regions required ($n_W = 3d, n_{com} = d$ would yield an overlap of 0.66).

²This work [174] was done in collaboration with Abtin Molavi, Aws Albarghouthi, and Swamit Tannu. Abtin Molavi helped set up the lattice surgery compiler to get the layout-dependent patch sizes and the impact of routing conflicts.

The requirement for syndromes to be processed faster than they are generated has motivated research to build fast and accurate hardware decoders for the surface code [8, 13, 213, 237, 265, 273, 284], especially for systems using superconducting qubit architectures due to their fast gate times. GPU-accelerated decoding has also been demonstrated for neutral atom systems [199]. Figure 5.4(a) shows the general structure of a real-time decoding system that uses accelerated hardware decoders. Syndromes are generated after readout, and these syndromes are buffered until a decoder can operate on them. Such hardware systems are necessary for ensuring that the backlog problem is avoided [254]. Figure 5.4(a) also shows that T_{decode} is dependent on not just the actual decoder latency but also the communication/memory access latencies required for storing and providing the syndromes to the decoder.

5.3.1 Decoder allocation is non-trivial

5.3.1.1 SWD is not scalable

Decoding in the SWD paradigm is inherently sequential. This makes SWD too slow and fundamentally unscalable for large systems. Figure 5.4(b) illustrates this issue for $5d$ rounds of pending syndromes from a single logical qubit. The figure shows slowdown as a function of buffer region size and normalized decoder latency $t_D = T_{\text{decode}}/T_{\text{cycle}}$. Slowdown accounts for both the initial $5d$ rounds of pending syndromes and the additional syndromes generated during their processing; we measure the time required for the decoder to catch up to the live syndrome stream. Buffer regions are expressed in multiples of code distance d , so a buffer of size d corresponds to d rounds of overlap between adjacent windows.

Larger buffer regions improve accuracy in windowed decoding [89, 233], but also increase slowdown. The results are clear: SWD avoids the backlog problem only for unrealistically small buffer sizes and very optimistic t_D . For instance, $t_D = 0.2$ in a superconducting system with $T_{\text{cycle}} \simeq 1 \mu\text{s}$ requires an average decoder latency of 200 ns — far below what any proposed hardware decoder has achieved. When $t_D > 0.8$, the slowdown diverges, meaning the decoder can never catch up with the incoming syndrome stream.

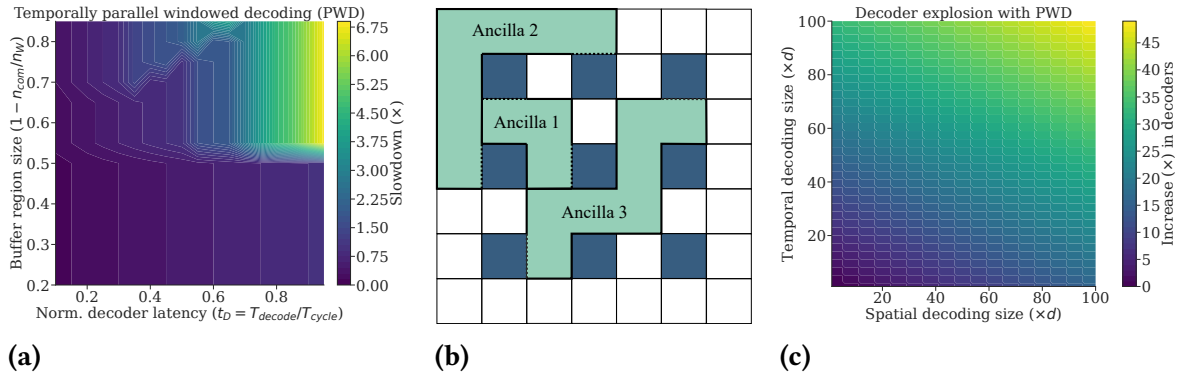


Figure 5.5: (a) Using simulations, effect of the normalized decoder latency t_D on the slowdown in processing 5d rounds of syndromes using Temporally parallel windowed decoding (PWD); (b) Logical qubits arranged in the EDPC layout with ancilla bridges of varying sizes to facilitate LS operations; (c) PWD is applicable in both space *and* time – this leads to an a variable and sudden increase in the number of decoders required for implementing PWD (assuming $t_W = 3d$).

5.3.1.2 PWD introduces non-determinism

Figure 5.5(a) shows the same initial problem used for the SWD paradigm in Figure 5.4(b): even for values of t_D close to one, the slowdown is still finite. This example clearly highlights the utility and scalability of the PWD paradigm – Figure 5.5(a) would be the same for any initial size of pending syndromes.

This instance of PWD exploits *temporal* parallelism: windows in time are executed in parallel to increase decoder throughput. However, PWD can also have a spatial interpretation. We now discuss how the spatial interpretation of PWD is non-deterministic and its interplay with the temporally PWD.

The impact of gate routing. Logical operations in the surface code are performed via lattice surgery, where qubits are merged through ancilla regions that form routing paths. The size of these ancilla bridges varies, leading to unpredictable decoding volumes. Figure 5.5(b) illustrates this in the EDPC layout [21], where blue squares are logical qubits and white squares are ancillas. Ancilla 1 connects nearby qubits with a short bridge of length 3. Ancilla 2 connects similarly adjacent qubits but requires a longer path of length 7 to avoid Ancilla 1. Ancilla 3, unaffected by others, also has length 7 since it spans distant qubits. Such routing conflicts and varying patch sizes create non-deterministic decoder demands,

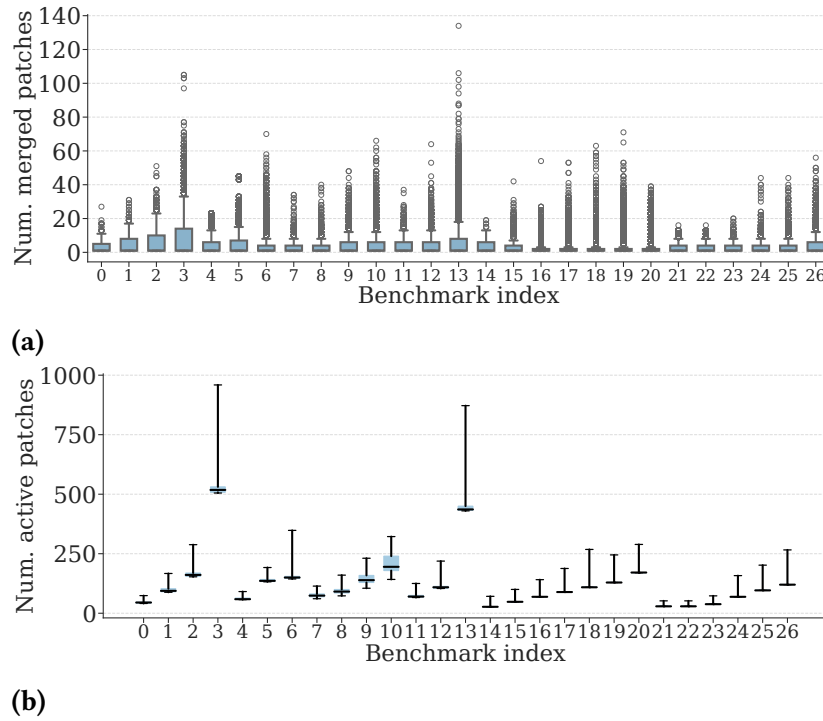


Figure 5.6: (a) Distribution of the number of patches merged to perform lattice surgery-based logical measurements for different workloads – most workloads require short-range interactions on average, but some large outliers are possible due to routing constraints; (b) Distribution of the decoder demand over time for different benchmarks – these error bars capture the fluctuations in active logical patches that require decoding during the computation.

as the number of required decoders scales linearly with patch size.

Interplay of temporal and spatial requirements. Consider the logical program shown in Figure 5.2. This lattice surgery operation could be between qubits that have either a large temporal decoding task (pending syndromes), a large spatial decoding task (due to the physical distance and the subsequent routing between them), or both. We attempt to capture this interplay between the spatial and temporal decoder requirements in Figure 5.5(c). In the simplest case, both qubits do not have any pending syndromes, and they are next to each other on the device, which means that they need at most a decoder each to decode all generated syndromes. However, if there is a spatial and/or temporal element to the decoding task, the number of decoders required can explode to up to $50\times$, just for two logical qubits!

5.3.1.3 Need for decoder scheduling

In the sections above, we showed that the number of decoders required under the PWD paradigm can be highly variable and, in some cases, untenable when both spatial and temporal decoding demands grow. Figure 5.6(a) highlights this variability in the spatial dimension: different benchmarks exhibit a wide range in the number of merged patches, with significant outliers. This directly affects the number of logical qubits active at any given time, as illustrated in Figure 5.6(b), which shows the distribution of active qubits across benchmarks. Since every logical qubit continuously produces syndromes that must be decoded, the key challenge becomes:

How do we allocate decoders for benchmarks with such diverse and dynamic demands?

The simplest allocation strategies are to provision (a) for the worst-case demand (i.e., the maximum number of active qubits from Figure 5.6(b)), or (b) for the average-case demand. Option (a) is prohibitively resource-intensive, since each decoder may require nearly all the resources of a single FPGA [273, 284]. Option (b) risks substantial slowdowns: spatial decoding tasks of large size would be delayed, reducing throughput and degrading program performance.

Thus, static allocation is insufficient. The problem of decoder allocation is better viewed as a *scheduling problem*, where a finite pool of decoders must be dynamically assigned to decoding tasks over time. Like CPU scheduling in classical computers, decoder scheduling must balance throughput, latency, and fairness, while avoiding bottlenecks from workload spikes.

5.3.2 Coarse-grained scheduling

We define decoder scheduling at two levels—coarse-grained and fine-grained—as illustrated in Figure 5.7. Coarse-grained scheduling prioritizes critical decodes and resolves conflicts between spatial and temporal tasks on the same qubits. For example, in Figure 5.7, the spatial decode on qubits Q_0, Q_1 from a lattice surgery operation takes precedence over the temporal decode. After coarse-grained allocation, remaining decoders are assigned to other qubits. In the figure, Q_2 and Q_N receive decoders while Q_3 does not, demonstrating

fine-grained scheduling, which selects among tasks from individual qubits. Because critical and spatial decodes impose rigid constraints, we adopt a single coarse-grained policy to maximize utilization, while fine-grained policies that operate in tandem.

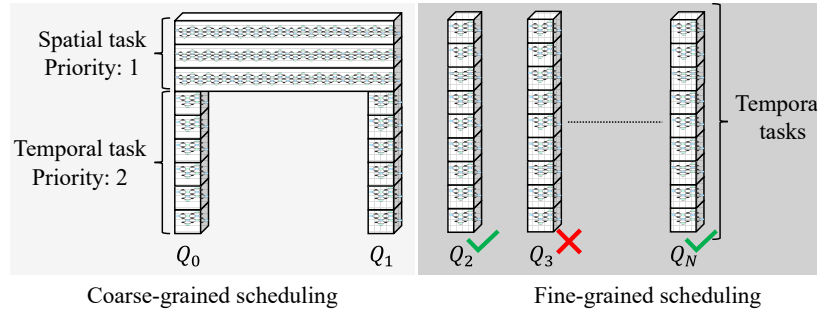


Figure 5.7: Distinction between coarse- and fine-grained scheduling. (Left) Coarse-grained decoder scheduling handles the prioritization between spatial and temporal decoding tasks. This includes handling critical decodes, which will always have some spatial component to them due to lattice surgery. (Right) Any decoders left over after handling critical decoding tasks are assigned to the temporal decoding tasks associated with logical qubits not involved in any operation in that time step. Scheduling decoders among this subset of qubits is what we refer to as fine-grained scheduling.

5.3.2.1 Decoding space-time volumes

Figures 5.8(a), 5.8(b), and 5.8(c), show space-time decoding volumes for three distinct scenarios under the PWD paradigm. Decoding volumes could be simple in both space and time, or require multiple decoders for a large volume in time, or have a large volume in *both* space and time. Each case shows how the decoding volume can be different in the same program, which requires decoders to be *elastic* such that an increase or decrease in the demand can be dealt with without any detrimental effects on the performance of a program.

5.3.2.2 Prioritizing between spatial and temporal parallelism

As shown in Figure 5.8(c), lattice surgery can require decoders for both spatial and temporal parallelism, raising a key scheduling question: **Which type of parallelism should take priority when both are needed simultaneously?**

Consider first the effect of decoder scarcity on spatial PWD. Suppose a large patch requires three decoders: the first layer uses all three for non-overlapping windows, while

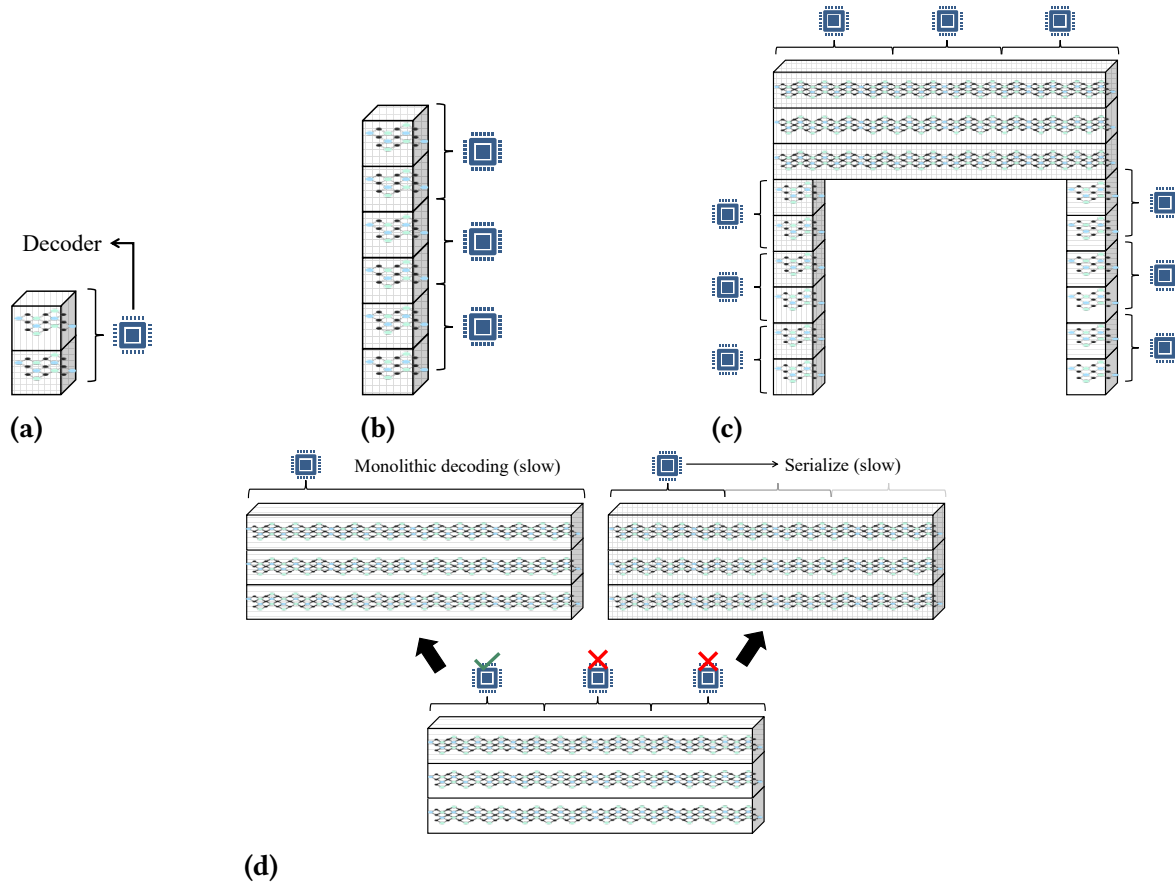
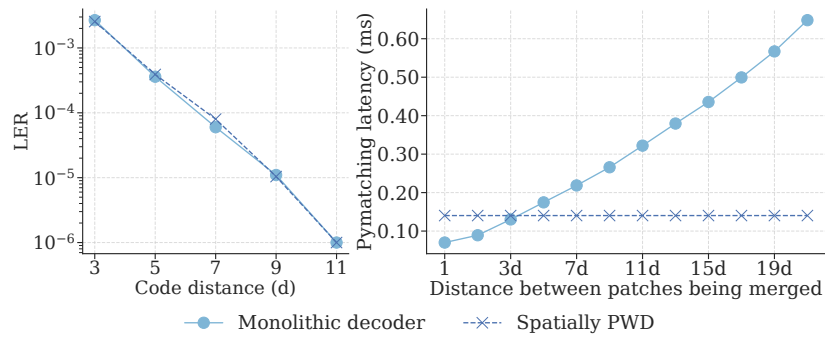
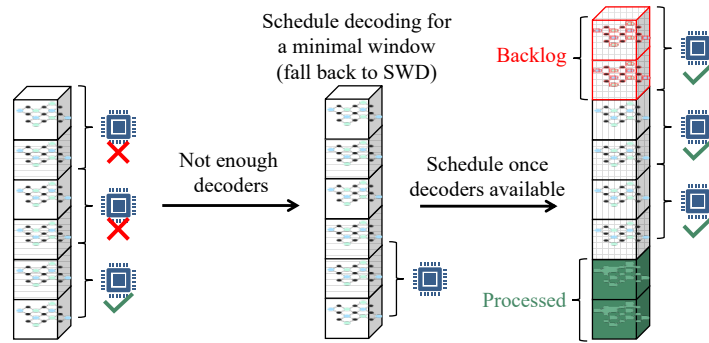


Figure 5.8: (a) Decoding volume of a single logical patch idling for a few rounds—only one decoder is needed; (b) Idling for multiple rounds creates a linear growth in windows, requiring temporally parallel windowed decoding (PWD); (c) A lattice surgery merge between distant patches temporarily increases decoder demand due to spatial parallelism, which may coincide with temporal parallelism (overlapping windows omitted for clarity); (d) Spatial PWD is not robust to decoder shortages—if two of three required decoders are unavailable, one must either decode the merged patch monolithically or serialize the protocol, both slow and undesirable. Deferring the task is also problematic, as the next slice may contain a critical decode that must wait for the deferred task, further slowing execution.

the second layer reuses two to handle overlaps and boundary errors [27, 89]. If only one decoder is available, the protocol cannot proceed as designed, forcing one of three fallbacks: (i) switch to a monolithic decoder, (ii) serialize window decoding, or (iii) defer decoding to the next slice. Option (iii) risks slowdown if followed by a critical non-Clifford operation. As Figure 5.9 shows, spatial PWD and monolithic decoding yield similar logical error



(a)



(b)

Figure 5.9: (a) (Left) Logical error rates (LER) for various lattice surgery configurations with and without spatially parallel window decoding (PWD) at circuit-level noise $p = 0.1\%$. (Right) Spatial parallelism does not affect LER but reduces decoder latency when merged patches are separated by $\geq 3d$ data qubits, assuming sufficient decoders for all distances (for $d = 11$, window size = $3d$). (b) Temporal PWD remains robust under decoder resource constraints: with only one of three decoders available, the decoding task can be processed incrementally by the single decoder. This creates a qubit backlog, which can be rapidly cleared once additional decoders become available via temporal parallelism.

rates (LERs), but monolithic decoding incurs far higher latency for large patches³. While demonstrated using PyMatching [114], the same trends hold for hardware decoders. Thus, fallback options (i) and (ii) degrade performance, and spatial PWD loses its latency advantage without concurrent decoder availability.

In contrast, temporally PWD is more resilient to decoder constraints. As a parallelized form of sliding windowed decoding (SWD), it can always fall back to SWD when decoders are

³Spatial PWD is only beneficial beyond $3d$ data qubits (or ~ 3 patches); smaller patches are treated as monolithic in our evaluations.

scarce. Available decoders are simply assigned to the oldest undecoded windows, ensuring correctness. Once more decoders free up, temporally PWD clears accumulated backlogs in *constant time*, provided sufficient parallelism is available. This adaptability is its main strength: **as long as delays do not exceed the system’s recovery capacity, efficient scheduling remains feasible despite temporary decoder shortages.**

5.3.2.3 Distinguishing between spatial decoding tasks

Lattice surgery operations that require spatially PWD can either be due to critical decodes arising due to non-Clifford gates, or due to other operations between Y-states [267]. Critical decodes must be serviced, which is why those spatial decoding tasks are given the highest priority, followed by any temporal decoding tasks needed for the critical decode. If there are non-critical spatial decoding tasks required at the same time step, they are given a priority lower than critical decodes but higher than that of decodes needed by logical qubits not involved in any operation in that time step. This allows spatial decoding tasks, critical or not, to be prioritized over all other temporal decoding tasks.

5.3.3 Fine-grained scheduling

In this section, we define fine-grained decoder scheduling policies. These policies act on decoders that are available after all other higher priority spatial decoding tasks have been allocated decoders. These fine-grained policies are similar to process scheduling policies used by modern operating systems where decoding tasks of individual qubits are analogous to processes and decoders are analogous to CPU cores.

5.3.3.1 Connection with real-time scheduling systems research

Decoder scheduling maps naturally to a **mixed-criticality, soft real-time model** [15, 263]: decoding windows are sporadic tasks with bursty arrivals; critical decodes (from non-Clifford gates) impose firm deadlines on dependent computation; non-critical decodes tolerate bounded delay; and the decoder pool represents a set of identical parallel processors. Our two-level scheduler mirrors hierarchical scheduling architectures [58]: coarse-grained scheduling handles criticality levels (critical > spatial > temporal), analogous to priority-based admission control; fine-grained policies allocate residual capacity among admitted tasks for backlog control.

While conventional real-time systems research often provides hard schedulability guarantees (e.g., rate-monotonic analysis [157]), this approach is unattainable for QEC decoders due to fundamental differences:

1. **Non-stationary arrivals:** Task generation depends on dynamic program structure—lattice surgery operations, gate scheduling, and routing decisions—rather than fixed periodic or sporadic arrival models with known minimum inter-arrival times.
2. **Data-dependent execution:** Decoding latency varies with syndrome patterns and error distributions, making tight worst-case execution time (WCET) bounds impractical. Even for fixed code distance, execution time can vary 2-3× based on error chain complexity.
3. **Elastic resource requirements:** Spatial decoding tasks require anywhere from 1 to 50+ decoders depending on patch size and routing distance—a form of moldable parallelism [171] uncommon in traditional real-time systems.
4. **Temporal dependencies:** Overlapping decoding windows create precedence constraints between consecutive tasks on the same logical qubit, violating the task independence assumption central to most schedulability analyses.

Despite these constraints, certain scheduling principles from queuing theory and soft real-time systems remain applicable. Work-conserving policies that account for task backlog and waiting time can prevent starvation and control queue growth under bursty, non-stationary arrivals [252]. Similarly, policies that prioritize tasks based on accumulated service deficits can provide fairness guarantees without requiring detailed arrival models [228]. These insights inform our fine-grained policy design, which we describe next.

5.3.3.2 Scheduling for individual qubits

Fine-grained scheduling determines when each logical qubit’s syndromes (current and pending) are provided access to a decoder. If a policy starves qubits of decoding, its measured syndromes will accumulate, leading to longer decoding latencies (since the total size of the decoding problem increases with time). This thus necessitates fine-grained policies to ensure fair access to decoders for all qubits that are not involved in lattice surgery operations.

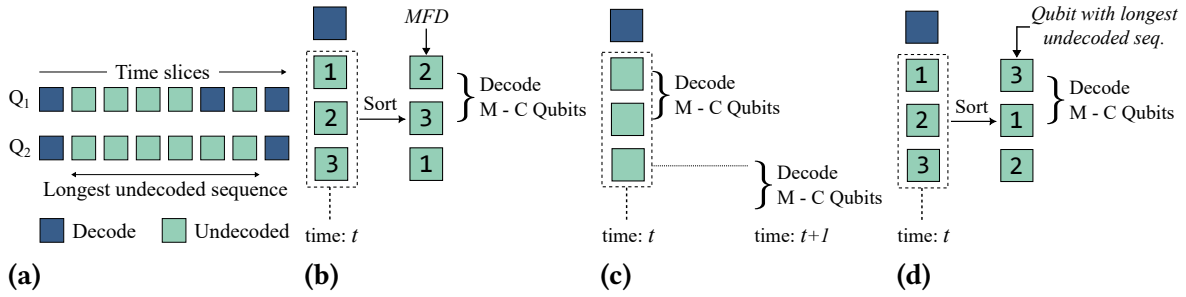


Figure 5.10: (a) Illustration of the longest undecoded sequence – Q_1 has the longest undecoded sequence before the last decode (each square is a decoding block/window); (b) MFD policy – undecoded qubits are sorted according to the number of critical decodes they are involved in after time slice t and $M - C$ qubits are selected from this sorted list; (c) RR policy – the $M - C$ qubits decoded in time slice t are not decoded in time slice $t + 1$; (d) MLS policy – undecoded qubits are sorted according to their undecoded sequence lengths and $M - C$ qubits are selected from this sorted list.

Longest undecoded sequence. To quantify the fairness of a fine-grained scheduling policy, we use a metric ‘*Longest Undecoded Sequence*’, which measures how well the decoders are servicing all logical qubits. A large undecoded sequence length implies that a qubit has been left undecoded for a long time. A qubit left undecoded for a long time will also require more processing from the decoder for it to be up to date with the latest syndrome. Figure 5.10(a) shows an example of determining the longest undecoded sequence length.

Consider an arbitrary time slice t in the execution of a quantum program. There are N logical qubits and M hardware decoders ($N > M$). All decoding scheduling policies will have two components: The first will assign the decoders necessary for all critical decodes C in the time slice t . The second will assign all the remaining $M - C$ hardware decoders to the $N - C$ qubits based on the scheduling policy used. We now discuss three decoder scheduling policies (all policies are illustrated in Figure 5.10(b) – Figure 5.10(d)):

Most frequently critically decoded (MFD). A logical qubit that consumes a significant number of T-states during the execution of a program would have a frequent requirement of critical decodes – leaving such a logical qubit undecoded for more than a few slices would make subsequent critical decodes take longer, thus slowing down computation. This motivates the MFD scheduling policy that prioritizes decoding of logical qubits that have numerous critical decodes in the future at any given time (Figure 5.10(b)). The MFD policy will ensure that future critical decodes have a minimized number of undecoded syndromes

for qubits with frequent critical decodes. The MFD policy is predicated on quantum programs being static in nature. While logical operations to perform S corrections during magic state consumption represent ‘dynamic’ instructions, they do not add any additional non-Clifford gates to the program, thus ensuring that any statically compiled program using the MFD policy will always work.

Round-robin (RR). Derived from CPU scheduling policies used in operating systems, the RR policy does not prioritize any specific logical qubits – rather, it chooses $M - C$ qubits in a round-robin manner (Figure 5.10(c)) in every time slice to ensure fairness for all qubits in the system. This policy was also alluded to in prior works [36, 55] for scheduling decoders for different logical qubits.

Minimize longest undecoded sequence (MLS). The longest undecoded sequence length at any time slice is an indicator of how well the fine-grained scheduling policy is servicing all qubits in the system. We use this as the base for the MLS policy, which greedily minimizes the longest undecoded sequence at every time slice (Figure 5.10(d)). The MLS policy works as follows: at any time slice t , qubits are sorted on the basis of their current undecoded sequence lengths. Then, $M - C$ qubits with the largest undecoded sequence lengths are assigned hardware decoders. The MLS policy mimics longest-queue-first (LQF) scheduling [70], known to prevent starvation and control backlog growth under bursty demand without strong arrival distribution assumptions.

5.3.3.3 Magic state cultivation factories

Magic state cultivation [98] is a protocol that generates high-quality non-Clifford (magic) states by using one or more smaller, more error-prone logical qubits. These cultivation protocols do not require high-performance decoders during the state generation process. Instead, they rely on **post-selection**, where attempts are *discarded* as soon as a syndrome bit-flip is detected. Post-selection is well-suited for non-deterministic state preparation, providing exponential error suppression at the cost of success probability [98, 153]. While some protocols support active error correction [41], we assume fully post-selected cultivation protocols for our evaluation, resulting in no decoder overhead during factory operation. However, the final output magic state is stored in a high-distance logical qubit for long-term

use, and thus requires decoding to ensure reliability after successful preparation. We include these magic state storage patches in our analysis.

5.3.4 Evaluations

5.3.4.1 Methodology

Compiler. We use the Lattice Surgery Compiler (LSC) [267] to generate Lattice Surgery Instructions (LLI) Intermediate Representations (IR) of workloads that can be executed on an error-corrected quantum computer using the Surface code with Lattice Surgery. LSC can generate IR that denote Lattice Surgery instructions from the QASM [52] representation of a workload. The Lattice Surgery instructions generated by LSC are a combination of Clifford operations and multi-body measurements used for implementing Pauli-Product Rotations (PPR) [152]. LSC handles mapping and routing based on the layout provided to the compiler. We configure LSC to use a ‘wave’ scheduling that maximizes the number of concurrent instructions executed in every time slice. LSC uses Gridsynth [219] to synthesize arbitrary rotations.

Layout. We perform all evaluations using the Edge-Disjoint Path Compilation layout (EDPC) [21] with a single routing lane.

Magic states. LSC abstracts distillation factories away – it is assumed that the magic state storage sites arranged along the borders of the EDPC layout get magic states within a specified time period. We assume that the magic state production throughput is high enough for generating magic states every alternate slice.

Other configurations. Y-states are produced using the twist-based initialization [95].

Simulation framework. Using the IR generated by LSC, we build a framework [221] that can parse the IR, determine the critical decodes in every slice, generate a timeline of all operations, and assign decoders to all logical qubits depending on the scheduling policy.

Benchmarks. We select benchmarks that are representative of large-scale programs that will run on FTQC systems. A majority of these will be building blocks for larger algorithms.

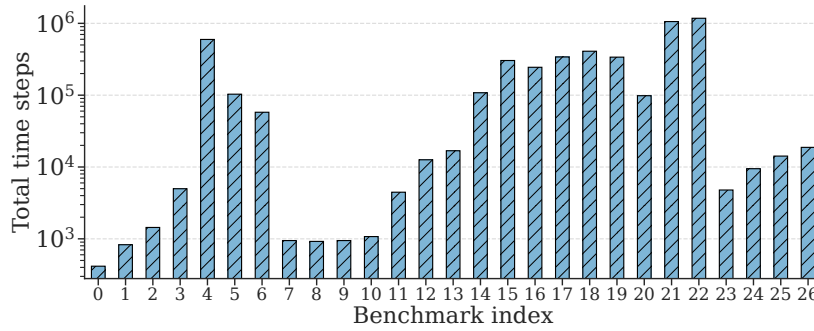


Figure 5.11: Total slices executed for each benchmark.

We use the following benchmarks (benchmark indices annotated in parentheses, benchmark size in terms of number of qubits specified after the hyphen):

adder-28 (0), adder-64 (1), adder-118 (2), adder-433 (3), bwt-37 (4), bwt-97 (5),
 ↪ heisenberg-100 (6), ising-34 (7), ising-42 (8), ising-66 (9), ising-98 (10),
 ↪ multiplier-45 (11), multiplier-75 (12), multiplier-350 (13), qft-20 (14),
 ↪ qft-40 (15), qft-60 (16), qft-80 (17), qft-100 (18), qft-120 (19), qft-160 (20),
 ↪ shor-9 (21), shor-15 (22), wstate-20 (23), wstate-40 (24), wstate-60 (25),
 ↪ wstate-80 (26)

Other software. Stim [97] was used for simulating stabilizer circuits to generate syndromes and error rates.

5.3.4.2 Benchmark statistics and baselines

Benchmark runtimes. LSC was configured to timeout after 10 minutes to keep simulation times tractable. Figure 5.11 shows the number of time steps for all benchmarks.

Concurrent critical decodes. Since all scheduling policies prioritize decoder allocation for any critical decode in a slice, the concurrency of critical decodes plays an important role in determining the number of decoders required for a benchmark. Figure 5.12 shows the difference between the peak and average critical decode concurrency. This highlights the importance and need for performant decoder scheduling policies – the limited concurrency of critical decodes in quantum programs makes efficient scheduling of decoders important for high decoder utilization.

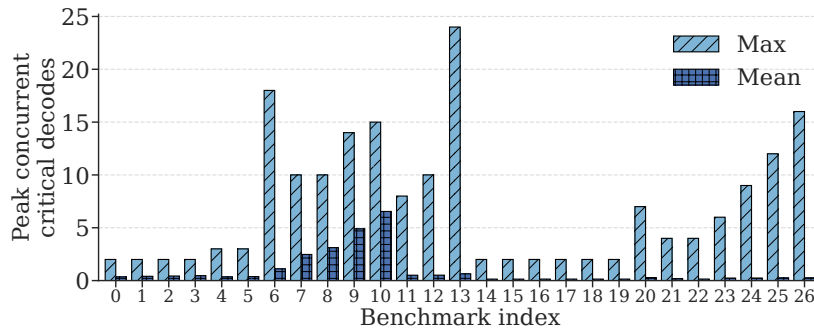


Figure 5.12: Peak and average concurrency for non-Clifford operations for all benchmarks – concurrency is sporadic.

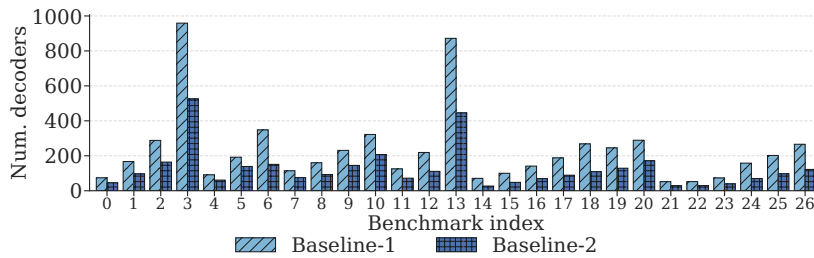


Figure 5.13: Number of decoders allocated for the baselines: Baselines-1 and 2 provision for the worst- and average-case decoder demands respectively.

Baselines. We assume a baseline system to allocate decoders for every logical qubit. Since the number of active logical qubits can vary through the course of a program, we consider the following baselines:

1. Baseline-1: Provisions for the worst-case number of active logical qubits.
2. Baseline-2: Provisions for the average-case number of active logical qubits.

Baseline-1 caters for all possible qubits that could be active for a given benchmark, and thus requires a very simple scheduling policy wherein syndromes for a logical qubit are passed to its pre-assigned decoder. On the other hand, Baseline-2 will require some scheduling policy for choosing which logical qubits are decoded at any given instant. We consider the Round-Robin (RR) policy as the baseline policy since its use has been suggested in prior works that allude to scheduling decoders to reduce resource overheads [36, 55]. Figure 5.13 shows the number of decoders configured for the two baselines for all benchmarks. As is to be expected, Baseline-1 requires significantly more decoders than Baseline-2.

5.3.4.3 Scheduling for the PWD paradigm

In our simulations of the PWD paradigm, we first assume that the normalized decoder latency is fixed at $t_D = 0.5$. This is in line with existing high-performance hardware decoders [8, 284]. Further research will likely reduce this latency further. We assess the impact of t_D on the simulation results in subsequent sections. For all evaluations, we assume both temporal and spatial windows have the same size of $n_W = 3d$. We use the coarse-grained policy presented in Section 5.3.2 in all evaluations and compare the MFD, RR, and MLS fine-grained scheduling policies with each other.

Minimum number of decoders needed to prevent any slowdowns. We first determine the minimum number of decoders required for all fine-grained policies and benchmarks to prevent any slowdowns in the computation. We determine this optimal number by performing a binary search on the number of decoders such that the final backlog for all logical qubits is zero.

Figure 5.14(a) shows the reduction in the number of decoders required when the optimal number is determined for the MLS policy relative to the two baselines. Compared to Baseline-1, the optimal decoder count is about **40% lower** than the counts determined for Baseline-1 across most workloads. However, compared to Baseline-2, the optimal decoder count is greater by up to 60% – highlighting that the optimal number of decoders must be more than the average-case number of logical qubits active for any workload.

Baseline-2 introduces significant slowdowns. As Baseline-2 has fewer decoders allocated than the optimal number, it is expected that Baseline-2 will incur slowdowns in computation due to decoder scarcity, regardless of the scheduling policies used. Figure 5.14(b) shows the slowdowns incurred by Baseline-2 across all benchmarks when the MLS and RR policies are used. For the RR policy, **the slowdown can be up to 10×**, highlighting the sub-optimality of Baseline-2 compared to the optimal number of decoders. Unsurprisingly, the slowdowns are higher for benchmarks which have a greater difference between the optimal number of decoders and the number allocated for Baseline-2.

Slowdowns are an important factor to consider when designing fault-tolerant quantum computing systems. Despite the speed-up they offer, FTQC systems still require considerable time to finish computations. For example, it is estimated that factoring a 2048-bit number will take about a week [96]. Slowdowns must thus be avoided to further increase this

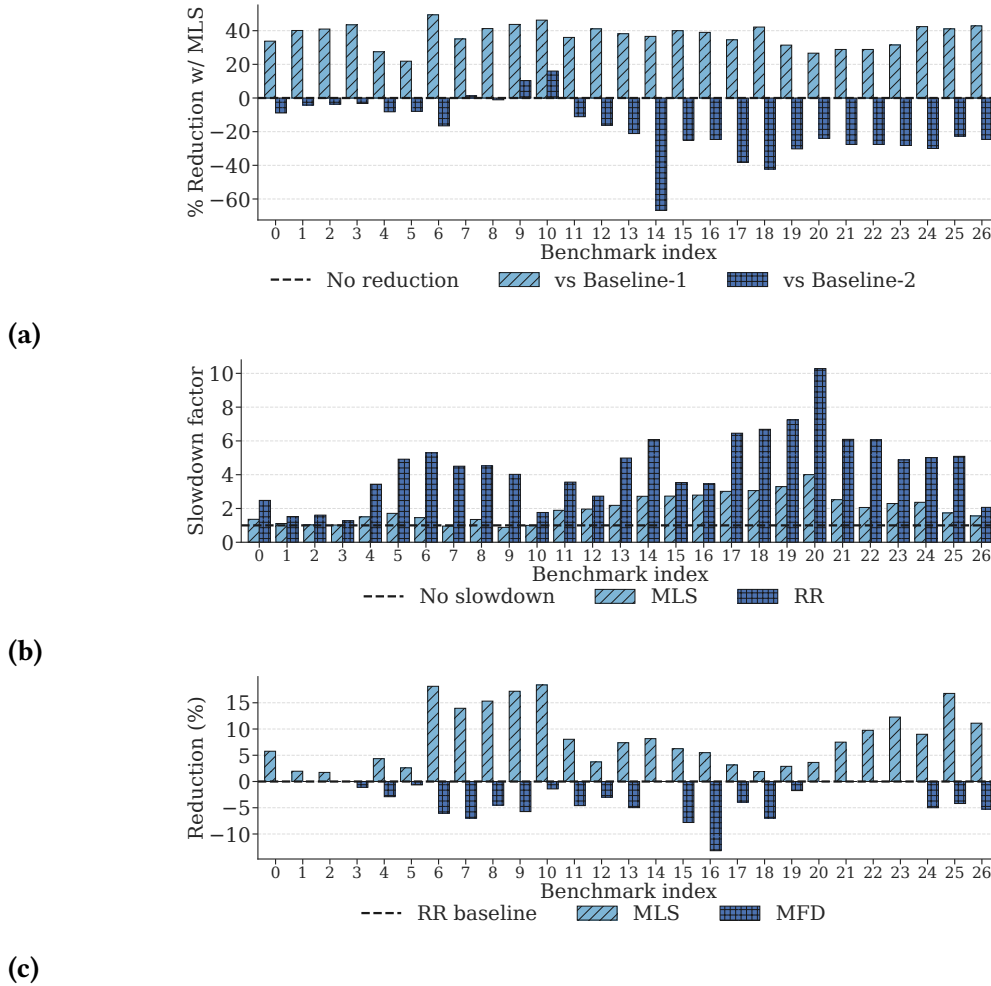


Figure 5.14: (a) The relative reduction (higher is better) in the number of decoders required by the MLS policy to prevent any slowdowns compared to the two baselines. Baseline-2 provisions for fewer decoders than what is required by the MLS policy; (b) Slowdown in computation when setting the number of decoders to Baseline-2 for the MLS and RR fine-grained policies; (c) The relative reduction in the minimum number of decoders required (higher is better) when using the MFD and MLS policies over the baseline RR policy. The MFD policy requires more decoders to prevent any slowdowns, while the MLS policy can reduce the number of required decoders by up to 19%;

computing time – a $2\times$ slowdown would increase that time to two weeks. Figure 5.14(b) also highlights that the MLS policy is more resilient to decoder pressure/scarcity compared to the RR policy. Also, Baseline-1 cannot incur any slowdowns since the number of decoders allocated to it is always more than the optimal number determined across all benchmarks.

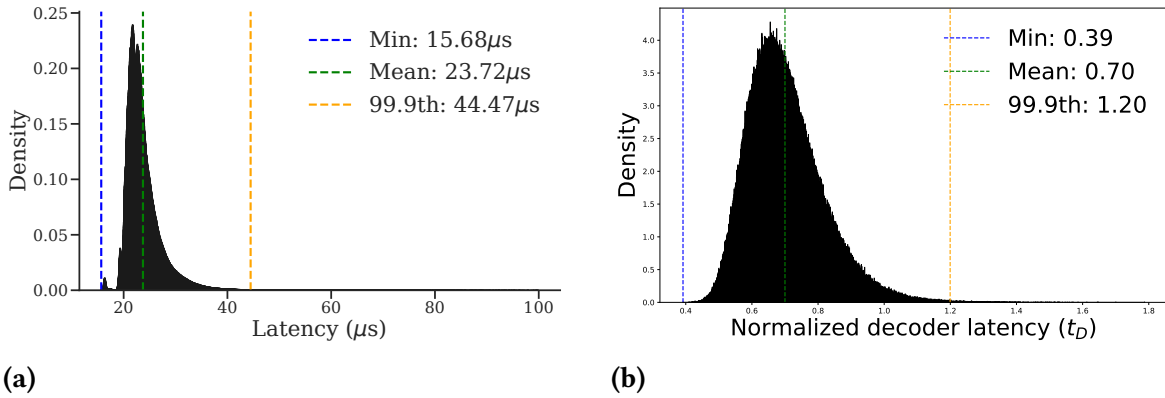


Figure 5.15: (a) Density histogram of latencies sampled from the PyMatching [114] decoder: the distribution can be approximated by a log-normal distribution with long tail latencies ($d = 9, p = 0.1\%$); (b) The density histogram used for our simulations with t_D sampled from a log-normal distribution.

Comparing the fine-grained scheduling policies. To further illustrate the differences between the fine-grained scheduling policies, we determine the reduction in the optimal number of decoders when the MLS and RR policies are used compared to the baseline RR policy, as shown in Figure 5.14(c). The MLS policy achieves a relative reduction of **up to 19%** compared to the RR policy, while the MFD policy is either as good or worse than the RR policy. This is likely because the MFD policy starves some qubits of decoding, thus requiring more decoders. Combined with the results of Figure 5.14(b), these results clearly show the benefits of the MLS policy over the other fine-grained policies.

Another important observation from Figure 5.14(c) is its correlation with Figure 5.12 – benchmarks that exhibit higher concurrency experience greater benefits with the MLS policy compared to benchmarks that are more serial.

5.3.4.4 Effect of decoder tail latencies

So far, we have performed all evaluations with a fixed normalized decoder latency $t_D = 0.5$ (where $t_D = \frac{T_{\text{decode}}}{T_{\text{cycle}}}$). However, decoders do not have a deterministic latency, and some decoding problems take longer to decode than others. In particular, some decoding tasks can have a long *tail latency*. This can result in higher decoder allocations, because slower decoders are unavailable for a longer period, reducing the throughput at which decoding tasks are processed. Figure 5.15(a) shows the distribution of latencies from the state-of-the-art

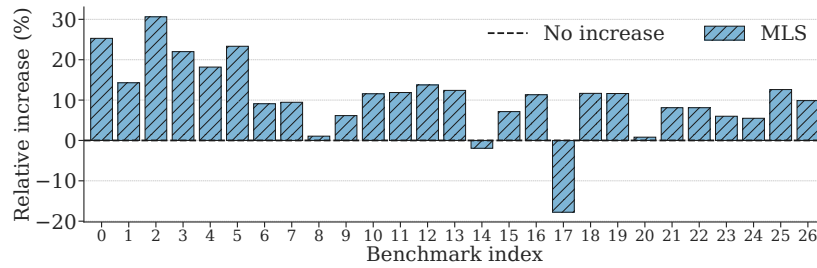


Figure 5.16: Relative increase in the number of optimal decoders required by the MLS policy to prevent slowdowns with longer non-uniform decoder latencies.

software decoder for the surface code, PyMatching [114] for a distance 9 ($d = 9$) surface code logical qubit and a circuit-level physical error rate of 0.1% (p). The distribution is centered around the mean but tail latencies are significant long. We approximate this distribution with a log-normal probability density function [64] of the form:

$$\text{PDF}(x; \mu, \sigma) = \frac{1}{x \sigma \sqrt{2\pi}} \exp\left(-\frac{(\ln(x) - \mu)^2}{2\sigma^2}\right), \quad x > 0$$

We model hardware decoder latencies with a log-normal distribution (see Figure 5.15(b)), which produces worst-case tail latencies longer than the syndrome generation cycle ($t_D > 1.0$). These long tails shift the optimal decoder count for all benchmarks compared with Figure 5.14(a): most benchmarks require more decoders under the MLS policy, while qft-20 (14), qft-80 (17) require fewer because some critical decodes sampled latencies below $t_d = 0.5$, reducing their decoder needs. **Overall, despite these longer, non-uniform latencies, we can still reduce decoder counts by 10–40% across all benchmarks.**

5.4 Synchronizing Logical Qubits

For a system composed of perfect couplers and qubits with a homogeneous QEC code, synchronization will not be necessary since all operations will occur in lockstep. However, due to the poor coding rate of the surface code, using a heterogeneous mix of QEC codes will yield better space-time trade-offs in future systems [242]. However, this heterogeneity, along with other factors, will result in logical qubits being desynchronized from one another.

5.4.1 Sources of Desynchronization

5.4.1.1 Using better codes for non-Clifford gates

Non-Clifford gates are a crucial component for universal FTQC. Distillation of non-Clifford T states using triorthogonal codes [153] was up until recently the most promising means of producing high-quality magic states for the surface code. However, recent work introducing magic state cultivation [98] has been shown to be capable of producing good quality magic states for a fraction of the space-time cost. However, these constructions, along with many others [34, 41, 54, 125, 208], use different codes and rely on teleporting logical qubits between a code that is better for producing magic states and the surface code. This automatically introduces desynchronization between patches involved in magic state production and other algorithmic logical qubits in the system, since the syndrome generation cycle time for color/other codes is different from the surface code, as shown in Figure 5.17(a). Quantum LDPC codes [30, 242] also rely on teleporting logical qubits to the surface code for executing non-Clifford gates, resulting in desynchronization.



Figure 5.17: (a) Teleporting logical qubits between different codes will require synchronization since every code has a different number of CNOTs in its syndrome generation cycle and hence has a different cycle duration; (b) Dropouts result in some qubits and/or coupler links being unusable (top). This can be circumvented by time-multiplexing of some qubits to construct syndrome generation circuits that have a duration that is longer than (but not a multiple of) the original duration (bottom).

5.4.1.2 Circuit construction in the presence of dropouts

Exploiting time-dynamics to construct error correction circuits that can tolerate defects/failed couplers and qubits in a lattice (called dropouts) have been proposed [59, 183]. These works are part of a broader set of works that seek to achieve fault-tolerance even

in the presence of fabrication defects [12, 151, 230, 244]. As shown in Figure 5.17(b), these constructions can result in patches with syndrome generation cycle times that are longer than other patches in the system, thus introducing desynchronization.

5.4.1.3 Twist-based lattice surgery

Measurements involving Pauli Y operators benefit greatly from twist-based lattice surgery [40, 154]. However, patches enabling twist-based Lattice Surgery on the surface code require additional CNOTs in the syndrome generation circuit, which will desynchronize them with regular surface code patches in the system.

5.4.1.4 Modular architectures

Industrial roadmaps [102, 122] and several works have explored chiplet-based modular architectures [185, 186, 216, 236] and chip-to-chip couplers [80, 222] for implementing such modular architectures. However, movement of patches or logical operations across chiplet boundaries could result in desynchronization of those patches from the remaining system due to higher CNOT latencies at the boundary of chiplets.

5.4.2 Synchronizing two patches

Two patches are desynchronized if they are at different points of their syndrome generation cycle, as shown in Figure 5.18(a). Patch P is ahead in time as it is executing the final Hadamard gate, while patch P' is still on its first CNOT, leading to a synchronization slack τ . Note that since τ is the phase difference in the syndrome generation cycles of P, P' , it is bounded by cycle time T_{cycle} since $\tau = \tau \% T_{\text{cycle}}$. This slack must be eliminated before Lattice Surgery. We now discuss policies that can synchronize patches P, P' .

5.4.2.1 *Passive synchronization*

The simplest way of synchronizing two patches would be to have the faster (leading) patch P wait for the slower (lagging) patch P' before starting the Lattice Surgery operation, as shown in Figure 5.18(b). This would essentially pause syndrome generation for patch P , thus exposing all data qubits in P to more decoherence errors. While techniques like Dynamical Decoupling (DD) [35, 56, 131, 132] can be used to reduce the impact of idling (decoherence)

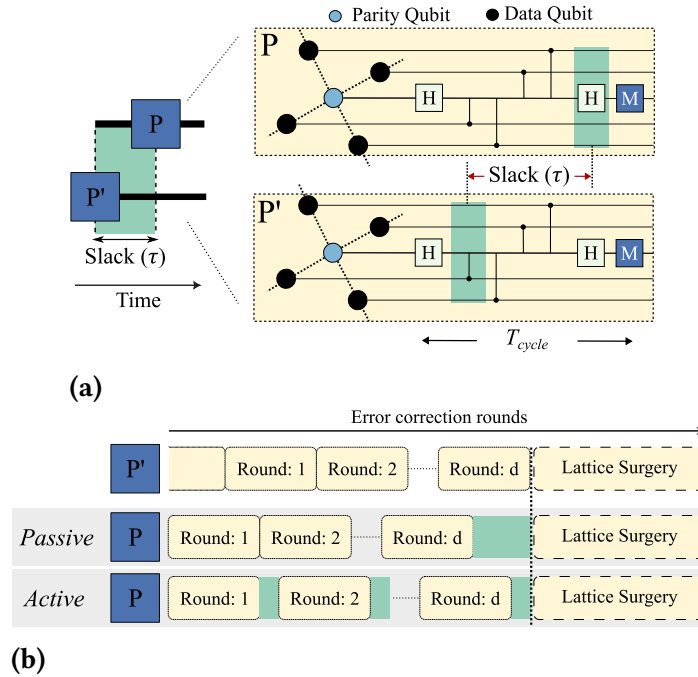


Figure 5.18: (a) Patches P and P' are desynchronized by a slack of τ . This corresponds to both patches being in different phases of their syndrome generation cycles (of duration $T_{cycle} = T_P, T_{P'}$); (b) P and P' can be synchronized by idling P for a period equal to τ right before Lattice Surgery (*Passive*), or by breaking τ into smaller segments and adding those idle periods after every round (*Active*) before Lattice Surgery.

errors, they can only partially mitigate such errors, not eradicate them. Secondly, DD sequences are physically applied on hardware and are not error-free – this reduces their efficacy in reducing idling errors. Work by Google [5, 6] shows how idling errors constitute a significant portion of the total error budget **despite** the use of DD. Finally, DD sequences are calibrated per data qubit [5] to optimize performance, which is not scalable as we increase the number of logical qubits.

5.4.2.2 Active synchronization

As an alternative to this *Passive* synchronization approach, we propose *Active* synchronization that achieves synchronization by **gradually** slowing down the leading logical qubit. Since logical operations are generally performed at the cadence of d rounds [84, 86, 152], the leading logical qubit can be slowed down by inserting idle periods before the start of (or after the end of) every round. Doing so breaks the total synchronization slack into

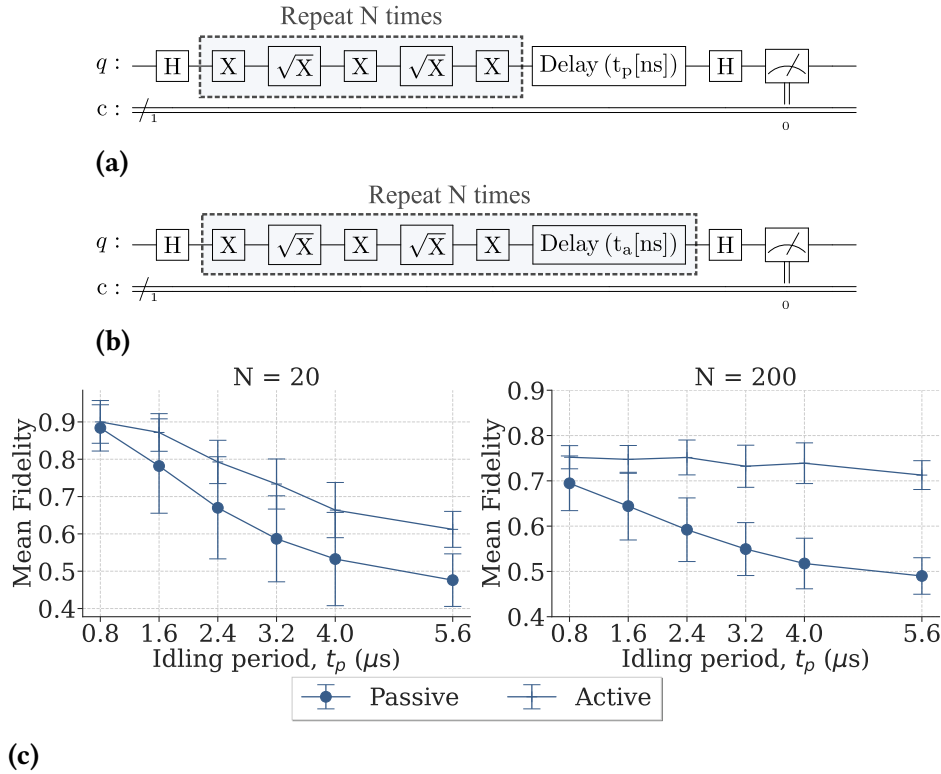


Figure 5.19: Circuits used to emulate the (a) *Passive* and (b) *Active* policies. The delay t_a is repeated N times to yield a net delay of t_p , resulting in both circuits having the same total runtime. Both circuits were run on 20 qubits on IBM Brisbane [120] for 20,000 shots. (c) Experimental mean fidelity across all 20 qubits for $N = 20, 200$ with different values of t_p ($t_a = t_p/N$).

smaller fragments and results in fewer errors occurring due to qubit idling, as shown in Figure 5.18(b).

Active synchronization attempts to decrease the impact of idling errors on the logical error rate by inserting the synchronization slack between multiple code cycles.

Insights from experiments on IBMQ. *Active* synchronization aims to enhance the mitigation provided by Dynamical Decoupling (DD) against idling errors by reducing idle periods experienced by data qubits. To demonstrate its effectiveness, we conducted two experiments on IBM Brisbane [120], a 127-qubit quantum computer. In the first experiment (Figure 5.19(a)), an idle period of t_p was added after N repetitions of a gate sequence, mimicking the *Passive* policy. In the second experiment (Figure 5.19(b)), the same idle period

was split equally across all N repetitions, with $t_a = t_p/N$, and added to the end of each gate sequence, emulating the *Active* policy. Both circuits had the same total duration and were applied to the first 20 qubits of the system, with X-X DD sequences inserted during all idle periods.

The benefits of dividing idle durations are clear in Figure 5.19(c), which shows the mean fidelity across all 20 qubits for $N = 20$ and $N = 200$. The circuit using the *Active* policy consistently outperformed the *Passive* policy. Moreover, as N increases, the idle period t_a becomes smaller, further improving performance compared to the *Passive* policy. While the improvement with the *Active* policy is modest for $t_p = 0.8\mu\text{s}$ at both $N = 20$ and $N = 200$, the benefits are expected to be more significant in error correction circuits involving more entangled qubits.

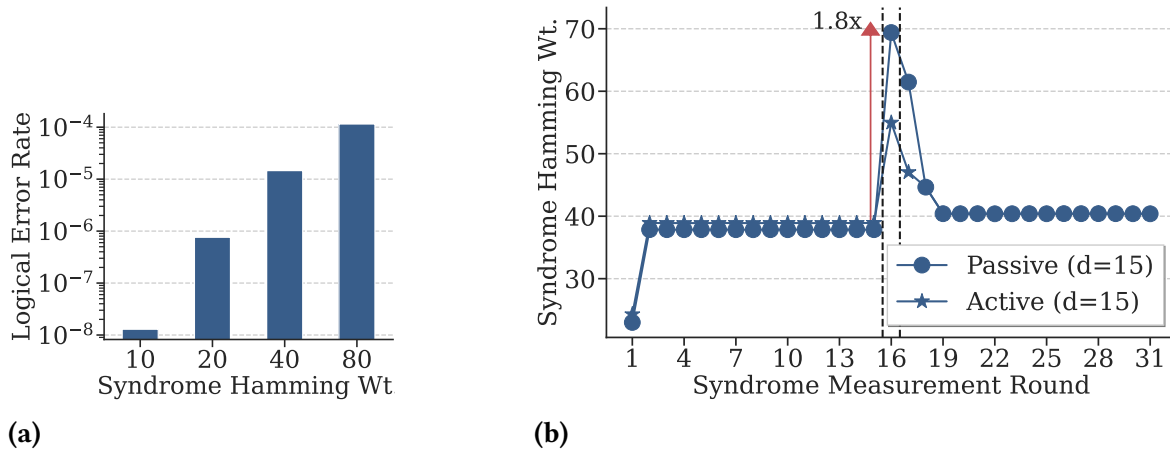


Figure 5.20: (a) Logical error rate is influenced by the syndrome hamming weight – higher weights result in more logical errors ($d = 15, p = 0.001$); (b) Syndrome hamming weights for all rounds before and after Lattice Surgery with *Passive* and *Active* synchronization ($\tau = 500\text{ns}$). The dashed vertical line shows the round during which Lattice Surgery is performed. *Passive* synchronization results in an almost 2x higher hamming weight during Lattice Surgery, resulting in a higher chance of logical errors compared to *Active* synchronization.

Why does the *Active* policy work? The experiments on IBM hardware highlighted the benefits of the *Active* policy on physical qubits. To evaluate its impact on logical qubits, we analyzed syndromes collected during Lattice Surgery operations under both *Active* and *Passive* policies. As shown in Figure 5.20(a), logical error rates increase with higher syndrome Hamming weights, as these are more likely to introduce uncorrectable errors.

Figure 5.20(b) illustrates the syndrome Hamming weight⁴ over QEC rounds for $d = 15$. During Lattice Surgery, there is a spike in Hamming weight, which is significantly higher under the *Passive* policy. Although the *Active* policy results in a slightly higher average Hamming weight before Lattice Surgery, the spike during the operation is much smaller, leading to better logical error rates. This improvement can be seen in Table 5.1, where the *Active* policy reduces undetectable errors across various code distances ($p = 10^{-3}$, $T_1 = 25\mu\text{s}$, $T_2 = 40\mu\text{s}$, 10^5 shots).

Table 5.1: Number of logical errors for different code distances d for the example shown in Figure 5.18(b).

Case	Number of Errors (slack=500ns)				Number of Errors (slack=1000ns)			
	$d = 7$	$d = 9$	$d = 11$	$d = 13$	$d = 7$	$d = 9$	$d = 11$	$d = 13$
<i>Passive</i>	278	89	22	5	403	114	31	13
<i>Active</i>	232	77	15	2	324	94	18	6
% Reduction	16.54	13.48	31.81	60	19.6	17.54	41.93	53.84

5.4.2.3 Active synchronization within a single round

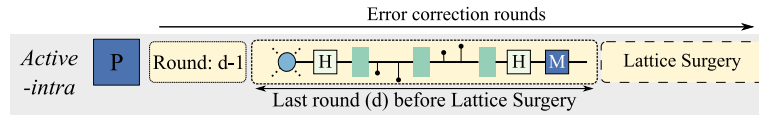


Figure 5.21: The *Active-intra* policy, where the slack is distributed within the last syndrome measurement round before Lattice Surgery.

Another variant of the *Active* policy is to distribute the total slack within a round rather than between multiple rounds. We call this the *Active-intra* policy. This can be seen in Figure 5.21 where the slack is distributed inside the final round before Lattice Surgery. An advantage of this policy is that synchronization can be performed within one round rather than multiple rounds. An obvious disadvantage is that this would add decoherence errors due to idling on the measure qubits as well as the data qubits.

⁴These syndromes correspond to detectors [97].

5.4.2.4 Synchronizing by running additional rounds

Rather than stopping syndrome generation for the leading patch P and idling for the slack duration, logical qubits can also be synchronized by running additional rounds. **This is not possible if both P and P' have the same syndrome generation cycle time ($T'_P = T_P$).** Figure 5.22 shows how patch P' with a cycle time of T'_P can be run for n rounds, while patch P with a cycle time of T_P can be run for m rounds to synchronize the two patches. In general, this synchronization policy can be defined with a Diophantine equation as:

$$n.T'_P = m.T_P + \tau; \quad (T_P \neq T'_P) \quad (5.1)$$

Here τ is the synchronization slack. To synchronize the two patches, we must iteratively find the smallest integer m such that $\frac{m.T_P + \tau}{T'_P}$ is an integer. As shown in Figure 5.23, the number of additional rounds required is highly dependent on the values of τ , T'_P , T_P , and in some cases, has no integral solution. As is evident from Figure 5.23, some combinations of τ , T'_P , T_P can require numerous rounds **per synchronization**, which will cause significant slowdown and reduce the per round error budget for other logical operations.

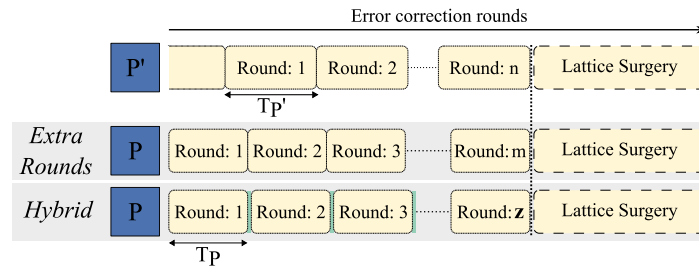


Figure 5.22: Extra Rounds: for patch P' with a cycle time of T'_P and P with a cycle time of T_P , synchronization before Lattice Surgery can be achieved by running $P(P')$ for $m(n)$ extra rounds; *Hybrid* policy: a small idling period is added after every round and P is run for z extra rounds.

5.4.2.5 Combining policies

It is impossible to synchronize patches P and P' by just running additional rounds if their syndrome generation cycle times are equal ($T_P = T'_P$). Thus, for $T_P = T'_P$, they will have to be synchronized by using either *Active* or *Passive* synchronization. However, when $T_P \neq T'_P$, there is an opportunity to further optimize the synchronization policy by running some

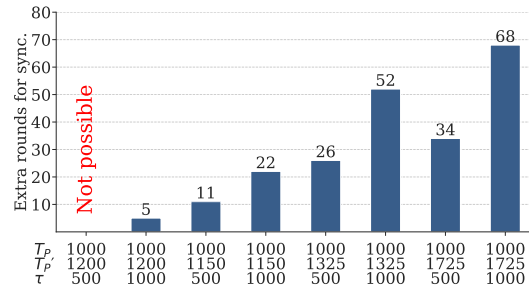


Figure 5.23: Different example configurations to show how running P for m extra rounds to achieve synchronization can be advantageous in some cases and impossible in others with the syndrome generation cycle times of P, P' as $T_P, T_{P'}$ (ns) and an initial slack τ (ns).

additional rounds of error correction **and** using the *Active* policy. This can be visualized in Figure 5.22, where the *Hybrid* policy inserts significantly smaller idling periods between the rounds while running for an additional z rounds. By combining the two policies, we can reduce the total idle period during which syndrome generation is paused compared to the pure *Active* policy implementation and also reduce the number of rounds required compared to the pure extra round strategy. To find an optimal *Hybrid* policy for patches P, and P' , we first define a slack tolerance, ϵ , which is the maximum duration for which syndrome generation can be paused. Using ϵ , the value of z can be computed by finding the smallest integer z for which –

$$\left\lceil \frac{z \cdot T_P + \tau}{T_{P'}} \right\rceil \times T_{P'} - (z \cdot T_P + \tau) < \epsilon; \quad (T_P \neq T_{P'}) \quad (5.2)$$

Table 5.2: Comparison of the idling period, extra rounds, and LER of different synchronization policies for the given configuration of patch cycle times and initial slack τ . The *Hybrid* policy achieves a significantly better LER by reducing the idling period and the number of extra rounds run.

$T_P = 1000\text{ns}, T_{P'} = 1325\text{ns}, \text{ffi} = 1000\text{ns}, \epsilon = 400\text{ns}, 20\text{M shots}$			
Synchronization Policy	<i>Active</i>	<i>Extra Rounds</i>	<i>Hybrid</i>
Idling period	1000 ns	0	300 ns
Num. extra rounds	0	52	4
LER ($d = 7$)	0.0014	0.0059	0.00095

As an example, we show how the three policies compare for a specific scenario where $t_P = 1000\text{ns}$, $t_{P'} = 1325\text{ns}$, $\tau = 800\text{ns}$, $\epsilon = 200\text{ns}$. Here, the *Hybrid* synchronization policy can reduce the idling duration to 175ns from 800ns and the number of rounds from 31 to

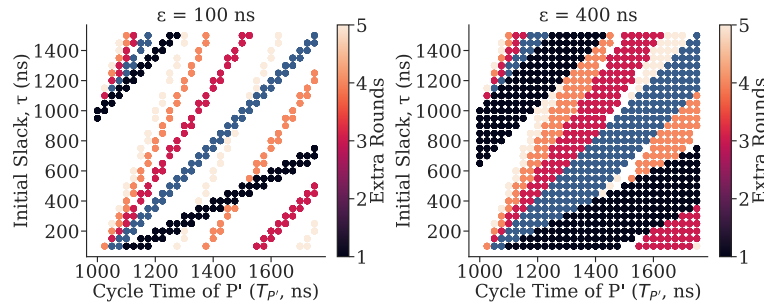


Figure 5.24: τ vs. $T_{P'}$, colored by the number of extra rounds (z). T_P was fixed at 1000ns. $\epsilon = 400$ ns allows more $\tau, T_P/T_{P'}$ combinations to be optimized with the *Hybrid* policy. White space indicates no solution was found.

3. This combination results in a logical error rate (LER) reduction of 1.71x compared to the pure *Active* policy and a 3.25x reduction compared to when only additional rounds are run. While this is a very specific case where the *Hybrid* policy works much better, there are cases where the *Hybrid* policy cannot do better than just running extra rounds or the *Active* policy.

5.4.2.6 Optimizing the *Hybrid* policy

A natural question that arises from Equation 5.2 is the selection of the slack tolerance (ϵ). As with the Diophantine equation defined in Equation 5.1, Equation 5.2 can only be solved iteratively, thus requiring some bounds to be set on z and ϵ . For this work, we set an upper bound on the number of extra rounds and the maximum tolerated slack with the *Hybrid* policy at 5 rounds and 400ns respectively. Note that this will depend on the noise profile of the system and the cycle times $T_P, T_{P'}$ of the patches being synchronized. Figure 5.24 shows the trade-off between using a smaller value of $\epsilon = 100$ ns and a larger one $\epsilon = 400$ ns. Using a smaller value of ϵ will thus be beneficial if there is a very limited variance in the slack (τ) and the cycle times ($T_P, T_{P'}$). Using larger values of ϵ allows for more combinations of $\tau, T_P, T_{P'}$ to be tackled with the *Hybrid* policy. Due to its more general application, we use a larger value of $\epsilon = 400$ ns for all evaluations.

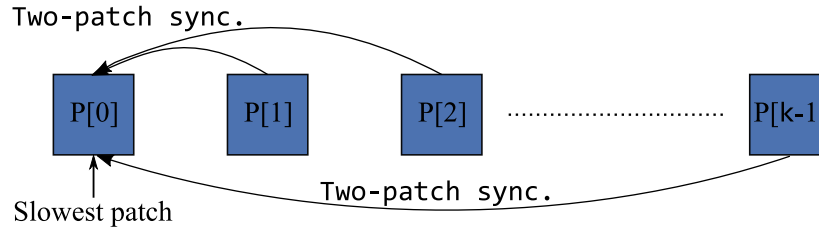


Figure 5.25: Active synchronization of k patches – every pair can be synchronized concurrently.

5.4.3 Synchronizing k patches

Generalizing two patch synchronization to an arbitrary number of patches can be done by identifying the slowest (most lagging) patch after sorting the patches based on the time needed for them to complete their current code cycle and then performing pair-wise synchronizations of all the patches with the slowest patch. All pair-wise synchronizations can be performed in parallel, which makes the synchronization of k patches take constant time.

5.4.4 Evaluations

5.4.4.1 Methodology

The simulation framework used in this work is a combination of our proposed framework `lattice-sim` and `Stim` [97]. `lattice-sim` generates stabilizer circuits compatible with `Stim` and can also perform Lattice Surgery between two patches, as shown in Figure 5.26 (`lattice-sim` can perform both X and Z basis Lattice Surgery). We will focus on the $X_P X_{P'}$, $X_P (Z_P Z_{P'}, Z_P)$ observables for Z (X) basis Lattice Surgery for the sake of brevity. While the use of heterogeneous codes will most likely be the dominant source of synchronization in FTQC systems, we restrict our evaluations to only surface code patches because decoders are faster and more accurate for the surface code compared to other codes such as the Color code. Building decoders that can accurately decode Lattice Surgery operations between different codes is an active field of research [229].

`lattice-sim` consists of a parser that can take QASM [52] circuits as an input to generate the corresponding stabilizer circuits (non-Clifford operations are not supported, as `Stim` is a Clifford simulator). `lattice-sim` also has a rich error modeling interface to add error annotations to the stabilizer circuit for realistic and device-specific simulations **with**

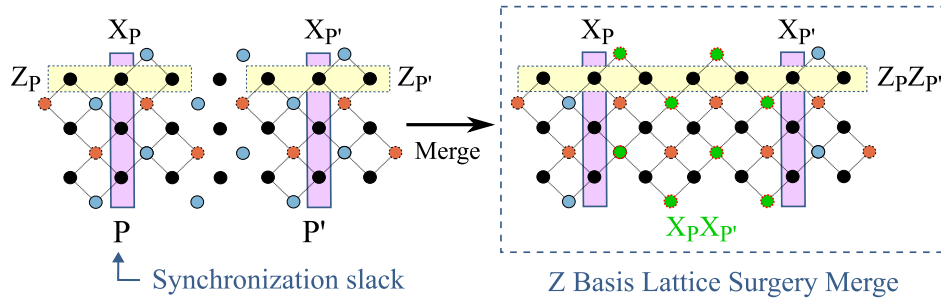


Figure 5.26: Two surface code patches P , P' are initialized, with some synchronization slack added to P . After $d + 1$ rounds, they are synchronized and then merged, after which the merged patch is run for another $d + 1$ rounds. The observables X_P , $X_{P'}$, $Z_P Z_{P'}$, $X_P X_{P'}$ of the merged patch can be used by Stim to compute the Logical Error Rate (LER). Note that for the Z basis Lattice Surgery, the $X_P X_{P'}$ observable corresponds to the logical measurement between P and P' and is computed by measuring the green X measure qubits [84].

circuit-level noise of $p = 10^{-3}$. `lattice-sim` also annotates idling errors based on the idle periods experienced by the qubits after every operation. The idling error model is based on the Pauli twirl approximation [93, 257]. Idling errors were inserted as single Pauli error channels [81], with p_x , p_y , and p_z defined as: $p_x = p_y = \frac{1 - e^{-\tau/T_1}}{4}$; $p_z = \frac{1 - e^{-\tau/T_2}}{2} - p_x$.

Note that the idling error model is conservative and does not model crosstalk, spectator effects, and leakage, which are typically observed on real hardware.

Stim Configuration. `pymatching2` [112] was used for decoding error syndromes. All circuits were sampled for 100M shots. The patches were initialized and run for $d + 1$ rounds, after which they were merged using Lattice Surgery and run for another $d + 1$ rounds.

Hardware Configurations. Table 5.3 shows the hardware configurations used during simulations for evaluating different synchronization policies. The cycle time is the total time taken (gates + measurement + reset) for generating one round of syndromes.

Table 5.3: System parameters used in evaluations.

Technology	T_1, T_2	1Q gates	2Q gates	Readout	Cycle time T_{cycle}
IBM [120, 121]	200 μ s, 150 μ s	50ns	70ns	1500ns	$\sim$ 1900ns
Google [6]	25 μ s, 40 μ s	35ns	42ns	660ns	$\sim$ 1100ns
QuEra [24, 193]	4s, 1.5s	5 μ s	200 μ s	1ms	$\sim$ 2ms

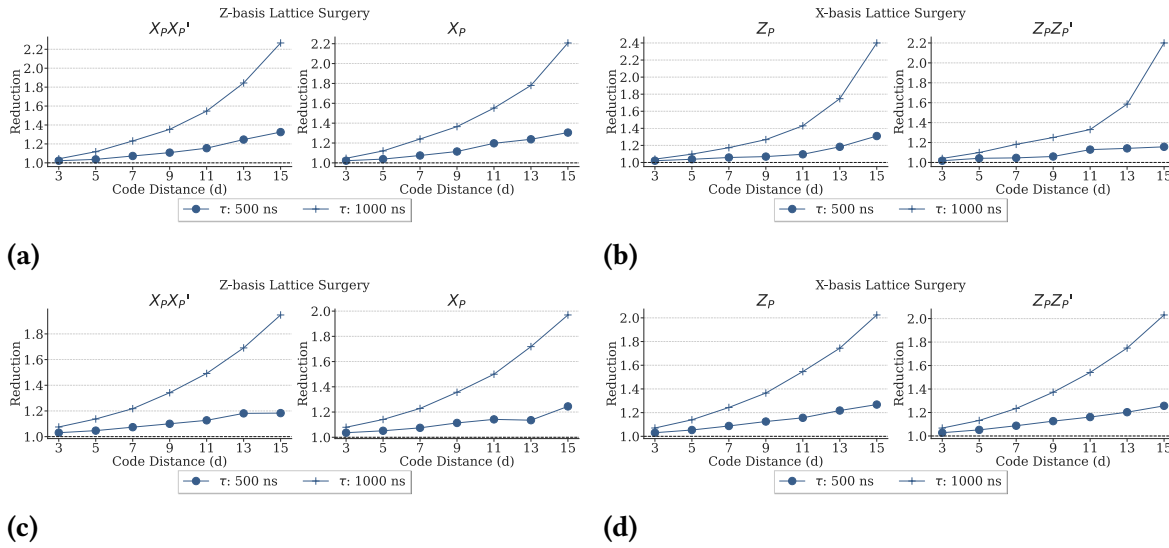


Figure 5.27: Reduction in the logical error rate for **one synchronization** when using *Active* synchronization instead of *Passive* for different code distances with a system configuration similar to (a)-(b) IBM systems, and (c)-(d) Google systems for both X and Z basis Lattice Surgery with $p = 10^{-3}$. The synchronization slack (τ) was set at 500ns and 1000ns.

Other software. Benchmarks were generated and derived from MQTBench [212]. Resource estimation was done using the Azure Quantum Resource Estimation tool [22].

5.4.4.2 Active vs. Passive synchronization

If the syndrome generation cycle times for patches P and P' are equal ($T_P = T_{P'}$), the *Active* and *Passive* policies are the only viable strategies to synchronize the two patches. Figure 5.27 shows the reduction⁵ in the logical error rate when *Active* synchronization is used in place of *Passive* before merging patches P and P' for IBM ((a) – (b)) and Google-like((c) – (d)) systems. **Since the worst-case slack (τ) is bounded by the cycle time $\tau \leq T_{\text{cycle}}$, we bounded $\tau \leq 1000\text{ns}$ to emulate superconducting systems since their cycle time T_{cycle} is greater than 1000ns (Table 5.3). Figure 5.27 shows that the **Active policy achieves a reduction of up to 2.4x in the logical error rate.****

⁵Reduction = $\frac{\text{LogicalErrorRate}_{\text{Passive}}}{\text{LogicalErrorRate}_{\text{Active}}}$.

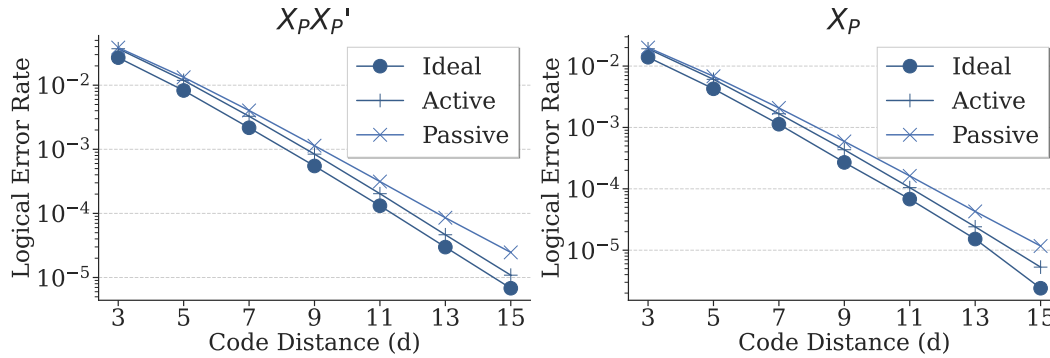


Figure 5.28: Logical error rate of the $X_P X_{P'}$ and X_P observables with Z basis Lattice Surgery after $2d + 2$ rounds for an ideal system that does not require any synchronizations and a system either using *Active* or *Passive* synchronization. The worst-case synchronization slack of $\tau = 1000\text{ns}$ was chosen for this experiment with an IBM configuration ($p = 10^{-3}$).

5.4.4.3 Cost of synchronization

Compared to an ideal system in which all patches are always perfectly synchronized, what is the cost of synchronization in a more realistic system? Figure 5.28 shows the logical error rate for three systems: an ideal system that requires no synchronizations, and systems using the *Passive* and *Active* policies. For a worst-case synchronization slack of 1000ns , *Active* synchronization can achieve a logical error rate that is much closer to the ideal system as compared to the *Passive* policy, thus highlighting its effectiveness as a synchronization policy.

5.4.4.4 Will running additional rounds help the *Active* policy?

Active synchronization works by dividing the slack into smaller idle periods and slowing the leading logical patch gradually. If the number of rounds over which the slack is distributed is increased, the idle periods before every round become even smaller. To evaluate the effect of additional rounds on the efficacy of *Active* synchronization, we added R additional rounds before Lattice Surgery was performed, thus allowing the slack to be distributed over $d + 1 + R$ rounds for the *Active* policy. The *Passive* policy was also used after $d + 1 + R$ rounds. This setup corresponds to starting the synchronization process *earlier* in time, since more rounds are being used to distribute the slack for the *Active* policy. Figure 5.29(a) shows how the benefit of *Active* synchronization diminishes as additional rounds are run, especially for $\tau = 1000\text{ns}$ (averaged over both $X_P X_{P'}$, $Z_P Z_{P'}$ observables, with the IBM

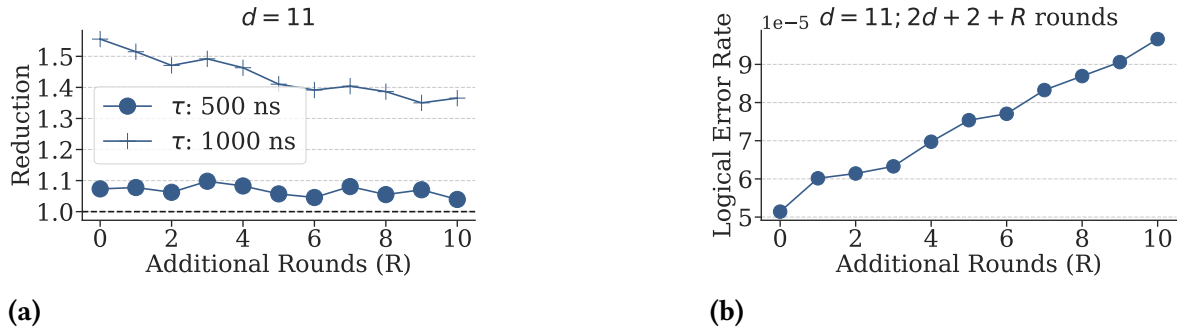


Figure 5.29: (a) Reduction in the LER compared to the *Passive* policy when the *Active* policy distributes its slack over $d + 1 + R$ rounds rather than $d + 1$ rounds; (b) Influence of the number of rounds run on the LER (without any slack/idling).

configuration). From Figure 5.29, we can establish that increasing the number of rounds over which the *Active* policy operates has diminishing returns. This can be understood from Figure 5.29(b), where the LER increases with the number of additional rounds run, even without the presence of any synchronization slack and idling. This is because decoders are imperfect, and these imperfections lead to error accumulating after every round.

5.4.5 The *Extra Rounds* and *Hybrid* policies

So far, we have shown how the *Active* policy is superior to the *Passive* policy, especially for large τ . If the syndrome generation cycle times T_P, T'_P of the two patches are different, additional policies are possible. Figure 5.30 shows the reduction in the LER for different policies. We varied the slack tolerance for the *Hybrid* policy from 100 – 400ns and used all combinations of the cycle times – $T_P = 1000\text{ns}, T'_P = 1050/1100/1150\text{ns}$. These cycle times are representative of cycle times consisting of 1/2/3 additional CNOTs compared to the surface code, as would be the case for color/qLDPC codes. The results were obtained as an average over all combinations of cycle times and the $X_P X_{P'}, Z_P Z_{P'}$ observables (IBM configuration). Figure 5.30 shows that for larger values of τ , the *Hybrid* policy works much better, since it allows the slack to be reduced significantly. Furthermore, higher values of ϵ allow more leeway in selecting the number of extra rounds to be run, allowing significantly better LERs. Conversely, for smaller values of τ , Figure 5.30 suggests that the *Active/Passive* policies would be more advantageous. Table 5.4 shows that the reduction is even higher at $3.4\times$ for $d = 15$.

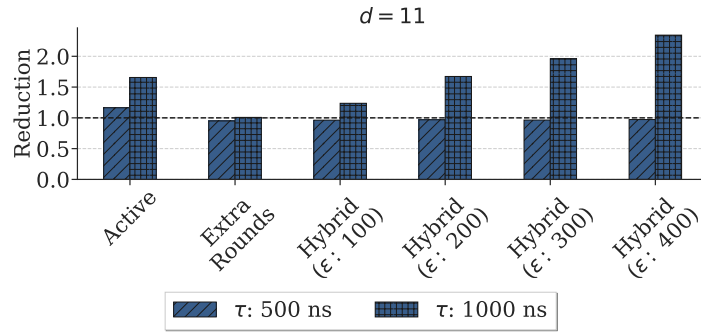


Figure 5.30: Reduction in the LER compared to the *Passive* policy for different initial slacks τ and tolerances ϵ (in ns). The *Hybrid* policy works more effectively for larger τ .

Table 5.4: Average reduction in the LER for all policies compared to the *Passive* policy for $d = 11, 13, 15$.

	d	Active	Extra Rounds	Hybrid ($\epsilon = 400\text{ns}$)
LER Reduction ($\tau = 1000\text{ns}$)	d = 15	2.14	1.63	3.4
	d = 13	1.87	1.29	2.72
	d = 11	1.65	1.07	2.34

Table 5.5: Additional rounds needed for synchronization with the *Hybrid* policy – these result in an increase in the LER.

Initial Slack τ (ms)	$\tau = 0.2$	$\tau = 0.6$	$\tau = 1.0$	$\tau = 1.6$	$\tau = 2.0$
Hybrid ($\epsilon = 0.1\text{ms}$)	9	3	6	8	12
Hybrid ($\epsilon = 0.4\text{ms}$)	5	3	5	8	10

5.4.6 Synchronization in neutral atom platforms

So far, we have discussed results on superconducting systems. Neutral atom systems have considerably higher T_1, T_2 times (Table 5.3) which reduces the impact of idling errors. Figure 5.31 shows the reduction in the LER of all policies for different initial synchronization slacks compared to the *Passive* policy⁶. *Active* synchronization achieves a marginal $\sim 2\%$ reduction in the LER over the *Passive* policy, but the *Hybrid* policy leads to an increase in the LER due to more rounds being run, as shown in Table 5.5. These results indicate that the *Active/Passive* policies are always better on neutral atom systems due to their long coherence times.

⁶The results were averaged over the $X_P X_{P'}, Z_P Z_{P'}$ observables and all combinations of $T_P = 2\text{ms}$, $T'_P = 2.2\text{ms}, 2.4\text{ms}, 2.6\text{ms}$.

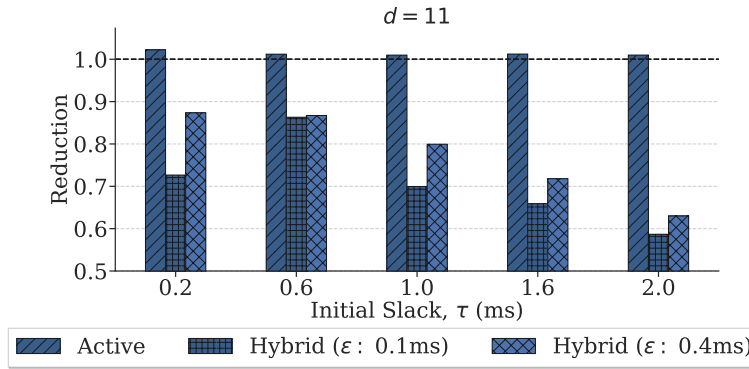


Figure 5.31: Reduction in the LER compared to the *Passive* policy with the use of the *Active* and *Hybrid* policies on a neutral atom system. Running additional rounds, even with the *Hybrid* policy, is detrimental to the LER.

5.5 Summary

In this chapter, we identified and addressed two classical control problems that emerge when scaling a fault-tolerant quantum computer beyond a single logical qubit: resource contention among decoders and temporal misalignment between logical qubits participating in joint operations.

Decoder provisioning and scheduling. We showed that neither SWD nor static decoder allocation is sufficient at scale. SWD is fundamentally sequential and cannot sustain the throughput required by superconducting hardware even at optimistic decoder latencies. PWD resolves the throughput problem but introduces non-deterministic decoder demand: the number of decoders required by a single logical operation can spike to $50\times$ or more due to spatial parallelism from routing-driven patch sizes. Static allocation strategies—provisioning for the worst case or the average case—are either too expensive or introduce significant program slowdowns (up to $10\times$ for Baseline-2 under the RR policy).

We proposed a two-level scheduling framework that addresses this variability. Coarse-grained scheduling prioritizes critical spatial decodes arising from non-Clifford operations, ensuring that magic state consumption is never delayed. Fine-grained scheduling allocates the remaining decoder capacity across idling logical qubits using one of three policies: MFD, RR, and MLS. Of these, the MLS policy—which greedily minimizes the longest undecoded sequence—achieves the best performance, reducing the minimum number of decoders needed to prevent slowdowns by up to 19% compared to round-robin and up to 40% below

worst-case provisioning. These reductions hold even in the presence of log-normal tail latencies, with 10–40% savings sustained across all benchmarks.

Synchronization of logical qubits. We identified four sources of desynchronization in realistic FTQC systems: the use of heterogeneous codes for non-Clifford gate production (e.g., color codes, qLDPC codes), dropout-adapted syndrome circuits with longer cycle times, twist-based lattice surgery, and chiplet-boundary latencies in modular architectures. In all cases, a synchronization slack τ must be eliminated before two patches can participate in lattice surgery.

We defined and evaluated four synchronization policies. The *Passive* policy idles the leading patch, concentrating all decoherence exposure in a single interval. The *Active* policy distributes the slack across d QEC rounds, reducing the idle period per round and the Hamming weight spike at the lattice surgery boundary. On superconducting hardware, *Active* synchronization achieves up to $2.4\times$ reduction in logical error rate compared to *Passive*. When the two patches have different syndrome generation cycle times, the *Extra Rounds* policy avoids idling entirely but can require an impractical number of additional rounds. The *Hybrid* policy combines the two: it runs a small number of extra rounds to reduce the residual slack and then distributes that residual using the *Active* approach, achieving up to $3.4\times$ LER reduction at $d = 15$. On neutral atom platforms, long coherence times reduce the impact of idling, making *Active* or *Passive* preferable—the extra rounds incurred by *Hybrid* increase the LER rather than reducing it.

Outlook. Together, the elastic decoder scheduling framework and the synchronization policies constitute the core of a quantum operating system for fault-tolerant machines. Just as a classical OS multiplexes CPU cores across processes and coordinates shared-memory access between threads, the QOS must multiplex a finite pool of accelerated decoders across a large ensemble of logical qubits and enforce timing alignment before inter-qubit operations. The analyses in this chapter suggest that neither problem is intractable: intelligent scheduling reduces decoder hardware requirements substantially below naive provisioning, and active synchronization keeps the logical error rate close to that of an ideally synchronized system. Realizing these policies in hardware will be an essential step toward practical, large-scale fault-tolerant computation.

Chapter 6

Related Work

A significant research effort is underway to create fault-tolerant quantum computers capable of solving complex, classically intractable problems. This research effort is across the stack – engineering better qubits, more accurate control and readout systems, efficient QEC codes and methods for logical operations, fast and accurate decoders, compilers, and useful algorithms that can exploit the computational power of quantum computers. In this chapter, we highlight related work that span the topics covered in this dissertation – control and readout systems, decoding, and systems support for the classical compute infrastructure.

6.1 Low-Level Control Hardware

6.1.1 Qubit control

Efficient qubit control is crucial to scale quantum computers. While most control platforms use discrete FPGAs and analog ICs, we expect Radio-Frequency Systems on Chips (RFSoc)-based and custom ASIC-based solutions to grow in the future.

Room-temperature control platforms. The dominant control paradigm synthesizes microwave pulses at room temperature and delivers them to the qubits over attenuated coaxial lines. A number of FPGA- and RFSoc-based platforms have made this approach programmable and increasingly dense. QICK [71, 241] and QubiC [276] are open-source RFSoc controllers that integrate high-speed DACs, ADCs, and a real-time processor on a single device, and subsequent work has extended them to frequency-multiplexed generation,

mixer-free readout, and multi-board synchronization at sub-100 ps alignment [71, 231]. Intel’s instruction-driven controller similarly replaces precomputed waveform tables with a direct digital synthesis pipeline driven by an instruction set [129]. These platforms confront the same bottleneck COMPAQT addresses—each generated channel consumes waveform-memory bandwidth that grows with the qubit count—but they treat the waveform as an incompressible stream. COMPAQT is complementary: its compressed-memory microarchitecture can be retrofit beneath the signal generators of such platforms to raise the number of channels a given device can drive.

Cryogenic CMOS controllers. To shorten the control path and reduce the number of wires crossing thermal stages, a second line of work moves control electronics into the cryostat. Cryo-CMOS controllers such as Intel’s Horse Ridge [277] and the cross-resonance-capable 4 K ASIC [39], along with low-power transmon controllers [14, 69], demonstrate in-fridge pulse generation, but all operate under a severe cooling-power budget in which active power scales with the data moved through the controller. This is precisely the regime in which COMPAQT’s compression yields a power benefit: by reducing waveform-memory traffic, COMPAQT lowers the dynamic power of a cryogenic ASIC controller, and our adaptive scheme trades a controlled amount of fidelity for further power savings. Hybrid photonic/CMOS schemes [155] attack the same wiring and power problem from a different angle, distributing shared pulse templates optically while programming envelopes locally—an architectural choice that, like COMPAQT, exploits redundancy across the waveforms that must be delivered.

Digital and SFQ-based control. A more radical departure replaces analog microwave drive with classical superconducting digital logic co-located with the qubits. Single-flux-quantum (SFQ) control irradiates qubits with trains of quantized voltage pulses [148, 180], and recent multi-chip-module demonstrations have reduced SFQ gate error by an order of magnitude by suppressing quasiparticle poisoning [156]. SFQ control sidesteps the high-bandwidth waveform memory entirely, but at the cost of new fidelity and fabrication challenges; it is best viewed as a longer-horizon alternative to the DAC-based control that COMPAQT optimizes for near- and intermediate-term systems.

Reducing what must be stored and streamed. Orthogonal to the hardware substrate, a body of compiler and instruction-set work reduces the quantity of distinct pulses a system must calibrate, store, and stream. Calibration- and expressivity-efficient instruction set design [146] shows that a small set of two-qubit gate types captures most of a continuous family’s expressivity while sharply cutting calibration overhead, and per-qubit basis-gate selection [150] and reconfigurable continuous ISAs [278] push further on this tradeoff. These techniques shrink the *number* of waveforms; COMPAQT instead shrinks the *footprint and bandwidth* of whichever waveforms remain, and the two are mutually reinforcing—a leaner instruction set produces a waveform set that is itself more amenable to the compile-time compression COMPAQT relies on.

Architectural studies of the control interface. Finally, architecture-level studies have characterized the control interface as a scalability bottleneck in its own right. QIsim [187] models a 10 K-qubit quantum-classical interface and exposes the wiring, power, and temperature tradeoffs of scaling it, while subsequent work on low-latency feedback [256] and tightly integrated hybrid execution [251] targets the round-trip latency between control and computation that error correction will demand. COMPAQT contributes to this broader effort by removing the waveform memory as a first-order limit on how many qubits a single control device can address.

6.1.2 Qubit readout

Qubit-state discrimination sits on the critical path of every measurement, so a discriminator must be simultaneously accurate, fast, and cheap enough to run on the readout hardware already present in a control system. Prior work tends to optimize one or two of these axes at the expense of the third. We organize the discussion around the accuracy–latency–cost tension that HERQULES targets and position our hierarchical matched-filter-plus-network design against each line of work.

Classical and statistical discriminators. The conventional approach integrates the readout signal against a matched filter and applies a linear threshold or a support-vector machine [91, 169]. These methods are lightweight and fast, and remain the default in most experimental setups, but they assume well-separated Gaussian distributions and degrade

in the presence of readout crosstalk, state-transition (T_1) events, and leakage. HERQULES retains a matched filter as its front end—inheriting this efficiency—but replaces the single linear decision boundary with a hierarchy that recovers the accuracy lost to these non-idealities.

Neural-network discriminators. To compensate for system-dependent non-idealities, a body of work applies neural networks to single-shot discrimination. Lienhard et al. [149] showed that fully connected feedforward networks reduce assignment error by up to 25% on frequency-multiplexed superconducting qubits, primarily by suppressing readout crosstalk, and related work has applied networks to spin [245], neutral atom [190], and trapped-ion [72] readout. These designs establish that learned discriminators are more accurate, but the networks are large enough that discrimination is typically performed in software off the FPGA, adding latency that is unacceptable for fast feedback and QEC. HERQULES is motivated directly by this gap: it attains comparable accuracy with a network small enough to run *on* the control FPGA, removing the software round-trip.

Hardware-accelerated discrimination. A more recent line of work attacks the latency problem by mapping discriminators onto FPGA fabric directly. Gautam et al. [92] deploy a quantized feedforward network on an RFSoc to discriminate five frequency-multiplexed qubits in under 50 ns, and report an even leaner SVM variant; KLiNQ [108] uses knowledge distillation to shrink the per-qubit network footprint. These efforts share HERQULES’s goal of low-latency, on-hardware discrimination and in several cases postdate and build on it; the distinguishing idea in HERQULES is architectural rather than purely an implementation optimization—a hierarchy of matched filters does most of the separation work, so the learned component can be small without sacrificing accuracy or requiring qubit-specific retraining.

Readout beyond binary discrimination. Discrimination is also being pushed beyond the two-state setting. Multi-level readout that resolves leakage to higher transmon states [189] is increasingly important because leakage is a dominant correlated error source for QEC, and learned temporal post-processors [130] and reservoir-computing discriminators [128] target the same accuracy ceiling from different algorithmic directions. Reinforcement learning has also been used to co-optimize the readout pulse itself rather than only the classifier [43]. These directions are complementary to HERQULES: the hierarchical-filter

structure is agnostic to the number of target states and provides a natural substrate into which multi-level or leakage-aware classification can be folded.

Readout as a system-level bottleneck. Finally, architecture-level work treats readout fidelity and latency as first-order constraints on system performance, since assignment error feeds directly into NISQ program fidelity and into the logical error rate of the surface code, and since slow readout lengthens the QEC cycle. Measurement-aware techniques such as measurement subsetting [57] improve effective fidelity at the program level, while our own readout-architecture studies [175] examine the security implications of these discriminators. HERQULES contributes to this system view by making high-accuracy discrimination cheap enough to run inline, so that readout no longer forces a trade-off between fidelity and the latency budget of fast feedback.

6.2 Decoding

With experimental demonstrations of QEC codes working in practice [6, 145], research on fast, accurate decoding has gained more popularity for real-time implementations. While most works target algorithmic optimizations, FPGA/ASIC/GPU implementations of decoders have recently become mainstream as well.

Cluster decoders. This class of decoders [61, 63, 113, 115, 142, 268] group nearby detection events in the decoding graph that are likely caused by the same underlying faults, iteratively growing and merging clusters until each can be locally corrected. Prior work on FPGA decoders using clustering/union-find are focused on the surface code [159–161, 271]. Our FPGA-tailored clustering decoder presented in Chapter 4 proposes a different bookkeeping and cluster solving technique compared to prior work on surface codes due to the added complexities of qLDPC codes.

Message-passing decoders. These algorithms exchange information iteratively between check and fault nodes in a graphical representation of the decoding matrix, with local update rules that map naturally onto parallel FPGA architectures [202, 260]. The standard example is Min-Sum belief propagation (BP), which often fails to converge for quantum LDPC codes [165, 207] in part due to loops and symmetries in the decoding graph. Recent

advances address these issues through decimation to break loops [65, 66, 74, 75, 101, 258, 279], incorporating memory to damp message oscillations [47, 191, 192], and decoder ensembling [141], while others learn modified message-update rules by pre-training [158]. The recently introduced Relay decoder [191] performs a sequence of BP runs (“legs”) with varying memory-strength parameters, progressively breaking trapping sets to improve convergence and accuracy. Relay shows the best accuracy-speed tradeoff for quantum LDPC codes, even on FPGAs [172, 191].

Ordered-statistics decoders. OSD algorithms [82, 83, 203] leverage an unconverged pre-decoder’s output by reprocessing bits according to their estimated reliability. Following Ref. [217], the standard OSD selects a set S of linearly independent columns of H , ordered by likelihood, with $|S| = \text{rank}(H)$. The corresponding submatrix $H_S \in \mathbb{F}_2^{M \times |S|}$ is inverted to obtain a correction $\mathbf{x} = H_S^{-1} \sigma$ supported on S . However, standard OSD is impractical on FPGAs [260] because detecting independence and inverting the large $\text{rank}(H) \times \text{rank}(H)$ matrix dominate runtime, even when optimized for FPGAs [16].

SAT solver-based decoders. Decoders using SAT solvers map the decoding problem using SAT reductions achieve minimum weight corrections, but are too slow to be considered for practical real-time applications [19, 198, 229, 282].

Search-based decoders. These decoders leverage fast tree/graph search algorithms for finding minimum-weight solutions. However, since traversing the entire decoding graph can be slow, practical implementations use BP as a guide [201], or aggressive pruning [17, 270]. While Tesseract [17] has fast implementations leveraging parallel processing, it along with other search-based decoders remain impractical for real-time implementations.

Pre-decoders Pre-decoders aim to reduce the average decoding latency by quickly decoding trivial syndromes. While we use BP as a pre-decoder in our FPGA-based clustering decoder, it is possible that other pre-decoders could improve its performance. They thus function as a filter that delegates hard-to-decode (but rare) syndromes to a second-level decoder (which will also be slower). AI-based pre-decoding [42] shows promise for the surface code by reducing decoding latencies to less than $1\mu\text{s}$ per round on average through aggressive parallel windowed decoding and using fast GPUs. Cryogenic pre-decoders aim

to reduce the communication latency incurred during syndrome transfer to the decoder by placing the pre-decoder close to the quantum hardware at cryogenic temperatures [137, 213]. Other pre-decoders use a variety of heuristics for identifying trivial syndromes [60, 237].

Matching decoders Matching decoders can be used iff any fault flips exactly two detectors on the decoding graph. These decoders are used extensively for the surface code for this reason, using the minimum-weight perfect matching algorithm. PyMatching [114] is a highly optimized version of the MWPM algorithm, and Wu et al. implemented a version of this decoder on FPGAs [272].

Machine learning-based decoders Machine learning was successfully leveraged for real-time operation as a pre-decoder by Chamberland et al. [42] using GPUs. Other neural network approaches have also been proposed [23, 100, 118, 165, 197] but reported long latencies and difficulty in generalizing to logical operations. Gu et al. proposed a convolutional neural network decoder that exploited the repeating structures present in the decoding graph for most QEC codes [107] which makes it more scalable than other machine learning approaches.

6.3 Systems for Quantum Computers

Systems research for the classical control system (encompassing control, readout, decoding, scheduling) has been limited due to the lack of actual systems and higher focus on downstream components. Some early work on quantum operating systems treated quantum hardware similarly to FPGAs [49], while later work used the same terminology for scheduling jobs on quantum cloud service providers [99]. Kim et al. presented a follow up work on the elastic decoder scheduling policies of Chapter 5 [133]. A high-level blueprint for networked decoders was presented in [160] for multi-logical qubit systems using lattice surgery. A heterogeneous architecture utilizing both surface and qLDPC codes was introduced in [242] but did not consider the cost of synchronizing logical qubits across code boundaries. Viszlai et al. [264] presented a speculation-based strategy to speed up windowed decoding for non-Clifford operations, but operated from a system-level rather than at a decoder-level.

Chapter 7

Conclusion and Future Work

“Yes, Mr. Frodo. It’s over now.”

— Samwise Gamgee, *The Return of the King*

We have now reached the end of this dissertation. We will provide a recap of the topics covered in this dissertation, and then discuss future work, and the outlook as we move into a new paradigm of computing.

7.1 Summary

In this dissertation, we worked throughout the control system stack necessary for realizing useful, utility-scale fault-tolerant quantum computers. The general theme throughout this dissertation was to make the control system more robust and scalable, especially as we increase the number of qubits from a few hundred to thousands over the next decade. Furthermore, while some solutions and ideas presented in this dissertation target specific qubit technologies or QEC codes, a lot of them are transferable to other technologies and codes as well. The ideas developed in this dissertation have also been used to generate tools that future researchers could find useful, and these tools are presented in the Appendix.

In Chapter 3, we presented COMPAQT and HERQULES: two optimizations to the qubit control and readout pipeline for superconducting qubits. COMPAQT reduces the size *and* bandwidth requirements of the waveform memory used for generating control pulses for superconducting qubits. Contrary to the usual usage of compression for reducing size,

COMPAQT leverages compression to reduce the memory bandwidth required for the simultaneous control of many qubits, which otherwise limits the scalability of platforms such as RFSocS. More specifically, we used a hardware-friendly integer discrete-cosine transform-based compression algorithm coupled with run-length encoding to compress single- and two-qubit gate pulses. Even though the compression algorithm was lossy, the overall effect on gate fidelity was miniscule, which made the control optimization extremely effective for increasing the number of qubits controlled by a single RFSocC-based controller by up to $5\times$. For cryogenic ASIC controllers, waveform compression reduces the size of the waveform memory, which directly reduces the power dissipated by the controllers. This reduction is extremely important in power-constrained cryogenic environments. Even though COMPAQT targets superconducting qubits, other qubit technologies like quantum dots and neutral atoms use modulated waveforms for control, which makes COMPAQT viable for those as well. HERQULES targets the reverse, qubit readout pipeline for qubit state classification. It achieves high readout accuracies without using complex, costly hardware. The primary driver behind HERQULES's high accuracy is a semi-supervised training algorithm that helps it identify thermal relaxation errors that occur during readout. These errors contribute to misclassifications of qubits that were in state '1' as state '0'. HERQULES uses lightweight matched filters for detecting these relaxation errors, and then uses a very tiny fully-connected neural network for reducing crosstalk errors encountered during readout. These optimizations make HERQULES orders of magnitude more scalable than other neural network based classifiers.

In Chapter 4, we introduced an FPGA-tailored clustering decoder for quantum LDPC codes. This clustering decoder uses belief propagation as a pre-decoder, by using its outputs to guide the initial clusters on the decoding graph. To make processing on an FPGA more efficient, the clustering decoder instantiates clusters as one dimensional binary bitmaps, with lengths equal to the total detector and faults in the decoding graph. This representation makes cluster growth and merge operations relatively straightforward using simple boolean logic. For solving a cluster to determine its validity (whether the detectors enclosed in the cluster have a support within that cluster), the clustering decoder uses a hardware-friendly linear system of equations solver over binary fields that is designed as a systolic array. This allows for fast FPGA clocks and a linear runtime complexity. While the proposed clustering decoder does not perform as well as pure message passing decoders like Relay, it gives us an important insight into what kind of decoders must be pursued in the future for real-time

usage. Furthermore, the clustering decoder could be useful for decoding logical operations and could also be improved by using Relay within a smaller cluster.

In Chapter 5, we moved further up the control stack and exposed two problems large-scale fault-tolerant quantum computers will face. The first is resource provisioning for decoding, and the second is the synchronization of logical qubits for executing logical operations. While Chapter 4 focused on a single decoder in isolation, a real FTQC system will require hundreds to thousands of decoders to decode all logical qubits in the system. Furthermore, these decoders will be custom, accelerated decoders implemented on FPGAs or ASICs. It is thus necessary from a scalability standpoint to know what the optimal number of decoders is for a given QEC code and application. However, for a given system decoding capacity, it is also important to know how decoding is scheduled, especially if the demand for decoding is variable. We introduce coarse- and fine-grained scheduling policies, that create a set of priorities that must be followed in order to prevent resource (decoder) starvation while ensuring an efficient use of all available decoders. Through the use of these scheduling policies, we were able to reduce the number of decoders by up to 40% compared to a simple worst-case provisioning of decoders. Just as utility-scale fault-tolerant quantum computers will require a system of decoders, logical operations between logical qubits on these computers will require those qubits to be synchronized. As systems scale, guaranteed synchronization between these logical qubits will become harder. This will be further exacerbated by the use of multiple QEC codes in the same system. We introduce three synchronization policies that aim to reduce the cost of synchronizing logical qubits – the *Active*, *Hybrid*, and *Passive* policies. The *Active* and *Hybrid* policies reduce the cost of synchronization by proactively inserting delays between QEC rounds, reducing the overall effect the synchronization idle time has on the logical error rate. While these policies were primarily evaluated for the surface code, they will be applicable for all kinds of codes and qubit technologies.

7.2 Future Work

Despite the contributions of this dissertation, there are many open and unexplored problems that must be addressed to build useful fault-tolerant quantum computers.

Real-time decoding graph partitioning. Chapter 5 presented elastic decoders, where decoding was scheduled on decoders based on predefined heuristics. However, decoders operate on decoding graphs which in turn are determined from the physical qubits that form the logical block. By dynamically assigning decoders to logical qubits, these graphs must be partitioned in real time. The need for partitioning, however, is not specific to scheduling: logical operations themselves reshape the decoding graph. A lattice surgery merge or split reconfigures the surface-code matching graph at runtime, and spatially parallel decoders must therefore have their windows—and the artificial boundaries between them—generated on the fly rather than assumed at compile time, an aspect left open by existing parallel- window decoding schemes [234]. This is especially important for qLDPC codes, where complex ancilla systems are used for measuring Pauli operators [51, 280]: these constructions induce time-varying decoding graphs with little geometric locality, so the planar window boundaries that suffice for the surface code do not transfer, and partitioning becomes a substantially harder problem. Because partition boundaries cut correlated error mechanisms, partition quality trades directly against decoding accuracy, making real-time partitioning a joint accuracy–latency optimization rather than a purely geometric one.

Handling heterogeneous decoders. Research on pre-decoders, along with logical operations that utilize ancilla systems [50, 51], introduces multiple types of decoders in the same system. For example, NVIDIA’s AI-based pre-decoders [42] use a transformer architecture on high-end GPUs to resolve common syndromes at sub-microsecond latencies, escalating only the residual, harder syndromes to a slower but more accurate second-level decoder. Coupling such a pre-decoder with a global decoder that may itself be implemented on separate hardware (to avoid long decoder tail latencies) makes scheduling substantially more complex. The decoders no longer differ only in speed and accuracy: they occupy different points in a latency-and-locality hierarchy, with a pre-decoder placed close to the QPU while a GPU-based global decoder may reside at room temperature or on a separate accelerator, where the data-movement latency between them consumes part of the real-time budget. Scheduling must therefore decide not only *which* decoder handles a syndrome but *where* it runs and at what communication cost—a joint placement-and-scheduling problem. This heterogeneity also interacts with the active/idle asymmetry exploited in Chapter 5: lightweight pre-decoders may suffice for idle memory qubits, while accurate decoders are reserved for active qubits on the critical path and for magic states that demand very low

logical error rates. Finally, because pre-decoders have low average but high tail latency on escalation, a real-time system must provision for the tail rather than the mean, requiring synergy between all components.

Minimizing control system latencies for magic state production. Magic states are the most important algorithmic resource for executing programs on a fault-tolerant quantum computer. Preparing these states using techniques such as cultivation [98] requires a tight classical control loop: cultivation is post-selected, growing a state and then accepting or rejecting it based on stabilizer and GHZ checks and complementary-gap thresholding, restarting on rejection. Because the acceptance probability decreases sharply with code distance, the time to produce a usable state is variable and data-dependent. This variability makes magic state production not only a throughput concern but a primary source of the very desynchronization studied earlier in this chapter, since a consumer waiting on a cultivated state cannot know in advance when it will arrive. Reducing the classical control latency in this loop therefore has a dual benefit: it improves cultivation throughput and it tightens the timing variance that synchronization policies must absorb. Crucially, the feedback budget is set by the same per-round cycle time ($\sim 1\ \mu\text{s}$ for superconducting qubits) that the control and readout optimizations of Chapter 3 target, so the low-latency waveform control and qubit-state discrimination developed there feed directly into this loop. Understanding the impact of classical control latency on the efficacy of these protocols is an open question, and co-optimizing the control system to reduce this feedback latency across waveform generation, readout, and decoding is an interesting problem.

Fault tolerance of the classical control infrastructure. As the number of logical qubits grows to thousands and the classical control infrastructure expands to match—racks of FPGAs, ASICs, and GPUs, connected by high-speed networks. The classical hardware itself will experience failures at non-negligible rates. A single decoder failure today stalls one logical qubit; at scale, classical failures could corrupt Pauli frame tracking, mis-route syndrome data, or silently drop critical decode outputs. Making the classical control infrastructure fault-tolerant with redundancy, error detection, and reconfiguration protocols is an open engineering and systems problem that has received almost no attention in the literature.

7.3 Closing Remarks

We are in exciting times in the world of computing. With AI and quantum computing, we are making fast progress towards solving problems that might have looked impossible a decade ago. While the field of quantum computing is still in its infancy, the progress made over the past five years is incredible. Most of the last decade was spent maturing qubit technologies, QEC codes, and developing algorithms that can exploit the computational advantage offered by quantum computers. The next decade should see rapid scaling in the number of qubits, more advances in QEC codes and decoding, and more experimental demonstrations of QEC. But scaling the number of qubits is only half the story. Every additional logical qubit must be controlled, read out, decoded, and coordinated, and the cost of these classical responsibilities grows with the machine. If the control system does not scale alongside the qubits, it (not just the qubits) becomes the bottleneck. This dissertation has worked to keep that from happening: by reducing the cost of control and readout, accelerating decoding, and introducing a runtime layer to orchestrate decoders and logical qubits, it advances the classical infrastructure on which a scalable fault-tolerant quantum computer will depend. Some of these ideas are useful today, but it is the forthcoming era of scaling that they truly target.

Remember to look up at the stars and not down at your feet.

— Stephen Hawking

Appendix A

Tools and Frameworks

In this appendix, we briefly describe software tools and frameworks created for evaluating designs and ideas presented in this dissertation.

A.1 HERQULES

HERQULES, presented in Chapter 3, uses a hierarchical approach for qubit state discrimination. The first level consists of two pairs of matched filters per qubit – one for discriminating between states 0 and 1, and the other for detecting $1 \rightarrow 0$ transitions occurring during readout. The second level is a small neural network that combines the outputs of the matched filters from all qubits. This achieves two things: selects the right matched filter outputs for all qubits, and detects crosstalk errors due to multiplexed readout.

HERQULES is available online at: <https://github.com/satvikmaurya/HERQULES>. We now describe the training and inference subroutines of HERQULES.

A.1.1 Training

Training proceeds in two stages, given in Algorithm 4. The matched filters in the first stage are constructed analytically from labeled calibration traces—each envelope is the variance-normalized mean difference of two trace populations, windowed to shorten the integration time—and are not trained. With the filters frozen, the second stage trains the $10 \rightarrow 10 \rightarrow 20 \rightarrow 32$ multilayer perceptron on their ten outputs against the true joint-state labels.

Algorithm 4: HERQULES training (two stages)

Require: Calibration traces per qubit q : ground X_0^q , excited X_1^q , relaxation X_r^q ;
window lengths w_q ; joint-state labels y

Ensure: Frozen filters $\{e_q^{01}, e_q^r\}$ and trained network $\mathcal{N}$

// Stage 1 - build matched filters (analytic, no training)

- 1 **for** each qubit $q \in \{1, \dots, 5\}$ **do**
- 2 balance class sizes of (X_0^q, X_1^q) by random subsampling
- 3 $f_q^{01} \leftarrow \text{mean}(X_0^q - X_1^q) / \text{var}(X_0^q - X_1^q)$ *// SNR-weighted envelope*
- 4 $f_q^r \leftarrow \text{mean}(X_r^q - X_0^q) / \text{var}(X_r^q - X_0^q)$ *// 1 $\rightarrow$ 0 detector*
- 5 $b_q \leftarrow \text{BoxCAR}(w_q)$ *// truncate integration window*
- 6 $e_q^{01} \leftarrow f_q^{01} \odot b_q$; $e_q^r \leftarrow f_q^r \odot b_q$
- 7 **end**

// Stage 2 - train the second-level network on filter outputs

- 8 **for** each training shot with traces $\{x_q\}$ **do**
- 9 $z \leftarrow [\langle x_q, e_q^{01} \rangle, \langle x_q, e_q^r \rangle]_{q=1}^5$ *// 10 features*
- 10 **end**
- 11 initialize MLP $\mathcal{N} : 10 \rightarrow 10 \rightarrow 20 \rightarrow 32$ (ReLU, Xavier)
- 12 **for** epoch = 1 **to** 100 **do**
- 13 $\eta \leftarrow 0.01 \times 0.1^{\lfloor t \in \{30, 60, 90\} : \text{epoch} \geq t \rfloor}$ *// $\times 0.1$ at epochs 30, 60, 90*
- 14 update $\mathcal{N}$ with Adam on cross-entropy loss, batch size 512
- 15 checkpoint $\mathcal{N}$ if validation accuracy improves
- 16 **end**
- 17 **return** best-validation $\mathcal{N}$ and frozen filters

A.1.2 Inference

Inference is strictly feed-forward, with no iterative computation, as shown in Algorithm 5. Each trace is reduced to its matched-filter outputs by one dot product per filter; the ten scalars pass through the network in a single forward pass; and the arg max of the 32-way softmax gives the joint state, from which per-qubit assignments are recovered by bit extraction. This keeps the discriminator within the latency budget of on-hardware readout.

A.2 decoder-bench: Benchmarking QEC Decoders

In Chapter 4, we presented an FPGA-tailored version of the clustering decoder for qLDPC codes. This research revealed a gap in the benchmarking of decoders – every researcher used

Algorithm 5: HERQULES inference (feed-forward)

Require: Readout traces $\{\mathbf{x}_q\}_{q=1}^5$; frozen filters $\{\mathbf{e}_q^{01}, \mathbf{e}_q^r\}$; network $\mathcal{N}$

Ensure: Per-qubit assignments $(s_1, \dots, s_5)$

```

1 for each qubit  $q \in \{1, \dots, 5\}$  do
  | // Level 1: one dot product per filter
2  |  $z_{2q-1} \leftarrow \langle \mathbf{x}_q, \mathbf{e}_q^{01} \rangle$ ;  $z_{2q} \leftarrow \langle \mathbf{x}_q, \mathbf{e}_q^r \rangle$ 
3  end
4   $\mathbf{p} \leftarrow \text{softmax}(\mathcal{N}(\mathbf{z}))$  // Level 2: single forward pass, 32-way
5   $k \leftarrow \arg \max_j p_j$  // joint five-qubit state
6  for  $q = 1$  to 5 do
  | // recover per-qubit bits
7  |  $s_q \leftarrow k \bmod 2$ ;  $k \leftarrow \lfloor k/2 \rfloor$ 
8  end
9 return  $(s_1, \dots, s_5)$ 

```

their own noise model and assumptions while evaluating decoders, which resulted in inconsistencies across different evaluations and papers. To standardize decoder benchmarking for QEC, we thus developed a framework decoder-bench [179].

decoder-bench is available online at: <https://github.com/satvikmaurya/decoder-bench>. We now give a brief overview of decoder-bench.

A.2.1 Trace Generation

The following information is needed to evaluate a QEC decoder:

- The parity check matrix H .
- Syndromes obtained by repeatedly¹ measuring the check qubits.
- A list of *priors*, or weights, which correspond to the probability of a fault happening. The length of this list is equal to the number of columns of H .
- The observable matrix A : This matrix maps the faults of the check matrix to the logical qubits' information stored in the code.

¹The number of repetitions is predefined in the QEC circuit for simulations. In reality, the number of repetitions of the syndrome generation circuit depends on the program.

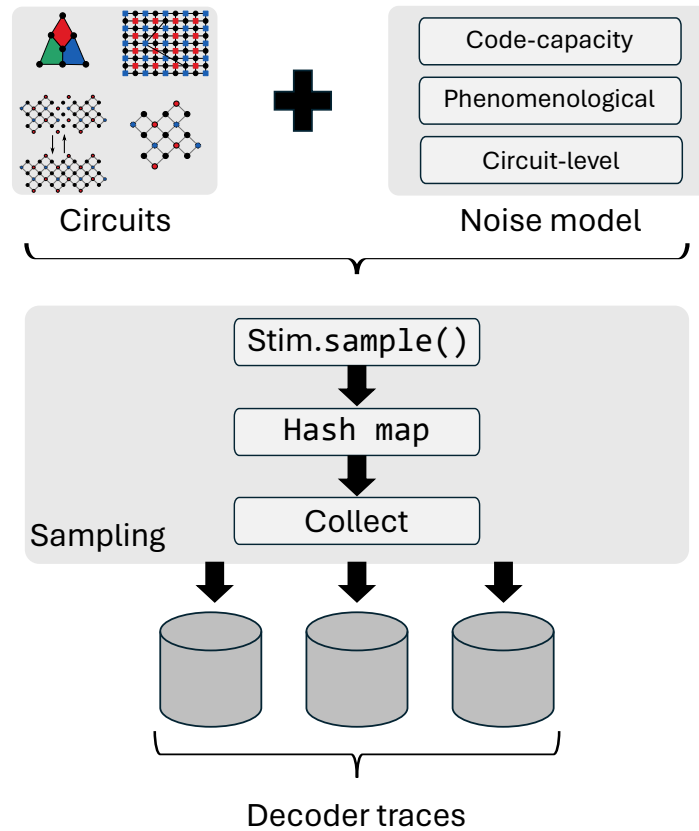


Figure A.1: decoder-bench: Given a circuit and noise model, decoder-bench generates the parity check matrix, priors, the observable matrix, and samples syndromes, observables using Stim. A hash map is used for ensuring only unique syndromes (and observables) are sampled.

- The measured observables Ω : The final measurement of the *logical* qubit(s) in the code.
A ‘trace’ dumped by decoder-bench is thus a dataset that includes all of the above information. The blocks used to generate these traces, as shown in Figure A.1, are described in greater detail below.

A.2.1.1 Circuits

decoder-bench takes a circuit-first approach, where circuits are defined using Stim [94]. This allows any stabilizer circuit that represents a QEC code to be used for benchmarking decoders. decoder-bench currently supports circuit generation for surface code, color code, and bivariate bicycle (qLDPC) code [29] memory experiments as well as surface code

lattice surgery experiments. These circuits can be generated for any code distance, and in any measurement basis (X/Z).

A.2.1.2 Noise models

`decoder-bench` supports three noise models: code-capacity, phenomenological, and circuit-level. The code-capacity and phenomenological noise models add depolarizing noise to the data qubits at the start of every round with the configured error rate p . The phenomenological noise model adds measurement errors with an error rate of p as well. The circuit-level noise model is derived from the SI1000 noise model used in [6]. This noise model assumes the use of superconducting qubits with a syndrome generation cycle time of 1000 ns. Error rates are operation-specific – for a configured physical error rate p ,

- Measurements have an error rate of $5p$.
- Single-qubit gates have an error rate of $p/10$.
- Two-qubit gates have an error rate of p .
- Resets have an error rate of $2p$.
- Idling qubits experience depolarizing errors with an error rate of $p/10$.

With a noise-annotated circuit, `decoder-bench` can generate the H, A matrices as well as the list of priors. Generating syndromes and observables is then performed by the sampling block.

A.2.1.3 Sampling

`decoder-bench` uses Stim [94] to sample syndromes and observables from a noise-annotated circuit. Since this sampling is random, the same syndrome can be sampled multiple times, especially for small code distances. To avoid this, `decoder-bench` hashes the syndromes it has seen already and stores them in a hash map. This allows `decoder-bench` to store only unique syndromes. Syndromes and observables have a 1:1 mapping, so discarding a duplicate syndrome discards the corresponding observable as well. This sampling methodology of ensuring datasets with unique syndromes is advantageous because:

- Decoders are tested on a wider variety of error patterns.
- Decoders are evaluated fairly on the same set of decoding problems.

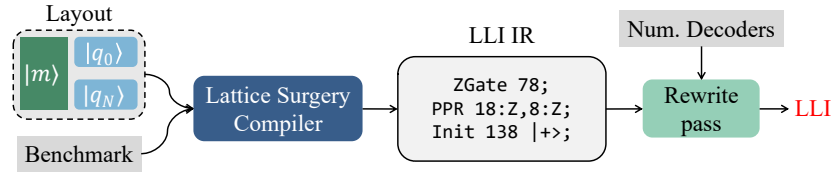


Figure A.2: Compiler workflow – our rewrite pass changes the IR schedule based on the number of available decoders.

- Decoders that target common/easy to decode syndromes are also compared fairly.
- Redundancies are eliminated from the datasets.

A.2.2 Trace Evaluation

Once the traces for a particular decoding problem have been generated, the accuracy of a decoder can be evaluated using the following methodology:

- Given an input syndrome, the parity check matrix, and the priors, the decoder makes a prediction $\hat{F}$ that satisfies $HF = \sigma$.
- The predicted state of the logical qubit (observable) is $\Omega' = A\hat{F}$.
- The measured observable stored in the trace dataset is Ω . A logical error (and thus a decoder error) is deemed to have happened if $\Omega \neq \Omega'$.

A.3 Elastic Decoders and Scheduling

In Chapter 5, we presented the notion of elastic decoders and introduced coarse- and fine-grained decoder scheduling policies. Figure A.2 shows the high-level methodology for decoder provisioning and scheduling. The source code is available at <https://github.com/satvikmaurya/decoder-resources>.

A.3.1 Scheduling

Each logical qubit accrues a *temporal* backlog of undecoded syndrome rounds, and each lattice-surgery operation additionally creates a *spatial* decoding task whose size scales with the routing distance between the participating patches. Algorithm 7 processes one

slice at a time: it generates the temporal and spatial tasks for that slice, assigns priorities (critical non-Clifford decodes highest, then Y-state injection, then routine decodes), and serves them from the decoder pool in priority order, with temporal tasks preferred over spatial ones. A task that cannot be fully served is either partially decoded or deferred to the next slice. Any decoders left over are spent on the remaining qubits according to the scheduling policy—MLS (decode the longest backlog first), RR (round-robin), or MFD (use a lookahead over the IR to favor the most frequently decoded qubits). Qubits left undecoded in a cycle accrue further backlog, and the simulator records both the decoder utilization and the longest backlog across the run.

A.3.2 Capacity planning

Given the scheduler, we then provision the smallest decoder pool that still meets the real-time deadline. Algorithm 6 binary-searches the pool size: for a candidate count it runs the scheduler and checks whether the resulting longest backlog stays within a fixed tolerance, shrinking the budget when feasible and growing it otherwise. The result is the minimum number of decoders the program requires under a given policy, which we show is far below the worst- and average-case baselines because active and idle qubits make very different demands on the pool.

Algorithm 6: Decoder capacity planning (provisioning)

Require: Benchmark IR; policy π ; backlog tolerance β

Ensure: Minimum decoder count N^* meeting the real-time deadline

```

1  $lo \leftarrow 1$ ;  $hi \leftarrow$  number of logical qubits;  $N^* \leftarrow hi$ 
2 while  $lo \leq hi$  do
    // binary search on pool size
3    $N \leftarrow \lfloor (lo + hi) / 2 \rfloor$ 
4    $backlog \leftarrow \text{SCHEDULE}(N, \pi).longestBacklog$  // Algorithm 7
5   if  $backlog \leq \beta$  then
6      $N^* \leftarrow N$ ;  $hi \leftarrow N - 1$  // feasible: try fewer
7   else
8      $lo \leftarrow N + 1$  // infeasible: need more
9   end
10 end
11 return  $N^*$ 

```

Algorithm 7: Elastic decoder scheduling (per program slice)

Require: Compiled IR slices $\{S_t\}$; decoder budget N ; policy $\pi \in \{\text{MLS}, \text{RR}, \text{MFD}\}$;
window size w

Ensure: Per-qubit backlogs and decoder utilization over time

```

1   $b_q \leftarrow 0$  for every logical qubit  $q$            // temporal syndrome backlog
2  for each slice  $S_t$  do
3       $pool \leftarrow N$ ;  $Q \leftarrow \emptyset$ ;  $heap \leftarrow \emptyset$            // fresh budget each cycle
4      for each operation in  $S_t$  do
5          if op is REQUESTMAGICSTATE then
6              | record pending non-Clifford operand
7          end
8          else if op is MULTIBODYMEASURE on operand  $\rho$  then
9              |  $p \leftarrow 1000$  if  $\rho$  non-Clifford (critical), else 5 if Y-injection, else 1
10             | push spatial task:  $\lceil \text{dist}(\rho)/w \rceil$  decoders, priority  $p$ 
11             | for  $q \in \rho$  with  $b_q > 0$  do
12                 | push temporal task:  $\lceil b_q/w \rceil$  decoders, priority  $p$ 
13             | end
14         end
15     end
16     // serve operation-driven tasks by priority (temporal before spatial)
17     while  $heap \neq \emptyset$  do
18         |  $\tau \leftarrow \text{pop-max}(heap)$ ;  $d \leftarrow \text{decoders}(\tau)$ 
19         | if  $d \leq pool$  then
20             | assign:  $pool \leftarrow pool - d$ ; reset  $b_q$ ; mark  $q \in Q$ 
21         | end
22         | else if  $\tau$  temporal then
23             | assign partial  $\min(pool, d)$ ; reduce  $b_q$ 
24         | end
25         | else
26             |  $\tau.\text{priority}++$ ; defer  $\tau$  to next slice
27         | end
28     end
29     // spend leftover decoders by policy  $\pi$ 
30     order undecoded qubits by  $\pi$ : longest backlog (MLS) / round-robin (RR) /
31     lookahead critical-decode count (MFD)
32     for  $q$  in that order while  $pool > 0$  do
33         |  $d \leftarrow \min(pool, \lceil b_q/w \rceil)$ ;  $pool \leftarrow pool - d$ ; reduce  $b_q$ ; mark  $q \in Q$ 
34     end
35      $b_q += 2\ell_{\text{dec}}$  for every  $q \notin Q$            // undecoded qubits accrue backlog
36     record utilization  $N - pool$  and  $\max_q b_q$ 
37 end
38 drain deferred critical tasks past the last slice (bounded)
39 return backlog history, decoder utilization, longest backlog  $\max_q b_q$ 

```

Bibliography

- [1] D. Aharonov and M. Ben-Or. Fault-tolerant quantum computation with constant error. In *Proceedings of the twenty-ninth annual ACM symposium on Theory of computing - STOC '97*, STOC '97, page 176–188. ACM Press, 1997.
- [2] Dorit Aharonov and Michael Ben-Or. Fault-tolerant quantum computation with constant error. In *Proceedings of the twenty-ninth annual ACM symposium on Theory of computing*, pages 176–188. ACM, 1997.
- [3] Nasir Ahmed, T_ Natarajan, and Kamisetty R Rao. Discrete cosine transform. *IEEE transactions on Computers*, 100(1):90–93, 1974.
- [4] Google Quantum AI. Exponential suppression of bit or phase errors with cyclic error correction. *Nature*, 595(7867):383–387, July 2021.
- [5] Google Quantum AI. Suppressing quantum errors by scaling a surface code logical qubit. *Nature*, 614(7949):676–681, February 2023.
- [6] Google Quantum AI. Quantum error correction below the surface code threshold, 2024.
- [7] Alaa R Alameldeen and David A Wood. Adaptive cache compression for high-performance processors. In *Proceedings. 31st Annual International Symposium on Computer Architecture, 2004.*, pages 212–223. IEEE, 2004.
- [8] Narges Alavisamani, Suhas Vittal, Ramin Ayanzadeh, Poulami Das, and Moinuddin Qureshi. Promatch: Extending the reach of real-time quantum error correction with adaptive predecoding, 2024.

- [9] Thomas Alexander, Naoki Kanazawa, Daniel J Egger, Lauren Capelluto, Christopher J Wood, Ali Javadi-Abhari, and David C McKay. Qiskit pulse: programming quantum computers through the cloud with pulses. *Quantum Science and Technology*, 5(4):044006, August 2020.
- [10] Constantia Alexandrou, Lena Funcke, Tobias Hartung, Karl Jansen, Stefan Kuehn, Georgios Polykratis, Paolo Stornati, and Xiaoyang Wang. Using classical bit-flip correction for error mitigation including 2-qubit correlations, 2021.
- [11] Frank Arute, Kunal Arya, Ryan Babbush, Dave Bacon, Joseph C. Bardin, Rami Barends, Rupak Biswas, Sergio Boixo, Fernando G. S. L. Brandao, David A. Buell, Brian Burkett, Yu Chen, Zijun Chen, Ben Chiaro, Roberto Collins, William Courtney, Andrew Dunsworth, Edward Farhi, Brooks Foxen, Austin Fowler, Craig Gidney, Marissa Giustina, Rob Graff, Keith Guerin, Steve Habegger, Matthew P. Harrigan, Michael J. Hartmann, Alan Ho, Markus Hoffmann, Trent Huang, Travis S. Humble, Sergei V. Isakov, Evan Jeffrey, Zhang Jiang, Dvir Kafri, Kostyantyn Kechedzhi, Julian Kelly, Paul V. Klimov, Sergey Knysh, Alexander Korotkov, Fedor Kostritsa, David Landhuis, Mike Lindmark, Erik Lucero, Dmitry Lyakh, Salvatore Mandrà, Jarrod R. McClean, Matthew McEwen, Anthony Megrant, Xiao Mi, Kristel Michielsen, Masoud Mohseni, Josh Mutus, Ofer Naaman, Matthew Neeley, Charles Neill, Murphy Yuezhen Niu, Eric Ostby, Andre Petukhov, John C. Platt, Chris Quintana, Eleanor G. Rieffel, Pedram Roushan, Nicholas C. Rubin, Daniel Sank, Kevin J. Satzinger, Vadim Smelyanskiy, Kevin J. Sung, Matthew D. Trevithick, Amit Vainsencher, Benjamin Villalonga, Theodore White, Z. Jamie Yao, Ping Yeh, Adam Zalcman, Hartmut Neven, and John M. Martinis. Quantum supremacy using a programmable superconducting processor. *Nature*, 574(7779):505–510, October 2019.
- [12] James M. Auger, Hussain Anwar, Mercedes Gimeno-Segovia, Thomas M. Stace, and Dan E. Browne. Fault-tolerance thresholds for the surface code with fabrication errors. *Physical Review A*, 96(4), October 2017.
- [13] Ben Barber, Kenton M. Barnes, Tomasz Bialas, Okan Buğdaycı, Earl T. Campbell, Neil I. Gillespie, Kauser Johar, Ram Rajan, Adam W. Richardson, Luka Skoric, Canberk Topal, Mark L. Turner, and Abbas B. Ziad. A real-time, scalable, fast and highly resource efficient decoder for a quantum computer, 2023.

- [14] Joseph C. Bardin, Ted White, Marissa Giustina, Kevin J. Satzinger, Kunal Arya, Pedram Roushan, Benjamin Chiaro, Julian Kelly, Zijun Chen, Brian Burkett, Yu Chen, Evan Jeffrey, Andrew Dunsworth, Austin Fowler, Brooks Foxen, Craig Gidney, Rob Graff, Paul Klimov, Josh Mutus, Matthew J. McEwen, Matthew Neeley, Charles J. Neill, Erik Lucero, Chris Quintana, Amit Vainsencher, Hartmut Neven, John Martinis, Trent Huang, Sayan Das, Daniel Thomas Sank, Ofer Naaman, Anthony Edward Megrant, and Rami Barends. Design and Characterization of a 28-nm Bulk-CMOS Cryogenic Quantum Controller Dissipating Less Than 2 mW at 3 K. *IEEE Journal of Solid-State Circuits*, 54(11):3043–3060, November 2019.
- [15] Sanjoy K. Baruah, Vincenzo Bonifaci, Gianlorenzo D’Angelo, Alberto Marchetti-Spaccamela, Suzanne van der Ster, and Leen Stougie. *Mixed-Criticality Scheduling of Sporadic Task Systems*, page 555–566. Springer Berlin Heidelberg, 2011.
- [16] Daniel Bascones, Francisco Garcia-Herrero, and Javier Valls. Exploring the FPGA and ASIC design space of belief propagation and ordered statistics decoders for quantum error correction codes. July 2025.
- [17] Laleh Aghababaie Beni, Oscar Higgott, and Noah Shetty. Tesseract: A search-based decoder for quantum error correction. *arXiv preprint arXiv:2503.10988*, 2025.
- [18] Paul Benioff. The computer as a physical system: A microscopic quantum mechanical hamiltonian model of computers as represented by turing machines. *Journal of statistical physics*, 22(5):563–591, 1980.
- [19] Lucas Berent, Lukas Burgholzer, Peter-Jan H.S. Derks, Jens Eisert, and Robert Wille. Decoding quantum color codes with maxsat. *Quantum*, 8:1506, October 2024.
- [20] E. Berlekamp, R. McEliece, and H. van Tilborg. On the inherent intractability of certain coding problems (corresp.). *IEEE Transactions on Information Theory*, 24(3):384–386, May 1978.
- [21] Michael Beverland, Vadym Kliuchnikov, and Eddie Schoute. Surface code compilation via edge-disjoint paths. *PRX Quantum*, 3(2), May 2022.
- [22] Michael E Beverland, Prakash Murali, Matthias Troyer, Krysta M Svore, Torsten Hoeffler, Vadym Kliuchnikov, Guang Hao Low, Mathias Soeken, Aarthi Sundaram, and Alexander Vaschillo. Assessing requirements to scale to practical quantum advantage. *arXiv preprint arXiv:2211.07629*, 2022.
- [23] John Blue, Harshil Avlani, Zhiyang He, Liu Ziyin, and Isaac L Chuang. Machine learning decoding of circuit-level noise for bivariate bicycle codes. *arXiv preprint arXiv:2504.13043*, 2025.

- [24] Dolev Bluvstein, Simon J. Evered, Alexandra A. Geim, Sophie H. Li, Hengyun Zhou, Tom Manovitz, Sepehr Ebadi, Madelyn Cain, Marcin Kalinowski, Dominik Hangleiter, J. Pablo Bonilla Ataides, Nishad Maskara, Iris Cong, Xun Gao, Pedro Sales Rodriguez, Thomas Karolyshyn, Giulia Semeghini, Michael J. Gullans, Markus Greiner, Vladan Vuletić, and Mikhail D. Lukin. Logical quantum processor based on reconfigurable atom arrays. *Nature*, 626(7997):58–65, December 2023.
- [25] Dolev Bluvstein, Simon J Evered, Alexandra A Geim, Sophie H Li, Hengyun Zhou, Tom Manovitz, Sepehr Ebadi, Madelyn Cain, Marcin Kalinowski, Dominik Hangleiter, et al. Logical quantum processor based on reconfigurable atom arrays. *Nature*, 626(7997):58–65, 2024.
- [26] Dolev Bluvstein, Alexandra A Geim, Sophie H Li, Simon J Evered, J Pablo Bonilla Ataides, Gefen Baranes, Andi Gu, Tom Manovitz, Muqing Xu, Marcin Kalinowski, et al. A fault-tolerant neutral-atom architecture for universal quantum computation. *Nature*, 649(8095):39–46, 2026.
- [27] Héctor Bombín, Chris Dawson, Ye-Hua Liu, Naomi Nickerson, Fernando Pastawski, and Sam Roberts. Modular decoding: parallelizable real-time decoding for quantum computers, 2023.
- [28] Stevan D Bradley. Optimizing a scheme for run length encoding. *Proceedings of the IEEE*, 57(1):108–109, 1969.
- [29] Sergey Bravyi, Andrew W Cross, Jay M Gambetta, Dmitri Maslov, Patrick Rall, and Theodore J Yoder. High-threshold and low-overhead fault-tolerant quantum memory. *Nature*, 627(8005):778–782, 2024.
- [30] Sergey Bravyi, Andrew W. Cross, Jay M. Gambetta, Dmitri Maslov, Patrick Rall, and Theodore J. Yoder. High-threshold and low-overhead fault-tolerant quantum memory. *Nature*, 627(8005):778–782, March 2024.
- [31] Sergey Bravyi, Sarah Sheldon, Abhinav Kandala, David C. McKay, and Jay M. Gambetta. Mitigating measurement errors in multiqubit experiments. *Phys. Rev. A*, 103:042605, Apr 2021.
- [32] N T Bronn, B Abdo, K Inoue, S Lekuch, A D Córcoles, J B Hertzberg, M Takita, L S Bishop, J M Gambetta, and J M Chow. Fast, high-fidelity readout of multiple qubits. *Journal of Physics: Conference Series*, 834:012003, May 2017.
- [33] KZ Bukhari, Georgi Kuzmanov, and Stamatis Vassiliadis. Dct and idct implementations on different fpga technologies. In *Proceedings of ProRISC*, pages 232–235. Citeseer, 2002.

- [34] Friederike Butt, Sascha Heußen, Manuel Rispler, and Markus Müller. Fault-tolerant code-switching protocols for near-term quantum processors. *PRX Quantum*, 5(2), May 2024.
- [35] Jonas Bylander, Simon Gustavsson, Fei Yan, Fumiki Yoshihara, Khalil Harrabi, George Fitch, David G. Cory, Yasunobu Nakamura, Jaw-Shen Tsai, and William D. Oliver. Noise spectroscopy through dynamical decoupling with a superconducting flux qubit. *Nature Physics*, 7(7):565–570, May 2011.
- [36] Ilkwon Byun, Junpyo Kim, Dongmoon Min, Ikki Nagaoka, Kosuke Fukumitsu, Iori Ishikawa, Teruo Tanimoto, Masamitsu Tanaka, Koji Inoue, and Jangwoo Kim. Xqsim: modeling cross-technology control processors for 10+k qubit quantum computers. In *Proceedings of the 49th Annual International Symposium on Computer Architecture, ISCA '22*. ACM, June 2022.
- [37] Madelyn Cain, Qian Xu, Robbie King, Lewis R. B. Picard, Harry Levine, Manuel Endres, John Preskill, Hsin-Yuan Huang, and Dolev Bluvstein. Shor’s algorithm is possible with as few as 10, 000 reconfigurable atomic qubits, 2026.
- [38] Laura Caune, Brendan Reid, Joan Camps, and Earl Campbell. Belief propagation as a partial decoder. *arXiv preprint arXiv:2306.17142*, 2023.
- [39] Sudipto Chakraborty, David J. Frank, Kevin Tien, Pat Rosno, Mark Yeck, Joseph A. Glick, Raphael Robertazzi, Ray Richetta, John F. Bulzacchelli, Devin Underwood, Daniel Ramirez, Dereje Yilma, Andrew Davies, Rajiv V. Joshi, Shawn D. Chambers, Scott Lekuch, Ken Inoue, Dorothy Wisnieff, Christian W. Baks, Donald S. Bethune, John Timmerwille, Thomas Fox, Peilin Song, Blake R. Johnson, Brian P. Gaucher, and Daniel J. Friedman. A cryo-cmos low-power semi-autonomous transmon qubit state controller in 14-nm finfet technology. *IEEE Journal of Solid-State Circuits*, 57(11):3258–3273, November 2022.
- [40] Christopher Chamberland and Earl T. Campbell. Circuit-level protocol and analysis for twist-based lattice surgery. *Physical Review Research*, 4(2), May 2022.
- [41] Christopher Chamberland and Kyungjoo Noh. Very low overhead fault-tolerant magic state preparation using redundant ancilla encoding and flag qubits. *npj Quantum Information*, 6(1), October 2020.
- [42] Christopher Chamberland, Jan Olle, Muyuan Li, Scott Thornton, and Igor Baratta. Fast and accurate ai-based pre-decoders for surface codes, 2026.
- [43] Aniket Chatterjee, Jonathan Schwinger, and Yvonne Y Gao. Enhanced qubit readout via reinforcement learning. *Physical Review Applied*, 23(5):054057, 2025.

- [44] Subiman Chatterjee and Kishor Sarawadekar. An optimized architecture of hevc core transform using real-valued dct coefficients. *IEEE Transactions on Circuits and Systems II: Express Briefs*, 65(12):2052–2056, 2018.
- [45] Yu Chen, D. Sank, P. O'Malley, T. White, R. Barends, B. Chiaro, J. Kelly, E. Lucero, M. Mariantoni, A. Megrant, C. Neill, A. Vainsencher, J. Wenner, Y. Yin, A. N. Cleland, and John M. Martinis. Multiplexed dispersive readout of superconducting phase qubits. *Applied Physics Letters*, 101(18):182601, October 2012.
- [46] Jerry M. Chow, A. D. Córcoles, Jay M. Gambetta, Chad Rigetti, B. R. Johnson, John A. Smolin, J. R. Rozen, George A. Keefe, Mary B. Rothwell, Mark B. Ketchen, and M. Steffen. Simple all-microwave entangling gate for fixed-frequency superconducting qubits. *Physical Review Letters*, 107(8), August 2011.
- [47] Dimitris Chytas, Nithin Raveendran, and Bane Vasic. Enhanced min-sum decoding of quantum codes using previous iteration dynamics. *arXiv preprint arXiv:2501.05021*, 2025.
- [48] A. D. Córcoles, Maika Takita, Ken Inoue, Scott Lekuch, Zlatko K. Mineev, Jerry M. Chow, and Jay M. Gambetta. Exploiting dynamic quantum circuits in a quantum algorithm with superconducting qubits. *Phys. Rev. Lett.*, 127:100501, Aug 2021.
- [49] Henry Corrigan-Gibbs, David J. Wu, and Dan Boneh. Quantum operating systems. In *Proceedings of the 16th Workshop on Hot Topics in Operating Systems*, HotOS '17, page 76–81. ACM, May 2017.
- [50] Alexander Cowtan and Simon Burton. Css code surgery as a universal construction. *Quantum*, 8:1344, May 2024.
- [51] Alexander Cowtan, Zhiyang He, Dominic J. Williamson, and Theodore J. Yoder. Parallel logical measurements via quantum code surgery. *PRX Quantum*, 7(2), May 2026.
- [52] Andrew W. Cross, Lev S. Bishop, John A. Smolin, and Jay M. Gambetta. Open quantum assembly language, 2017.
- [53] M. P. da Silva, C. Ryan-Anderson, J. M. Bello-Rivas, A. Chernoguzov, J. M. Dreiling, C. Foltz, F. Frachon, J. P. Gaebler, T. M. Gatterman, L. Grans-Samuelsson, D. Hayes, N. Hewitt, J. Johansen, D. Lucchetti, M. Mills, S. A. Moses, B. Neyenhuis, A. Paz, J. Pino, P. Siegfried, J. Strabley, A. Sundaram, D. Tom, S. J. Wernli, M. Zanner, R. P. Stutz, and K. M. Svore. Demonstration of logical qubits and repeated error correction with better-than-physical error rates, 2024.
- [54] Lucas Daguerre and Isaac H. Kim. Code switching revisited: low-overhead magic state preparation using color codes, 2024.

- [55] Poulami Das, Christopher A. Pattison, Srilatha Manne, Douglas M. Carmean, Krysta M. Svore, Moinuddin Qureshi, and Nicolas Delfosse. Afs: Accurate, fast, and scalable error-decoding for fault-tolerant quantum computers. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, April 2022.
- [56] Poulami Das, Swamit Tannu, Siddharth Dangwal, and Moinuddin Qureshi. Adapt: Mitigating idling errors in qubits via adaptive dynamical decoupling. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, MICRO '21, page 950–962, New York, NY, USA, 2021. Association for Computing Machinery.
- [57] Poulami Das, Swamit Tannu, and Moinuddin Qureshi. Jigsaw: Boosting fidelity of nisc programs via measurement subsetting. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, MICRO '21, page 937–949, New York, NY, USA, 2021. Association for Computing Machinery.
- [58] Robert I. Davis and Alan Burns. A survey of hard real-time scheduling for multiprocessor systems. *ACM Computing Surveys*, 43(4):1–44, October 2011.
- [59] Dripto M. Debroy, Matt McEwen, Craig Gidney, Noah Shutty, and Adam Zalcman. Luci in the surface code with dropouts, 2024.
- [60] Nicolas Delfosse. Hierarchical decoding to reduce hardware requirements for quantum computing, 2020.
- [61] Nicolas Delfosse, Vivien Londe, and Michael E Beverland. Toward a union-find decoder for quantum ldpc codes. *IEEE Transactions on Information Theory*, 68(5):3187–3199, 2022.
- [62] Nicolas Delfosse and Naomi H Nickerson. Almost-linear time decoding algorithm for topological codes. *arXiv preprint arXiv:1709.06218*, 2017.
- [63] Nicolas Delfosse and Naomi H Nickerson. Almost-linear time decoding algorithm for topological codes. *Quantum*, 5:595, 2021.
- [64] Nicolas Delfosse, Andres Paz, Alexander Vaschillo, and Krysta M. Svore. How to choose a decoder for a fault-tolerant quantum computer? the speed vs accuracy trade-off, 2023.
- [65] Antonio deMarti iOlius, Imanol Etxezarreta Martinez, Joschka Roffe, and Josu Etxezarreta Martinez. An almost-linear time decoding algorithm for quantum ldpc codes under circuit-level noise. *arXiv e-prints*, pages arXiv–2409, 2024.
- [66] Antonio deMarti iOlius and Josu Etxezarreta Martinez. The closed-branch decoder for quantum ldpc codes. *arXiv e-prints*, pages arXiv–2402, 2024.

- [67] David Deutsch. Quantum theory, the church–turing principle and the universal quantum computer. *Proceedings of the royal society of London. A. mathematical and physical sciences*, 400(1818):97–117, 1985.
- [68] David Deutsch and Richard Jozsa. Rapid solution of problems by quantum computation. *Proceedings of the royal society of London. Series A: Mathematical and Physical Sciences*, 439(1907):553–558, 1992.
- [69] Jeroen Petrus Gerardus Van Dijk, Bishnu Patra, Sushil Subramanian, Xiao Xue, Nodar Samkharadze, Andrea Corna, Charles Jeon, Farhana Sheikh, Esdras Juarez-Hernandez, Brando Perez Esparza, Huzaifa Rampurawala, Brent R. Carlton, Surej Ravikumar, Carlos Nieva, Sungwon Kim, Hyung-Jin Lee, Amir Sammak, Giordano Scappucci, Menno Veldhorst, Lieven M. K. Vandersypen, Edoardo Charbon, Stefano Pellerano, Masoud Babaie, and Fabio Sebastiano. A scalable cryo-CMOS controller for the wideband frequency-multiplexed control of spin qubits and transmons. *IEEE Journal of Solid-State Circuits*, 55(11):2930–2946, November 2020.
- [70] Antonis Dimakis and Jean Walrand. Sufficient conditions for stability of longest-queue-first scheduling: Second-order properties using fluid limits. *Advances in Applied probability*, 38(2):505–521, 2006.
- [71] Chunyang Ding, Martin Di Federico, Michael Hatridge, Andrew Houck, Sebastien Leger, Jeronimo Martinez, Connie Miao, David Schuster I, Leandro Stefanazzi, Chris Stoughton, Sara Sussman, Ken Treptow, Sho Uemura, Neal Wilcer, Helin Zhang, Chao Zhou, and Gustavo Cancelo. Experimental advances with the qick (quantum instrumentation control kit) for superconducting quantum hardware. *Physical Review Research*, 6(1), March 2024.
- [72] Zi-Han Ding, Jin-Ming Cui, Yun-Feng Huang, Chuan-Feng Li, Tao Tu, and Guang-Can Guo. Fast high-fidelity readout of a single trapped-ion qubit via machine-learning methods. *Physical Review Applied*, 12(1):014038, 2019.
- [73] David P. DiVincenzo. The physical implementation of quantum computation. *Fortschritte der Physik*, 48(9-11):771–783, 2000.
- [74] Julien Du Crest, Francisco Garcia-Herrero, Mehdi Mhalla, Valentin Savin, and Javier Valls. Check-agnosia based post-processor for message-passing decoding of quantum ldpc codes. *Quantum*, 8:1334, 2024.
- [75] Julien Du Crest, Mehdi Mhalla, and Valentin Savin. Stabilizer inactivation for message-passing decoding of quantum ldpc codes. In *2022 IEEE Information Theory Workshop (ITW)*, pages 488–493. IEEE, 2022.

- [76] Bryan Eastin and Emanuel Knill. Restrictions on transversal encoded quantum gate sets. *Physical Review Letters*, 102(11):110502, 2009.
- [77] D. J. Egger and F. K. Wilhelm. Adaptive hybrid optimal quantum control for imprecisely characterized systems. *Phys. Rev. Lett.*, 112:240503, Jun 2014.
- [78] Fang Fang, Tsuhan Chen, and Rob A Rutenbar. Lightweight floating-point arithmetic: Case study of inverse discrete cosine transform. *EURASIP Journal on Advances in Signal Processing*, 2002(9):1–14, 2002.
- [79] Richard P. Feynman. Simulating physics with computers. *International Journal of Theoretical Physics*, 21(6-7):467–488, 1982.
- [80] Mark Field, Angela Q. Chen, Ben Scharmann, Eyob A. Sete, Feyza Oruc, Kim Vu, Valentin Kosenko, Joshua Y. Mutus, Stefano Poletto, and Andrew Bestwick. Modular superconducting-qubit architecture with a multichip tunable coupler. *Physical Review Applied*, 21(5), May 2024.
- [81] Steven T. Flammia and Joel J. Wallman. Efficient estimation of pauli channels. *ACM Transactions on Quantum Computing*, 1(1):1–32, December 2020.
- [82] Marc PC Fossorier. Iterative reliability-based decoding of low-density parity check codes. *IEEE Journal on selected Areas in Communications*, 19(5):908–917, 2002.
- [83] Marc PC Fossorier and Shu Lin. Soft-decision decoding of linear block codes based on ordered statistics. *IEEE Transactions on information Theory*, 41(5):1379–1396, 1995.
- [84] Austin G. Fowler and Craig Gidney. Low overhead quantum computation using lattice surgery, 2019.
- [85] Austin G Fowler, Matteo Mariantoni, John M Martinis, and Andrew N Cleland. Surface codes: Towards practical large-scale quantum computation. *Physical Review A—Atomic, Molecular, and Optical Physics*, 86(3):032324, 2012.
- [86] Austin G. Fowler, Matteo Mariantoni, John M. Martinis, and Andrew N. Cleland. Surface codes: Towards practical large-scale quantum computation. *Physical Review A*, 86(3), September 2012.
- [87] Austin G Fowler, Ashley M Stephens, and Peter Groszkowski. High-threshold universal quantum computation on the surface code. *Physical Review A*, 80(5):052312, 2009.

- [88] David J. Frank, Sudipto Chakraborty, Kevin Tien, Pat Rosno, Thomas Fox, Mark Yeck, Joseph A. Glick, Raphael Robertazzi, Ray Richetta, John F. Bulzacchelli, Daniel Ramirez, Dereje Yilma, Andrew Davies, Rajiv V. Joshi, Shawn D. Chambers, Scott Lekuch, Ken Inoue, Devin Underwood, Dorothy Wisnieff, Chris Baks, Donald Bethune, John Timmerwilke, Blake R. Johnson, Brian P. Gaucher, and Daniel J. Friedman. A cryo-CMOS low-power semi-autonomous qubit state controller in 14nm FinFET technology. In *2022 IEEE International Solid- State Circuits Conference (ISSCC)*. IEEE, February 2022.
- [89] Sophia Fuhui Lin, Eric C Peterson, Krishanu Sankar, and Prasahnt Sivarajah. Spatially parallel decoding for multi-qubit lattice surgery. *Quantum Science and Technology*, 10(3):035007, April 2025.
- [90] Lena Funcke, Tobias Hartung, Karl Jansen, Stefan Kühn, Paolo Stornati, and Xiaoyang Wang. Measurement error mitigation in quantum computers through classical bit-flip correction. *Physical Review A*, 105(6), June 2022.
- [91] Jay Gambetta, WA Braff, A Wallraff, SM Girvin, and RJ Schoelkopf. Protocols for optimal readout of qubits using a continuous quantum nondemolition measurement. *Physical Review A*, 76(1):012325, 2007.
- [92] Pradeep Kumar Gautam, Shantharam Kalipatnapu, Ujjawal Singhal, Benjamin Lienhard, Vibhor Singh, Chetan Singh Thakur, et al. Low-latency machine learning fpga accelerator for multi-qubit-state discrimination. *arXiv preprint arXiv:2407.03852*, 2024.
- [93] Joydip Ghosh, Austin G. Fowler, and Michael R. Geller. Surface code with decoherence: An analysis of three superconducting architectures. *Phys. Rev. A*, 86:062318, Dec 2012.
- [94] Craig Gidney. Stim: a fast stabilizer circuit simulator. *Quantum*, 5:497, 2021.
- [95] Craig Gidney. Inplace access to the surface code y basis. *Quantum*, 8:1310, April 2024.
- [96] Craig Gidney. How to factor 2048 bit rsa integers with less than a million noisy qubits. *arXiv preprint arXiv:2505.15917*, 2025.
- [97] Craig Gidney and Martin Ekerå. How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits. *Quantum*, 5:433, April 2021.
- [98] Craig Gidney, Noah Shutty, and Cody Jones. Magic state cultivation: growing t states as cheap as cnot gates. 2024.
- [99] Emmanouil Giortamis, Francisco Romão, Nathaniel Tornow, and Pramod Bhatotia. QOS: Quantum operating system. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 429–447, Boston, MA, July 2025. USENIX Association.

- [100] Anqi Gong, Sebastian Cammerer, and Joseph M. Renes. Graph neural networks for enhanced decoding of quantum ldpc codes. In *2024 IEEE International Symposium on Information Theory (ISIT)*, page 2700–2705. IEEE, July 2024.
- [101] Anqi Gong, Sebastian Cammerer, and Joseph M Renes. Toward low-latency iterative decoding of qldpc codes under circuit-level noise. *arXiv preprint arXiv:2403.18901*, 2024.
- [102] Google Quantum AI. Google Quantum Roadmap, 2024. Accessed: September 18, 2025.
- [103] Quantum Computer Datasheet. <https://quantumai.google/hardware/datasheet/weber.pdf>. [Online; accessed 06-15-2021].
- [104] Daniel Gottesman. Stabilizer codes and quantum error correction, 1997.
- [105] Daniel Gottesman. The heisenberg representation of quantum computers. 1998.
- [106] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, STOC ’96, page 212–219, New York, NY, USA, 1996. Association for Computing Machinery.
- [107] Andi Gu, J Ataide, Mikhail D Lukin, and Susanne F Yelin. Scalable neural decoders for practical fault-tolerant quantum computation. *arXiv preprint arXiv:2604.08358*, 2026.
- [108] Xiaorang Guo, Tigran Bunarjyan, Dai Liu, Benjamin Lienhard, and Martin Schulz. Kling: Knowledge distillation-assisted lightweight neural network for qubit readout on fpga. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, pages 1–7. IEEE, 2025.
- [109] Akel Hashim, Ravi K. Naik, Alexis Morvan, Jean-Loup Ville, Bradley Mitchell, John Mark Kreikebaum, Marc Davis, Ethan Smith, Costin Iancu, Kevin P. O’Brien, Ian Hincks, Joel J. Wallman, Joseph Emerson, and Irfan Siddiqi. Randomized compiling for scalable quantum computing on a noisy superconducting quantum processor, 2021.
- [110] Johannes Heinsoo, Christian Kraglund Andersen, Ants Remm, Sebastian Krinner, Theodore Walter, Yves Salathé, Simone Gasparinetti, Jean-Claude Besse, Anton Potocnik, Andreas Wallraff, and Christopher Eichler. Rapid high-fidelity multiplexed readout of superconducting qubits. *Physical Review Applied*, 10(3), September 2018.
- [111] Rebecca Hicks, Bryce Kobrin, Christian W. Bauer, and Benjamin Nachman. Active readout-error mitigation. *Physical Review A*, 105(1), January 2022.

- [112] Oscar Higgott. Pymatching: A python package for decoding quantum codes with minimum-weight perfect matching. *ACM Transactions on Quantum Computing*, 3(3), jun 2022.
- [113] Oscar Higgott, Thomas C Bohdanowicz, Aleksander Kubica, Steven T Flammia, and Earl T Campbell. Improved decoding of circuit noise and fragile boundaries of tailored surface codes. *Physical Review X*, 13(3):031007, 2023.
- [114] Oscar Higgott and Craig Gidney. Sparse blossom: correcting a million errors per core second with minimum-weight matching, 2023.
- [115] Timo Hillmann, Lucas Berent, Armanda O Quintavalle, Jens Eisert, Robert Wille, and Joschka Roffe. Localized statistics decoding: A parallel decoding algorithm for quantum low-density parity-check codes. *arXiv preprint arXiv:2406.18655*, 10, 2024.
- [116] Clare Horsman, Austin G Fowler, Simon Devitt, and Rodney Van Meter. Surface code quantum computing by lattice surgery. *New Journal of Physics*, 14(12):123011, 2012.
- [117] Min-Hsiu Hsieh and François Le Gall. Np-hardness of decoding quantum error-correction codes. *Physical Review A*, 83(5), May 2011.
- [118] Gengyuan Hu, Wanli Ouyang, Chao-Yang Lu, Chen Lin, and Han-Sen Zhong. Efficient and universal neural-network decoder for stabilizer-based quantum error correction. *arXiv preprint arXiv:2502.19971*, 2025.
- [119] Jingwei Hu, Wen Wang, Kris Gaj, Donglong Chen, and Huaxiong Wang. Universal gaussian elimination hardware for cryptographic purposes. *Journal of Cryptographic Engineering*, 14(2):383–397, 2024.
- [120] IBM Brisbane. IBM Brisbane 127-qubit Quantum Computer, 2024. Accessed: October 18, 2024.
- [121] IBM Brisbane. IBM Fez 156-qubit Quantum Computer, 2024. Accessed: October 18, 2024.
- [122] IBM Quantum. IBM Quantum Roadmap, 2024. Accessed: September 18, 2025.
- [123] IBM Quantum Services. <https://quantum-computing.ibm.com/services?services=systems>.
- [124] Ibm readout error mitigation, 2022.
- [125] Tomohiro Itogawa, Yugo Takada, Yutaka Hirano, and Keisuke Fujii. Even more efficient magic state distillation by zero-level distillation, 2024.

- [126] Pavithran Iyer and David Poulin. Hardness of decoding quantum stabilizer codes. *IEEE Transactions on Information Theory*, 61(9):5209–5223, 2015.
- [127] Mohammad Reza Jokar, Richard Rines, Ghasem Pasandi, Haolin Cong, Adam Holmes, Yunong Shi, Massoud Pedram, and Frederic T. Chong. DigiQ: A scalable digital controller for quantum computers using sfq logic, 2022.
- [128] Robert Kent, Benjamin Lienhard, Gregory Lafyatis, and Daniel J Gauthier. Superconducting-qubit readout using next-generation reservoir computing. *Physical Review Applied*, 25(4):044009, 2026.
- [129] Nader Khammassi, Randy W Morris, Shavindra Premaratne, Florian Luthi, Felix Borjans, Satoshi Suzuki, Robert Flory, Linda Patricia Osuna Ibarra, Lester Lampert, and Anne Y Matsuura. A scalable microarchitecture for efficient instruction-driven signal synthesis and coherent qubit control. *arXiv preprint arXiv:2205.06851*, 2022.
- [130] Saeed A Khan, Ryan Kaufman, Boris Mesits, Michael Hatridge, and Hakan E Türeci. Practical trainable temporal postprocessor for multistate quantum measurement. *PRX Quantum*, 5(2):020364, 2024.
- [131] K. Khodjasteh and D. A. Lidar. Fault-tolerant quantum dynamical decoupling. *Phys. Rev. Lett.*, 95:180501, Oct 2005.
- [132] Kaveh Khodjasteh and Daniel A. Lidar. Performance of deterministic dynamical decoupling schemes: Concatenated and periodic pulse sequences. *Phys. Rev. A*, 75:062310, Jun 2007.
- [133] Dongmin Kim, Jeonggeun Seo, Youngsun Han, and Yongtae Kim. Constraint-optimal driven allocation for scalable quantum error correction decoder scheduling. *Physical Review A*, 113(4):042459, 2026.
- [134] Jihye Kim, Byungdu Oh, Yonuk Chong, Euyheon Hwang, and Daniel K Park. Quantum readout error mitigation via deep learning. *New Journal of Physics*, 24(7):073009, July 2022.
- [135] Yosep Kim, Alexis Morvan, Long B Nguyen, Ravi K Naik, Christian Jünger, Larry Chen, John Mark Kreikebaum, David I Santiago, and Irfan Siddiqi. High-fidelity three-qubit toffoli gate for fixed-frequency superconducting qubits. *Nature Physics*, pages 1–6, 2022.
- [136] A Yu Kitaev. Fault-tolerant quantum computation by anyons. *Annals of Physics*, 303(1):2–30, 2003.

- [137] Alexander Knapen, Guanchen Tao, Jacob Mack, Tomas Bruno, Mehdi Saligane, Dennis Sylvester, Qirui Zhang, and Gokul Subramanian Ravi. Pinball: A cryogenic predecoder for quantum error correction decoding under circuit-level noise. In *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, page 1–17. IEEE, January 2026.
- [138] Emanuel Knill, Raymond Laflamme, and Wojciech H Zurek. Resilient quantum computation: error models and thresholds. In *Proceedings of the Royal Society of London A: Mathematical, Physical and Engineering Sciences*, volume 454, pages 365–384. The Royal Society, 1998.
- [139] Emanuel Knill, Raymond Laflamme, and Wojciech H. Zurek. Resilient quantum computation: error models and thresholds. *Proceedings of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences*, 454(1969):365–384, January 1998.
- [140] Jens Koch, Terri M Yu, Jay Gambetta, Andrew A Houck, David I Schuster, Johannes Majer, Alexandre Blais, Michel H Devoret, Steven M Girvin, and Robert J Schoelkopf. Charge-insensitive qubit design derived from the cooper pair box. *Physical Review A—Atomic, Molecular, and Optical Physics*, 76(4):042319, 2007.
- [141] Stergios Koutsoumpas, Hasan Sayginel, Mark Webster, and Dan E Browne. Automorphism ensemble decoding of quantum ldpc codes. *arXiv preprint arXiv:2503.01738*, 2025.
- [142] Alexey A Kovalev and Leonid P Pryadko. Fault tolerance of quantum low-density parity check codes with sublinear distance scaling. *Physical Review A*, 87(2):020304, 2013.
- [143] P. Krantz, M. Kjaergaard, F. Yan, T. P. Orlando, S. Gustavsson, and W. D. Oliver. A quantum engineer's guide to superconducting qubits. *Applied Physics Reviews*, 6(2):021318, June 2019.
- [144] Sebastian Krinner, Nathan Lacroix, Ants Remm, Agustin Di Paolo, Elie Genois, Catherine Leroux, Christoph Hellings, Stefania Lazar, Francois Swiadek, Johannes Herrmann, et al. Realizing repeated quantum error correction in a distance-three surface code. *arXiv preprint arXiv:2112.03708*, 2021.

- [145] N. Lacroix, A. Bourassa, F. J. H. Heras, L. M. Zhang, J. Bausch, A. W. Senior, T. Edlich, N. Shutty, V. Sivak, A. Bengtsson, M. McEwen, O. Higgott, D. Kafri, J. Claes, A. Morvan, Z. Chen, A. Zalcman, S. Madhuk, R. Acharya, L. Aghababaie Beni, G. Aigeldinger, R. Alcaraz, T. I. Andersen, M. Ansmann, F. Arute, K. Arya, A. Asfaw, J. Atalaya, R. Babbush, B. Ballard, J. C. Bardin, A. Bilmes, S. Blackwell, J. Bovaird, D. Bowers, L. Brill, M. Broughton, D. A. Browne, B. Buchea, B. B. Buckley, T. Burger, B. Burkett, N. Bushnell, A. Cabrera, J. Campero, H.-S. Chang, B. Chiaro, L.-Y. Chih, A. Y. Cleland, J. Cogan, R. Collins, P. Conner, W. Courtney, A. L. Crook, B. Curtin, S. Das, S. Demura, L. De Lorenzo, A. Di Paolo, P. Donohoe, I. Drozdov, A. Dunsworth, A. Eickbusch, A. Moshe Elbag, M. Elzouka, C. Erickson, V. S. Ferreira, L. Flores Burgos, E. Forati, A. G. Fowler, B. Foxen, S. Ganjam, G. Garcia, R. Gasca, É. Genois, W. Giang, D. Gilboa, R. Gosula, A. Grajales Dau, D. Graumann, A. Greene, J. A. Gross, T. Ha, S. Habegger, M. Hansen, M. P. Harrigan, S. D. Harrington, S. Heslin, P. Heu, R. Hiltermann, J. Hilton, S. Hong, H.-Y. Huang, A. Huff, W. J. Huggins, E. Jeffrey, Z. Jiang, X. Jin, C. Joshi, P. Juhas, A. Kabel, H. Kang, A. H. Karamlou, K. Kechedzhi, T. Khaira, T. Khattar, M. Khezri, S. Kim, P. V. Klimov, B. Kobrin, A. N. Korotkov, F. Kostritsa, J. Mark Kreikebaum, V. D. Kurilovich, D. Landhuis, T. Lange-Dei, B. W. Langle, P. Laptev, K.-M. Lau, J. Ledford, K. Lee, B. J. Lester, L. Le Guevel, W. Yan Li, Y. Li, A. T. Lill, W. P. Livingston, A. Lochar, E. Lucero, D. Lundahl, A. Lunt, A. Maloney, S. Mandrà, L. S. Martin, O. Martin, C. Maxfield, J. R. McClean, S. Meeks, A. Megrant, K. C. Miao, R. Molavi, S. Molina, S. Montazeri, R. Movassagh, C. Neill, M. Newman, A. Nguyen, M. Nguyen, C.-H. Ni, M. Y. Niu, L. Oas, W. D. Oliver, R. Orosco, K. Ottosson, A. Pizzuto, R. Potter, O. Pritchard, C. Quintana, G. Ramachandran, M. J. Reagor, R. Resnick, D. M. Rhodes, G. Roberts, E. Rosenberg, E. Rosenfeld, E. Rossi, P. Roushan, K. Sankaragomathi, H. F. Schurkus, M. J. Shearn, A. Shorter, V. Shvarts, S. Small, W. Clarke Smith, S. Springer, G. Sterling, J. Suchard, A. Szasz, A. Sztein, D. Thor, E. Tomita, A. Torres, M. Mert Torunbalci, A. Vaishnav, J. Vargas, S. Vdovichev, G. Vidal, C. Vollgraf Heidweiller, S. Waltman, J. Waltz, S. X. Wang, B. Ware, T. Weidel, T. White, K. Wong, B. W. K. Woo, M. Woodson, C. Xing, Z. Jamie Yao, P. Yeh, B. Ying, J. Yoo, N. Yosri, G. Young, Y. Zhang, N. Zhu, N. Zobrist, H. Neven, P. Kohli, A. Davies, S. Boixo, J. Kelly, C. Jones, C. Gidney, and K. J. Satzinger. Scaling and logic in the colour code on a superconducting quantum processor. *Nature*, 645(8081):614–619, May 2025.
- [146] Lingling Lao, Prakash Murali, Margaret Martonosi, and Dan Browne. Designing calibration and expressivity-efficient instruction sets for quantum computing. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, pages 846–859, 2021.
- [147] Ang Li, Samuel Stein, Sriram Krishnamoorthy, and James Ang. Qasmbench: A low-level qasm benchmark suite for nist evaluation and simulation, 2021.

- [148] Kangbo Li, R. McDermott, and Maxim G. Vavilov. Hardware-efficient qubit control with single-flux-quantum pulse sequences. *Physical Review Applied*, 12(1), July 2019.
- [149] Benjamin Lienhard, Antti Vepsäläinen, Luke C.G. Gavia, Cole R. Hoffer, Jack Y. Qiu, Diego Ristè, Matthew Ware, David Kim, Roni Winik, Alexander Melville, Bethany Niedzielski, Jonilyn Yoder, Guilhem J. Ribeill, Thomas A. Ohki, Hari K. Krovi, Terry P. Orlando, Simon Gustavsson, and William D. Oliver. Deep-neural-network discrimination of multiplexed superconducting-qubit states. *Physical Review Applied*, 17(1), January 2022.
- [150] Sophia Fuhui Lin, Sara Sussman, Casey Duckering, Pranav S Mundada, Jonathan M Baker, Rohan S Kumar, Andrew A Houck, and Frederic T Chong. Let each quantum bit choose its basis gates. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 1042–1058. IEEE, 2022.
- [151] Sophia Fuhui Lin, Joshua Vizlai, Kaitlin N. Smith, Gokul Subramanian Ravi, Charles Yuan, Frederic T. Chong, and Benjamin J. Brown. Codesign of quantum error-correcting codes and modular chiplets in the presence of defects. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS '24*, page 216–231. ACM, April 2024.
- [152] Daniel Litinski. A game of surface codes: Large-scale quantum computing with lattice surgery. *Quantum*, 3:128, March 2019.
- [153] Daniel Litinski. Magic State Distillation: Not as Costly as You Think. *Quantum*, 3:205, December 2019.
- [154] Daniel Litinski and Felix von Oppen. Lattice surgery with a twist: Simplifying clifford gates of surface codes. *Quantum*, 2:62, May 2018.
- [155] Bowen Liu and Zhaoran Rena Huang. A cryogenic hybrid photonic/cmos controller architecture for scalable superconducting qubit control. *arXiv preprint arXiv:2606.10114*, 2026.
- [156] Chuan-Hong Liu, Andrew Ballard, David Olaya, Daniel R Schmidt, John Biesecker, Tammy Lucas, Joel Ullom, Shravan Patel, Owen Rafferty, Alexander Opremcak, et al. Single flux quantum-based digital control of superconducting qubits in a multichip module. *PRX quantum*, 4(3):030310, 2023.
- [157] Chung Laung Liu and James W Layland. Scheduling algorithms for multiprogramming in a hard-real-time environment. *Journal of the ACM (JACM)*, 20(1):46–61, 1973.

- [158] Ye-Hua Liu and David Poulin. Neural belief-propagation decoders for quantum error-correcting codes. *Physical review letters*, 122(20):200501, 2019.
- [159] Namitha Liyanage, Yue Wu, Alexander Deters, and Lin Zhong. Scalable quantum error correction for surface codes using FPGA. In *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*, volume 1, pages 916–927. IEEE, 2023.
- [160] Namitha Liyanage, Yue Wu, Emmet Houghton, and Lin Zhong. Network-integrated decoding system for real-time quantum error correction with lattice surgery. *arXiv preprint arXiv:2504.11805*, 2025.
- [161] Namitha Liyanage, Yue Wu, Siona Tagare, and Lin Zhong. Fpga-based distributed union-find decoder for surface codes. *IEEE Transactions on Quantum Engineering*, 5:1–18, 2024.
- [162] Christoph Loeffler, Adriaan Ligtenberg, and George S Moschytz. Practical fast 1-d dct algorithms with 11 multiplications. In *International Conference on Acoustics, Speech, and Signal Processing*,, pages 988–991. IEEE, 1989.
- [163] Thomas Lubinski, Sonika Johri, Paul Varosy, Jeremiah Coleman, Luning Zhao, Jason Necaie, Charles H. Baldwin, Karl Mayer, and Timothy Proctor. Application-oriented performance benchmarks for quantum computing. *IEEE Transactions on Quantum Engineering*, 4:1–32, 2023.
- [164] Christopher M. *Pattern Recognition and Machine Learning*. Information Science and Statistics. Springer, New York, NY, 1 edition, August 2006.
- [165] Arshpreet Singh Maan, Francisco Garcia Herrero, Alexandru Paler, and Valentin Savin. Decoding correlated errors in quantum LDPC codes. 2025. *arXiv:2510.14060*.
- [166] C. Macklin, K. O’Brien, D. Hover, M. E. Schwartz, V. Bolkhovsky, X. Zhang, W. D. Oliver, and I. Siddiqi. A near-quantum-limited josephson traveling-wave parametric amplifier. *Science*, 350(6258):307–310, October 2015.
- [167] Easwar Magesan, J. M. Gambetta, and Joseph Emerson. Scalable and robust randomized benchmarking of quantum processes. *Physical Review Letters*, 106(18), May 2011.
- [168] Easwar Magesan, Jay M. Gambetta, A. D. Córcoles, and Jerry M. Chow. Machine learning for discriminating quantum measurement trajectories and improving readout. *Phys. Rev. Lett.*, 114:200501, May 2015.
- [169] Easwar Magesan, Jay M Gambetta, Antonio D Córcoles, and Jerry M Chow. Machine learning for discriminating quantum measurement trajectories and improving readout. *Physical review letters*, 114(20):200501, 2015.

- [170] JF Marques, BM Varbanov, MS Moreira, Hany Ali, Nandini Muthusubramanian, Christos Zachariadis, Francesco Battistel, Marc Beekman, Nadia Haider, Wouter Vlothuizen, et al. Logical-qubit operations in an error-detecting surface code. *Nature Physics*, 18(1):80–86, 2022.
- [171] Lukasz Masko, Pierre–Francois Dutot, Gregory Mounie, Denis Trystram, and Marek Tudruj. *Scheduling Moldable Tasks for Dynamic SMP Clusters in SoC Technology*, page 879–887. Springer Berlin Heidelberg, 2006.
- [172] Thilo Maurer, Markus Bühler, Michael Kröner, Frank Haverkamp, Tristan Müller, Drew Vandeth, and Blake R. Johnson. Real-time decoding of the gross code memory with FPGAs, 2025.
- [173] Satvik Maurya, Thilo Maurer, Markus Bühler, Drew Vandeth, and Michael E Beverland. Fpga-tailored algorithms for real-time decoding of quantum ldpc codes. *arXiv preprint arXiv:2511.21660*, 2025.
- [174] Satvik Maurya, Abtin Molavi, Aws Albarghouthi, and Swamit Tannu. A case for elastic quantum error correction decoders. In *Proceedings of the 21st European Conference on Computer Systems*, EUROSYS ’26, page 514–531. ACM, April 2026.
- [175] Satvik Maurya, Chaithanya Naik Mude, Benjamin Lienhard, and Swamit Tannu. Understanding side-channel vulnerabilities in superconducting qubit readout architectures. In *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, volume 1, pages 1177–1183. IEEE, 2024.
- [176] Satvik Maurya, Chaithanya Naik Mude, William D. Oliver, Benjamin Lienhard, and Swamit Tannu. Scaling qubit readout with hardware efficient machine learning architectures. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*, ISCA ’23, page 1–13. ACM, 2023.
- [177] Satvik Maurya and Swamit Tannu. Compagt: Compressed waveform memory architecture for scalable qubit control. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, page 1059–1077. IEEE, October 2022.
- [178] Satvik Maurya and Swamit Tannu. Synchronization for fault-tolerant quantum computers. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, SIGARCH ’25, page 1370–1385. ACM, 2025.
- [179] Satvik Maurya, Joshua Visslai, Nithin Raveendran, Poulami Das, and Swamit Tannu. decoder-bench: Benchmarking decoders for quantum error correction. In *2025 IEEE International Symposium on Workload Characterization (IISWC)*, pages 286–295. IEEE, 2025.

- [180] R. McDermott and M. G. Vavilov. Accurate qubit control with single flux quantum pulses. *Phys. Rev. Applied*, 2:014007, Jul 2014.
- [181] R McDermott, M G Vavilov, B L T Plourde, F K Wilhelm, P J Liebermann, O A Mukhanov, and T A Ohki. Quantum–classical interface based on single flux quantum digital logic. *Quantum Science and Technology*, 3(2):024004, January 2018.
- [182] M. McEwen, D. Kafri, Z. Chen, J. Atalaya, K. J. Satzinger, C. Quintana, P. V. Klimov, D. Sank, C. Gidney, A. G. Fowler, F. Arute, K. Arya, B. Buckley, B. Burkett, N. Bushnell, B. Chiaro, R. Collins, S. Demura, A. Dunsworth, C. Erickson, B. Foxen, M. Giustina, T. Huang, S. Hong, E. Jeffrey, S. Kim, K. Kechedzhi, F. Kostritsa, P. Laptev, A. Megrant, X. Mi, J. Mutus, O. Naaman, M. Neeley, C. Neill, M. Niu, A. Paler, N. Redd, P. Roushan, T. C. White, J. Yao, P. Yeh, A. Zalcman, Yu Chen, V. N. Smelyanskiy, John M. Martinis, H. Neven, J. Kelly, A. N. Korotkov, A. G. Petukhov, and R. Barends. Removing leakage-induced correlated errors in superconducting quantum error correction. *Nature Communications*, 12(1), March 2021.
- [183] Matt McEwen, Dave Bacon, and Craig Gidney. Relaxing hardware requirements for surface code circuits using time-dynamics. *Quantum*, 7:1172, November 2023.
- [184] David C. McKay, Christopher J. Wood, Sarah Sheldon, Jerry M. Chow, and Jay M. Gambetta. Efficient Z-Gates for Quantum Computing. *Physical Review A*, 96(2):022330, August 2017. arXiv: 1612.00858 version: 2.
- [185] Evan McKinney, Girgis Falstin, Israa G. Yusuf, Gaurav Agarwal, Michael Hatridge, and Alex K. Jones. Towards error budgeting for superconducting modular quantum architecture designs, 2024.
- [186] Evan McKinney, Mingkang Xia, Chao Zhou, Pinlei Lu, Michael Hatridge, and Alex K. Jones. Co-designed architectures for modular superconducting quantum computers. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, page 759–772. IEEE, February 2023.
- [187] Dongmoon Min, Junpyo Kim, Junhyuk Choi, Ilkwon Byun, Masamitsu Tanaka, Koji Inoue, and Jangwoo Kim. Qisim: Architecting 10+ k qubit qc interfaces toward quantum supremacy. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*, pages 1–16, 2023.
- [188] Chris Monroe, David M Meekhof, Barry E King, Wayne M Itano, and David J Wineland. Demonstration of a fundamental quantum logic gate. *Physical review letters*, 75(25):4714, 1995.
- [189] Chaithanya Naik Mude, Satvik Maurya, Benjamin Lienhard, and Swamit Tannu. Efficient and scalable architectures for multi-level superconducting qubit readout. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, pages 1–7. IEEE, 2025.

- [190] Chaithanya Naik Mude, Linipun Phuttitarn, Satvik Maurya, Kunal Sinha, Mark Saffman, and Swamit Tannu. Enabling fast and accurate neutral atom readout through image denoising. *arXiv preprint arXiv:2510.25982*, 2025.
- [191] Tristan Müller, Thomas Alexander, Michael E Beverland, Markus Bühler, Blake R Johnson, Thilo Maurer, and Drew Vandeth. Improved belief propagation is sufficient for real-time decoding of quantum memory. *arXiv preprint arXiv:2506.01779*, 2025.
- [192] Kevin Murphy, Yair Weiss, and Michael I Jordan. Loopy belief propagation for approximate inference: An empirical study. *arXiv preprint arXiv:1301.6725*, 2013.
- [193] Áron Márton and János K. Asbóth. Optimal number of stabilizer measurement rounds in an idling surface code patch, 2024.
- [194] Yasunobu Nakamura, Yu A Pashkin, and JS Tsai. Coherent control of macroscopic quantum states in a single-cooper-pair box. *nature*, 398(6730):786–788, 1999.
- [195] C. Neill, P. Roushan, K. Kechedzhi, S. Boixo, S. V. Isakov, V. Smelyanskiy, A. Megrant, B. Chiaro, A. Dunsworth, K. Arya, R. Barends, B. Burkett, Y. Chen, Z. Chen, A. Fowler, B. Foxen, M. Giustina, R. Graff, E. Jeffrey, T. Huang, J. Kelly, P. Klimov, E. Lucero, J. Mutus, M. Neeley, C. Quintana, D. Sank, A. Vainsencher, J. Wenner, T. C. White, H. Neven, and J. M. Martinis. A blueprint for demonstrating quantum supremacy with superconducting qubits. *Science*, 360(6385):195–199, April 2018.
- [196] Michael A Nielsen and Isaac L Chuang. *Cambridge series on information and the natural sciences: Quantum computation and quantum information*. Cambridge University Press, Cambridge, England, October 2000.
- [197] Vukan Ninkovic, Ognjen Kundacina, Dejan Vukobratovic, Christian Häger, and Alexandre Graell i Amat. Decoding quantum ldpc codes using graph neural networks. In *GLOBECOM 2024 - 2024 IEEE Global Communications Conference*, page 3479–3484. IEEE, December 2024.
- [198] Mohammadreza Noormandipour and Tobias Haug. Maxsat decoders for arbitrary css codes, 2024.
- [199] NVIDIA. NVIDIA and QuEra Decode Quantum Errors with AI, 2025. Accessed: June 18, 2025.
- [200] Nissim Ofek, Andrei Petrenko, Reinier Heeres, Philip Reinhold, Zaki Leghtas, Brian Vlastakis, Yehan Liu, Luigi Frunzio, Steven M Girvin, Liang Jiang, et al. Extending the lifetime of a quantum bit with error correction in superconducting circuits. *Nature*, 536(7617):441–445, 2016.

- [201] Kai R Ott, Bence Hetényi, and Michael E Beverland. Decision-tree decoders for general quantum ldpc codes. *arXiv preprint arXiv:2502.16408*, 2025.
- [202] Alptekin Pamuk. An FPGA implementation architecture for decoding of polar codes. In *2011 8th International symposium on wireless communication systems*, pages 437–441. IEEE, 2011.
- [203] Pavel Panteleev and Gleb Kalachev. Degenerate quantum ldpc codes with good finite length performance. *Quantum*, 5:585, 2021.
- [204] Kun Hee Park, Yung Szen Yap, Yuanzheng Paul Tan, Christoph Hufnagel, Long Hoang Nguyen, Karn Hwa Lau, Stavros Efthymiou, Stefano Carrazza, Rangga P. Budoyo, and Rainer Dumke. Icarus-q: A scalable rfsoc-based control system for superconducting quantum computers, 2021.
- [205] Gennady Pekhimenko, Vivek Seshadri, Onur Mutlu, Michael A Kozuch, Phillip B Gibbons, and Todd C Mowry. Base-delta-immediate compression: Practical data compression for on-chip caches. In *2012 21st international conference on parallel architectures and compilation techniques (PACT)*, pages 377–388. IEEE, 2012.
- [206] Thomas Picot, A Lupășcu, S Saito, CJPM Harmans, and JE Mooij. Role of relaxation in the quantum measurement of a superconducting qubit using a nonlinear oscillator. *Physical Review B*, 78(13):132508, 2008.
- [207] David Poulin and Yeojin Chung. On the iterative decoding of sparse quantum codes. *Quantum Information & Computation*, 8(10):987–1000, 2008.
- [208] Hendrik Poulsen Nautrup, Nicolai Friis, and Hans J. Briegel. Fault-tolerant interface between quantum memories and quantum processors. *Nature Communications*, 8(1), November 2017.
- [209] John Preskill. Quantum computing in the nisc era and beyond. *Quantum*, 2:79, 2018.
- [210] Quantinuum. Quantinuum Unveils Accelerated Roadmap to Achieve Universal, Fully Fault-Tolerant Quantum Computing by 2030, 2025. Accessed: September 18, 2025.
- [211] QuERA. QuERA Quantum Roadmap, 2026. Accessed: June 18, 2026.
- [212] Nils Quetschlich, Lukas Burgholzer, and Robert Wille. MQT Bench: Benchmarking software and design automation tools for quantum computing. *Quantum*, 2023. MQT Bench is available at <https://www.cda.cit.tum.de/mqtbench/>.

- [213] Gokul Subramanian Ravi, Jonathan M. Baker, Arash Fayyazi, Sophia Fuhui Lin, Ali Javadi-Abhari, Massoud Pedram, and Frederic T. Chong. Better than worst-case decoding for quantum error correction. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. ACM, January 2023.
- [214] T Recommendation. 81: Digital compression and coding of continuous-tone still images—requirements and guidelines. Technical report, Technical report, CCITT, 1992.
- [215] Chad Rigetti and Michel Devoret. Fully microwave-tunable universal gates in superconducting qubits with linear couplings and fixed transition frequencies. *Physical Review B*, 81(13), April 2010.
- [216] Santiago Rodrigo, Sergi Abadal, Eduard Alarcon, and Carmen G. Almudever. Will quantum computers scale without inter-chip comms? a structured design exploration to the monolithic vs distributed architectures quest. In *2020 XXXV Conference on Design of Circuits and Integrated Systems (DCIS)*, page 1–6. IEEE, November 2020.
- [217] Joschka Roffe, David R. White, Simon Burton, and Earl Campbell. Decoding across the quantum low-density parity-check code landscape. *Physical Review Research*, 2(4), December 2020.
- [218] Katharina Rojan, Daniel M. Reich, Igor Dotsenko, Jean-Michel Raimond, Christiane P. Koch, and Giovanna Morigi. Arbitrary-quantum-state preparation of a harmonic oscillator via optimal control. *Phys. Rev. A*, 90:023824, Aug 2014.
- [219] Neil J. Ross and Peter Selinger. Optimal ancilla-free clifford+t approximation of z-rotations, 2014.
- [220] Colm A Ryan, Blake R Johnson, Jay M Gambetta, Jerry M Chow, Marcus P Da Silva, Oliver E Dial, and Thomas A Ohki. Tomography via correlation of noisy measurement records. *Physical Review A*, 91(2):022118, 2015.
- [221] Satvik Maurya and Abtin Molavi and Aws Albarghouthi and Swamit Tannu. Decoder Resource Estimator, 2026. Available at <https://doi.org/10.5281/zenodo.18555904>.
- [222] Eyob A. Sete, Nicolas Didier, Angela Q. Chen, Shobhan Kulshreshtha, Riccardo Manti, and Stefano Poletto. Parametric-resonance entangling gates with a tunable coupler. *Physical Review Applied*, 16(2), August 2021.
- [223] Eyob A. Sete, John M. Martinis, and Alexander N. Korotkov. Quantum theory of a bandpass purcell filter for qubit readout. *Physical Review A*, 92(1), July 2015.

- [224] Sha Shen, Weiwei Shen, Yibo Fan, and Xiaoyang Zeng. A unified 4/8/16/32-point integer idct architecture for multiple video coding standards. In *2012 IEEE International Conference on Multimedia and Expo*, pages 788–793. IEEE, 2012.
- [225] Yunong Shi, Nelson Leung, Pranav Gokhale, Zane Rossi, David I. Schuster, Henry Hoffmann, and Frederic T. Chong. Optimized compilation of aggregated instructions for realistic quantum computers. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. ACM, April 2019.
- [226] Peter W Shor. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th annual symposium on foundations of computer science*, pages 124–134. Ieee, 1994.
- [227] Peter W. Shor. Scheme for reducing decoherence in quantum computer memory. *Physical Review A*, 52(4):R2493–R2496, October 1995.
- [228] Madhavapeddi Shreedhar and George Varghese. Efficient fair queuing using deficit round-robin. *IEEE/ACM Transactions on networking*, 4(3):375–385, 1996.
- [229] Noah Shutty and Christopher Chamberland. Decoding merged color-surface codes and finding fault-tolerant clifford circuits using solvers for satisfiability modulo theories. *Physical Review Applied*, 18(1):014072, 2022.
- [230] Adam Siegel, Armands Strikis, Thomas Flatters, and Simon Benjamin. Adaptive surface code for quantum error correction in the presence of temporary or permanent defects. *Quantum*, 7:1065, July 2023.
- [231] Agustin Silva and Alvaro Orgaz-Fuertes. Manarat: A scalable qick-based control system for superconducting quantum processors supporting synchronized control of 10 flux-tunable qubits, 2025.
- [232] Ashish Singhadia, Meghan Mamillapalli, and Indrajit Chakrabarti. Hardware-efficient 2d-dct/idct architecture for portable hevc-compliant devices. *IEEE Transactions on Consumer Electronics*, 66(3):203–212, 2020.
- [233] Luka Skoric, Dan E. Browne, Kenton M. Barnes, Neil I. Gillespie, and Earl T. Campbell. Parallel window decoding enables scalable fault tolerant quantum computation. *Nature Communications*, 14(1), November 2023.
- [234] Luka Skoric, Dan E Browne, Kenton M Barnes, Neil I Gillespie, and Earl T Campbell. Parallel window decoding enables scalable fault tolerant quantum computation. *Nature Communications*, 14(1):7040, 2023.

- [235] Alistair W. R. Smith, Kiran E. Khosla, Chris N. Self, and M. S. Kim. Qubit readout error mitigation with bit-flip averaging. *Sci. Adv.*, 7(47):abi8009, 2021.
- [236] Kaitlin N. Smith, Gokul Subramanian Ravi, Jonathan M. Baker, and Frederic T. Chong. Scaling superconducting quantum computers with chiplet architectures. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 1092–1109, 2022.
- [237] Samuel C. Smith, Benjamin J. Brown, and Stephen D. Bartlett. Local predecoder to reduce the bandwidth and latency of quantum error correction. *Physical Review Applied*, 19(3), March 2023.
- [238] A. Spörl, T. Schulte-Herbrüggen, S. J. Glaser, V. Bergholm, M. J. Storcz, J. Ferber, and F. K. Wilhelm. Optimal control of coupled josephson qubits. *Phys. Rev. A*, 75:012302, Jan 2007.
- [239] A. M. Steane. Simple quantum error-correcting codes. *Physical Review A*, 54(6):4741–4751, December 1996.
- [240] Leandro Stefanazzi, Kenneth Treptow, Neal Wilcer, Chris Stoughton, Collin Bradford, Sho Uemura, Silvia Zorzetti, Salvatore Montella, Gustavo Cancelo, Sara Sussman, et al. The qick (quantum instrumentation control kit): Readout and control for qubits and detectors. *Review of Scientific Instruments*, 93(4):044709, 2022.
- [241] Leandro Stefanazzi, Kenneth Treptow, Neal Wilcer, Chris Stoughton, Collin Bradford, Sho Uemura, Silvia Zorzetti, Salvatore Montella, Gustavo Cancelo, Sara Sussman, Andrew Houck, Shefali Saxena, Horacio Arnaldi, Ankur Agrawal, Helin Zhang, Chunyang Ding, and David I. Schuster. The qick (quantum instrumentation control kit): Readout and control for qubits and detectors. *Review of Scientific Instruments*, 93(4), April 2022.
- [242] Samuel Stein, Shifan Xu, Andrew W. Cross, Theodore J. Yoder, Ali Javadi-Abhari, Chenxu Liu, Kun Liu, Zeyuan Zhou, Charles Guinn, Yufei Ding, Yongshan Ding, and Ang Li. Architectures for heterogeneous quantum error correction codes, 2024.
- [243] Vladimir M. Stojanović, A. Fedorov, A. Wallraff, and C. Bruder. Quantum-control approach to realizing a toffoli gate in circuit QED. *Physical Review B*, 85(5), February 2012.
- [244] Armands Strikis, Simon C. Benjamin, and Benjamin J. Brown. Quantum computing is scalable on a planar array of qubits with fabrication defects. *Physical Review Applied*, 19(6), June 2023.

- [245] Tom Struck, Javed Lindner, Arne Hollmann, Floyd Schauer, Andreas Schmidbauer, Dominique Bougeard, and Lars R Schreiber. Robust and fast post-processing of single-shot spin qubit detection events with a neural network. *Scientific Reports*, 11(1):16203, 2021.
- [246] Neereja Sundaresan, Theodore J. Yoder, Youngseok Kim, Muyuan Li, Edward H. Chen, Grace Harper, Ted Thorbeck, Andrew W. Cross, Antonio D. Córcoles, and Maika Takita. Demonstrating multi-round subsystem quantum error correction using matching and maximum likelihood decoders. *Nature Communications*, 14(1), May 2023.
- [247] Vivienne Sze, Madhukar Budagavi, and Gary J Sullivan. High efficiency video coding (hevc). In *Integrated circuit and systems, algorithms and architectures*, volume 39, page 40. Springer, 2014.
- [248] Swamit S Tannu, Douglas M Carmean, and Moinuddin K Qureshi. Cryogenic-dram based memory system for scalable quantum computers: A feasibility study. In *Proceedings of the International Symposium on Memory Systems*, pages 189–195, 2017.
- [249] Swamit S. Tannu, Zachary A. Myers, Prashant J. Nair, Douglas M. Carmean, and Moinuddin K. Qureshi. Taming the instruction bandwidth of quantum computers via hardware-managed error correction. In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture*. ACM, October 2017.
- [250] Swamit S Tannu and Moinuddin K Qureshi. Mitigating measurement errors in quantum computers by exploiting state-dependent bias. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, pages 279–290, 2019.
- [251] Chenning Tao, Liqiang Lu, Size Zheng, Li-Wen Chang, Minghua Shen, Hanyu Zhang, Fangxin Liu, Kaiwen Zhou, and Jianwei Yin. Qtenon: Towards low-latency architecture integration for accelerating hybrid quantum-classical computing. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, pages 299–312, 2025.
- [252] Leandros Tassiulas and Anthony Ephremides. Stability properties of constrained queueing systems and scheduling policies for maximum throughput in multihop radio networks. In *29th IEEE Conference on Decision and Control*, pages 2130–2132. IEEE, 1990.
- [253] Barbara M Terhal. Quantum error correction for quantum memories. *Reviews of Modern Physics*, 87(2):307–346, 2015.
- [254] Barbara M. Terhal. Quantum error correction for quantum memories. *Reviews of Modern Physics*, 87(2):307–346, April 2015.

- [255] Mats O. Tholén, Riccardo Borgani, Giuseppe Ruggero Di Carlo, Andreas Bengtsson, Christian Križan, Marina Kudra, Giovanna Tancredi, Jonas Bylander, Per Delsing, Simone Gasparinetti, and David B. Haviland. Measurement and control of a superconducting quantum processor with a fully-integrated radio-frequency system on a chip, 2022.
- [256] Wuwei Tian, Liqiang Lu, Siwei Tan, Yun Liang, Tingting Li, Kaiwen Zhou, Xinghui Jia, and Jianwei Yin. Artery: Fast quantum feedback using branch prediction. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, pages 285–298, 2025.
- [257] Yu Tomita and Krysta M. Svore. Low-distance surface codes under realistic quantum noise. *Phys. Rev. A*, 90:062320, Dec 2014.
- [258] Kento Tsubouchi, Hayata Yamasaki, and Shiro Tamiya. Degeneracy cutting: A local and efficient post-processing for belief propagation decoding of quantum low-density parity-check codes. *arXiv preprint arXiv:2510.08695*, 2025.
- [259] George Turin. An introduction to matched filters. *IRE transactions on Information theory*, 6(3):311–329, 1960.
- [260] Javier Valls, Francisco Garcia-Herrero, Nithin Raveendran, and Bane Vasić. Syndrome-based min-sum vs osd-0 decoders: FPGA implementation and analysis for quantum ldpc codes. *IEEE Access*, 9:138734–138743, 2021.
- [261] Lieven MK Vandersypen, Matthias Steffen, Gregory Breyta, Costantino S Yannoni, Mark H Sherwood, and Isaac L Chuang. Experimental realization of shor’s quantum factoring algorithm using nuclear magnetic resonance. *Nature*, 414(6866):883–887, 2001.
- [262] Richard Versluis, Stefano Poletto, Nader Khammassi, Brian Tarasinski, Nadia Haider, David J Michalak, Alessandro Bruno, Koen Bertels, and Leonardo DiCarlo. Scalable quantum circuit and control for a superconducting surface code. *Physical Review Applied*, 8(3):034021, 2017.
- [263] Steve Vestal. Preemptive scheduling of multi-criticality systems with varying degrees of execution time assurance. In *28th IEEE International Real-Time Systems Symposium (RTSS 2007)*, page 239–243. IEEE, December 2007.
- [264] Joshua Vizslai, Jason D Chadwick, Sarang Joshi, Gokul Subramanian Ravi, Yanjing Li, and Frederic T Chong. Swiper: Minimizing fault-tolerant quantum program latency via speculative window decoding. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, pages 1386–1401, 2025.

- [265] Suhas Vittal, Poulami Das, and Moinuddin Qureshi. Astrea: Accurate quantum error-decoding via practical minimum-weight perfect-matching. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*, ISCA '23. ACM, June 2023.
- [266] T. Walter, P. Kurpiers, S. Gasparinetti, P. Magnard, A. Potočnik, Y. Salathé, M. Pechal, M. Mondal, M. Oppliger, C. Eichler, and A. Wallraff. Rapid high-fidelity single-shot dispersive readout of superconducting qubits. *Physical Review Applied*, 7(5), May 2017.
- [267] George Watkins, Hoang Minh Nguyen, Keelan Watkins, Steven Pearce, Hoi-Kwan Lau, and Alexandru Paler. A high performance compiler for very large scale surface code computations. *Quantum*, 8:1354, May 2024.
- [268] Stasiu Wolanski and Ben Barber. Ambiguity clustering: an accurate and efficient decoder for qldpc codes. *arXiv preprint arXiv:2406.14527*, 2024.
- [269] W. K. Wootters and W. H. Zurek. A single quantum cannot be cloned. *Nature*, 299(5886):802–803, October 1982.
- [270] Yue Wu, Binghong Li, Kathleen Chang, Shruti Puri, and Lin Zhong. Minimum-weight parity factor decoder for quantum error correction. *arXiv preprint arXiv:2508.04969*, 2025.
- [271] Yue Wu, Namitha Liyanage, and Lin Zhong. Lego: Qec decoding system architecture for dynamic circuits, 2024.
- [272] Yue Wu, Namitha Liyanage, and Lin Zhong. Micro blossom: Accelerated minimum-weight perfect matching decoding for quantum error correction. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 639–654, 2025.
- [273] Yue Wu, Namitha Liyanage, and Lin Zhong. Micro blossom: Accelerated minimum-weight perfect matching decoding for quantum error correction. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ASPLOS '25, page 639–654. ACM, March 2025.
- [274] Lei Xie, Jidong Zhai, ZhenXing Zhang, Jonathan Allcock, Shengyu Zhang, and Yi-Cong Zheng. Suppressing ZZ crosstalk of quantum computers through pulse and scheduling co-optimization. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. ACM, February 2022.
- [275] Xilinx Ultrascale. <https://docs.xilinx.com/v/u/en-US/ds890-ultrascale-overview>.

- [276] Yilun Xu, Gang Huang, Jan Balewski, Ravi Naik, Alexis Morvan, Bradley Mitchell, Kasra Nowrouzi, David I Santiago, and Irfan Siddiqi. Qubic: An open-source fpga-based control and measurement system for superconducting quantum information processors. *IEEE Transactions on Quantum Engineering*, 2:1–11, 2021.
- [277] Xiao Xue, Bishnu Patra, Jeroen P. G. van Dijk, Nodar Samkharadze, Sushil Subramanian, Andrea Corna, Brian Paquelet Wuetz, Charles Jeon, Farhana Sheikh, Esdras Juarez-Hernandez, Brando Perez Esparza, Huzaifa Rampurawala, Brent Carlton, Surej Ravikumar, Carlos Nieva, Sungwon Kim, Hyung-Jin Lee, Amir Sammak, Giordano Scappucci, Menno Veldhorst, Fabio Sebastiano, Masoud Babaie, Stefano Pellerano, Edoardo Charbon, and Lieven M. K. Vandersypen. CMOS-based cryogenic control of silicon quantum circuits. *Nature*, 593(7858):205–210, May 2021.
- [278] Zhaohui Yang, Dawei Ding, Qi Ye, Cupjin Huang, Jianxin Chen, and Yuan Xie. Reconfigurable quantum instruction set computers for high performance attainable on hardware. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 1523–1546, 2026.
- [279] Hanwen Yao, Waleed Abu Laban, Christian Häger, Alexandre Graell i Amat, and Henry D Pfister. Belief propagation decoding of quantum ldpc codes with guided decimation. In *2024 IEEE International Symposium on Information Theory (ISIT)*, pages 2478–2483. IEEE, 2024.
- [280] Theodore J. Yoder, Eddie Schoute, Patrick Rall, Emily Pritchett, Jay M. Gambetta, Andrew W. Cross, Malcolm Carroll, and Michael E. Beverland. Tour de gross: A modular quantum computer based on bivariate bicycle codes, 2025.
- [281] Youwei Zhao, Yangsen Ye, He-Liang Huang, Yiming Zhang, Dachao Wu, Huijie Guan, Qingling Zhu, Zuolin Wei, Tan He, Sirui Cao, et al. Realizing an error-correcting surface code with superconducting qubits. *arXiv preprint arXiv:2112.13505*, 2021.
- [282] Kaiwen Zhou, Liqiang Lu, Debin Xiang, Chenning Tao, Anbang Wu, Jingwen Leng, Fangxin Liu, Mingshuai Chen, and Jianwei Yin. Vegapunk: Accurate and fast decoding for quantum ldpc codes with online hierarchical algorithm and sparse accelerator. In *Proceedings of the 2025 58th IEEE/ACM International Symposium on Microarchitecture, MICRO 2025*, page 719–732. ACM, October 2025.
- [283] Zhiwei Zhou and Zhongliang Pan. Effective hardware accelerator for 2d dct/idct using improved loeffler architecture. *IEEE Access*, 10:11011–11020, 2022.
- [284] Abbas B. Ziad, Ankit Zalawadiya, Canberk Topal, Joan Camps, György P. Gehér, Matthew P. Stafford, and Mark L. Turner. Local clustering decoder: a fast and adaptive hardware decoder for the surface code, 2024.